

CH 5 WORKING WITH SPRING DATA JPA

Practical No. 1: Introduction to JPA - Entity Class and Persisting a Single Entity

Objective:

To understand the requirements of a JPA entity class (no-arg constructor, @Entity, @Id, @GeneratedValue),

configure a relational datasource in application.properties, and persist/retrieve an entity using Spring Data JPA's

CrudRepository.

Class Name(s) with Attributes:

Employee (@Entity)

- empId : int (@Id @GeneratedValue)
- empName : String
- department : String
- salary : double

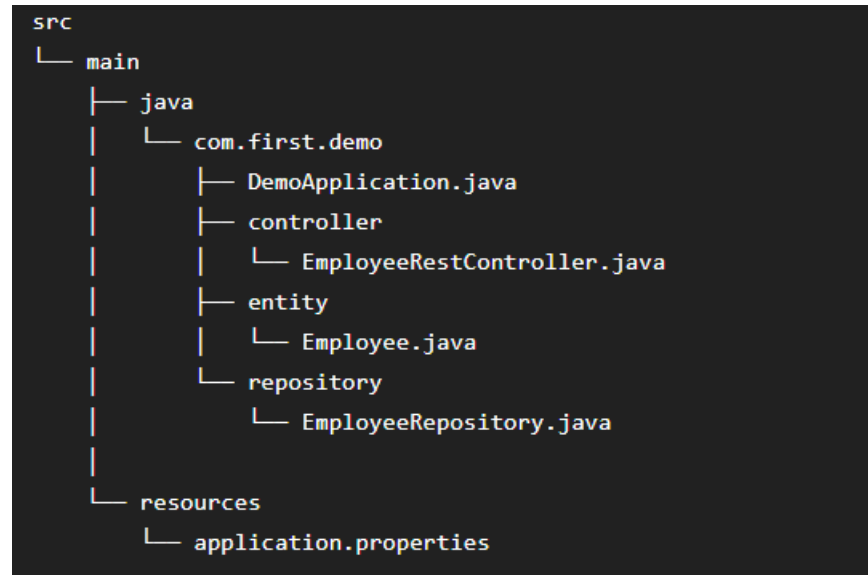
EmployeeRepository (interface extends CrudRepository<Employee, Integer>)

- Inherited: save(), findAll(), findById(), deleteById()

EmployeeRestController (@RestController)

- employeeRepository : EmployeeRepository

PROJECT STRUCTURE



Dependencies

- Spring Web
- Spring Data JPA
- MySQL Driver

Database

Database Name: employee_db

Table: employee (created automatically by Hibernate)

Employee.java(ENTITY model class)

```
package com.first.demo.entity;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
```

```
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;

@Entity
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer empId;

    private String empName;
    private String department;
    private double salary;

    // No-Argument Constructor
    public Employee() {
    }

    // Parameterized Constructor
    public Employee(Integer empId, String empName, String department, double salary) {
        this.empId = empId;
        this.empName = empName;
        this.department = department;
        this.salary = salary;
    }

    // Getter and Setter for empId
```

```
public Integer getEmpId() {  
    return empId;  
}  
  
public void setEmpId(Integer empId) {  
    this.empId = empId;  
}  
  
// Getter and Setter for empName  
public String getEmpName() {  
    return empName;  
}  
  
public void setEmpName(String empName) {  
    this.empName = empName;  
}  
  
// Getter and Setter for department  
public String getDepartment() {  
    return department;  
}  
  
public void setDepartment(String department) {  
    this.department = department;  
}
```



```
// Getter and Setter for salary  
public double getSalary() {  
    return salary;  
}  
  
public void setSalary(double salary) {  
    this.salary = salary;  
}  
}
```

EmployeeRepository.java(Repository file)

```
package com.first.demo.repository;  
  
import org.springframework.data.repository.CrudRepository;  
import com.first.demo.entity.Employee;  
  
public interface EmployeeRepository extends CrudRepository<Employee, Integer> {  
  
}
```

EmployeeController.java(Repository file)

```
package com.first.demo.controller;

import java.util.Optional;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;
import com.first.demo.entity.Employee;
import com.first.demo.repository.EmployeeRepository;
import org.springframework.web.bind.annotation.RequestBody;

@RestController
@RequestMapping("/api/employees")
public class EmployeeRestController {

    @Autowired
    private EmployeeRepository employeeRepository;

    // POST - Save Employee
    @PostMapping
    public Employee addEmployee(@RequestBody Employee employee) {
        return employeeRepository.save(employee);
    }
}
```

```
// GET - Get All Employees

@GetMapping
public Iterable<Employee> getAllEmployees() {

    return employeeRepository.findAll();

}

// GET - Get Employee By ID

@GetMapping("/{id}")
public Optional<Employee> getEmployee(@PathVariable Integer id) {

    return employeeRepository.findById(id);

}

// PUT - Update Employee

@PutMapping("/{id}")
public Employee updateEmployee(@PathVariable Integer id,

                                @RequestBody Employee employee) {

    employee.setEmpId(id);

    return employeeRepository.save(employee);

}

// DELETE - Delete Employee

@DeleteMapping("/{id}")
public String deleteEmployee(@PathVariable Integer id) {

    employeeRepository.deleteById(id);

    return "Employee Deleted Successfully";

}

}
```

Application.properties(all about url of databases and hibernate)

```
spring.application.name=demo

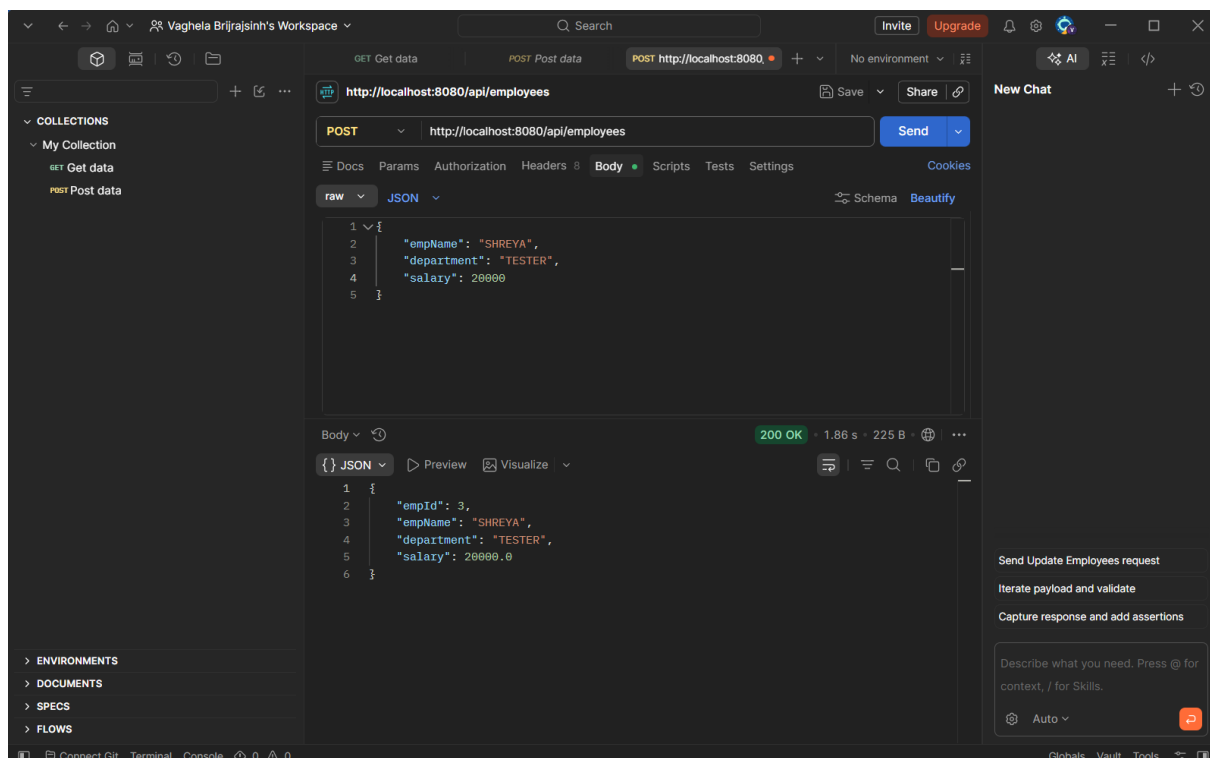
spring.datasource.url=jdbc:mysql://localhost:3306/employee_db
spring.datasource.username=root
spring.datasource.password=

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
```

API'S TESTING USING POSTMAN

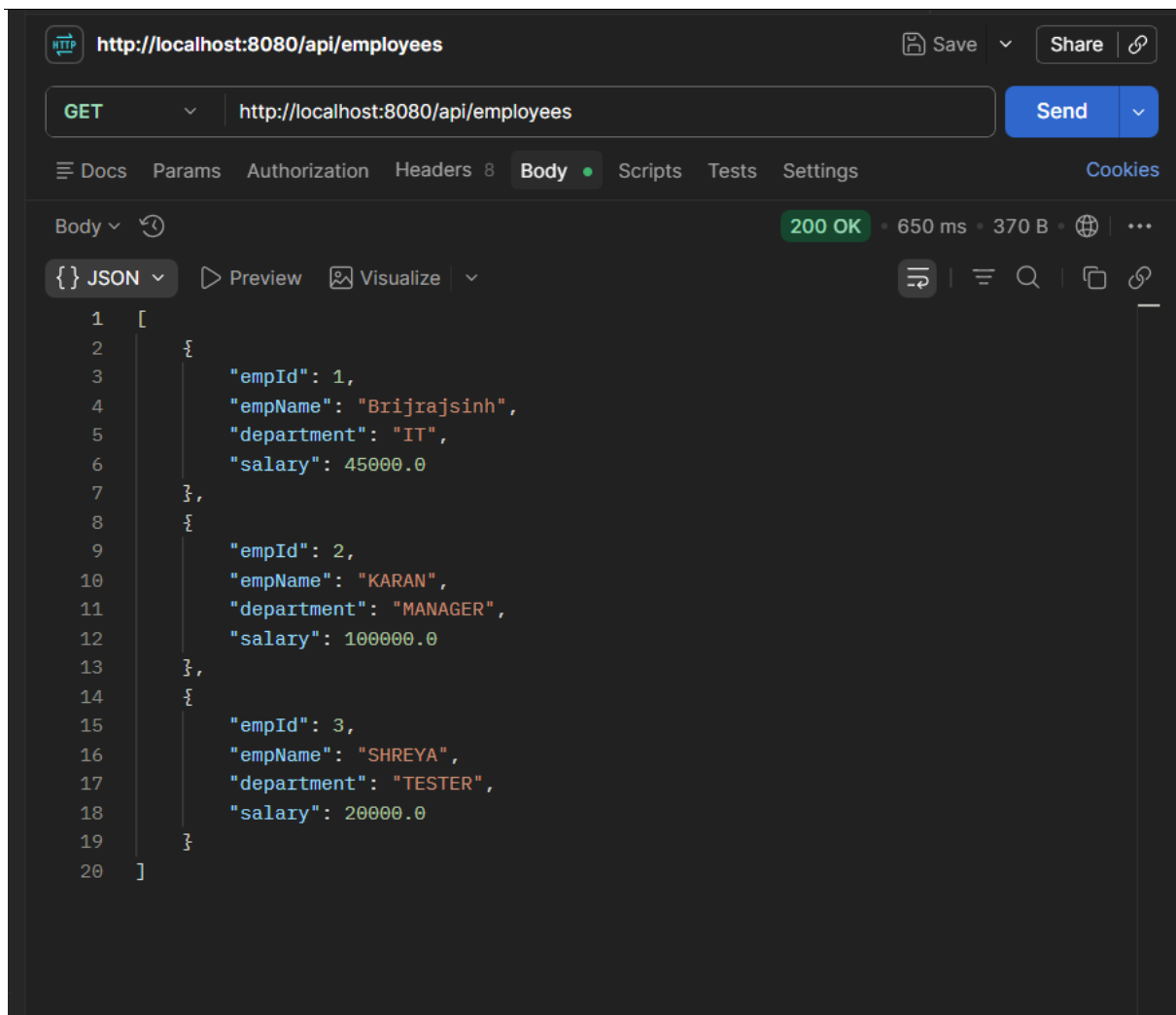
1. **POST API – Add Employee** : The POST API inserts a new employee into the database. The employee ID is generated automatically by MySQL.

POST <http://localhost:8080/api/employees>



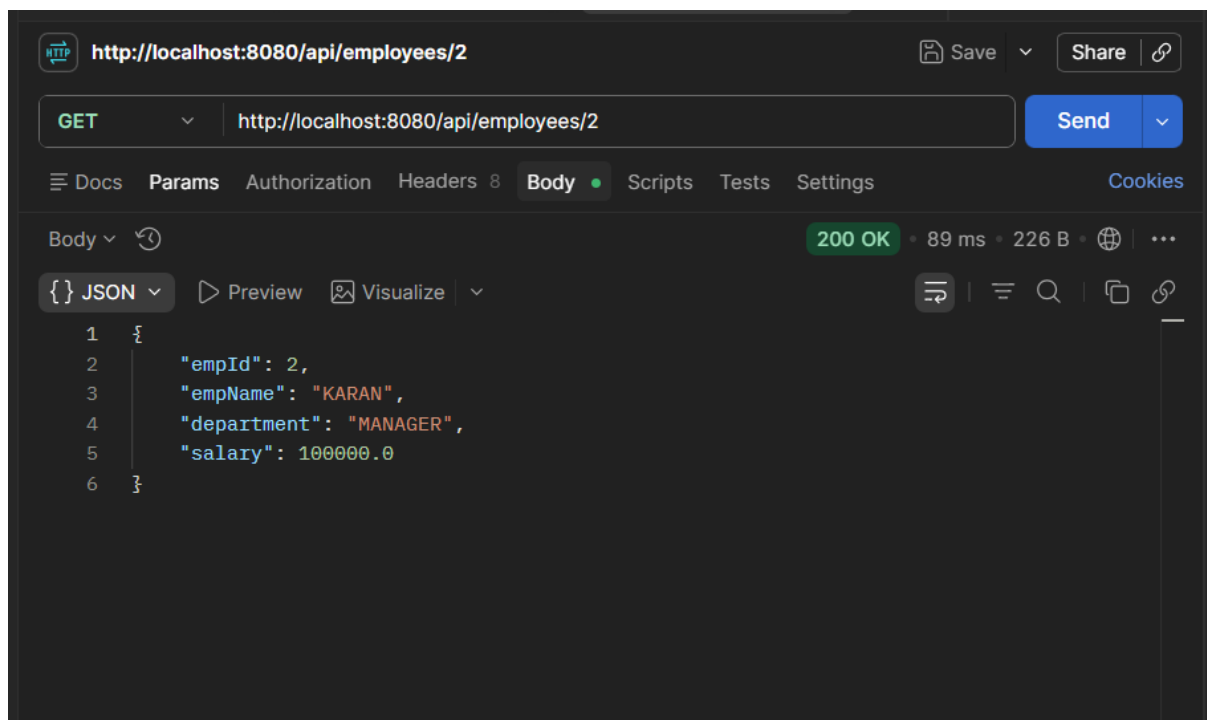
2. **GET API (All Employees)** : GET API – Retrieve All Employees The GET API retrieves all employee records stored in the database.

GET <http://localhost:8080/api/employees>



3. **GET BY ID** – Retrieve Employee by ID Retrieves a specific employee using the employee ID.

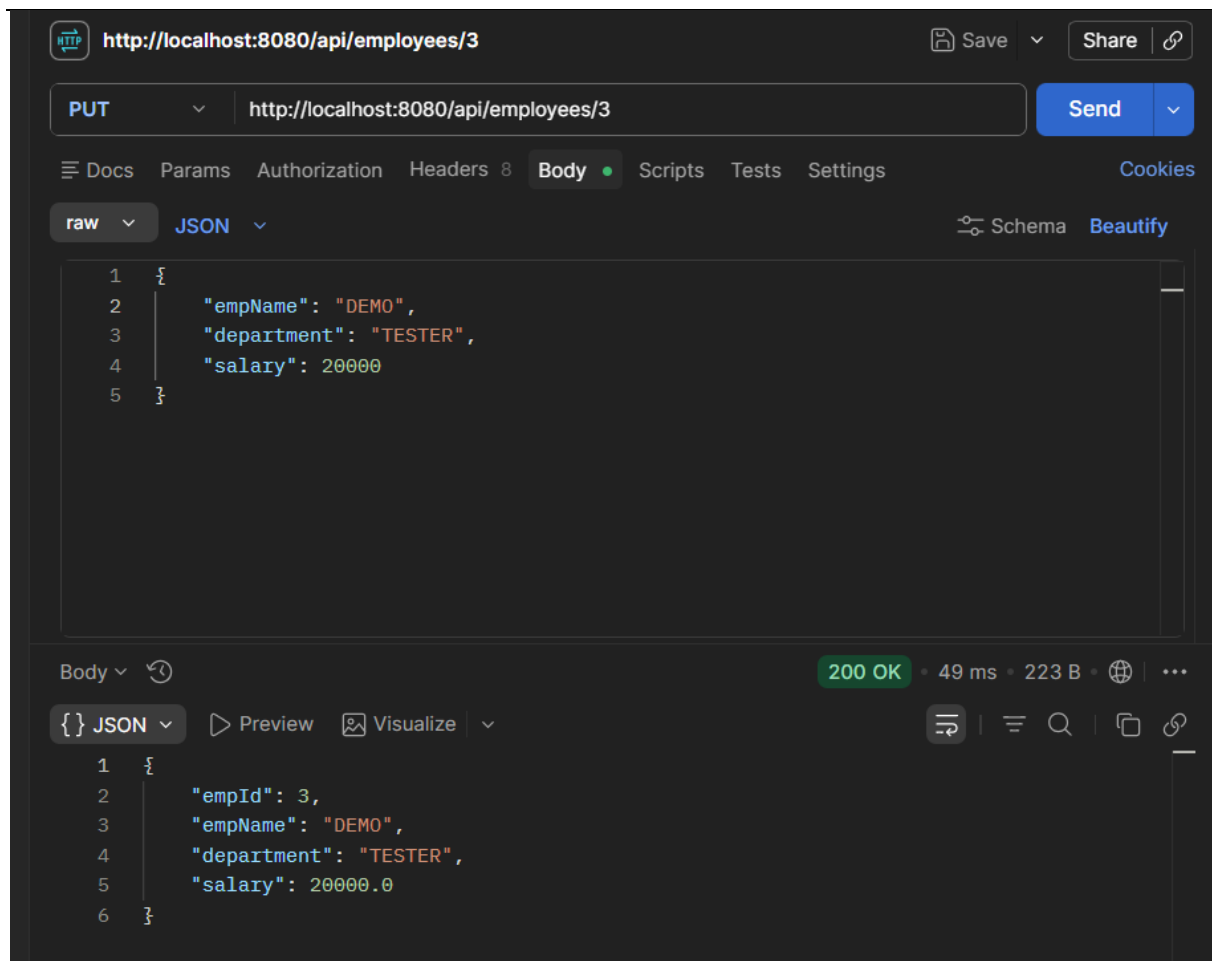
GET <http://localhost:8080/api/employees/2>



4. **PUT** :- PUT API – UPDATE Employee Updates the employee information using the employee ID.

PUT <http://localhost:8080/api/employees/3>

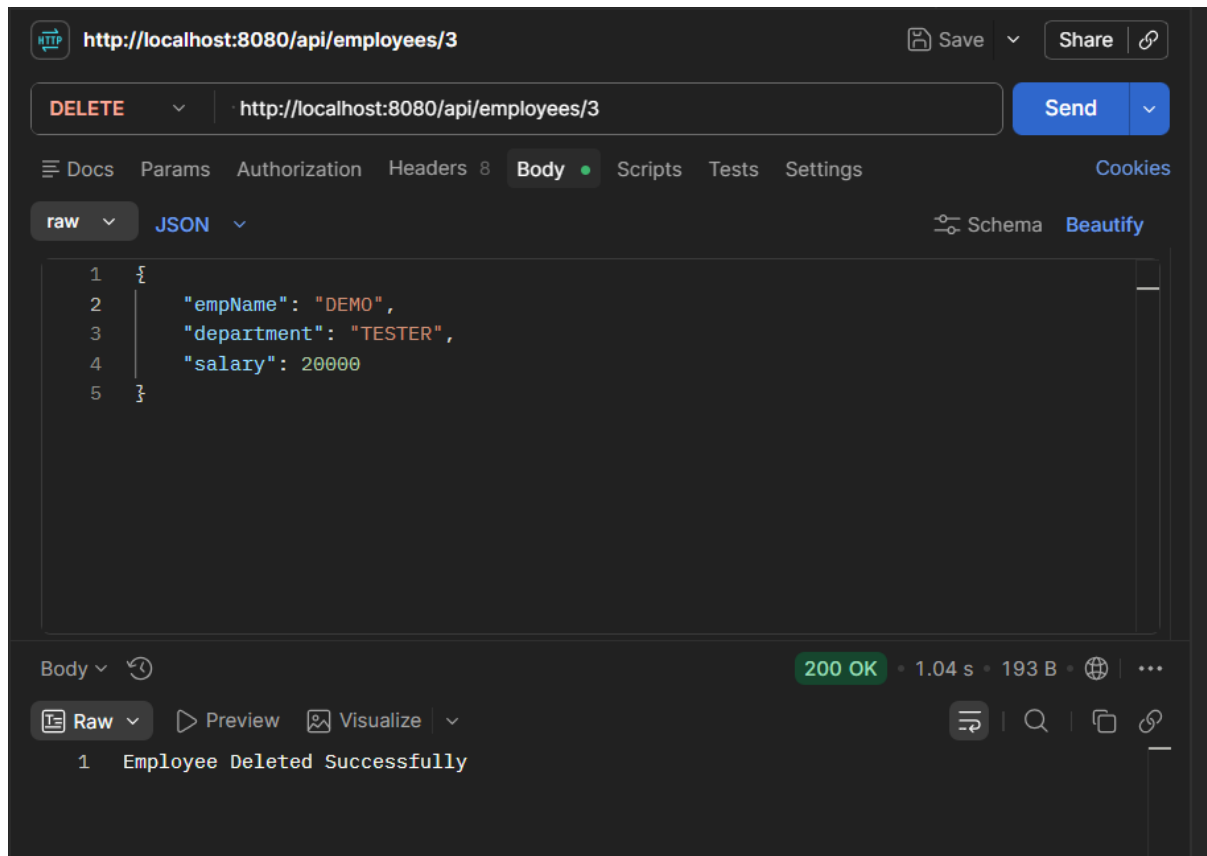
SEE AND VIEW IN LIST OF ALL EMPLOYEE'S AT ID 2 IT WAS SHERYA AFTER UPDATE IT IS DEMO



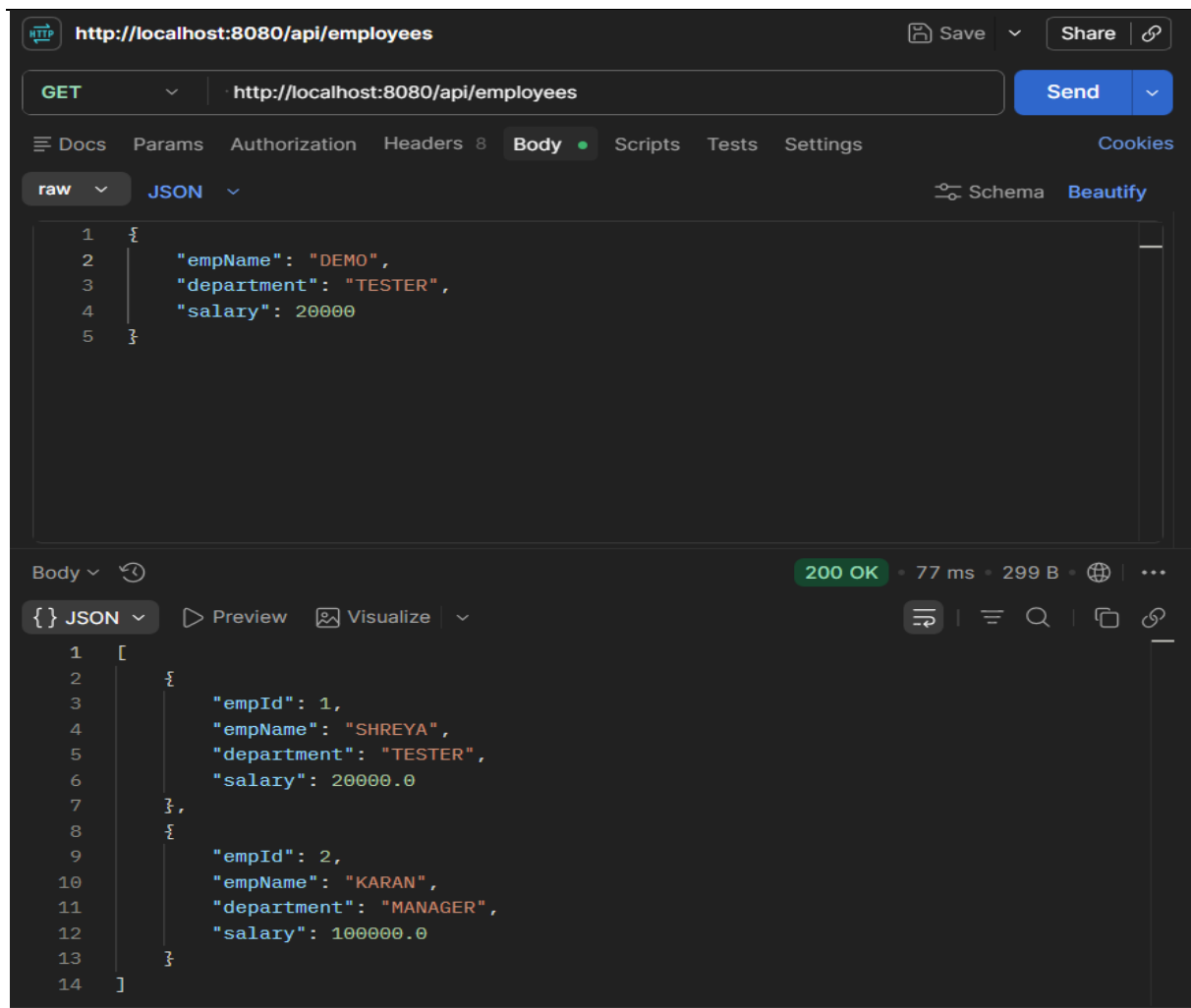
5. DELETE API – DELETE Employee Deletes the employee record from the database.

DELETE <http://localhost:8080/api/employees/3>

SEE THE ID WITH 3 WILL BE DELETED FROM LIST



NOW THIS TO CHECK WHETHER DELETED OR NOT :



The screenshot shows a REST client interface with a GET request to `http://localhost:8080/api/employees`. The response is a JSON array of two employee objects. The status is 200 OK, with a response time of 77 ms and a size of 299 B.

```
1 {
2   "empName": "DEMO",
3   "department": "TESTER",
4   "salary": 20000
5 }
```

```
1 [
2   {
3     "empId": 1,
4     "empName": "SHREYA",
5     "department": "TESTER",
6     "salary": 20000.0
7   },
8   {
9     "empId": 2,
10    "empName": "KARAN",
11    "department": "MANAGER",
12    "salary": 100000.0
13  }
14 ]
```

Database Output

TABLE'S AND DATA STORING SUCCEFULLY

WE JUST NEED TO CREATE THE DATABASE HIBERNATE IT SELF WILL CREATE TABLE FROM MODULE OR ENTITY CLASS

←

Server: MySQL:3306 » Database: employee_db » Table: employee

Browse

Structure

SQL

Search

Insert

Export

Import

Privileges

Operations

✓ Showing rows 0 - 1 (2 total, Query took 0.0011 seconds.)

SELECT * FROM `employee`

☐ Profiling

[Edit inline]

[Edit]

[Explain SQL]

[Create PHP code]

[Refresh]

☐ Show all

|

Number of rows: 25

|

Filter rows: Search this table

|

Sort by key: None

Extra options

← T →

▼

emp_id

department

emp_name

salary

☐

Edit

Copy

Delete

1

IT

Brijrajsinh

45000

☐

Edit

Copy

Delete

2

MANAGER

KARAN

100000

Practical No. 2: Spring Data Repository Hierarchy - CrudRepository vs PagingAndSortingRepository vs JpaRepository

Objective:

To compare the three levels of the Spring Data repository hierarchy covered in this chapter — CrudRepository

(basic CRUD), PagingAndSortingRepository (adds paging/sorting), and JpaRepository (adds JPA-specific batch

operations) — by rewriting the same repository at each level and observing the additional methods available.

Class Name(s) with Attributes:

Product (@Entity)

- productId : int (@Id @GeneratedValue)
- productName : String
- category : String
- price : double
- quantity : int

ProductRepository (progressively extends CrudRepository → PagingAndSortingRepository → JpaRepository)

ProductRestController (@RestController)

- productRepository : ProductRepository

PROJECT STRUCTURE**Dependencies**

- Spring Web
- Spring Data JPA
- MySQL Driver

Database

Database Name: product_db

Table: product (created automatically by Hibernate)

Product.java (ENTITY model class)

```
package com.example.second.entity;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;

@Entity
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int productId;

    private String productName;
    private String category;
    private double price;
    private int quantity;

    public Product() {
    }

    public int getProductId() {
        return productId;
    }
}
```

```
}

public void setProductId(int productId) {
    this.productId = productId;
}

public String getProductName() {
    return productName;
}

public void setProductName(String productName) {
    this.productName = productName;
}

public String getCategory() {
    return category;
}

public void setCategory(String category) {
    this.category = category;
}

public double getPrice() {
    return price;
}

public void setPrice(double price) {
```

```
        this.price = price;
    }

    public int getQuantity() {
        return quantity;
    }

    public void setQuantity(int quantity) {
        this.quantity = quantity;
    }
}
```

ProductRepository.java (Repository file)

```
package com.example.second.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import com.example.second.entity.Product;

public interface ProductRepository
    extends JpaRepository<Product, Integer> {

}
```


ProductRestController.java (REST Controller file)

```
package com.example.second.controller;

import java.util.List;

import org.springframework.data.domain.Sort;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import com.example.second.entity.Product;
import com.example.second.repository.ProductRepository;

@RestController
@RequestMapping("/api/products")
public class ProductRestController {

    private final ProductRepository productRepository;

    public ProductRestController(ProductRepository productRepository) {
        this.productRepository = productRepository;
    }

    // POST - Save Product
    @PostMapping
    public ResponseEntity<Product> addProduct(
        @RequestBody Product product) {
        Product savedProduct = productRepository.save(product);
```

```
        return new ResponseEntity<>(savedProduct, HttpStatus.CREATED);
    }

    // GET - Get All Products
    @GetMapping
    public List<Product> getAllProducts() {
        return productRepository.findAll();
    }

    // GET - Get Product By ID
    @GetMapping("/{id:\\d+}")
    public ResponseEntity<Product> getProductById(
        @PathVariable int id) {
        return productRepository.findById(id)
            .map(product -> ResponseEntity.ok(product))
            .orElse(ResponseEntity.notFound().build());
    }

    // GET - Sort Products By Price
    @GetMapping("/sorted")
    public List<Product> getSortedProducts() {
        return productRepository.findAll(Sort.by("price"));
    }

    // POST - Batch Insert Products
    @PostMapping("/batch")
    public ResponseEntity<List<Product>> addProductsInBatch(
```

```
@RequestBody List<Product> products) {

    List<Product> savedProducts =

        productRepository.saveAll(products);

    productRepository.flush();

    return new ResponseEntity<>(savedProducts, HttpStatus.CREATED);

}


// PUT - Update Product
@PutMapping("/{id}")
public Product updateProduct(@PathVariable Integer id,
    @RequestBody Product product) {
    product.setProductId(id);
    return repository.save(product);
}


// DELETE - Delete Product
@DeleteMapping("/{id:\\d+}")
public ResponseEntity<Void> deleteProduct(@PathVariable int id) {
    if (productRepository.existsById(id)) {
        productRepository.deleteById(id);
        return ResponseEntity.noContent().build();
    }
    return ResponseEntity.notFound().build();
}
}
```

Application.properties (database and Hibernate configuration)

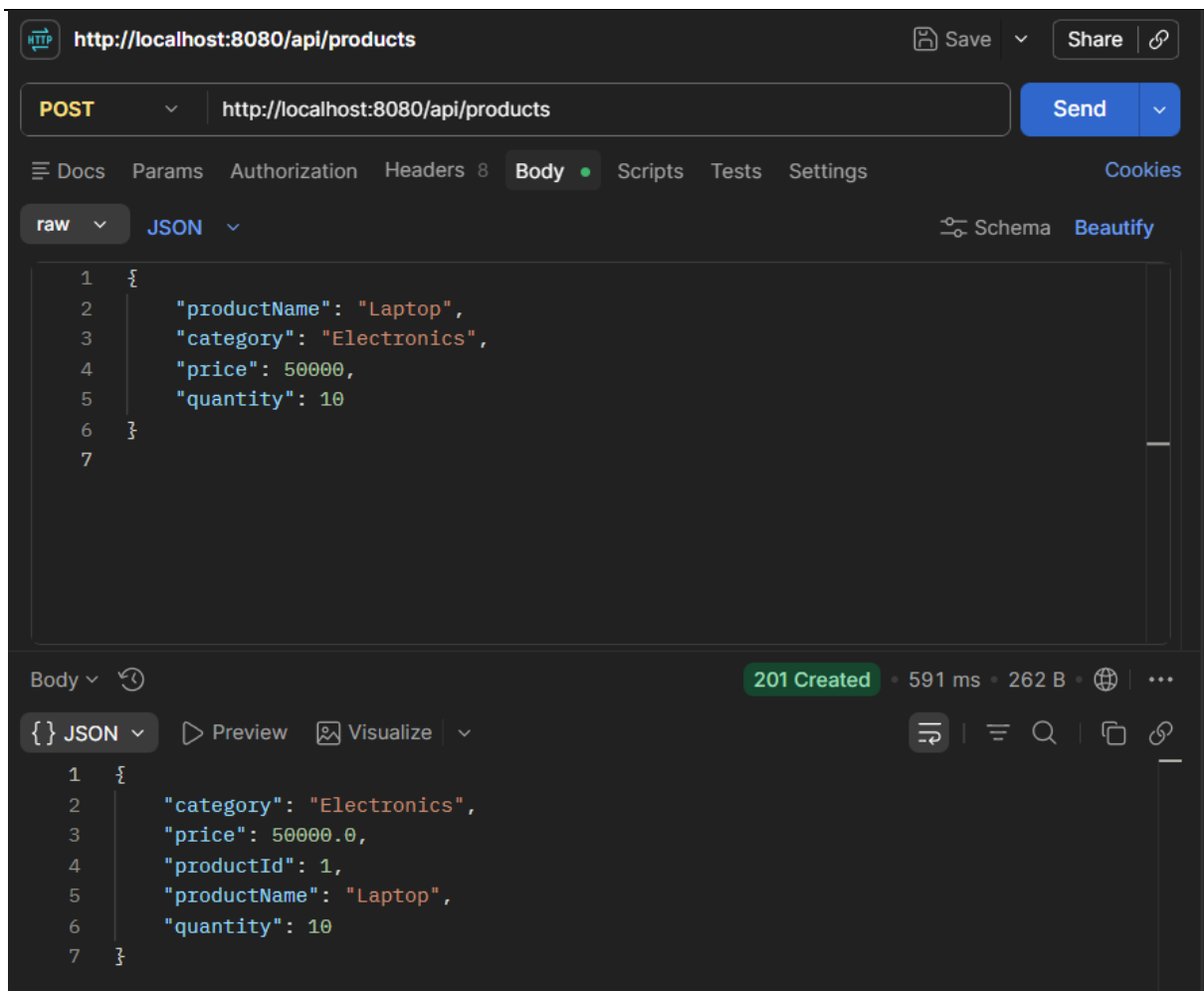
```
spring.application.name=second
spring.datasource.url=jdbc:mysql://localhost:3306/product_db
spring.datasource.username=root
spring.datasource.password=

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
```

API'S TESTING USING POSTMAN

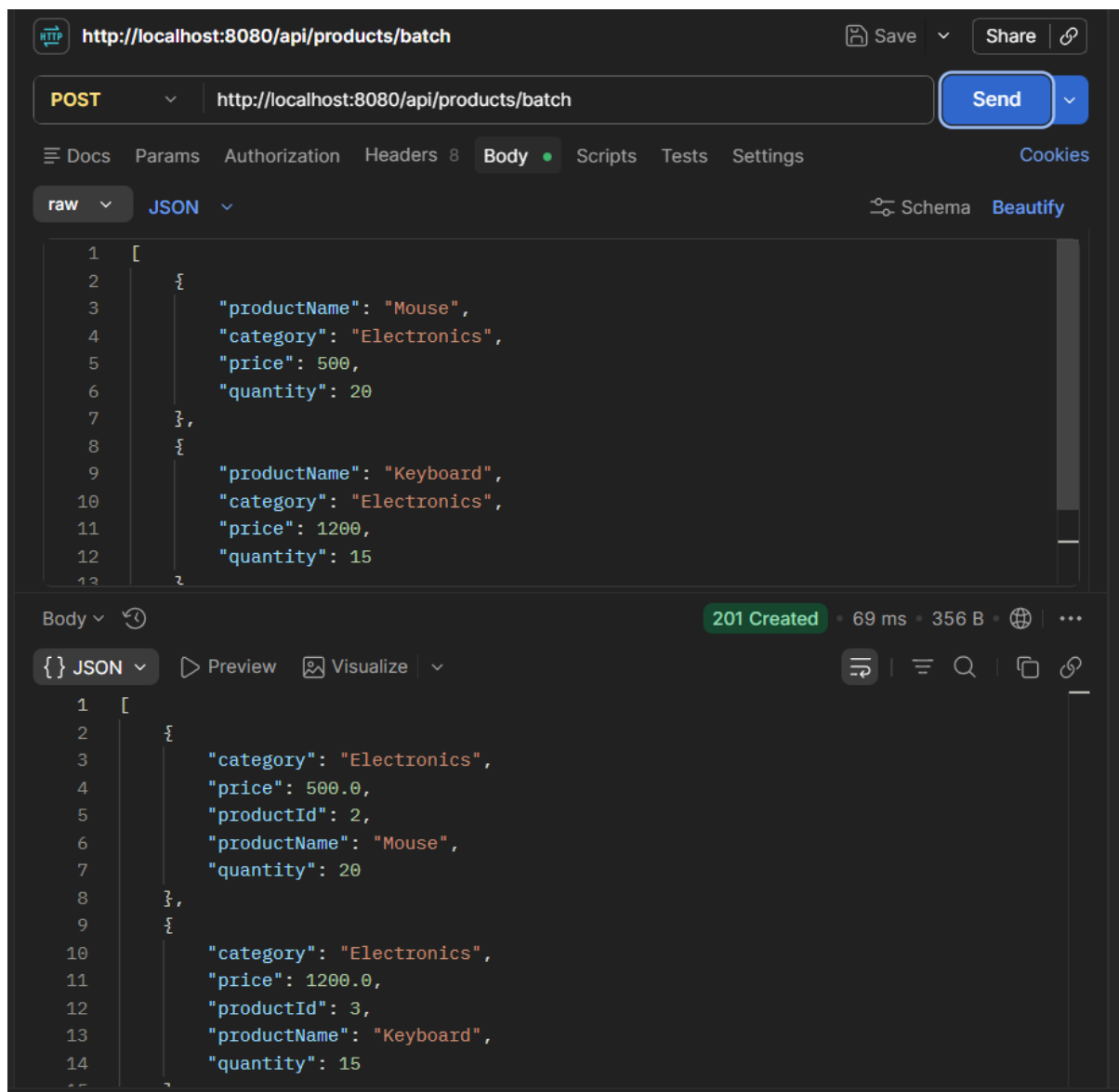
1. **POST API – Add Product:** The POST API inserts a new product into the database. The product ID is generated automatically by MySQL.

POST <http://localhost:8080/api/products>



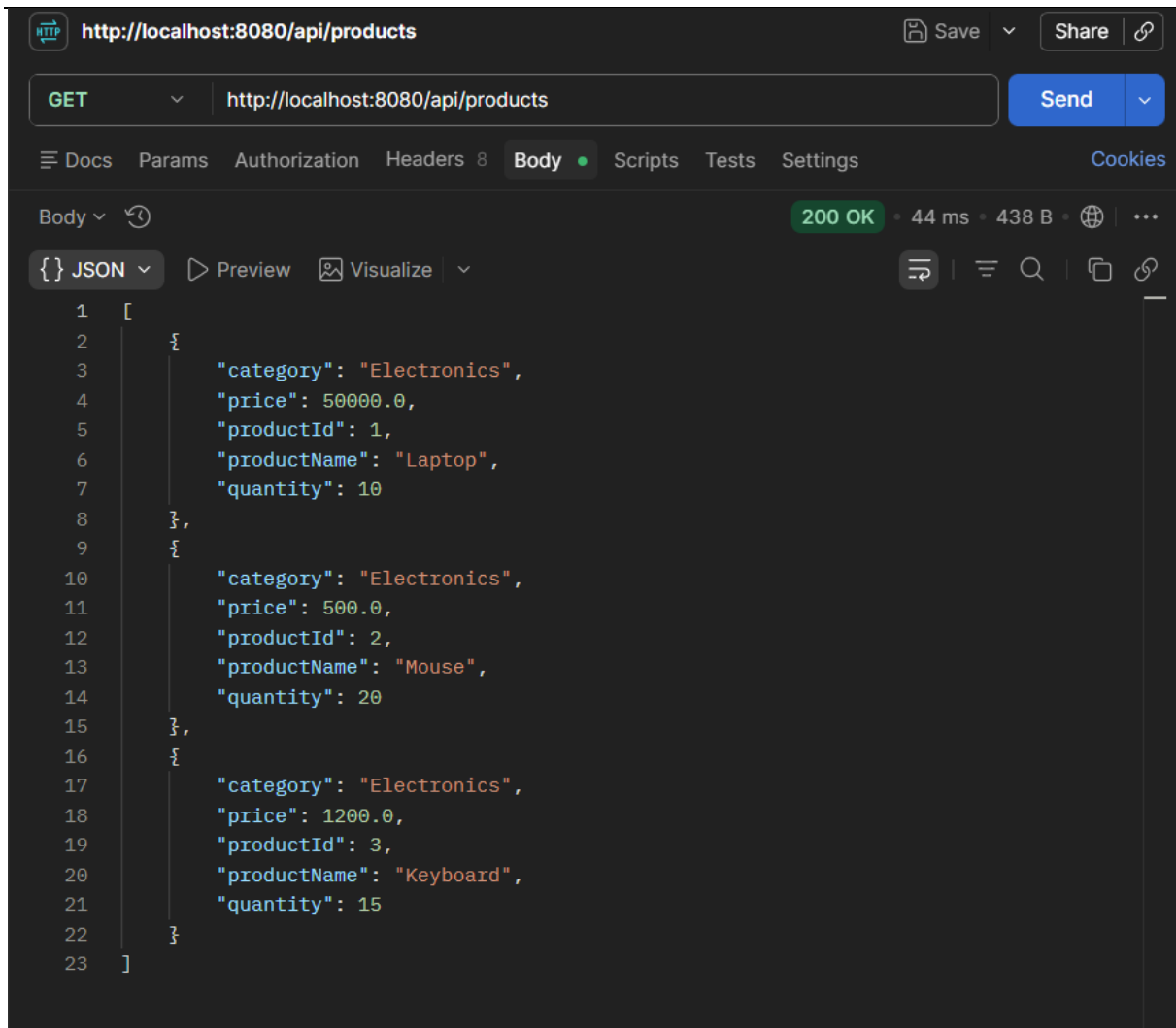
2. **POST API – Batch Insert:** The batch POST API inserts multiple products using the `saveAll()` method of `JpaRepository`.

POST <http://localhost:8080/api/products/batch>



3. **GET API (All Products):** The GET API retrieves all product records stored in the database.

GET <http://localhost:8080/api/products>



```
HTTP http://localhost:8080/api/products Save Share

GET http://localhost:8080/api/products Send

Docs Params Authorization Headers 8 Body Scripts Tests Settings Cookies

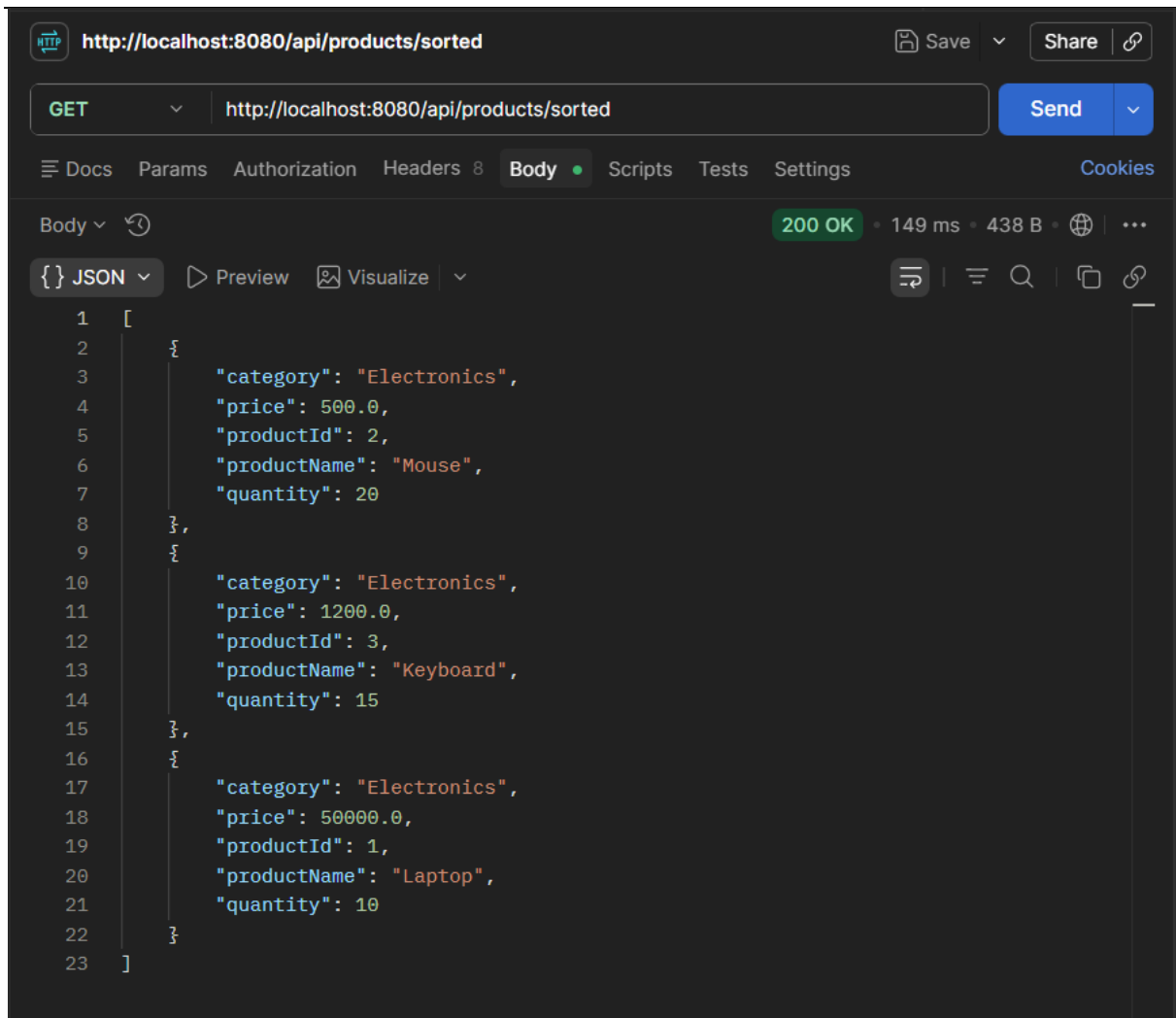
Body 200 OK • 44 ms • 438 B

{ } JSON Preview Visualize

1 [
2   {
3     "category": "Electronics",
4     "price": 50000.0,
5     "productId": 1,
6     "productName": "Laptop",
7     "quantity": 10
8   },
9   {
10    "category": "Electronics",
11    "price": 500.0,
12    "productId": 2,
13    "productName": "Mouse",
14    "quantity": 20
15  },
16  {
17    "category": "Electronics",
18    "price": 1200.0,
19    "productId": 3,
20    "productName": "Keyboard",
21    "quantity": 15
22  }
23 ]
```

4. **GET API – Sort Products By Price:** The GET API retrieves products sorted by price using the `findAll(Sort)` method.

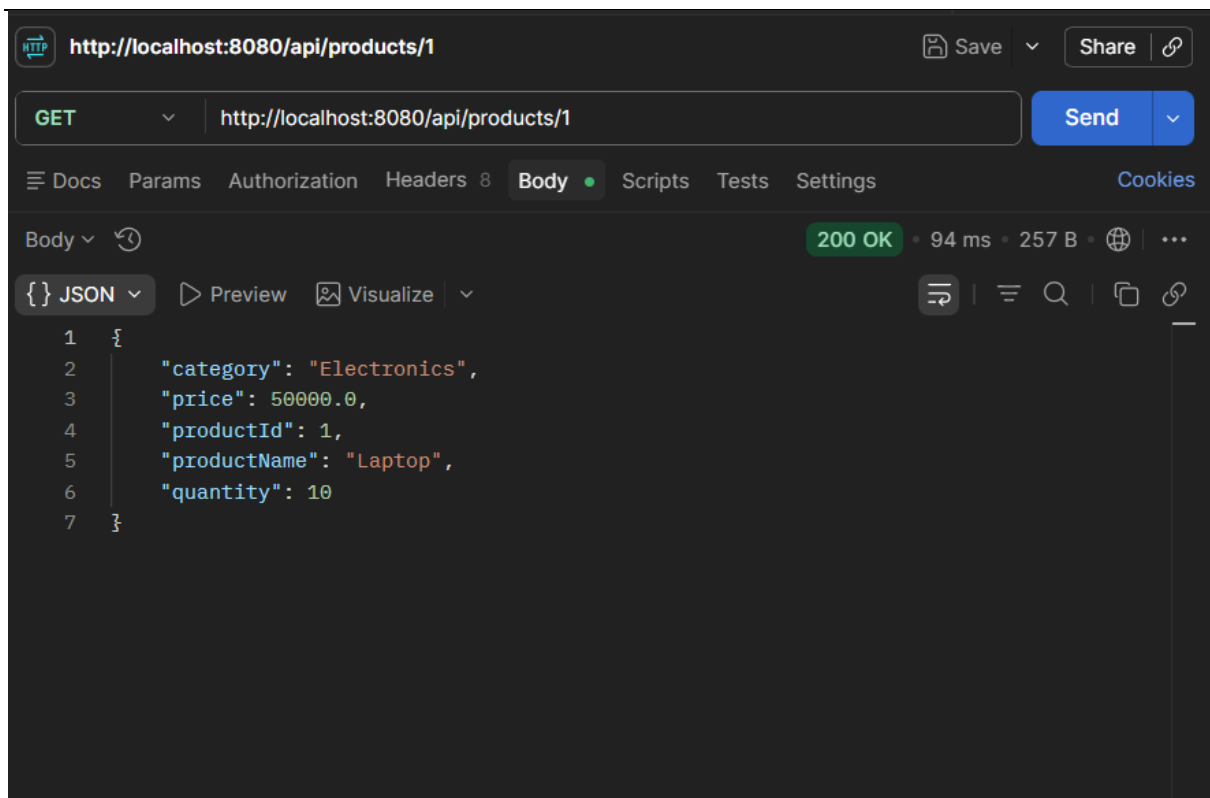
GET <http://localhost:8080/api/products/sorted>



```
1  [
2    {
3      "category": "Electronics",
4      "price": 500.0,
5      "productId": 2,
6      "productName": "Mouse",
7      "quantity": 20
8    },
9    {
10     "category": "Electronics",
11     "price": 1200.0,
12     "productId": 3,
13     "productName": "Keyboard",
14     "quantity": 15
15   },
16   {
17     "category": "Electronics",
18     "price": 50000.0,
19     "productId": 1,
20     "productName": "Laptop",
21     "quantity": 10
22   }
23 ]
```

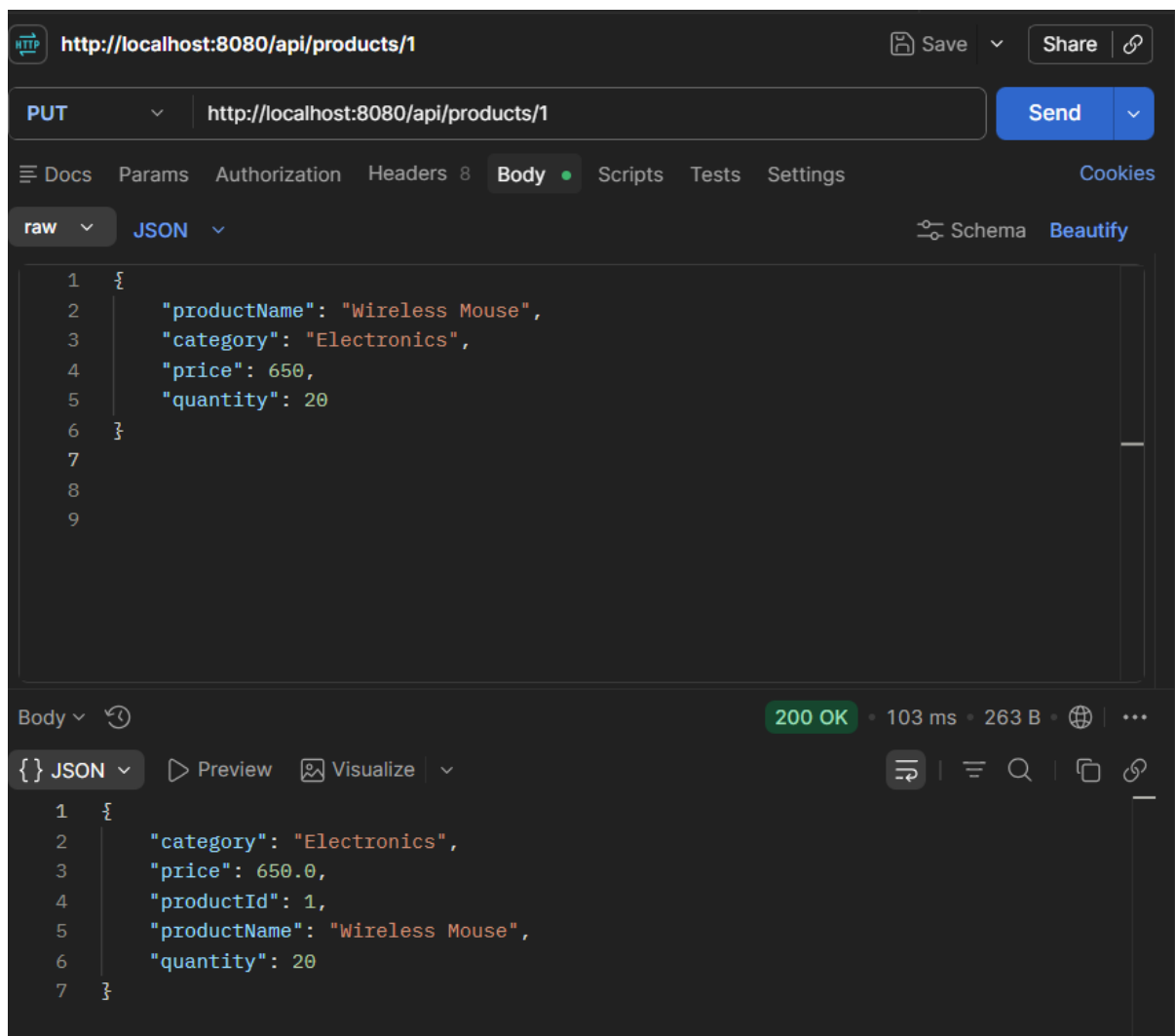
5. GET BY ID – Retrieve Product by ID: Retrieves a specific product using the product ID.

GET <http://localhost:8080/api/products/1>

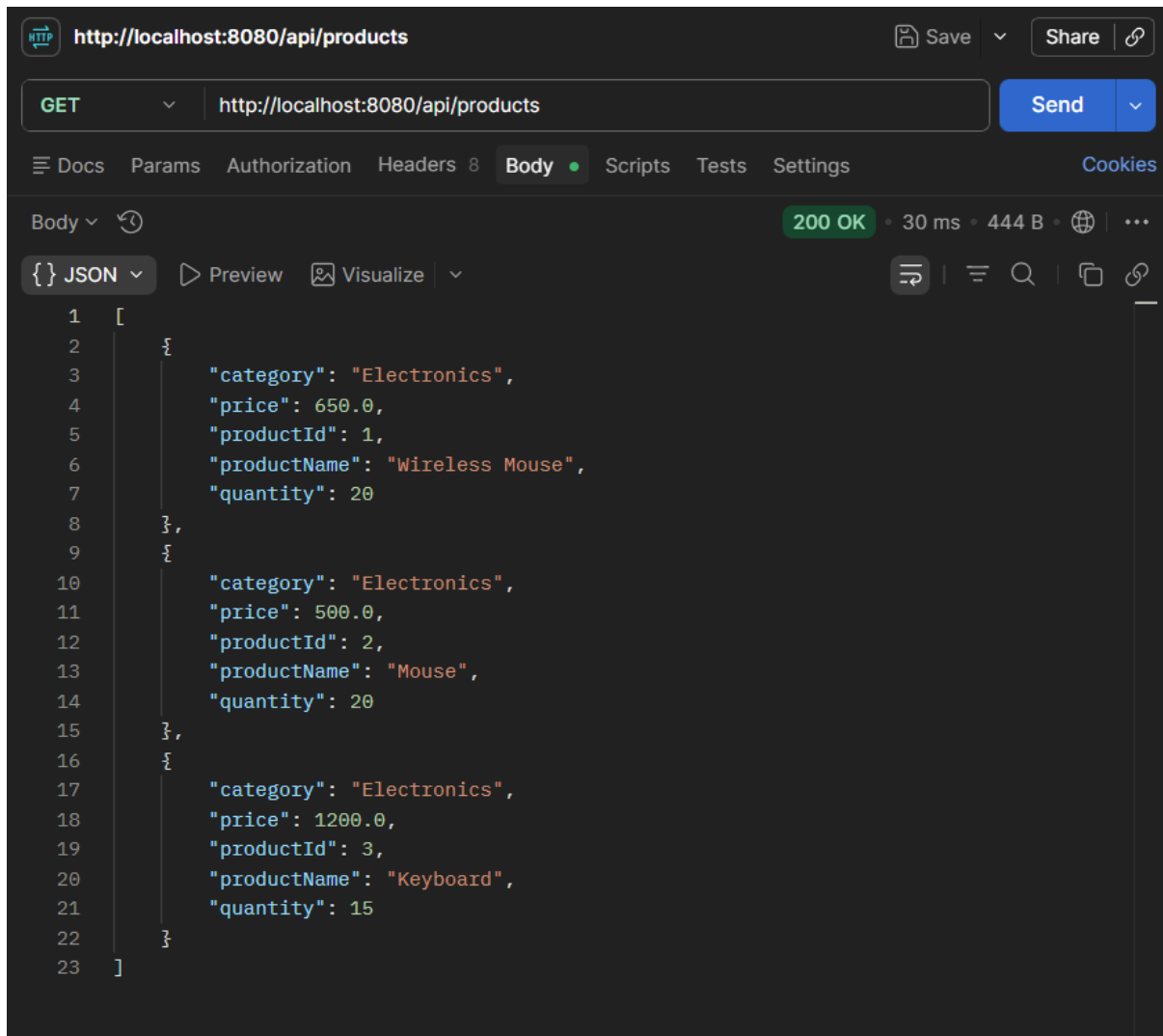


6. **PUT API – UPDATE Product:** Updates the product information using the product ID.

PUT <http://localhost:8080/api/products/1>

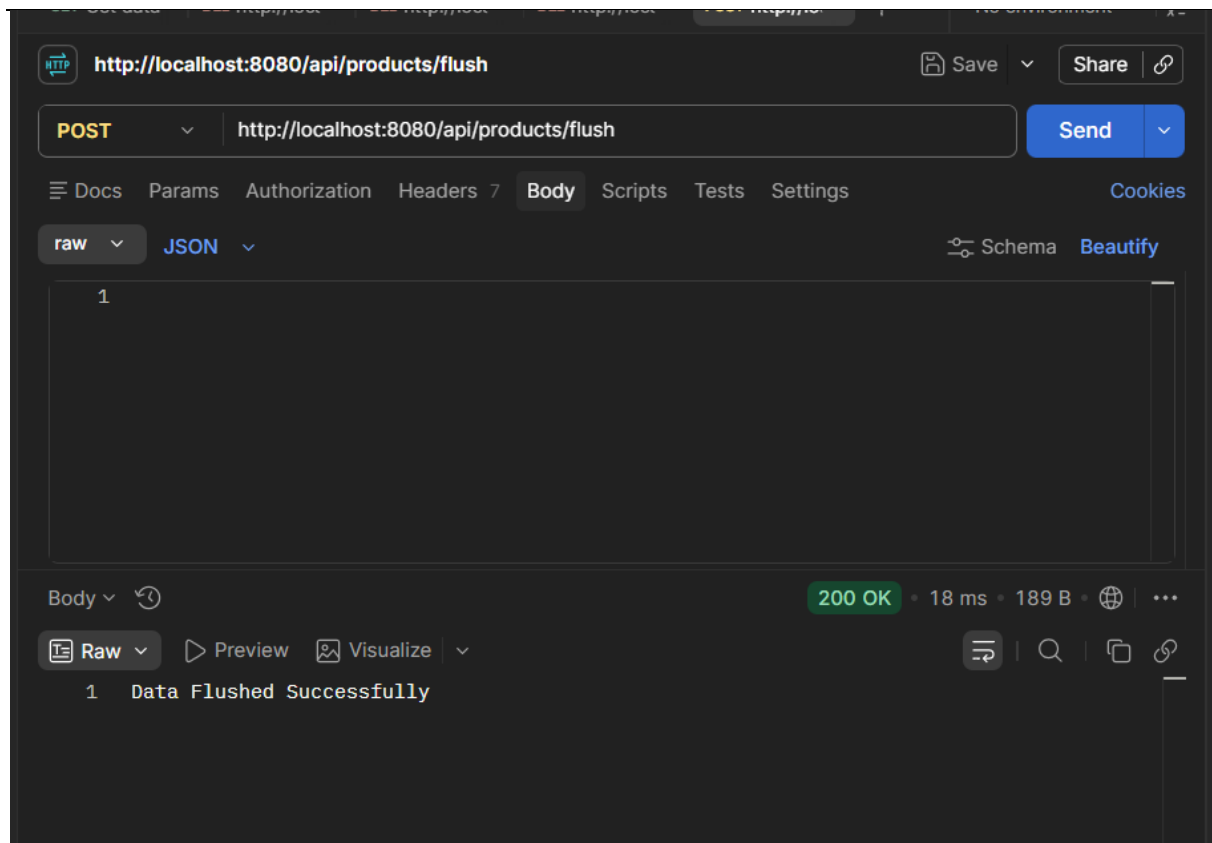


Verified removed laptop and replace the wireless mouse:-



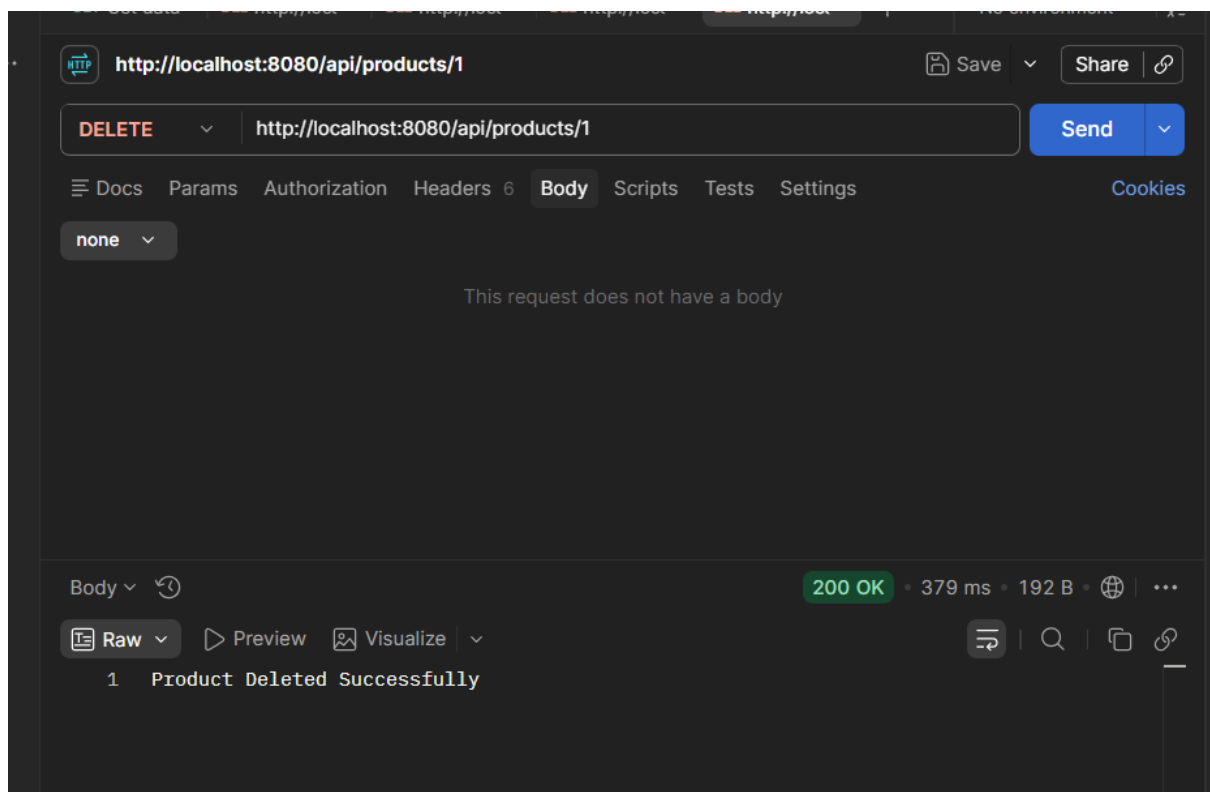
```
1  [
2      {
3          "category": "Electronics",
4          "price": 650.0,
5          "productId": 1,
6          "productName": "Wireless Mouse",
7          "quantity": 20
8      },
9      {
10         "category": "Electronics",
11         "price": 500.0,
12         "productId": 2,
13         "productName": "Mouse",
14         "quantity": 20
15     },
16     {
17         "category": "Electronics",
18         "price": 1200.0,
19         "productId": 3,
20         "productName": "Keyboard",
21         "quantity": 15
22     }
23 ]
```

7. **Flush:-** Force Hibernate/JPA to immediately send the pending changes from memory (persistence context) to the database.

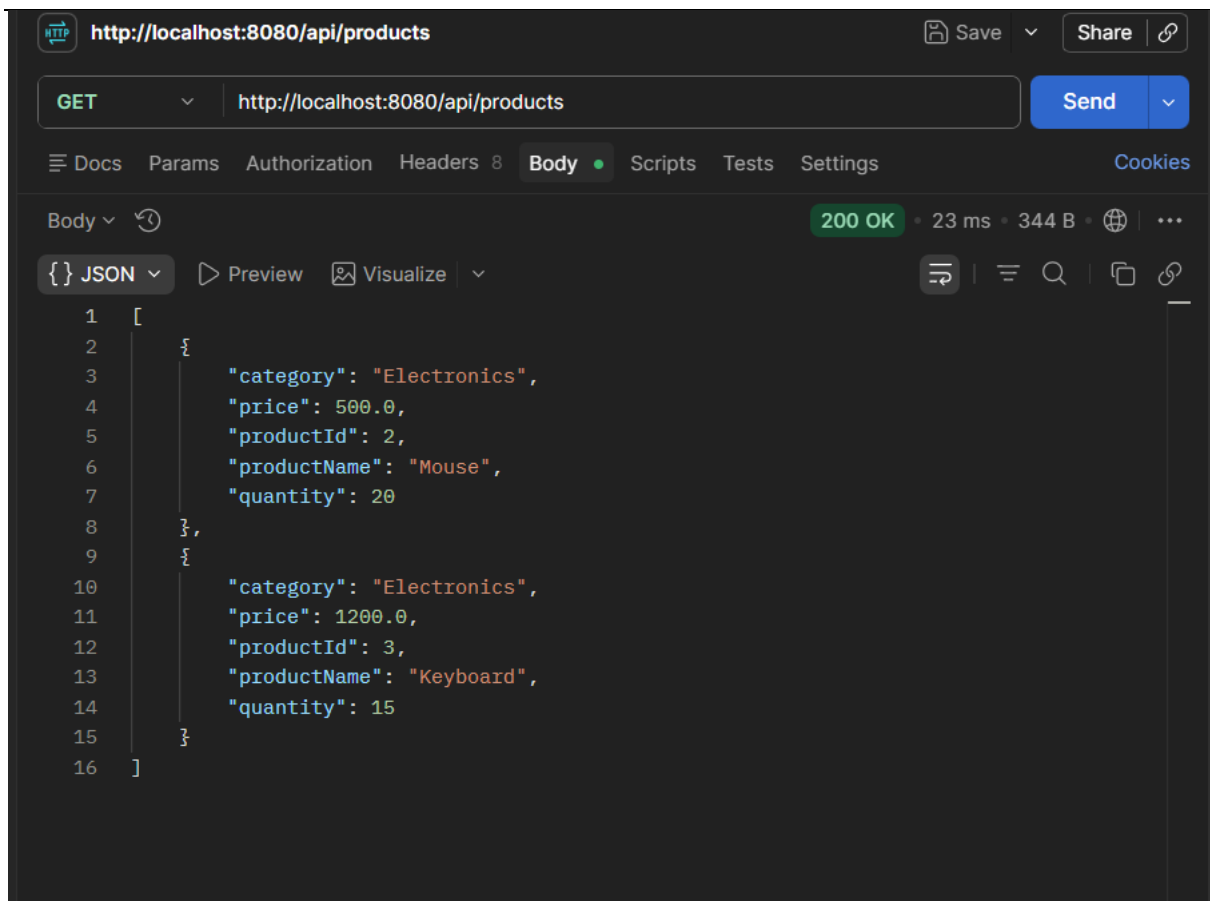


8. **DELETE API – DELETE Product:** Deletes the product record from the database.

DELETE <http://localhost:8080/api/products/1>



Verified removed firstid:-



DATABASE OUTPUT

PRODUCT TABLE AND DATA STORED SUCCESSFULLY

WE JUST NEED TO CREATE THE DATABASE. HIBERNATE ITSELF WILL CREATE THE TABLE FROM THE ENTITY CLASS.

Server: MySQL 3306 » Database: productdb » Table: product

Browse Structure SQL Search Insert Export Import Privileges Operations Triggers

✓ Showing rows 0 - 3 (4 total, Query took 0.0008 seconds.)

`SELECT * FROM `product``

☐ Profiling [[Edit inline](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

☐ Show all | Number of rows: 25 | Filter rows: | Sort by key: None

Extra options

				product_id	category	price	product_name	quantity
☐				2	Electronics	500	Mouse	20
☐				3	Electronics	1200	Keyboard	15
☐				4	Electronics	55000	Laptop	10
☐				5	Accessories	2500	Headphones	30

☐ Check all | With selected: Edit Copy Delete Export

Practical No. 3: Custom Query Methods – findBy, countBy and @Query (JPQL)

Objective:

To derive query methods automatically from method names (query derivation) and to write custom JPQL queries

using @Query for requirements that cannot be expressed by naming conventions alone, as covered in the Spring

Data JPA query-methods section of this chapter.

Class Name(s) with Attributes:

Employee (reuse from Practical 1)

- empId, empName, department, salary

EmployeeRepository (additional derived & custom methods)

- List<Employee> findByDepartment(String department)
- long countByDepartment(String department)
- List<Employee> findBySalaryGreaterThan(double salary)
- @Query("SELECT e FROM Employee e WHERE e.salary BETWEEN :min AND :max")

PROJECT STRUCTURE

```
src
├── main
│   └── java
│       └── com.example.third
│           ├── entity
│           │   └── Employee.java
│           ├── repository
│           │   └── EmployeeRepository.java
│           ├── controller
│           │   └── EmployeeRestController.java
│           └── ThirdApplication.java
```

Dependencies

- Spring Web
- Spring Data JPA
- MySQL Driver

Database

Database Name: employee_db

Table: product (created automatically by Hibernate)

Employee.java (ENTITY model class)

```
package com.example.third.entity;
```

```
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Table;

@Entity
@Table(name = "employee")
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer empId;

    private String empName;
    private String department;
    private double salary;

    public Employee() {
    }

    public Employee(Integer empId, String empName, String department, double salary) {
        this.empId = empId;
        this.empName = empName;
        this.department = department;
        this.salary = salary;
    }
}
```

```
public Integer getEmpId() {  
    return empId;  
}  
  
public void setEmpId(Integer empId) {  
    this.empId = empId;  
}  
  
public String getEmpName() {  
    return empName;  
}  
  
public void setEmpName(String empName) {  
    this.empName = empName;  
}  
  
public String getDepartment() {  
    return department;  
}  
  
public void setDepartment(String department) {  
    this.department = department;  
}  
  
public double getSalary() {  
    return salary;  
}
```

```
}

public void setSalary(double salary) {
    this.salary = salary;
}
}
```

EmployeeRepository.java (Repository file)

```
package com.example.third.repository;

import java.util.List;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;

import com.example.third.entity.Employee;

public interface EmployeeRepository extends JpaRepository<Employee, Integer> {

    List<Employee> findByDepartment(String department);

    long countByDepartment(String department);

    List<Employee> findBySalaryGreaterThan(double salary);
}
```

```
@Query("SELECT e FROM Employee e WHERE e.salary BETWEEN :min AND :max")
List<Employee> findBySalaryRange(@Param("min") double min,
                                @Param("max") double max);
}
```

EmployeeRestController.java (REST Controller file)

```
package com.example.third.controller;

import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

import com.example.third.entity.Employee;
import com.example.third.repository.EmployeeRepository;

@RestController
@RequestMapping("/api/employees")
public class EmployeeRestController {

    @Autowired
    private EmployeeRepository repository;

    // Add Employee
    @PostMapping
    public Employee addEmployee(@RequestBody Employee employee) {
```

```
        return repository.save(employee);
    }

    @PostMapping("/batch")
    public List<Employee> addEmployees(@RequestBody List<Employee> employees) {
        return repository.saveAll(employees);
    }

    // Get All Employees
    @GetMapping
    public List<Employee> getAllEmployees() {
        return repository.findAll();
    }

    // Get Employees By Department
    @GetMapping("/department/{department}")
    public List<Employee> getDepartment(@PathVariable String department) {
        return repository.findByDepartment(department);
    }

    // Count Employees By Department
    @GetMapping("/count/{department}")
    public long countDepartment(@PathVariable String department) {
        return repository.countByDepartment(department);
    }

    // Get Employees With Salary Greater Than
```

```
@GetMapping("/salary-above")

public List<Employee> salaryAbove(@RequestParam double salary) {

    return repository.findBySalaryGreaterThan(salary);

}

// Get Employees Between Salary Range

@GetMapping("/salary-range")

public List<Employee> salaryRange(@RequestParam double min,

                                @RequestParam double max) {

    return repository.findBySalaryRange(min, max);

}

// Update Employee

@PutMapping("/{id}")

public Employee updateEmployee(@PathVariable Integer id,

                               @RequestBody Employee employee) {

    employee.setEmpId(id);

    return repository.save(employee);

}

// Delete Employee

@DeleteMapping("/{id}")

public String deleteEmployee(@PathVariable Integer id) {

    repository.deleteById(id);

    return "Employee Deleted Successfully";

}

}
```

Application.properties (database and Hibernate configuration)

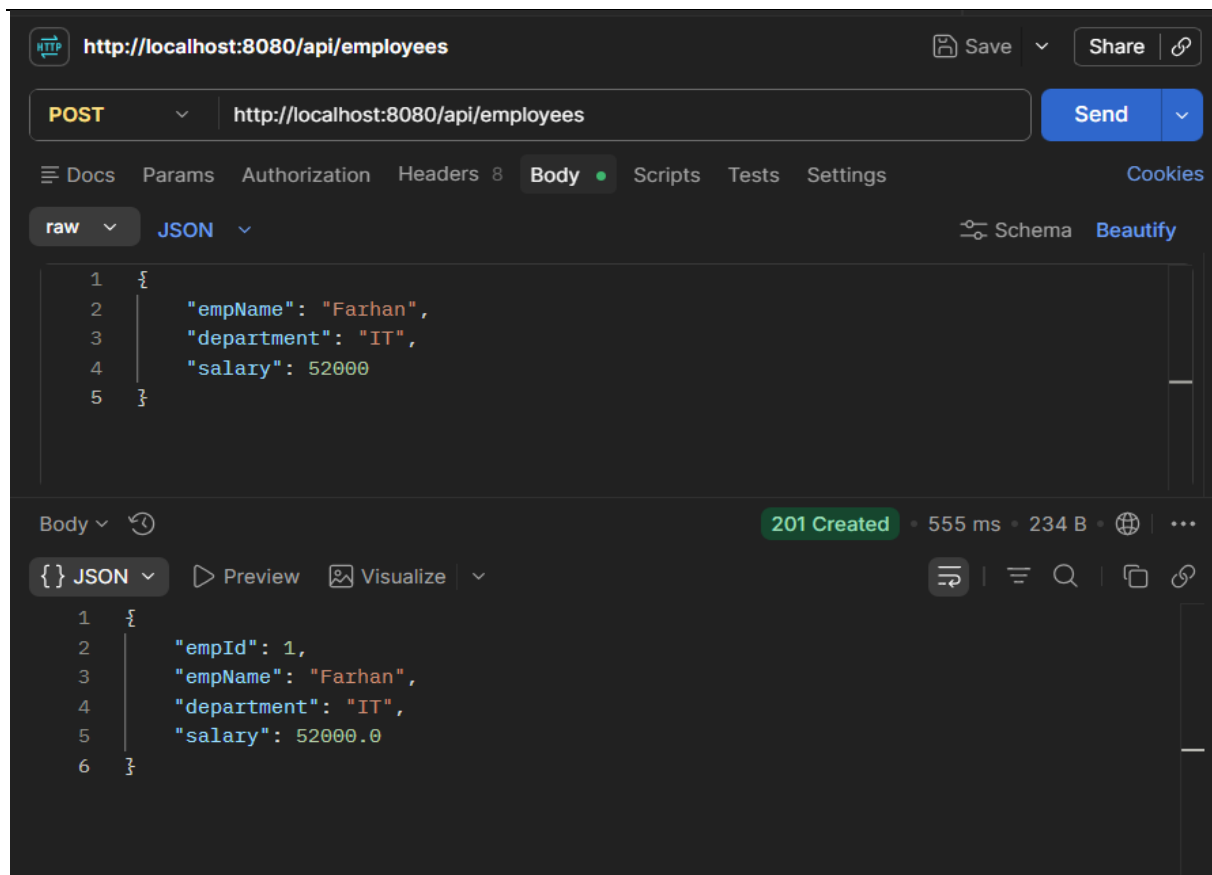
```
spring.application.name=third
spring.datasource.url=jdbc:mysql://localhost:3306/employee_db
spring.datasource.username=root
spring.datasource.password=

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
```

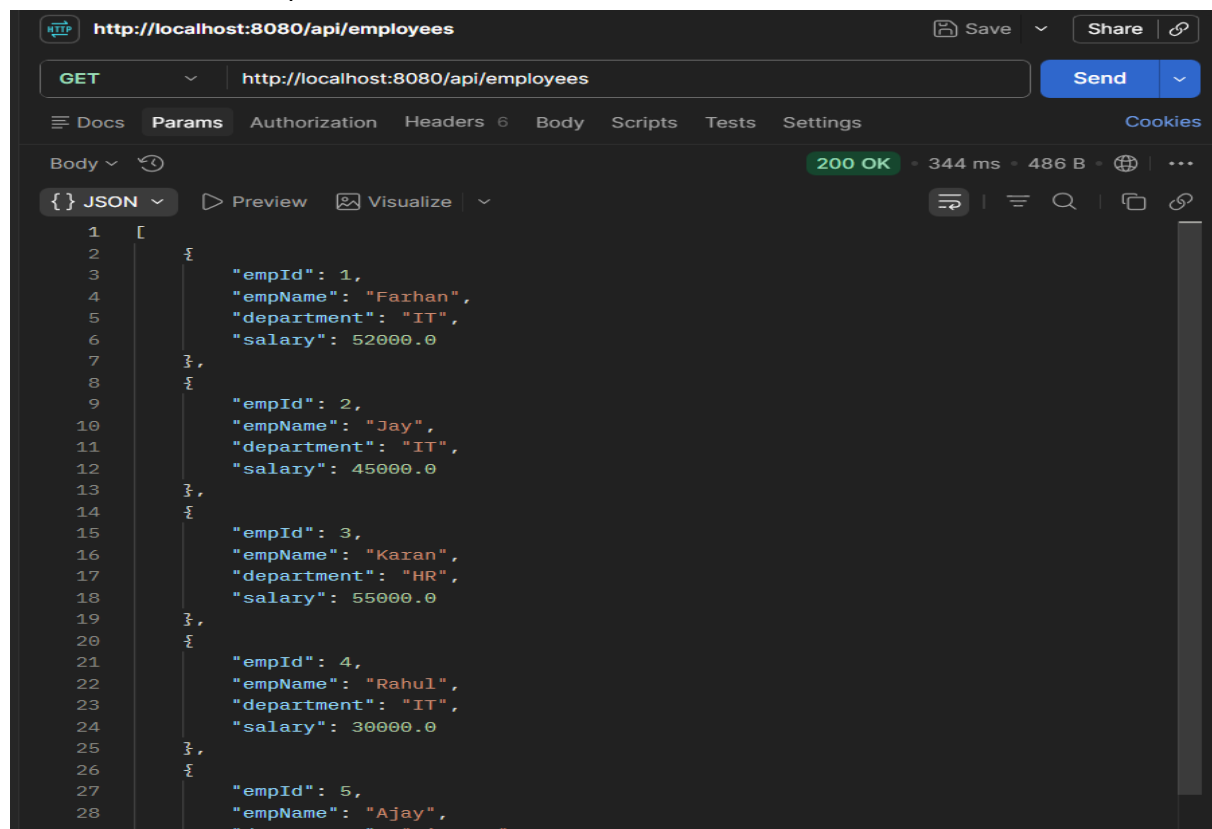
API'S TESTING USING POSTMAN

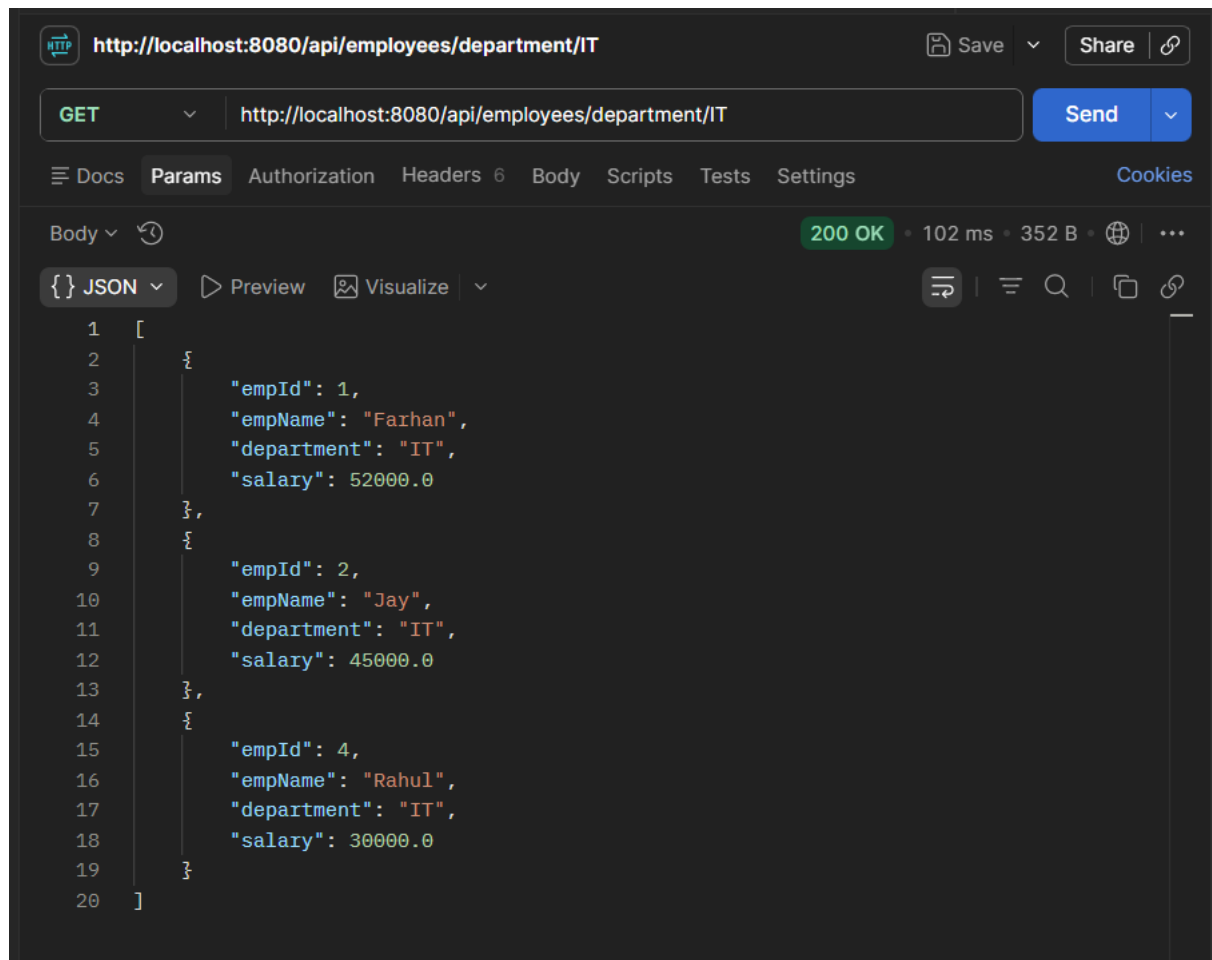
1. **POST API – Add Employee** : The POST API inserts employee data into database.

POST <http://localhost:8080/api/>



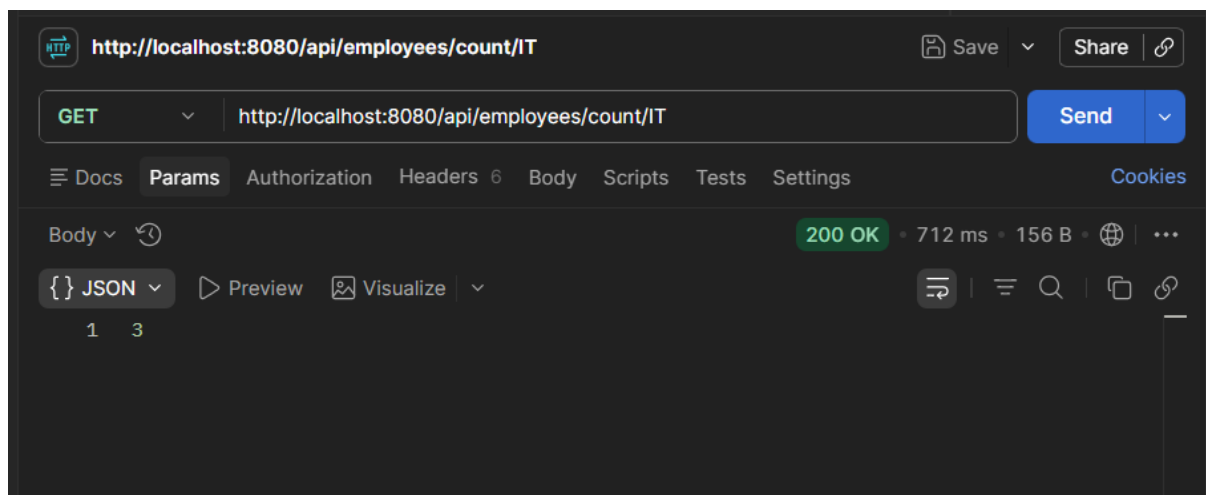
After Add some multiple records: -



2. GET API – Test Department Query : Get employee by department find ITGET <http://localhost:8080/api/employees/department/IT>

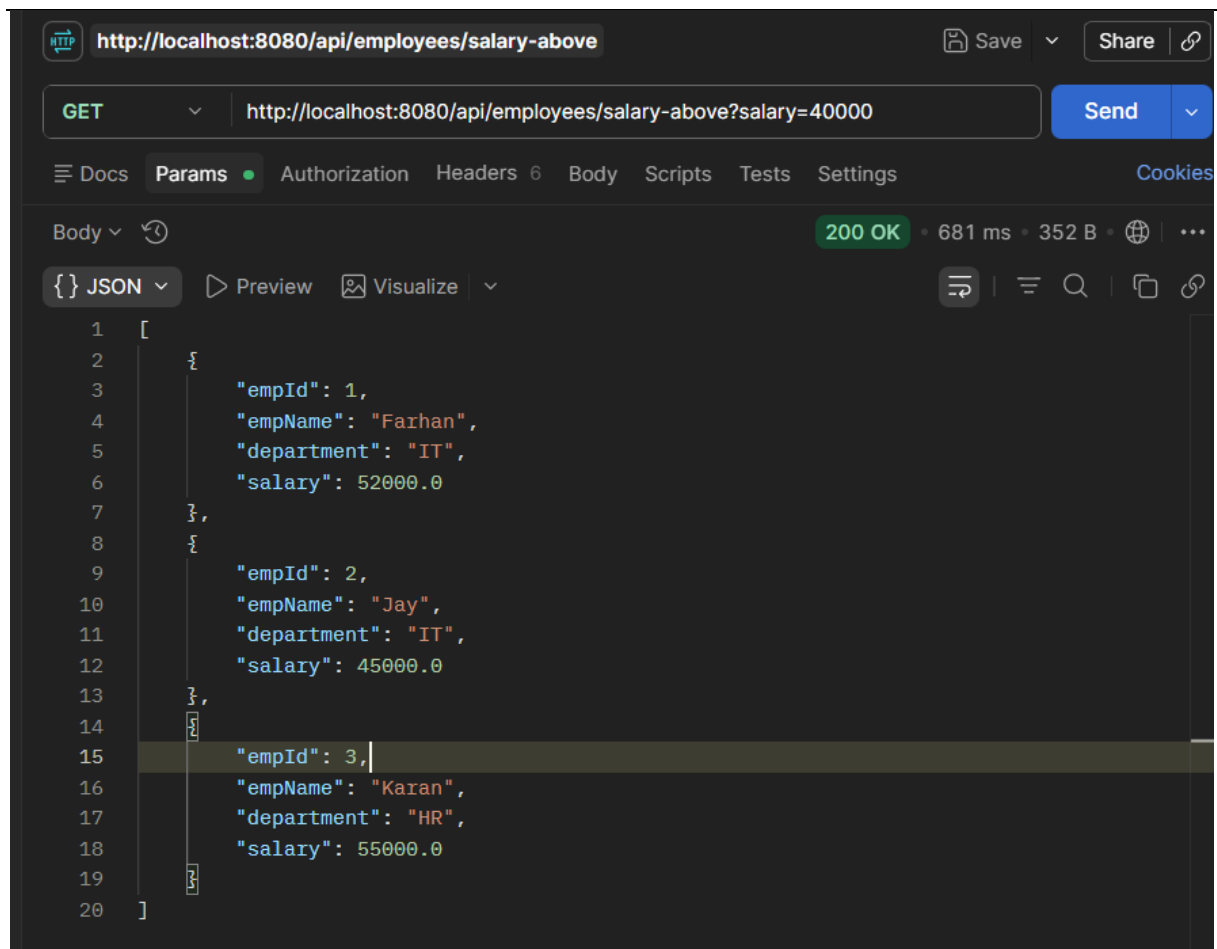
-
- 3. GET API (Count Employees in department IT):** The GET API retrieves all IT department of employees stored in the database.

GET <http://localhost:8080/api/employees/count/IT>



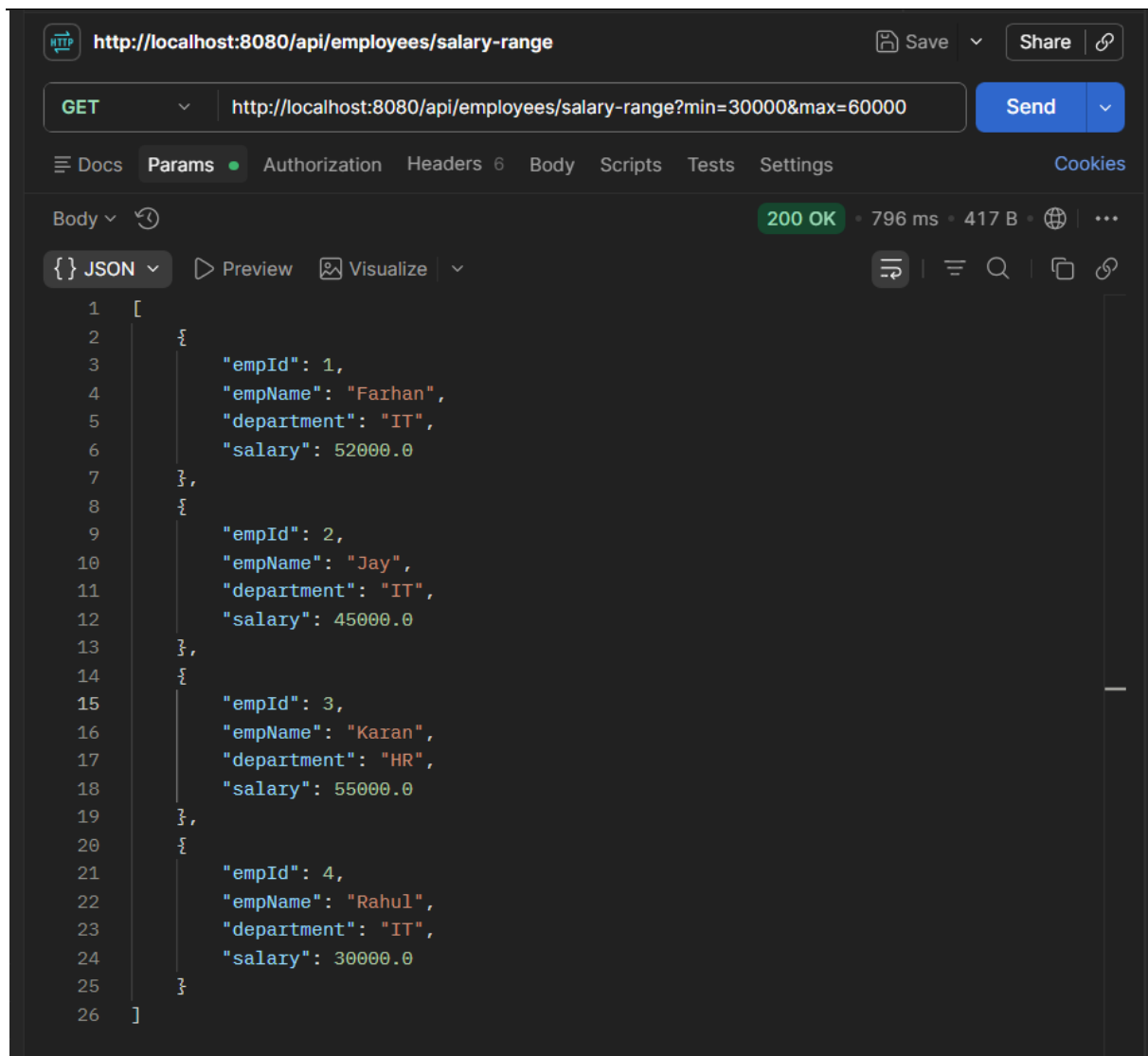
- 4. GET API –Employee salary above 40000:-** The GET API retrieves the above salary of employee greater than 40000.

GET <http://localhost:8080/api/employees/salary-above?salary=40000>



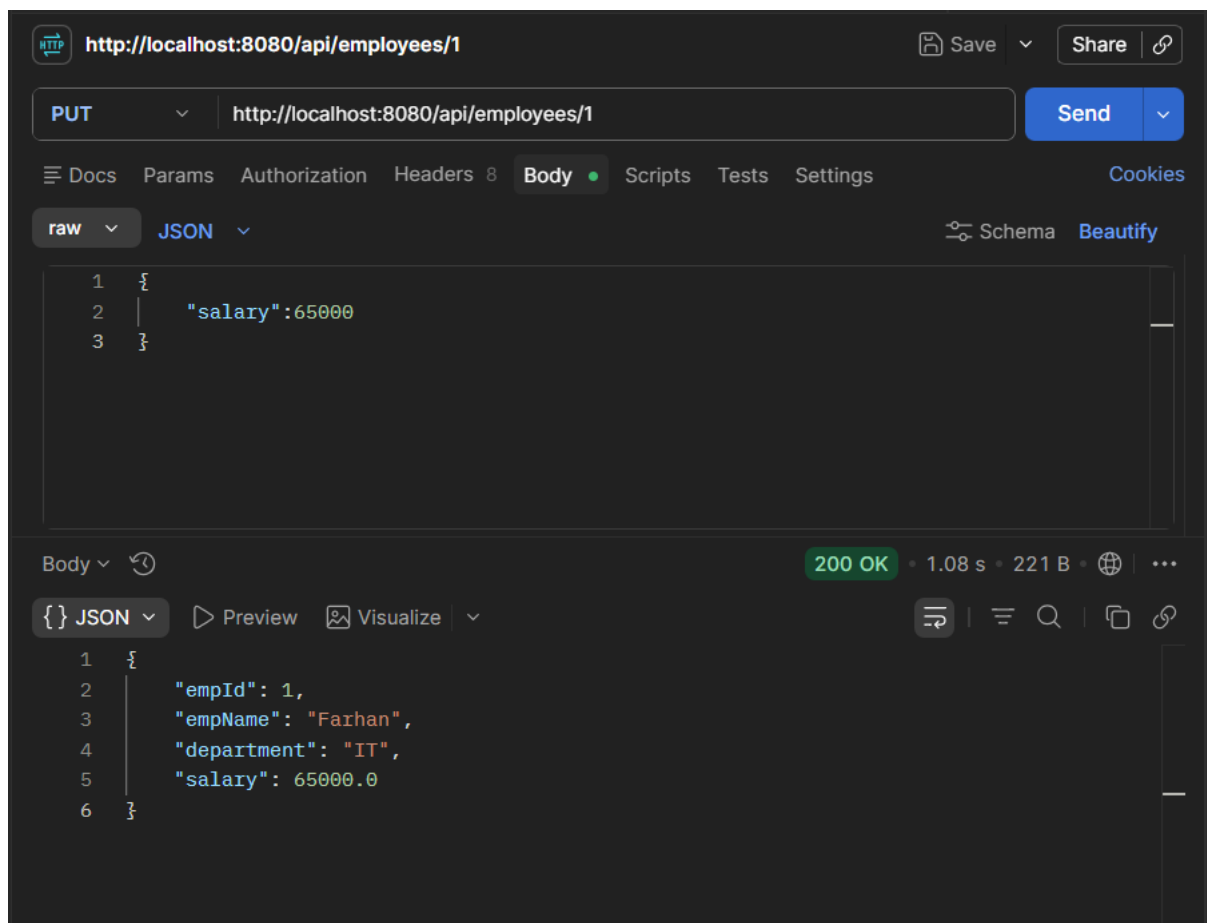
5. **GET API** – Employees salary range query : Retrieves a salary range between 30000 to 60000.

GET <http://localhost:8080/api/employees/salary-range?min=30000&max=60000>



6. **PUT API – UPDATE Salary** : Update the Employee salary of Farhan whose Id is 1 52000 to 65000.

PUT <http://localhost:8080/api/employees/1>

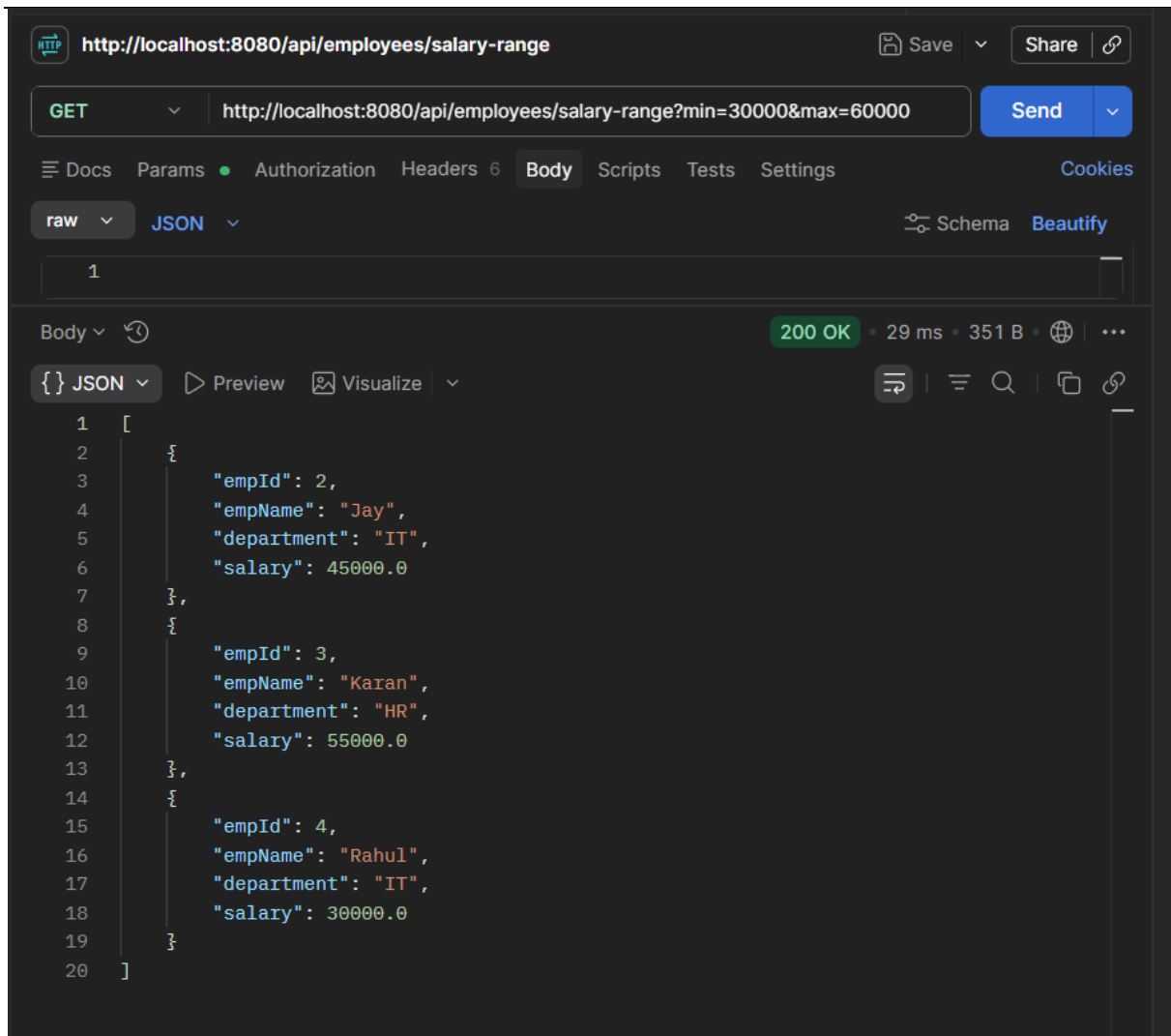


Check Salary Range After PUT:

Before salary update is in the range of 30k-60k now after update it is out of range.

GET <http://localhost:8080/api/employees/salary-range?min=30000&max=60000>

Show Farhan is not shown In list



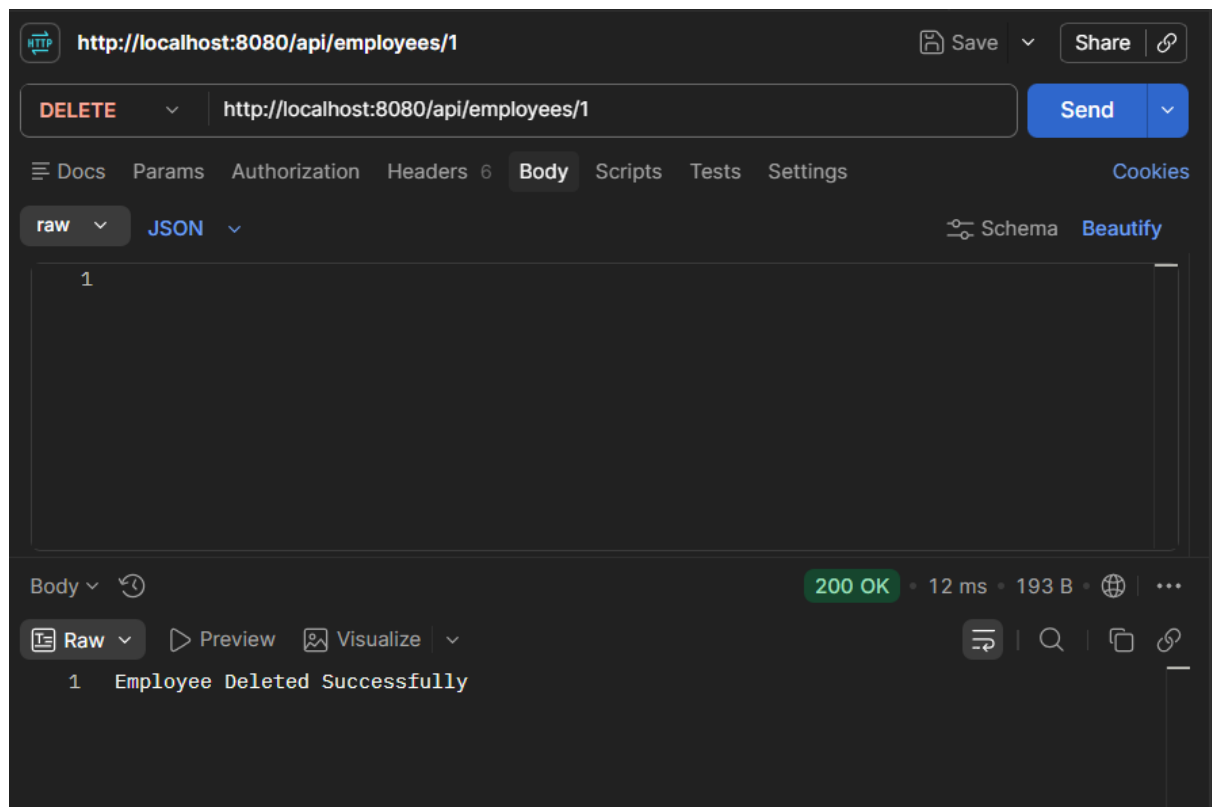
The screenshot shows an HTTP client interface with the following details:

- URL:** `http://localhost:8080/api/employees/salary-range`
- Method:** GET
- Query Parameters:** `?min=30000&max=60000`
- Status:** 200 OK
- Response Time:** 29 ms
- Response Size:** 351 B
- Response Format:** JSON
- Response Body:**

```
[
  {
    "empId": 2,
    "empName": "Jay",
    "department": "IT",
    "salary": 45000.0
  },
  {
    "empId": 3,
    "empName": "Karan",
    "department": "HR",
    "salary": 55000.0
  },
  {
    "empId": 4,
    "empName": "Rahul",
    "department": "IT",
    "salary": 30000.0
  }
]
```

7. DELETE API – DELETE Emp id:- Deletes the employee record from the database.

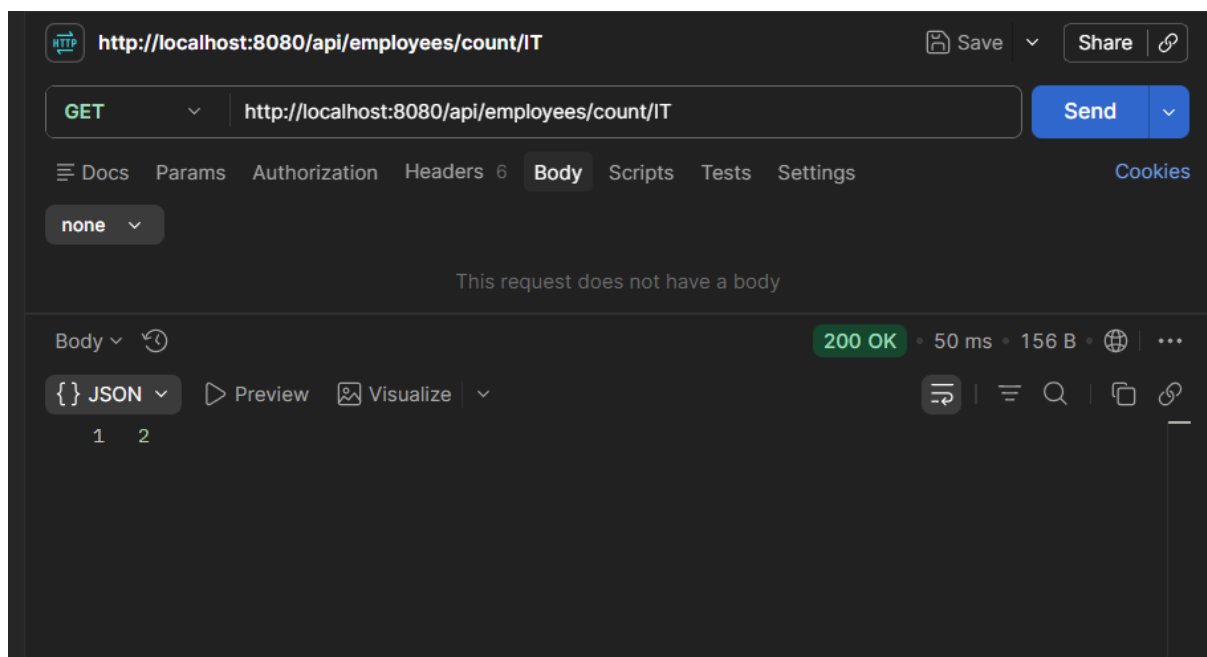
DELETE <http://localhost:8080/api/employees/1>



8. **GET API:-** Verify countByDepartment("IT") After DELETE the first ID.

GET <http://localhost:8080/api/employees/count/IT>

Before it is 3 after Farhan removed it Change to 2.



DATABASE OUTPUT

EMPLOYEE TABLE AND DATA STORED SUCCESSFULLY

WE JUST NEED TO CREATE THE DATABASE. HIBERNATE ITSELF WILL CREATE THE TABLE FROM THE ENTITY CLASS.

Server: MySQL:3306 » Database: employee_db » Table: employee

Browse Structure SQL Search Insert Export Import Privileges Operations Triggers

Showing rows 0 - 3 (4 total, Query took 0.0006 seconds.)

`SELECT * FROM `employee``

☐ Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

		emp_id	department	emp_name	salary
☐	Edit Copy Delete	2	IT	Jay	45000
☐	Edit Copy Delete	3	HR	Karan	55000
☐	Edit Copy Delete	4	IT	Rahul	30000
☐	Edit Copy Delete	5	Finance	Ajay	20000

☐ Check all With selected: Edit Copy Delete Export

Practical No. 4: One-to-One Mapping – Three Worked Examples

1. User and User profile

Objective:

To implement the One-to-One relationship using @OneToOne, @JoinColumn (owning side) and mappedBy

(inverse side), with cascading so that saving a parent entity also persists its associated entity. Three independent

one-to-one example pairs are implemented to reinforce the pattern across different domains; full CRUD is built

and tested for the primary example (User–UserProfile).

Class Name(s) with Attributes:

User (@Entity — Example 1, owning side)

- userId : int (@Id @GeneratedValue)
- username : String
- email : String
- userProfile : UserProfile (@OneToOne, cascade=ALL, @JoinColumn(name="profile_id"))

UserProfile (@Entity — Example 1, inverse side)

- profileId : int (@Id @GeneratedValue)
- address : String
- phone : String
- dob : LocalDate
- user : User (@OneToOne(mappedBy="userProfile"))

Employee (@Entity — Example 2, owning side)

- empId : int (@Id @GeneratedValue)
- empName : String

- department : String
- passport : Passport (@OneToOne, cascade=ALL, @JoinColumn(name="passport_id"))

Passport (@Entity — Example 2, inverse side)

- passportId : int (@Id @GeneratedValue)
- passportNumber : String
- issuingCountry : String
- expiryDate : LocalDate
- employee : Employee (@OneToOne(mappedBy="passport"))

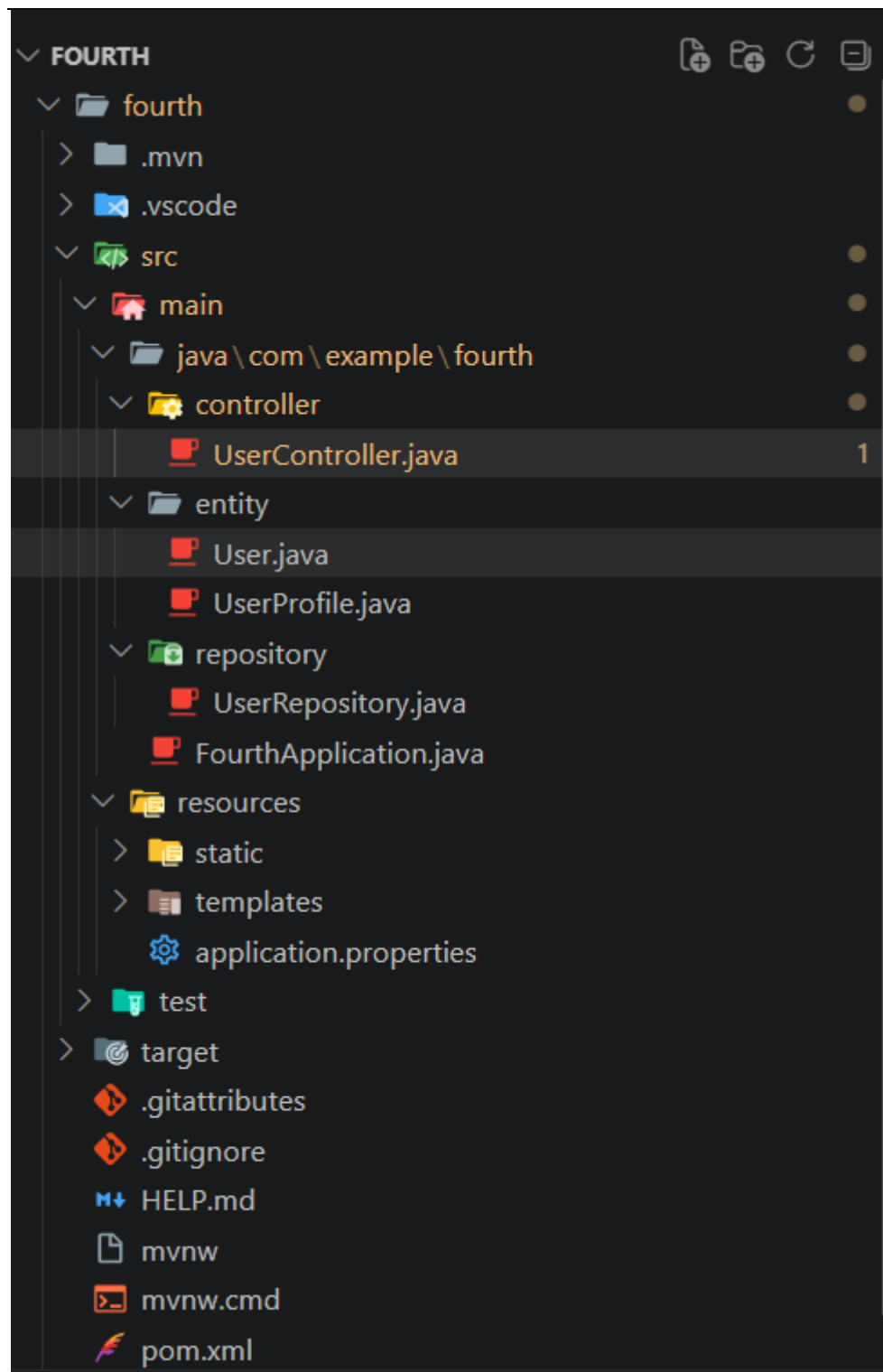
Country (@Entity — Example 3, owning side)

- countryId : int (@Id @GeneratedValue)
- countryName : String
- capital : Capital (@OneToOne, cascade=ALL, @JoinColumn(name="capital_id"))

Capital (@Entity — Example 3, inverse side)

- capitalId : int (@Id @GeneratedValue)
- cityName : String
- population : long
- country : Country (@OneToOne(mappedBy="capital"))

PROJECT STRUCTURE



Dependencies

- Spring Web
- Spring Data JPA
- MySQL Driver

Database

Database Name: fourth

Table: 1.user

2. user_profile

UserProfile.java (ENTITY model class)

```
package com.example.fourth.entity;  
  
import jakarta.persistence.*;  
  
@Entity  
public class UserProfile {  
  
    @Id  
    @GeneratedValue(strategy = GenerationType.IDENTITY)  
    private int profileId;  
  
    private String phone;  
  
    private String city;
```

```
private String address;

private String dob;

public UserProfile() {
}

public int getProfileId() {
    return profileId;
}

public void setProfileId(int profileId) {
    this.profileId = profileId;
}

public String getPhone() {
    return phone;
}

public void setPhone(String phone) {
    this.phone = phone;
}

public String getCity() {
    return city;
}
```

```
public void setCity(String city) {  
    this.city = city;  
}  
  
public String getAddress() {  
    return address;  
}  
  
public void setAddress(String address) {  
    this.address = address;  
}  
  
public String getDob() {  
    return dob;  
}  
  
public void setDob(String dob) {  
    this.dob = dob;  
}  
}
```


User.java (Entity model class)

```
package com.example.fourth.entity;

import jakarta.persistence.*;

@Entity
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int userId;

    private String username;

    private String email;

    @OneToOne(cascade=CascadeType.ALL)
    @JoinColumn(name="profile_id")
    private UserProfile userProfile;

    public int getUserId(){
        return userId;
    }

    public void setUserId(int userId){
```

```
this.userId=userId;
}

public String getUsername(){
    return username;
}

public void setUsername(String username){
    this.username=username;
}

public String getEmail(){
    return email;
}

public void setEmail(String email){
    this.email=email;
}

public UserProfile getUserProfile(){
    return userProfile;
}

public void setUserProfile(UserProfile userProfile){
    this.userProfile=userProfile;
}
}
```

UserRepository.java (Repository file)

```
package com.example.fourth.repository;

import org.springframework.data.jpa.repository.JpaRepository;

import com.example.fourth.entity.User;

public interface UserRepository
extends JpaRepository<User,Integer>{

}
```

UserRestController.java (REST controller file)

```
package com.example.fourth.controller;

import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

import com.example.fourth.entity.User;
import com.example.fourth.repository.UserRepository;

@RestController
@RequestMapping("/api/users")
public class UserController {
```

```
@Autowired
```

```
private UserRepository repository;
```

```
// Add User
```

```
@PostMapping
```

```
public User addUser(@RequestBody User user) {
```

```
    return repository.save(user);
```

```
}
```

```
// Get All Users
```

```
@GetMapping
```

```
public List<User> getAllUsers() {
```

```
    return repository.findAll();
```

```
}
```

```
// Get User By ID
```

```
@GetMapping("/{id}")
```

```
public User getUser(@PathVariable Integer id) {
```

```
    return repository.findById(id).orElse(null);
```

```
}
```

```
// Update User
```

```
@PutMapping("/{id}")
```

```
public User updateUser(@PathVariable Integer id, @RequestBody User user) {
```

```
    user.setUserId(id);
```

```
    return repository.save(user);
```

```
}

// Delete User
@DeleteMapping("/{id}")
public String deleteUser(@PathVariable Integer id) {

    repository.deleteByld(id);
    return "User Deleted Successfully";
}
}
```

Application.properties (database and Hibernate configuration)

```
spring.application.name=fourth
spring.datasource.url=jdbc:mysql://localhost:3306/fourth

spring.datasource.username=root

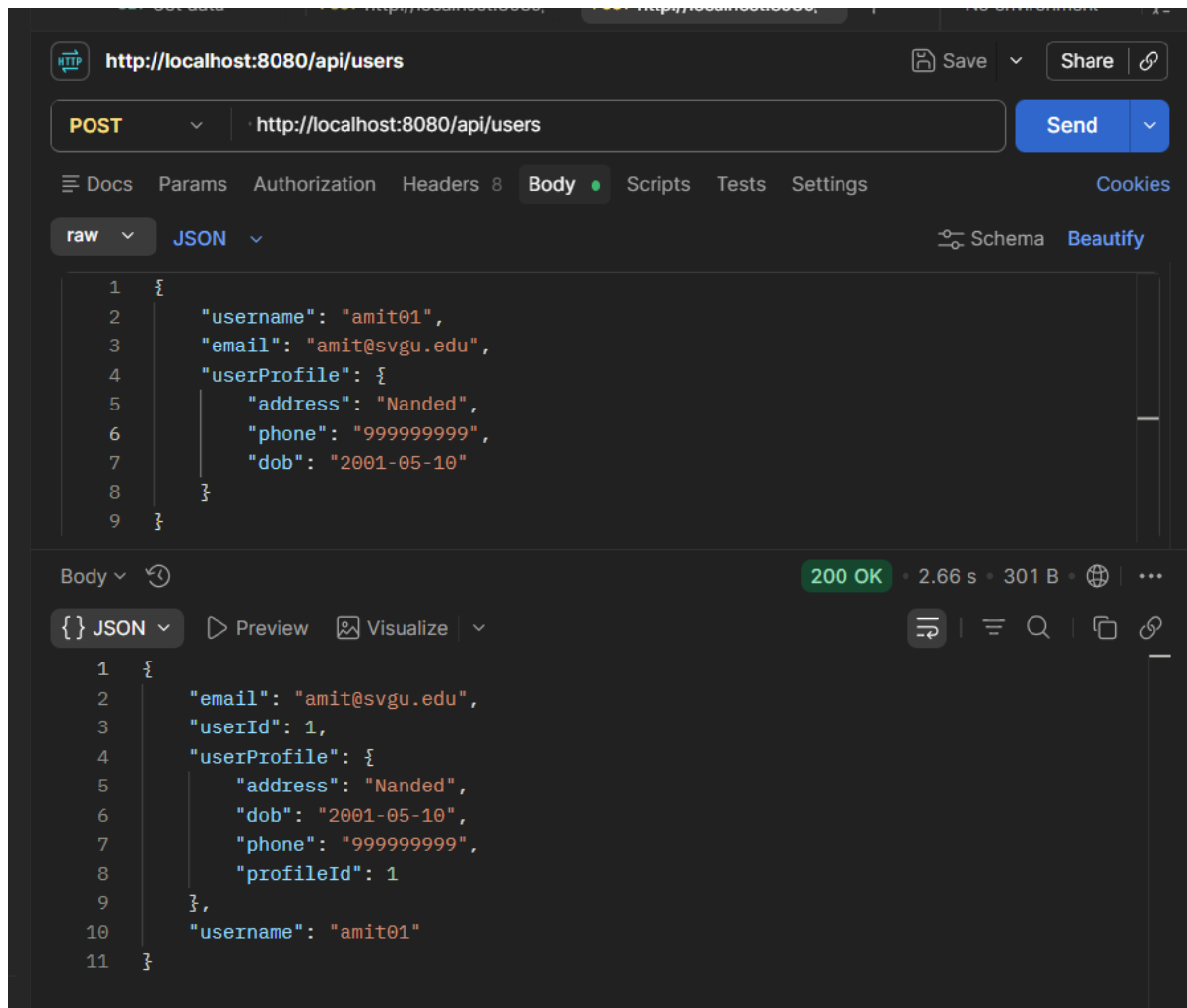
spring.datasource.password=

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

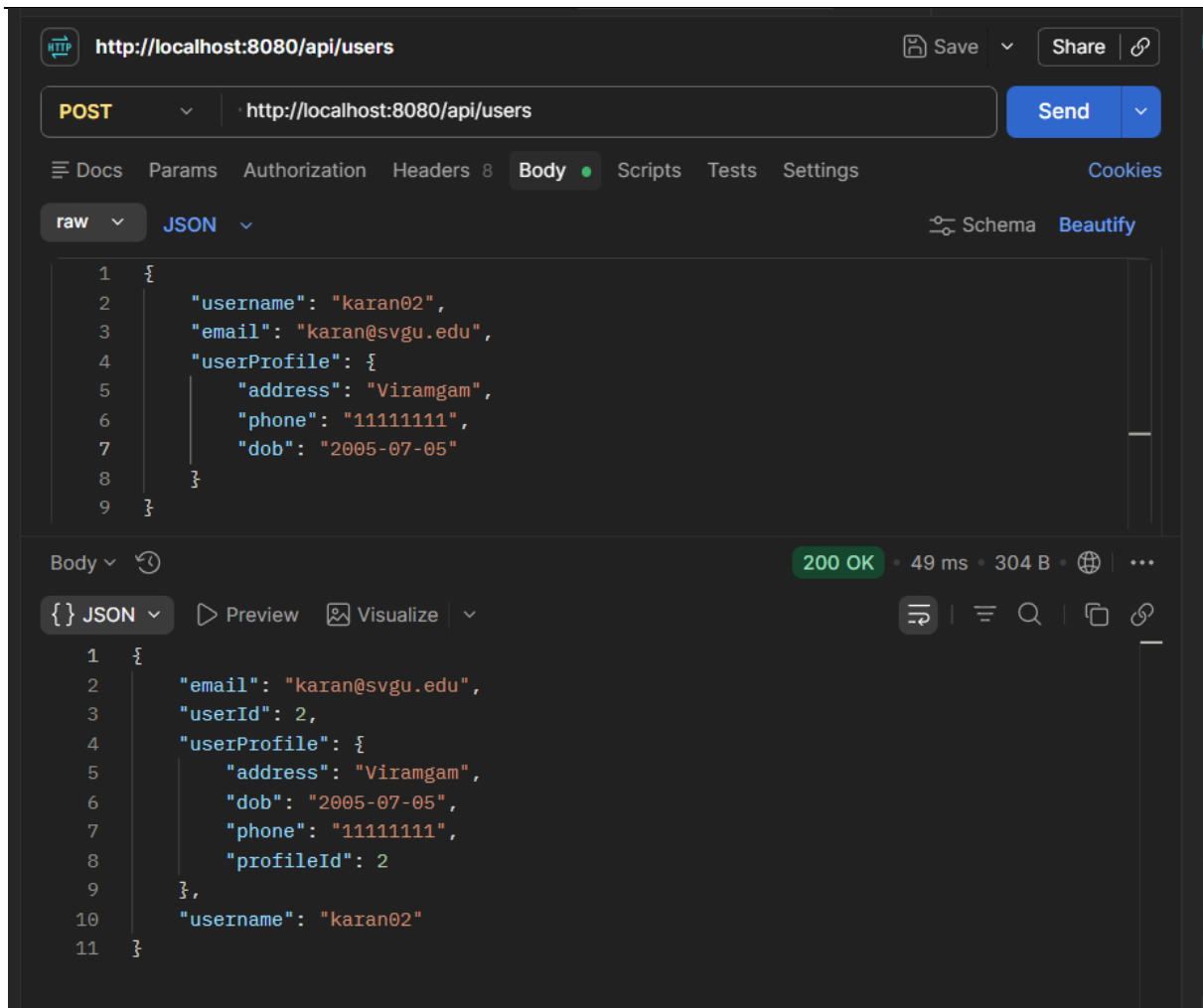
API'S TESTING USING POSTMAN

1. **POST API – Add Users** : The POST API inserts users data into database.

POST <http://localhost:8080/api/users>



Add One more record:-



http://localhost:8080/api/users

POST http://localhost:8080/api/users

Body

```
1 {
2   "username": "karan02",
3   "email": "karan@svgu.edu",
4   "userProfile": {
5     "address": "Viramgam",
6     "phone": "11111111",
7     "dob": "2005-07-05"
8   }
9 }
```

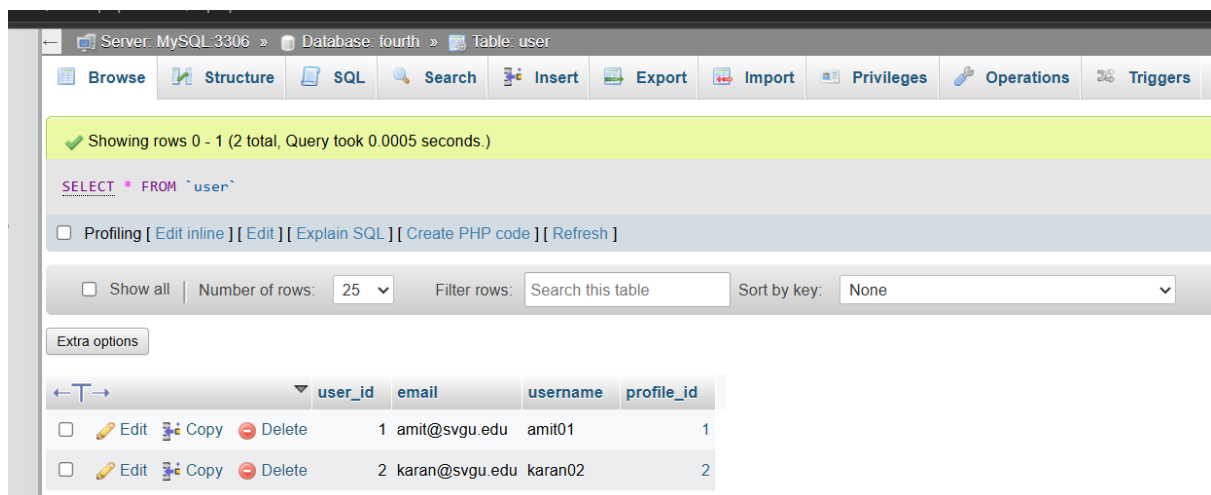
Body

```
1 {
2   "email": "karan@svgu.edu",
3   "userId": 2,
4   "userProfile": {
5     "address": "Viramgam",
6     "dob": "2005-07-05",
7     "phone": "11111111",
8     "profileId": 2
9   },
10  "username": "karan02"
11 }
```

200 OK • 49 ms • 304 B

Verify in Database:-

Both tables are created and data inserted..



Server: MySQL 3306 » Database: fourth » Table: user

Showing rows 0 - 1 (2 total, Query took 0.0005 seconds.)

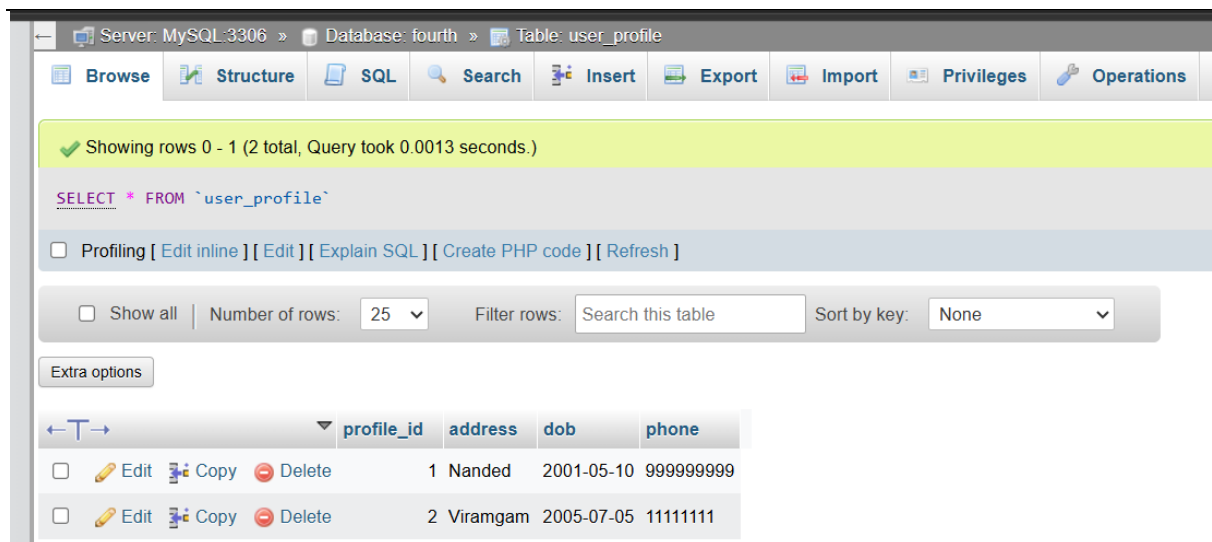
SELECT * FROM `user`

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

	user_id	email	username	profile_id
☐ Edit Copy Delete	1	amit@svgu.edu	amit01	1
☐ Edit Copy Delete	2	karan@svgu.edu	karan02	2



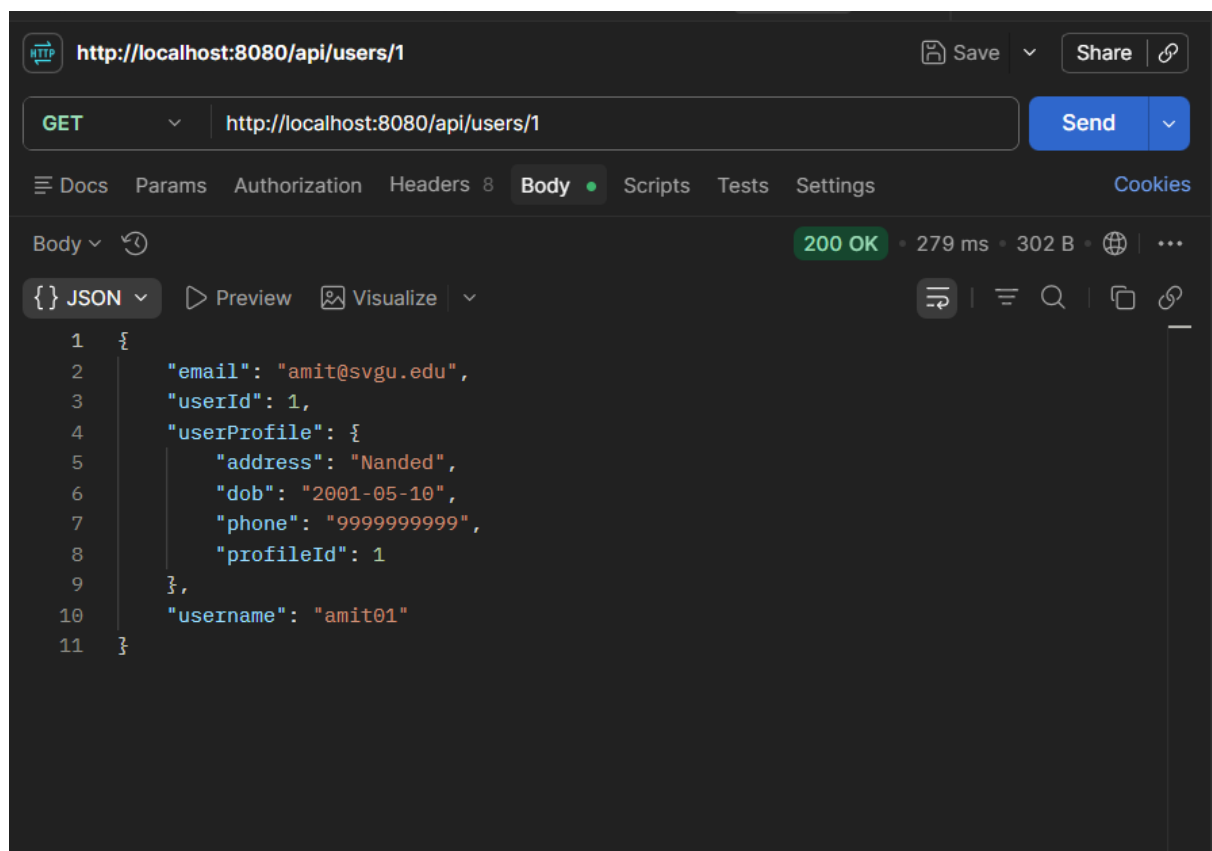
The screenshot shows the MySQL Workbench interface. At the top, the server is MySQL:3306, the database is fourth, and the table is user_profile. The SQL tab is active, showing the query: `SELECT * FROM `user_profile``. Below the query, there are options for Profiling, Edit inline, Edit, Explain SQL, Create PHP code, and Refresh. A status bar indicates 'Showing rows 0 - 1 (2 total, Query took 0.0013 seconds.)'. Below this, there are controls for 'Show all', 'Number of rows' (set to 25), 'Filter rows' (Search this table), and 'Sort by key' (set to None). An 'Extra options' button is also present. The table data is displayed below, with columns: profile_id, address, dob, and phone. There are two rows of data.

	profile_id	address	dob	phone
☐ Edit ☐ Copy ☐ Delete	1	Nanded	2001-05-10	999999999
☐ Edit ☐ Copy ☐ Delete	2	Viramgam	2005-07-05	11111111

2. **GET API – Retrieve user by id:** The GET API retrieves a specific user record using the user ID.

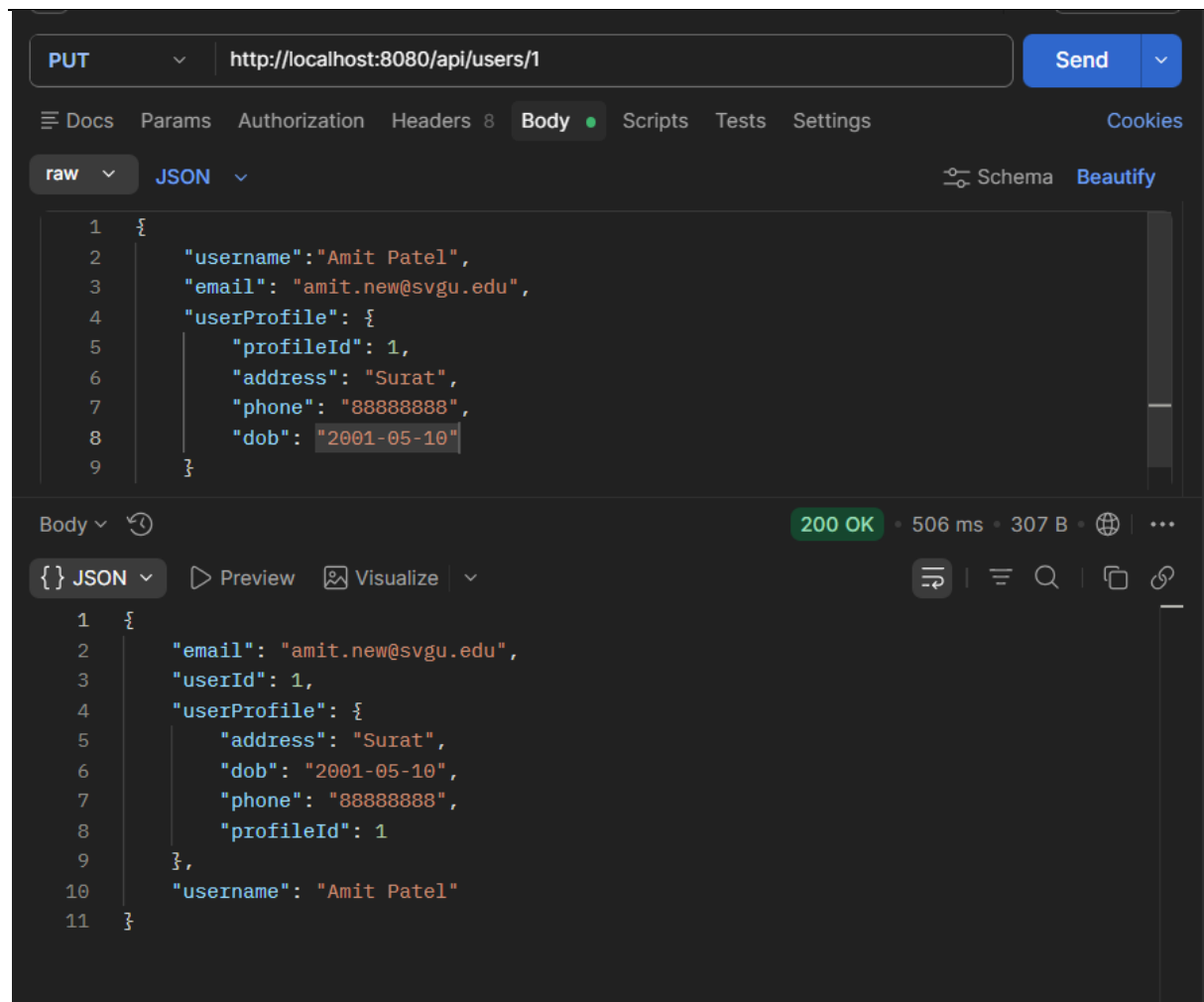
The associated UserProfile details are displayed along with the User object because of the One-to-One relationship.

GET <http://localhost:8080/api/users/1>



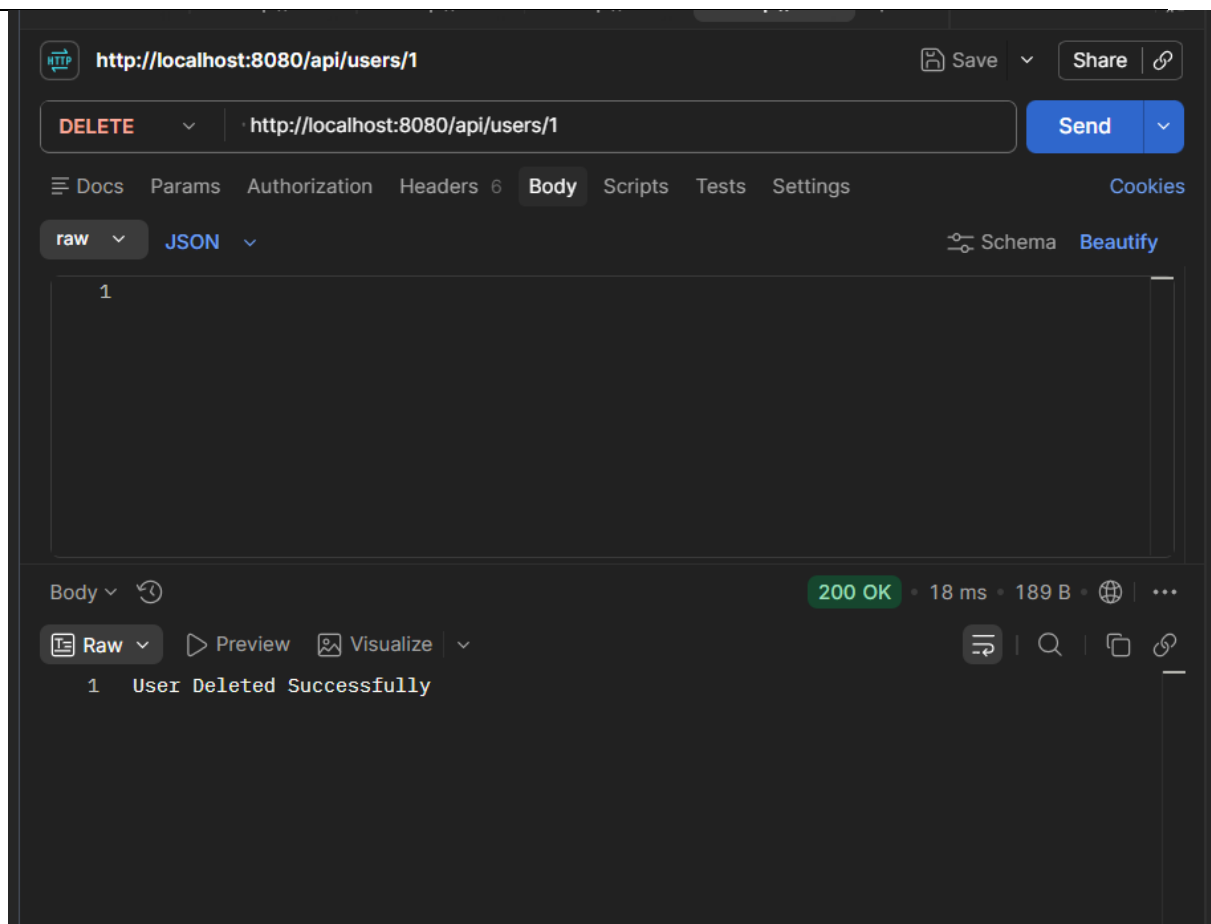
3. PUT API – UPDATE address and phone number :

PUT <http://localhost:8080/api/users/1>



4. DELETE API – DELETE user id:- Deletes the record from the database.

DELETE <http://localhost:8080/api/users/1>



DATABASE OUTPUT

WE JUST NEED TO CREATE THE DATABASE. HIBERNATE ITSELF WILL CREATE THE TABLE FROM THE ENTITY CLASS.

After deleted first id show both tables:

User profile Table:-

Server: MySQL:3306 » Database: fourth » Table: user_profile

Showing rows 0 - 0 (1 total, Query took 0.0003 seconds.)

```
SELECT * FROM `user_profile`
```

☐ Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

☐ Show all | Number of rows: 25 | Filter rows: Search this table

Extra options

	profile_id	address	dob	phone
☐ Edit Copy Delete	2	Viramgam	2005-07-05	111111111

User table: -

Server: MySQL:3306 » Database: fourth » Table: user

Showing rows 0 - 0 (1 total, Query took 0.0003 seconds.)

```
SELECT * FROM `user`
```

☐ Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

☐ Show all | Number of rows: 25 | Filter rows: Search this table

Extra options

	user_id	email	username	profile_id
☐ Edit Copy Delete	2	karan@svgu.edu	karan02	2

☐ Check all | With selected: Edit Copy Delete Export

2.EMPLOYEE AND PASSPORT

Project Structure:-

```
fourth
|
|-- src
|   |-- main
|       |-- java
|           |-- com.example.fourth
|               |-- controller
|                   |-- EmployeeRestController.java
|               |-- entity
|                   |-- Employee.java
|                   |-- Passport.java
|               |-- repository
|                   |-- EmployeeRepository.java
|               |-- FourthApplication.java
|           |-- resources
|               |-- application.properties
|
|-- pom.xml
```

Dependencies

- Spring Web
- Spring Data JPA
- MySQL Driver

Database

Database Name: fourth

Table: 1.employee

2. Passport

Employee.java (ENTITY model class)

```
package com.example.fourth.entity;

import com.fasterxml.jackson.annotation.JsonManagedReference;
import jakarta.persistence.*;

@Entity

public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer empId;

    private String empName;
```

```
private String department;

@OneToOne(cascade = CascadeType.ALL)
@JoinColumn(name = "passport_id")
@JsonManagedReference
private Passport passport;

public Employee() {
}

public Integer getEmpId() {
    return empId;
}

public void setEmpId(Integer empId) {
    this.empId = empId;
}

public String getEmpName() {
    return empName;
}

public void setEmpName(String empName) {
    this.empName = empName;
}

public String getDepartment() {
```

```
        return department;
    }

    public void setDepartment(String department) {
        this.department = department;
    }

    public Passport getPassport() {
        return passport;
    }

    public void setPassport(Passport passport) {
        this.passport = passport;
    }
}
```

Passport.java (Entity file)

```
package com.example.fourth.entity;

import java.time.LocalDate;

import com.fasterxml.jackson.annotation.JsonBackReference;
import jakarta.persistence.*;
```



```
@Entity

public class Passport {

    @Id

    @GeneratedValue(strategy = GenerationType.IDENTITY)

    private Integer passportId;

    private String passportNumber;

    private String issuingCountry;

    private LocalDate expiryDate;

    @OneToOne(mappedBy = "passport")

    @JsonBackReference

    private Employee employee;

    public Passport() {

    }

    public Integer getPassportId() {

        return passportId;

    }

    public void setPassportId(Integer passportId) {

        this.passportId = passportId;

    }

}
```

```
public String getPassportNumber() {  
    return passportNumber;  
}  
  
public void setPassportNumber(String passportNumber) {  
    this.passportNumber = passportNumber;  
}  
  
public String getIssuingCountry() {  
    return issuingCountry;  
}  
  
public void setIssuingCountry(String issuingCountry) {  
    this.issuingCountry = issuingCountry;  
}  
  
public LocalDate getExpiryDate() {  
    return expiryDate;  
}  
  
public void setExpiryDate(LocalDate expiryDate) {  
    this.expiryDate = expiryDate;  
}  
  
public Employee getEmployee() {  
    return employee;  
}
```

```
}

    public void setEmployee(Employee employee) {
        this.employee = employee;
    }
}
```

EmployeeRepository.java (Repository file)

```
package com.example.fourth.repository;

import org.springframework.data.jpa.repository.JpaRepository;

import com.example.fourth.entity.Employee;

public interface EmployeeRepository extends JpaRepository<Employee, Integer> {

}
```

EmployeeRestController.java (REST controller file)

```
package com.example.fourth.controller;

import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;
```

```
import com.example.fourth.entity.Employee;
import com.example.fourth.repository.EmployeeRepository;

@RestController
@RequestMapping("/api/employees-passport")
public class EmployeeRestController {

    @Autowired
    private EmployeeRepository repository;

    // Add Employee
    @PostMapping
    public Employee addEmployee(@RequestBody Employee employee) {
        return repository.save(employee);
    }

    // Get All Employees
    @GetMapping
    public List<Employee> getAllEmployees() {
        return repository.findAll();
    }

    // Get Employee By ID
    @GetMapping("/{id}")
    public Employee getEmployee(@PathVariable Integer id) {
        return repository.findById(id).orElse(null);
    }
}
```

```
}

// Update Employee
@PutMapping("/{id}")
public Employee updateEmployee(@PathVariable Integer id,
                               @RequestBody Employee employee) {

    employee.setEmpld(id);
    return repository.save(employee);
}

// Delete Employee
@DeleteMapping("/{id}")
public String deleteEmployee(@PathVariable Integer id) {
    repository.deleteByld(id);
    return "Employee Deleted Successfully";
}
}
```

Application.properties (database and Hibernate configuration)

```
spring.application.name=fourth
spring.datasource.url=jdbc:mysql://localhost:3306/fourth

spring.datasource.username=root
```

```
spring.datasource.password=
```

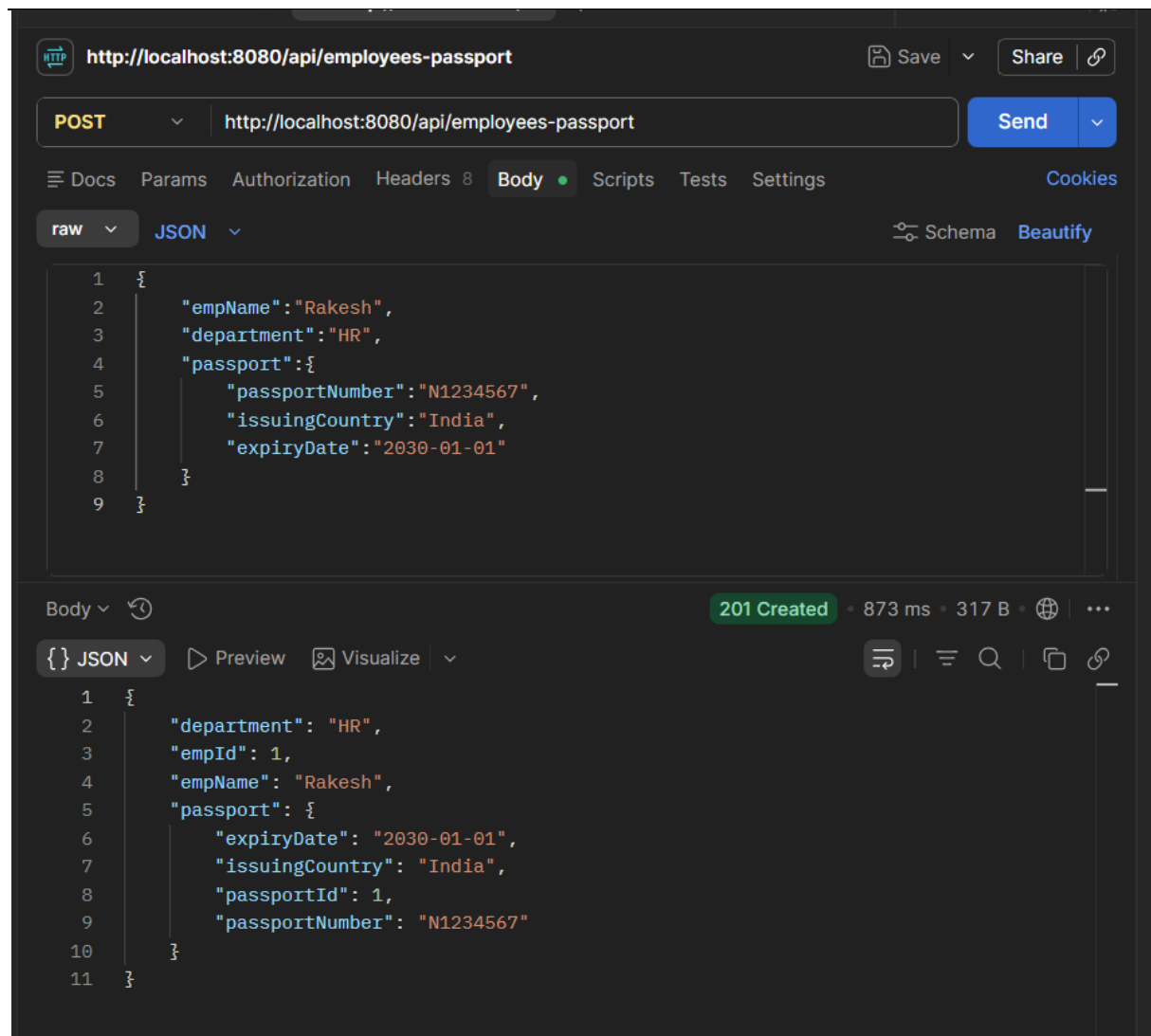
```
spring.jpa.hibernate.ddl-auto=update
```

```
spring.jpa.show-sql=true
```

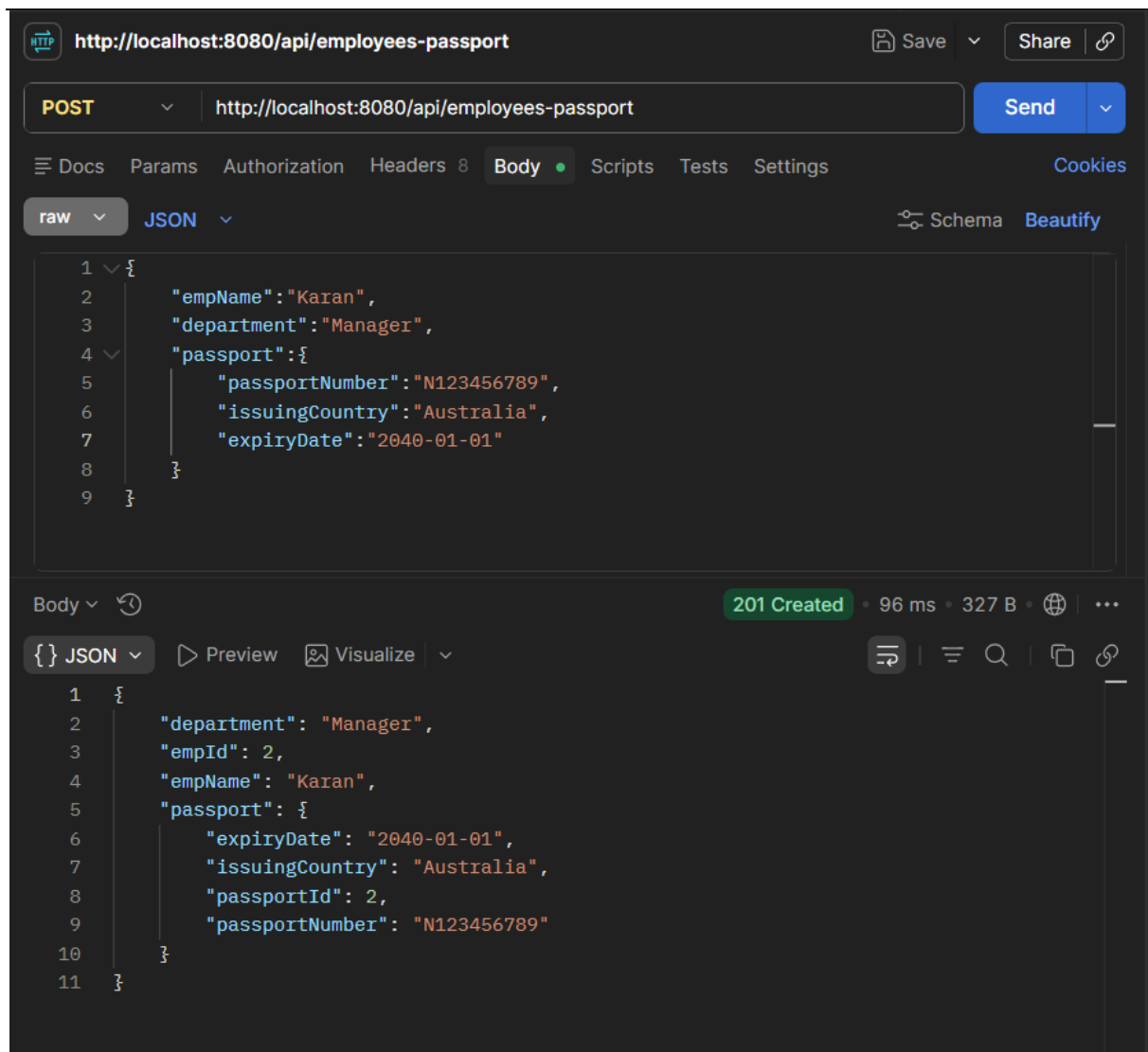
API'S TESTING USING POSTMAN

1. **POST API – Add Employee** : The POST API inserts emp data into database.

POST <http://localhost:8080/api/users>



After added one more record:-



Verify in Database:-

Both tables are created and data inserted..

Server: MySQL 3306 » Database: fourth » Table: employee

[Browse](#) [Structure](#) [SQL](#) [Search](#) [Insert](#) [Export](#) [Import](#) [Privileges](#)

✓ Showing rows 0 - 1 (2 total, Query took 0.0009 seconds.)

```
SELECT * FROM `employee`
```

☐ Profiling [[Edit inline](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

☐ Show all | Number of rows: 25 | Filter rows: Sort by key: None

Extra options

				emp_id	department	emp_name	passport_id
☐	Edit	Copy	Delete	1	HR	Rakesh	1
☐	Edit	Copy	Delete	2	Manager	Karan	2

Server: MySQL 3306 » Database: fourth » Table: passport

[Browse](#) [Structure](#) [SQL](#) [Search](#) [Insert](#) [Export](#) [Import](#) [Privileges](#) [Operations](#) [Triggers](#)

✓ Showing rows 0 - 1 (2 total, Query took 0.0006 seconds.)

```
SELECT * FROM `passport`
```

☐ Profiling [[Edit inline](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

☐ Show all | Number of rows: 25 | Filter rows: Sort by key: None

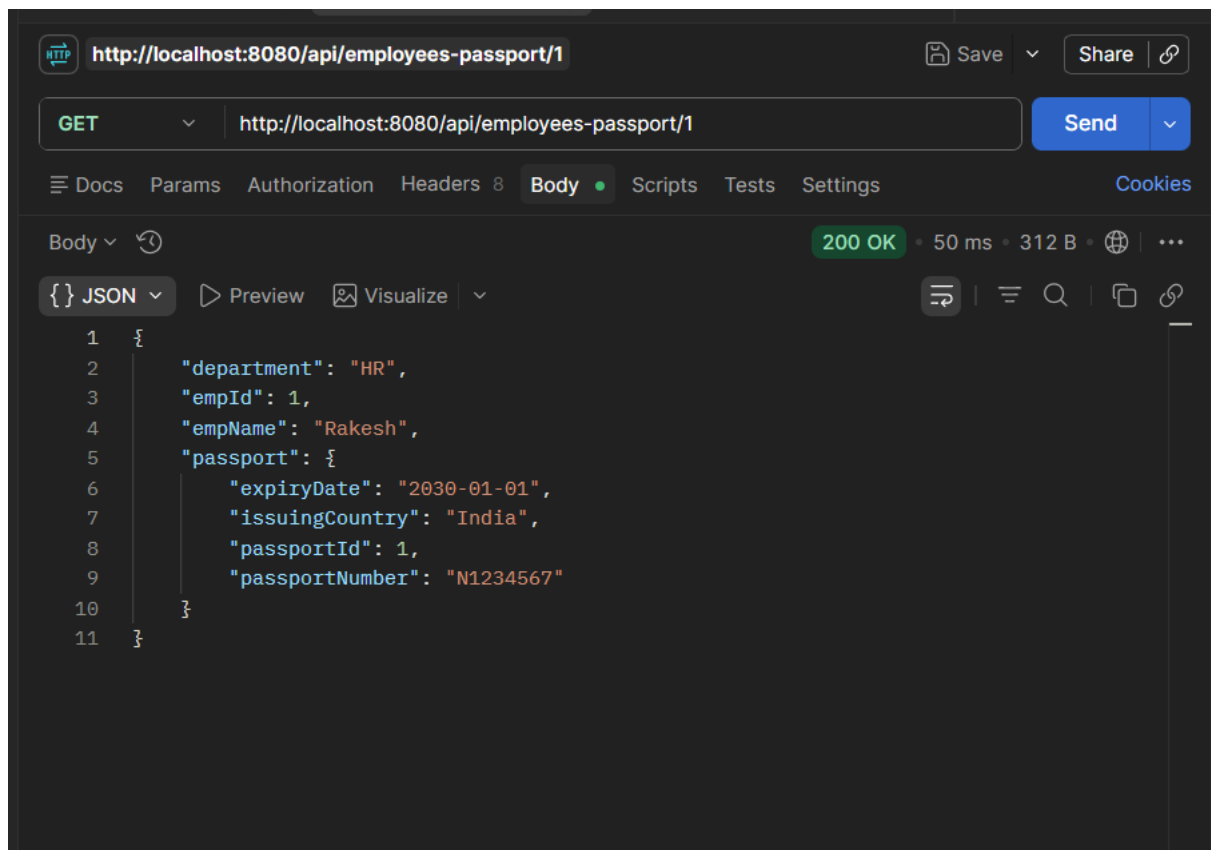
Extra options

				passport_id	expiry_date	issuing_country	passport_number
☐	Edit	Copy	Delete	1	2030-01-01	India	N1234567
☐	Edit	Copy	Delete	2	2040-01-01	Australia	N123456789

2. **GET API – Retrieve user by id:** The GET API retrieves a specific user record using the user ID.

The associated Passport details are displayed along with the Employee object because of the One-to-One relationship.

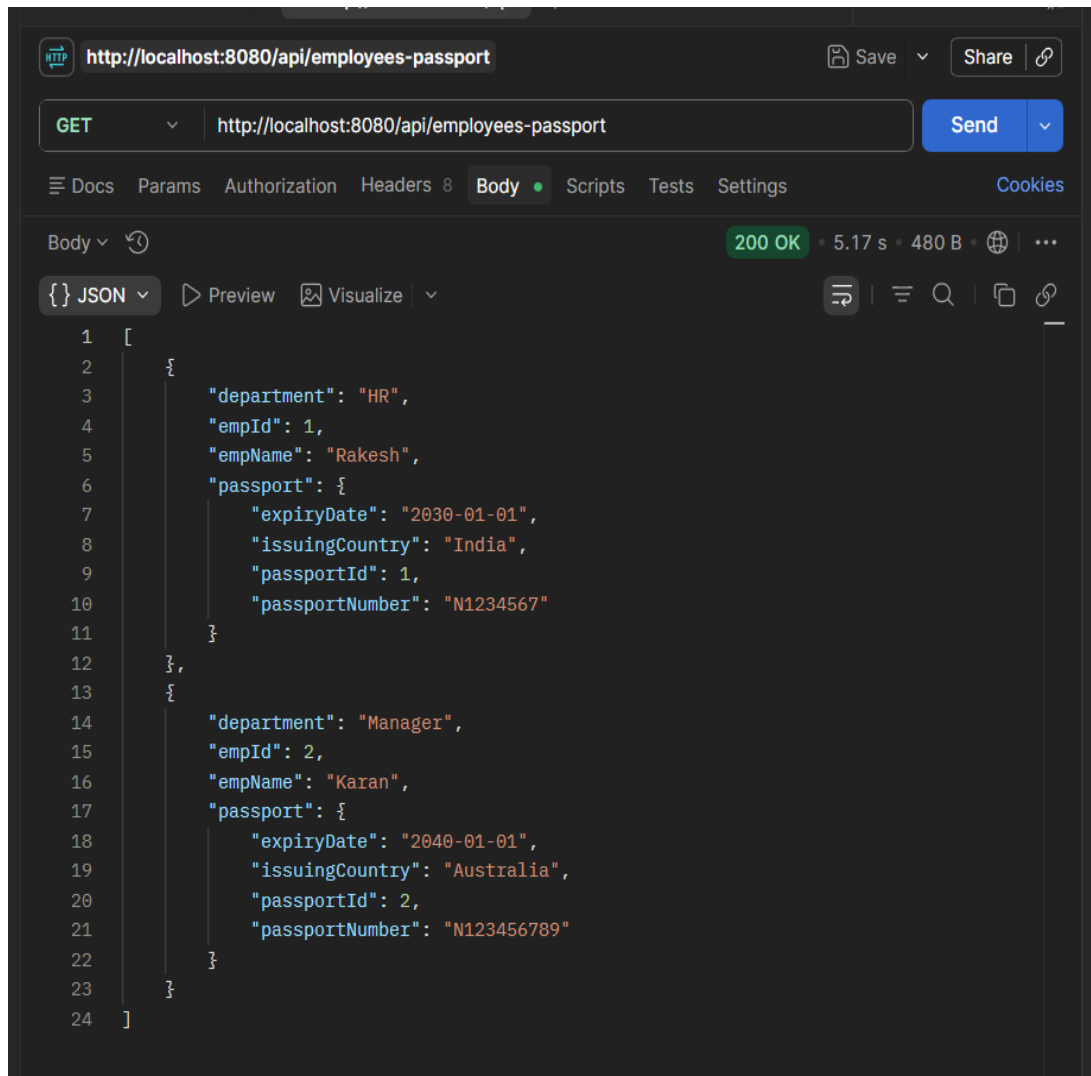
GET <http://localhost:8080/api/employees-passport/1>



3. **GET API – Retrieve All employee:** The GET API retrieves all employees from database.

Each Employee record includes its associated Passport details because of the **One-to-One** relationship..

GET <http://localhost:8080/api/employees-passport>



4. PUT API – UPDATE address and phone number :

The **PUT API** updates an existing **Employee** record using the employee ID. It allows modification of the Employee details such as **employee name** and **department**, along with the associated **Passport** details including **passport number**, **issuing country**, and **expiry date**. Because of the **One-to-One** relationship, both Employee and Passport information are updated together in a single request.

PUT <http://localhost:8080/api/employees-passport/1>

The screenshot displays a REST client interface with a PUT request to `http://localhost:8080/api/employees-passport/1`. The request body is a JSON object containing employee and passport details. The response is a 200 OK status with a JSON body that shows the updated data.

Request:

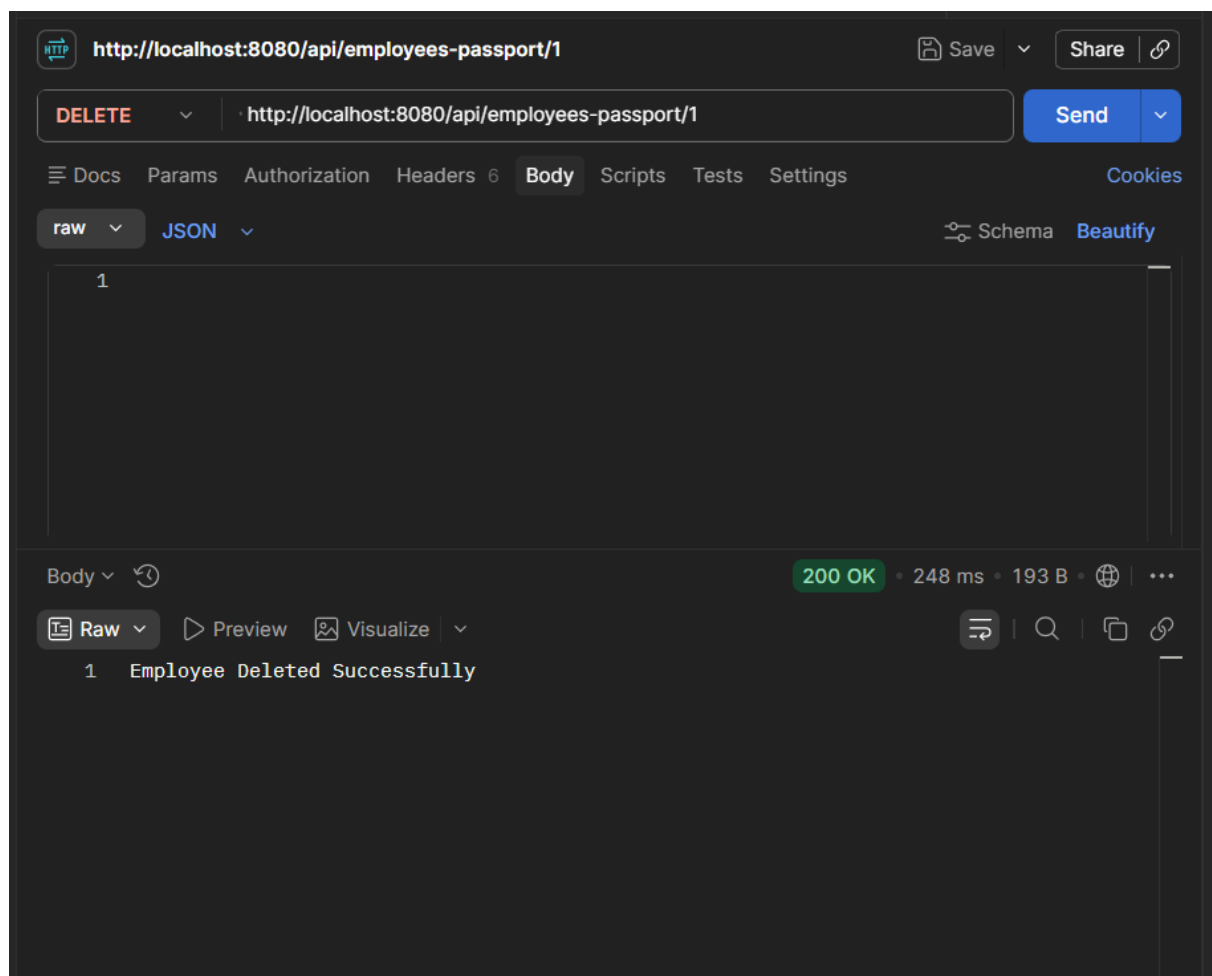
```
1 {
2   "empName": "Rakesh Kumar",
3   "department": "Finanace",
4   "passport": {
5     "passportId": 1,
6     "passportNumber": "N7654321",
7     "issuingCountry": "India",
8     "expiryDate": "2035-05-10"
9   }
10 }
```

Response:

```
1 {
2   "department": "Finanace",
3   "empId": 1,
4   "empName": "Rakesh Kumar",
5   "passport": {
6     "expiryDate": "2035-05-10",
7     "issuingCountry": "India",
8     "passportId": 1,
9     "passportNumber": "N7654321"
10   }
11 }
```

5. DELETE API – DELETE employee by id:- The **DELETE API** deletes a specific Employee record using the employee ID.

DELETE <http://localhost:8080/api/employees-passport/1>

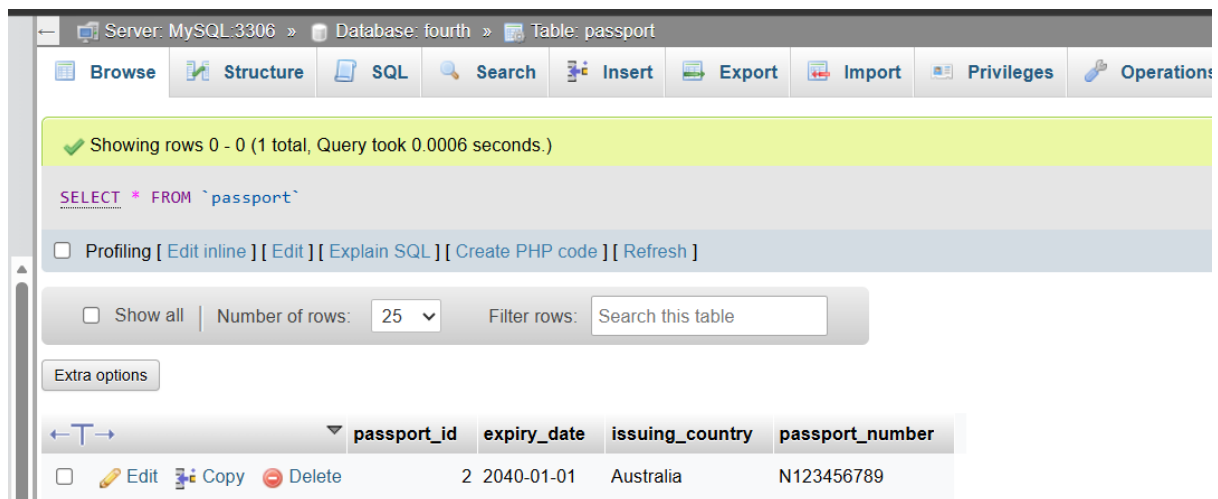


DATABASE OUTPUT

WE JUST NEED TO CREATE THE DATABASE. HIBERNATE ITSELF WILL CREATE THE TABLE FROM THE ENTITY CLASS.

After deleted first id show both tables:

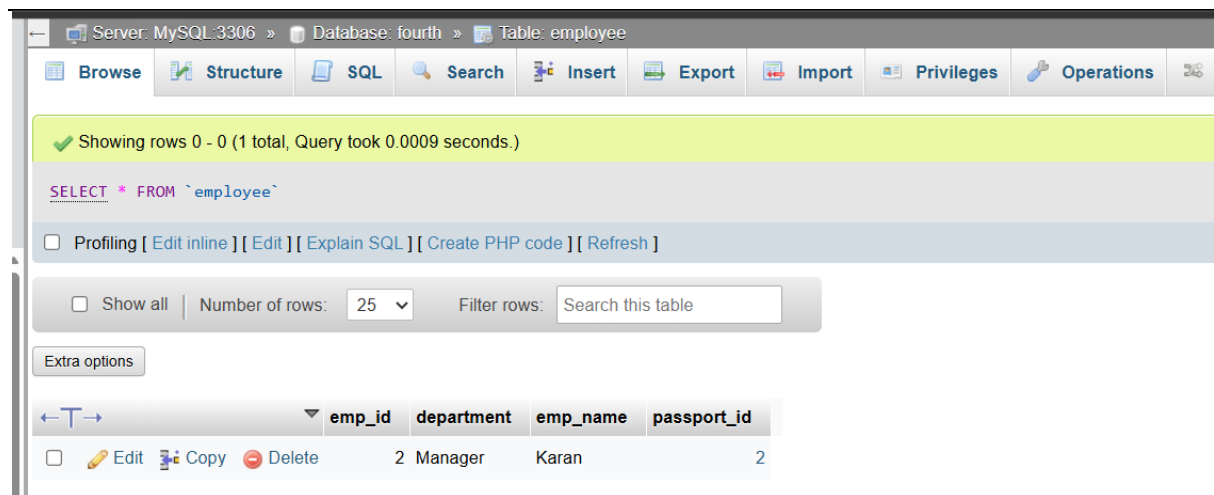
Passport Table:-



The screenshot shows the MySQL Workbench interface. The top toolbar includes buttons for Browse, Structure, SQL, Search, Insert, Export, Import, Privileges, and Operations. The main area displays a query result for the 'passport' table. A green status bar indicates 'Showing rows 0 - 0 (1 total, Query took 0.0006 seconds.)'. Below this, the SQL query 'SELECT * FROM `passport`' is shown. A toolbar for the query includes 'Profiling', 'Edit inline', 'Edit', 'Explain SQL', 'Create PHP code', and 'Refresh'. Below the query, there are controls for 'Show all', 'Number of rows' (set to 25), and a 'Filter rows' search box. An 'Extra options' button is also present. The table structure is shown with columns: passport_id, expiry_date, issuing_country, and passport_number. The data row shows: 2, 2040-01-01, Australia, and N123456789. At the bottom, there are icons for Edit, Copy, and Delete.

passport_id	expiry_date	issuing_country	passport_number
2	2040-01-01	Australia	N123456789

Employee table: -



3. Country and capital

Project Structure: -

```
src
├── main
│   ├── java
│   │   └── com.example.fourth
│   │       ├── controller
│   │       │   └── CountryRestController.java
│   │       ├── entity
│   │       │   ├── Country.java
│   │       │   └── Capital.java
│   │       ├── repository
│   │       │   └── CountryRepository.java
│   │       └── FourthApplication.java
│   └── resources
│       └── application.properties
```

Dependencies

- Spring Web
- Spring Data JPA
- MySQL Driver

Database

Database Name: fourth

Table: 1. Capital

2. Country

Capital.java (ENTITY model class)

```
package com.example.fourth.entity;

import com.fasterxml.jackson.annotation.JsonBackReference;
import jakarta.persistence.*;

@Entity
public class Capital {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer capitalId;

    private String cityName;
```



```
private long population;

@OneToOne(mappedBy = "capital")
@JsonBackReference
private Country country;

public Capital() {
}

public Integer getCapitalId() {
    return capitalId;
}

public void setCapitalId(Integer capitalId) {
    this.capitalId = capitalId;
}

public String getCityName() {
    return cityName;
}

public void setCityName(String cityName) {
    this.cityName = cityName;
}

public long getPopulation() {
    return population;
}
```

```
}

public void setPopulation(long population) {
    this.population = population;
}

public Country getCountry() {
    return country;
}

public void setCountry(Country country) {
    this.country = country;
}
}
```

Passport.java (Entity Class model)

```
package com.example.fourth.entity;

import java.time.LocalDate;

import com.fasterxml.jackson.annotation.JsonBackReference;
import jakarta.persistence.*;

@Entity
public class Passport {
```

```
@Id

@GeneratedValue(strategy = GenerationType.IDENTITY)

private Integer passportId;

private String passportNumber;

private String issuingCountry;

private LocalDate expiryDate;

@OneToOne(mappedBy = "passport")

@JsonBackReference

private Employee employee;

public Passport() {
}

public Integer getPassportId() {
    return passportId;
}

public void setPassportId(Integer passportId) {
    this.passportId = passportId;
}

public String getPassportNumber() {
    return passportNumber;
}

public void setPassportNumber(String passportNumber) {
```

```
        this.passportNumber = passportNumber;
    }

    public String getIssuingCountry() {
        return issuingCountry;
    }

    public void setIssuingCountry(String issuingCountry) {
        this.issuingCountry = issuingCountry;
    }

    public LocalDate getExpiryDate() {
        return expiryDate;
    }

    public void setExpiryDate(LocalDate expiryDate) {
        this.expiryDate = expiryDate;
    }

    public Employee getEmployee() {
        return employee;
    }

    public void setEmployee(Employee employee) {
        this.employee = employee;
    }
}
```

CountryRepository.java (Repository file)

```
package com.example.fourth.repository;

import org.springframework.data.jpa.repository.JpaRepository;

import com.example.fourth.entity.Country;

public interface CountryRepository extends JpaRepository<Country, Integer> {
}
```

CountryRestController.java (REST controller file)

```
package com.example.fourth.controller;

import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

import com.example.fourth.entity.Country;
import com.example.fourth.repository.CountryRepository;
```

```
@RestController
@RequestMapping("/api/countries")
public class CountryRestController {

    @Autowired
    private CountryRepository repository;

    // Add Country
    @PostMapping
    public Country addCountry(@RequestBody Country country) {
        return repository.save(country);
    }

    // Get All Countries
    @GetMapping
    public List<Country> getAllCountries() {
        return repository.findAll();
    }

    // Get Country By ID
    @GetMapping("/{id}")
    public Country getCountry(@PathVariable Integer id) {
        return repository.findById(id).orElse(null);
    }

    // Update Country
```

```
@PutMapping("/{id}")
public Country updateCountry(@PathVariable Integer id,
                             @RequestBody Country country) {

    country.setCountryId(id);
    return repository.save(country);
}

// Delete Country
@DeleteMapping("/{id}")
public String deleteCountry(@PathVariable Integer id) {
    repository.deleteById(id);
    return "Country Deleted Successfully";
}
}
```

Application.properties (database and Hibernate configuration)

```
spring.application.name=fourth
spring.datasource.url=jdbc:mysql://localhost:3306/fourth

spring.datasource.username=root

spring.datasource.password=
```

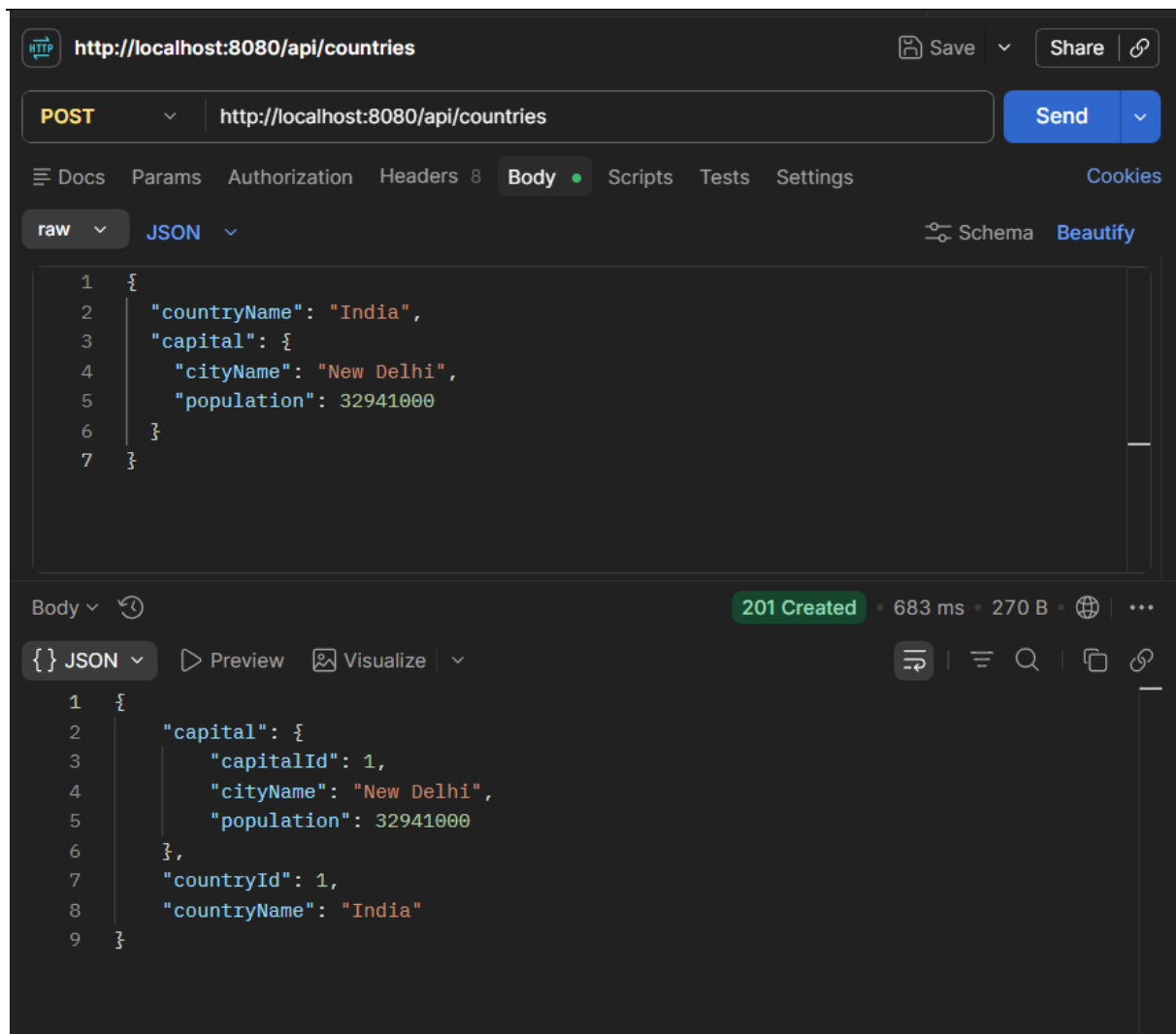
```
spring.jpa.hibernate.ddl-auto=update
```

```
spring.jpa.show-sql=true
```

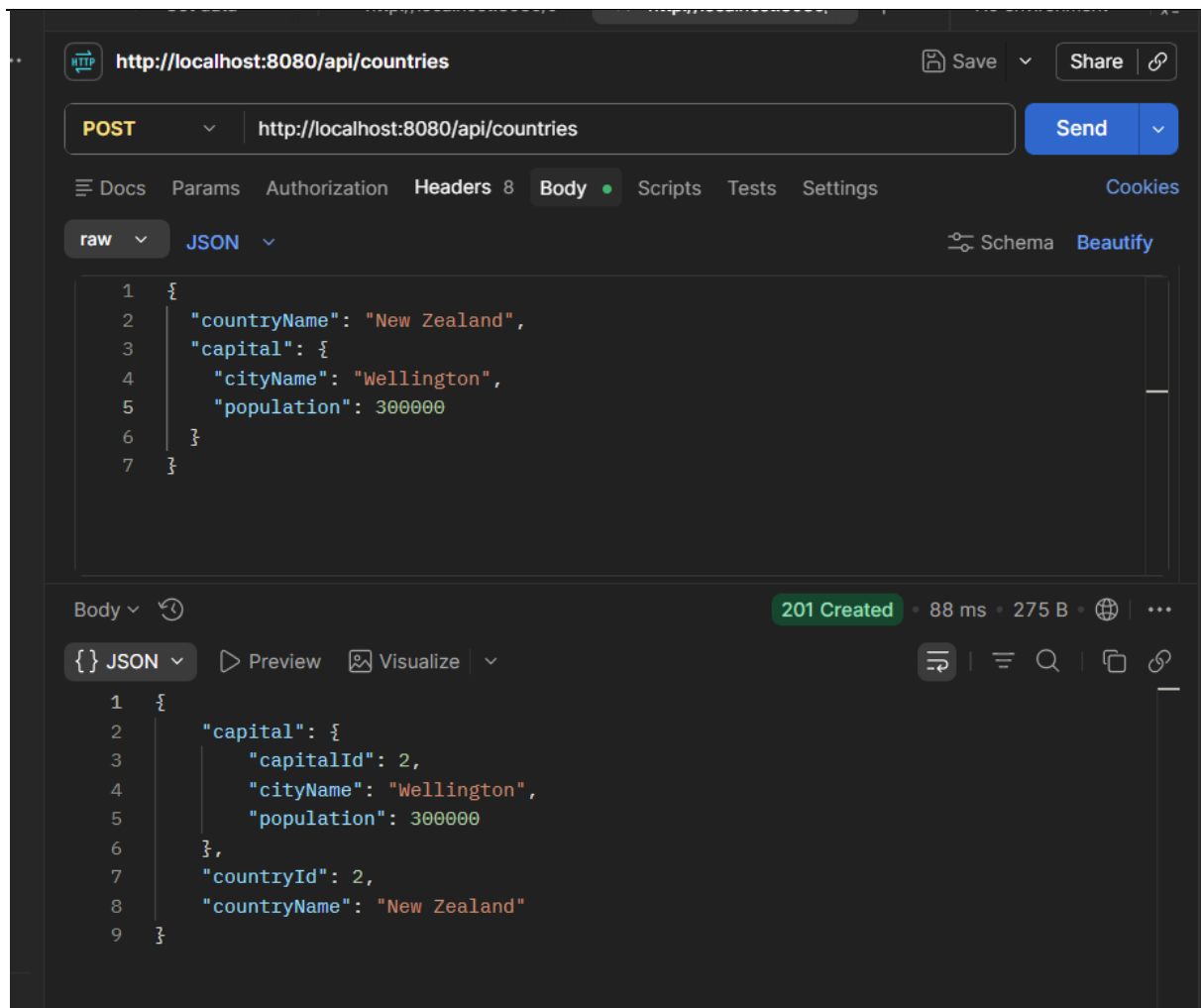
API'S TESTING USING POSTMAN

1. **POST API** – Add Country : The **POST API** inserts Country data into the database

POST <http://localhost:8080/api/countries>



After added one more record:-



Verify in Database:-

Both tables are created and data inserted..

Server: MySQL:3306 » Database: fourth » Table: capital

[Browse](#) [Structure](#) [SQL](#) [Search](#) [Insert](#) [Export](#) [Import](#) [Privileges](#)

✓ Showing rows 0 - 1 (2 total, Query took 0.0004 seconds.)

```
SELECT * FROM `capital`
```

☐ Profiling [[Edit inline](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

			capital_id	city_name	population
☐	Edit	Copy	Delete	1 New Delhi	32941000
☐	Edit	Copy	Delete	2 Wellington	300000

Server: MySQL:3306 » Database: fourth » Table: country

[Browse](#) [Structure](#) [SQL](#) [Search](#) [Insert](#) [Export](#) [Import](#) [P](#)

✓ Showing rows 0 - 1 (2 total, Query took 0.0004 seconds.)

```
SELECT * FROM `country`
```

☐ Profiling [[Edit inline](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: N

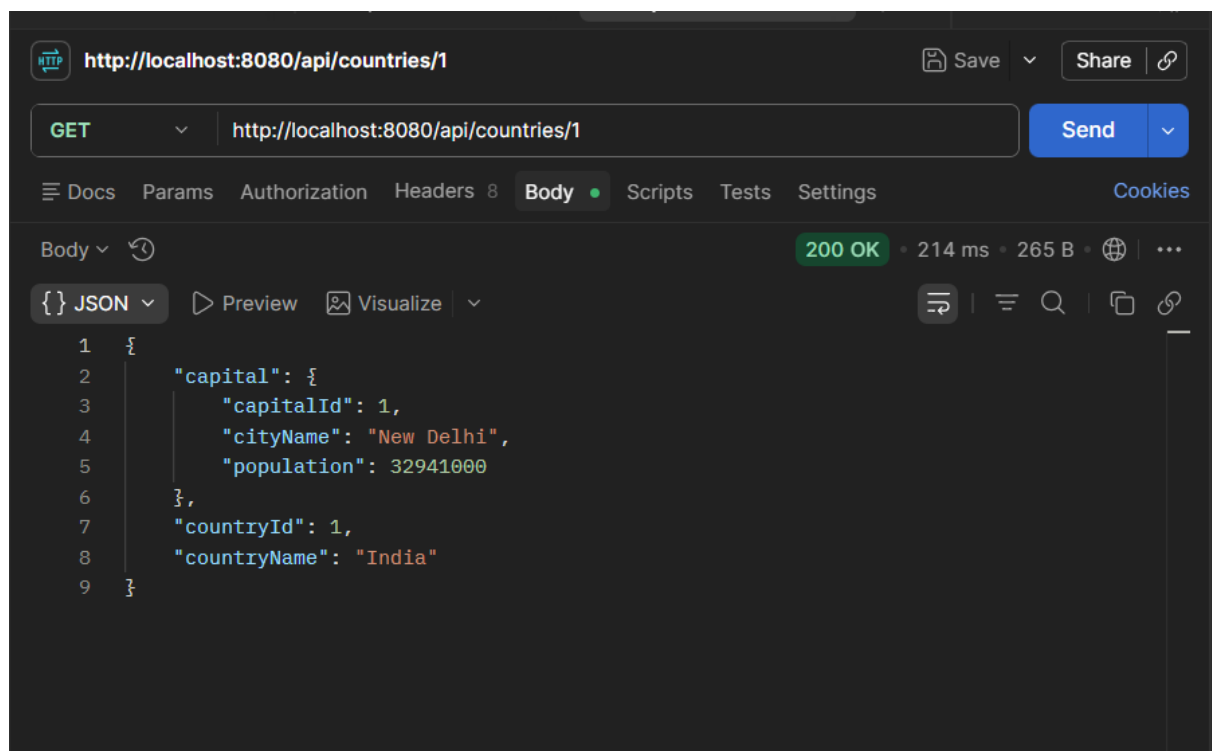
Extra options

				country_id	country_name	capital_id
☐	Edit	Copy	Delete	1	India	1
☐	Edit	Copy	Delete	2	New Zealand	2

2. **GET API – Retrieve by capital ID:** The **GET API** retrieves a specific Country record using the country ID.

The associated **Capital** details are displayed along with the **Country** object because of the One-to-One relationship.

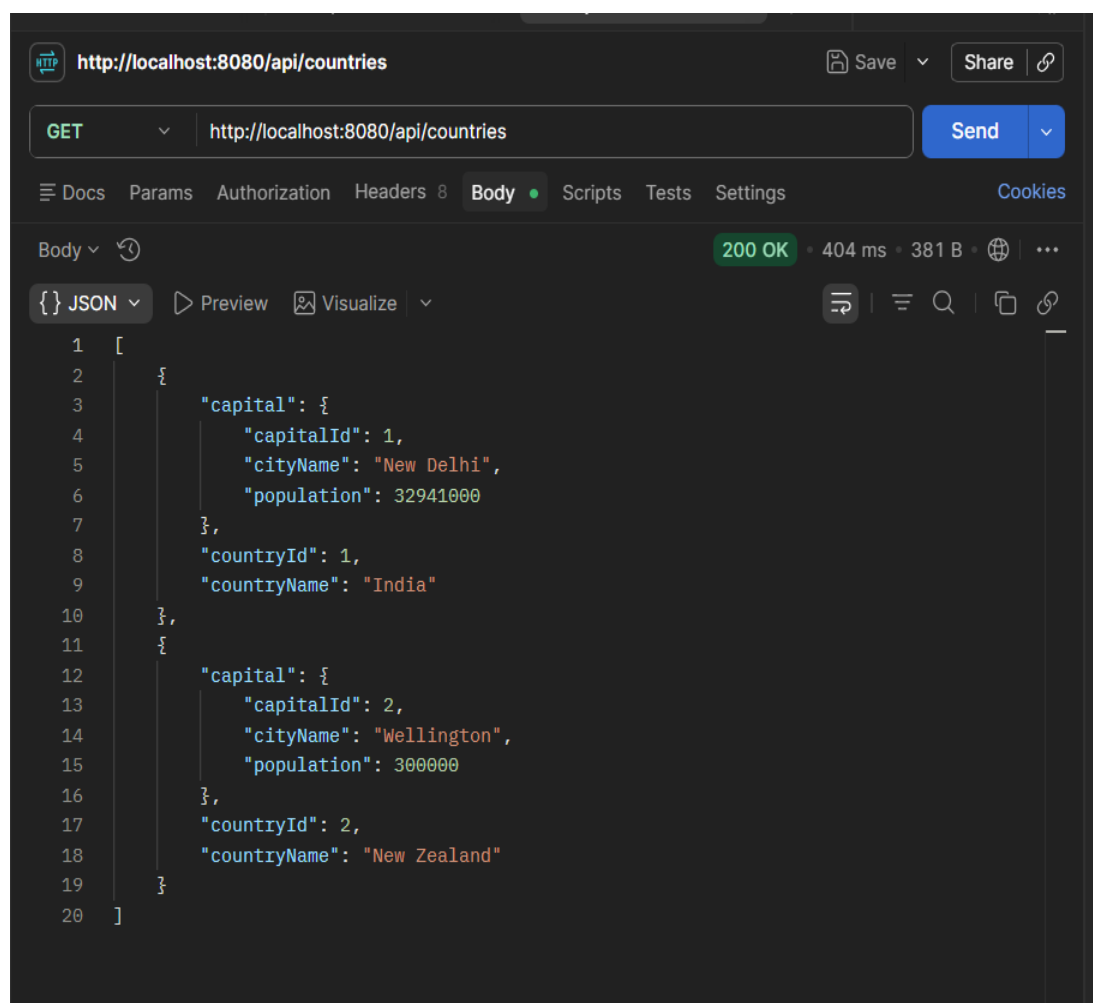
GET <http://localhost:8080/api/countries/1>



3. **GET API – Retrieve All countries:** The **GET API** retrieves all Countries from the database.

Each Employee record includes its associated Passport details because of the **One-to-One** relationship..

GET <http://localhost:8080/api/countries>

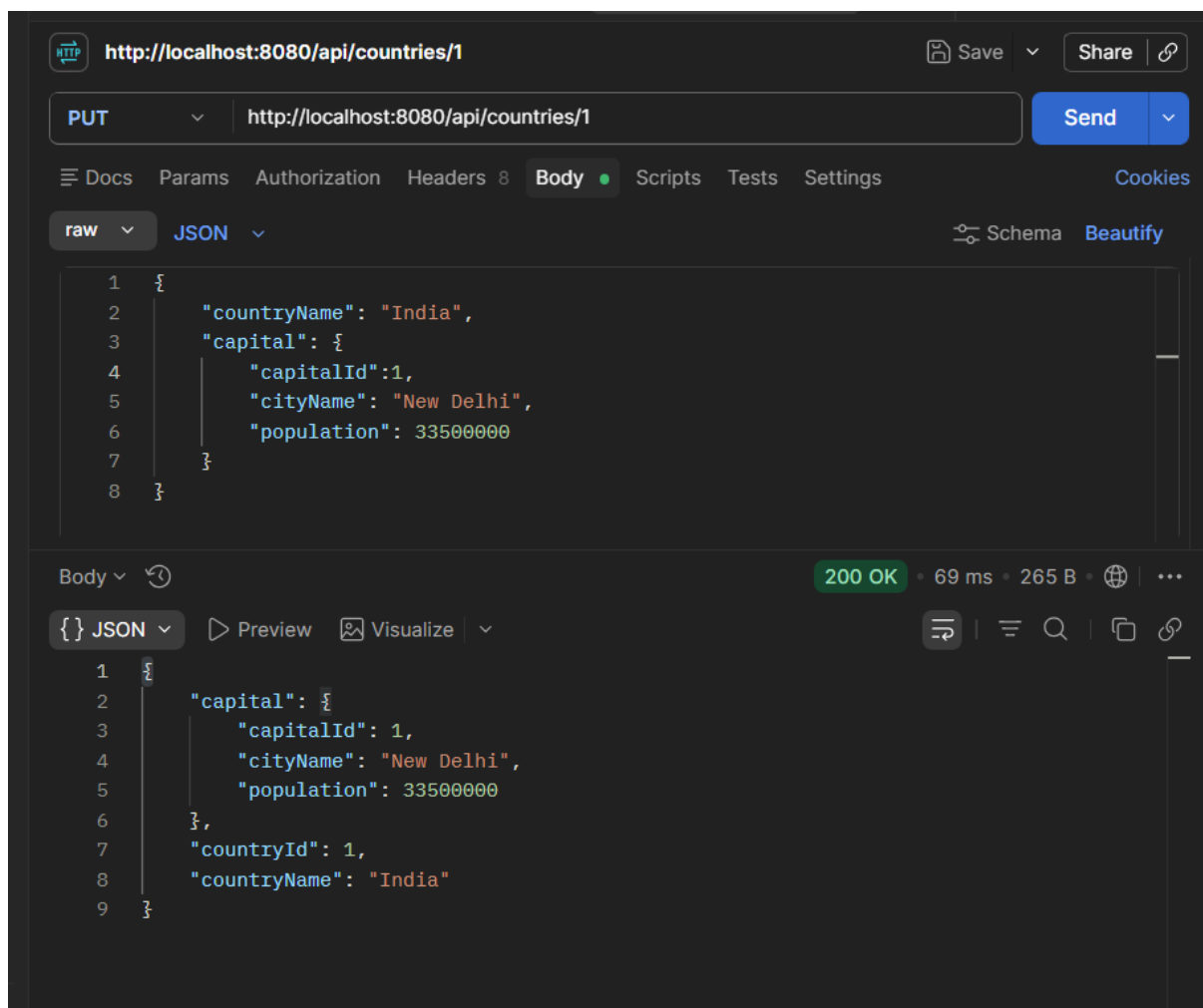


```
HTTP http://localhost:8080/api/countries
GET http://localhost:8080/api/countries
200 OK • 404 ms • 381 B • ...
JSON Preview Visualize
1 [
2   {
3     "capital": {
4       "capitalId": 1,
5       "cityName": "New Delhi",
6       "population": 32941000
7     },
8     "countryId": 1,
9     "countryName": "India"
10  },
11  {
12    "capital": {
13      "capitalId": 2,
14      "cityName": "Wellington",
15      "population": 300000
16    },
17    "countryId": 2,
18    "countryName": "New Zealand"
19  }
20 ]
```

4. PUT API – UPDATE The population:

The **PUT API** updates an existing Country record using the country ID. It allows modification of the **Country Name** along with the associated **Capital** details such as **City Name** and **Population**. Because of the One-to-One relationship, both Country and Capital information are updated together in a single request.

PUT <http://localhost:8080/api/countries/1>

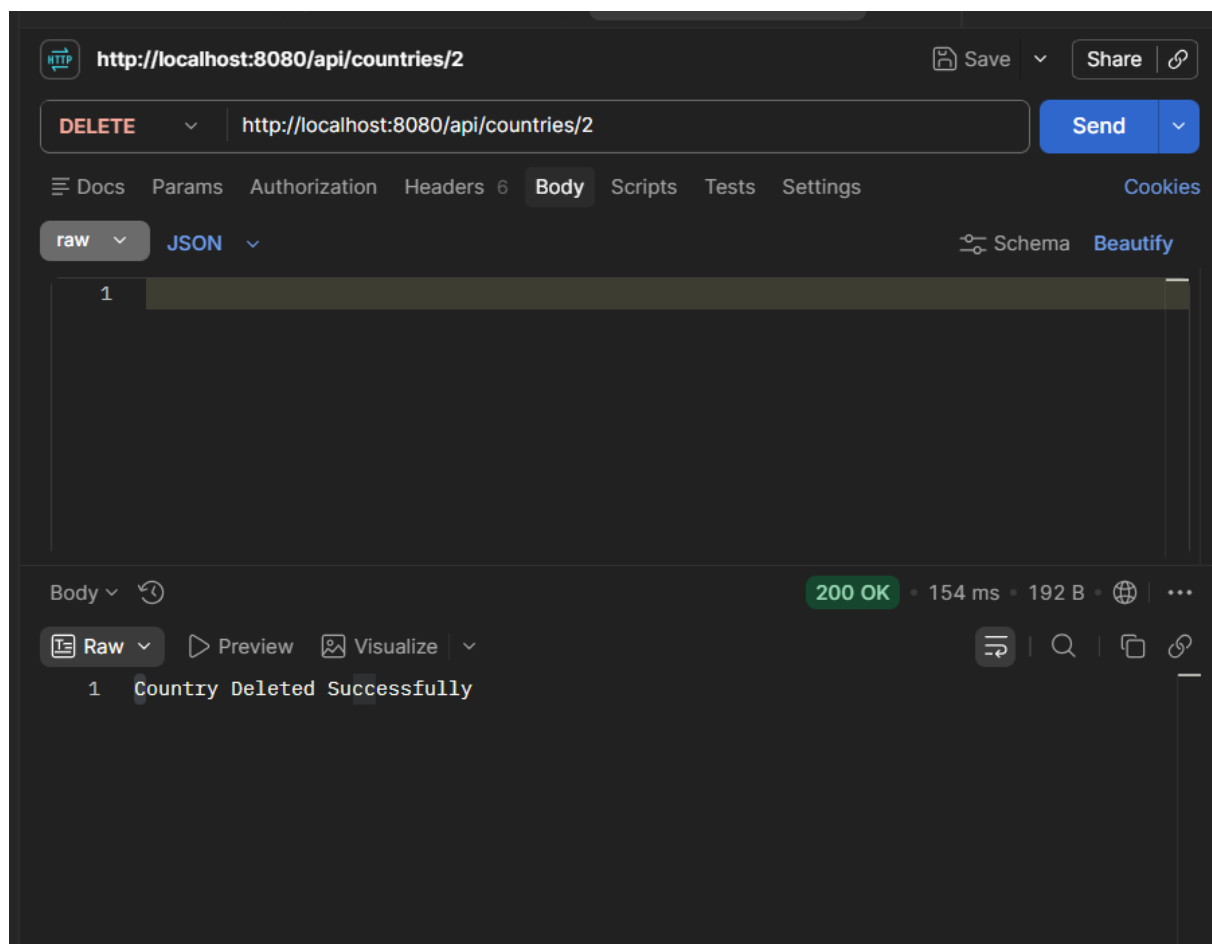


5. DELETE API – DELETE countryby id:- The **DELETE API** deletes a specific Country record using the country ID.

Due to **CascadeType.ALL**, the associated **Capital** record is also deleted automatically.

DELETE <http://localhost:8080/api/countries/2>

Here deleted New Zealand at id 2.



DATABASE OUTPUT

WE JUST NEED TO CREATE THE DATABASE. HIBERNATE ITSELF WILL CREATE THE TABLE FROM THE ENTITY CLASS.

Capital Table:-



Server: MySQL:3306 » Database: fourth » Table: capital

Showing rows 0 - 0 (1 total, Query took 0.0007 seconds.)

```
SELECT * FROM `capital`
```

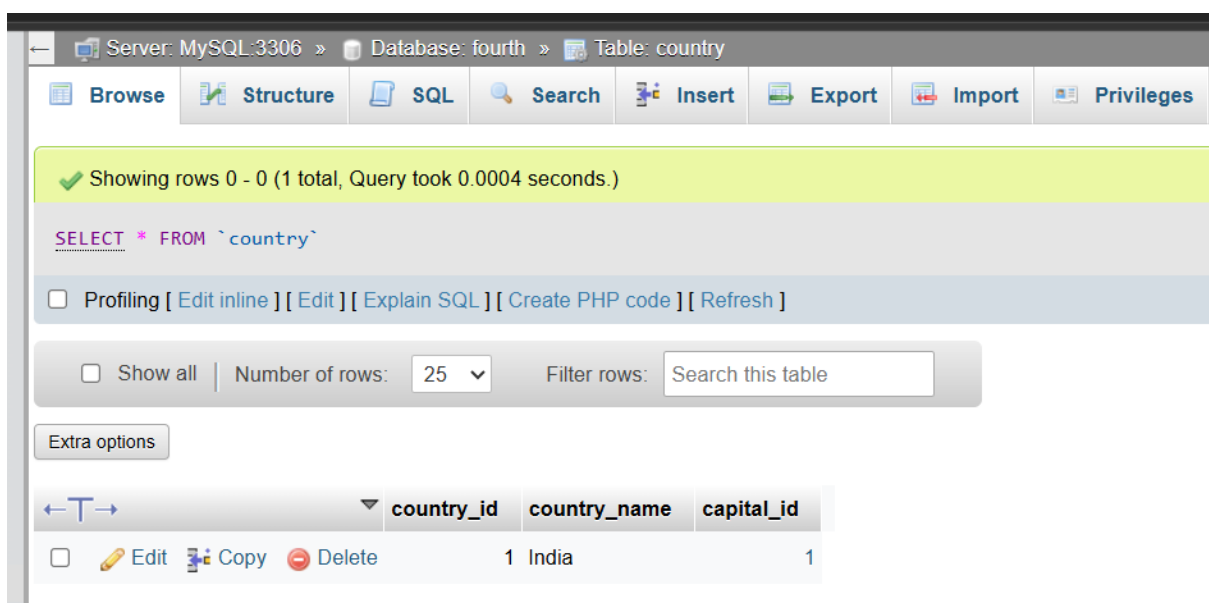
☐ Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

☐ Show all | Number of rows: 25 | Filter rows: Search this table

Extra options

	capital_id	city_name	population
☐ Edit <input type="image"/> Copy <input type="image"/> Delete	1	Delhi	33500000

Country table: -



Server: MySQL:3306 » Database: fourth » Table: country

Showing rows 0 - 0 (1 total, Query took 0.0004 seconds.)

```
SELECT * FROM `country`
```

☐ Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

☐ Show all | Number of rows: 25 | Filter rows: Search this table

Extra options

	country_id	country_name	capital_id
☐ Edit <input type="image"/> Copy <input type="image"/> Delete	1	India	1

Practical No. 5: One-to-Many / Many-to-One Mapping **- Department and Employee**

Objective:-

To implement a bidirectional One-to-Many (Department → Employees) and Many-to-One (Employee →

Department) relationship using mappedBy on the parent side and @JoinColumn on the child side, and to persist a

parent with a list of children in one request.

Class Name(s) with Attributes:

Department (@Entity — parent, One side)

- deptId : int (@Id @GeneratedValue)
- deptName : String
- employees : List<Employee> (@OneToMany(mappedBy="department", cascade=ALL))

Employee (@Entity — child, Many side)

- empId : int (@Id @GeneratedValue)
- empName : String
- salary : double
- department : Department (@ManyToOne @JoinColumn(name="dept_id"))

PROJECT STRUCTURE

```
src/main/java
|
|— controller
|   └─ DepartmentRestController.java
|
|— entity
|   └─ Department.java
|   └─ Employee.java
|
|— repository
|   └─ DepartmentRepository.java
|   └─ EmployeeRepository.java
|
└─ FifthApplication.java
```

Dependencies

- Spring Web
- Spring Data JPA
- MySQL Driver

Database

Database Name: fifth

Table: 1. Department

2.Employee

Department.java (ENTITY model class)

```
package com.example.fifth.entity;

import java.util.ArrayList;
import java.util.List;
import com.fasterxml.jackson.annotation.JsonManagedReference;

import jakarta.persistence.*;

@Entity
public class Department {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer deptId;

    private String deptName;

    @OneToMany(mappedBy = "department", cascade = CascadeType.ALL)
    @JsonManagedReference
    private List<Employee> employees = new ArrayList<>();

    public Department() {
    }

    public Integer getDeptId() {
        return deptId;
    }
}
```

```
}

public void setDeptId(Integer deptId) {
    this.deptId = deptId;
}

public String getDeptName() {
    return deptName;
}

public void setDeptName(String deptName) {
    this.deptName = deptName;
}

public List<Employee> getEmployees() {
    return employees;
}

public void setEmployees(List<Employee> employees) {
    this.employees = employees;
}
}
```

Employee.java (Entity model class)

```
package com.example.fifth.entity;
```

```
import com.fasterxml.jackson.annotation.JsonBackReference;

import jakarta.persistence.*;

@Entity
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer empId;

    private String empName;

    private double salary;

    @ManyToOne
    @JoinColumn(name = "dept_id")
    @JsonBackReference
    private Department department;

    public Employee() {
    }

    public Integer getEmpId() {
        return empId;
    }
}
```

```
public void setEmpId(Integer empId) {  
    this.empId = empId;  
}  
  
public String getEmpName() {  
    return empName;  
}  
  
public void setEmpName(String empName) {  
    this.empName = empName;  
}  
  
public double getSalary() {  
    return salary;  
}  
  
public void setSalary(double salary) {  
    this.salary = salary;  
}  
  
public Department getDepartment() {  
    return department;  
}  
  
public void setDepartment(Department department) {  
    this.department = department;  
}  
}
```

DepartmentRepository.java (Repository file)

```
package com.example.fifth.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import com.example.fifth.entity.Department;

public interface DepartmentRepository extends JpaRepository<Department, Integer> {
}
```

EmployeeRepository.java (Repository file)

```
package com.example.fifth.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import com.example.fifth.entity.Employee;

public interface EmployeeRepository extends JpaRepository<Employee, Integer> {
}
```

DepartmentRestController.java (REST controller file)

```
package com.example.fifth.controller;

import java.util.List;
```

```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;
import com.example.fifth.entity.*;
import com.example.fifth.repository.*;

@RestController
@RequestMapping("/api/departments")
public class DepartmentRestController {

    @Autowired
    private DepartmentRepository repository;

    // Add Department with Employees
    @PostMapping
    public Department addDepartment(@RequestBody Department department) {

        for (Employee employee : department.getEmployees()) {
            employee.setDepartment(department);
        }

        return repository.save(department);
    }

    // Get Department by ID
    @GetMapping("/{id}")
```



```
public Department getDepartment(@PathVariable Integer id) {

    return repository.findById(id).orElse(null);
}

// Get Employees of a Department
@GetMapping("/{id}/employees")
public List<Employee> getEmployees(@PathVariable Integer id) {

    Department department = repository.findById(id).orElse(null);

    if (department == null) {
        return null;
    }

    return department.getEmployees();
}

// Update Employee of Department
@PutMapping("/{deptId}/employees/{empId}")
public Department updateEmployee(@PathVariable Integer deptId,
                                  @PathVariable Integer empId,
                                  @RequestBody Employee updatedEmployee) {

    Department department = repository.findById(deptId).orElse(null);

    if (department == null) {
```

```
        return null;
    }

    for (Employee employee : department.getEmployees()) {

        if (employee.getEmpId().equals(empId)) {
            employee.setEmpName(updatedEmployee.getEmpName());
            employee.setSalary(updatedEmployee.getSalary());
            break;
        }
    }

    return repository.save(department);
}

// Delete Employee from Department
@DeleteMapping("/{deptId}/employees/{empId}")
public String deleteEmployee(@PathVariable Integer deptId,
                             @PathVariable Integer empId) {

    Department department = repository.findById(deptId).orElse(null);

    if (department == null) {
        return "Department Not Found";
    }
}
```

```
        department.getEmployees().removeIf(employee ->
employee.getEmpId().equals(empId));

        repository.save(department);

        return "Employee Deleted Successfully";
    }

    // Delete Department
    @DeleteMapping("/{id}")
    public String deleteDepartment(@PathVariable Integer id) {

        repository.deleteById(id);

        return "Department Deleted Successfully";
    }
}
```

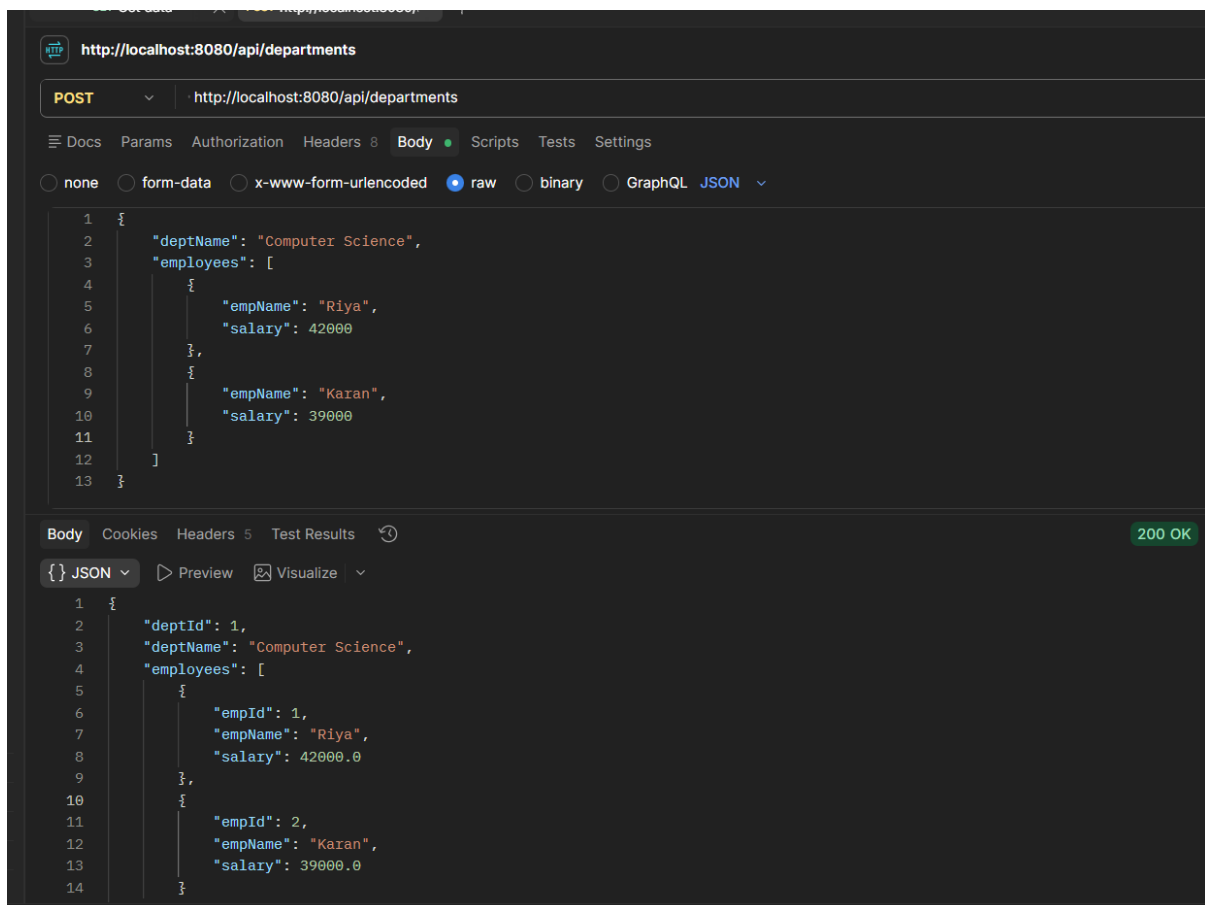
Application.properties (database and Hibernate configuration)

```
spring.application.name=fifth
spring.datasource.url=jdbc:mysql://localhost:3306/fifth
spring.datasource.username=root
spring.datasource.password=
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

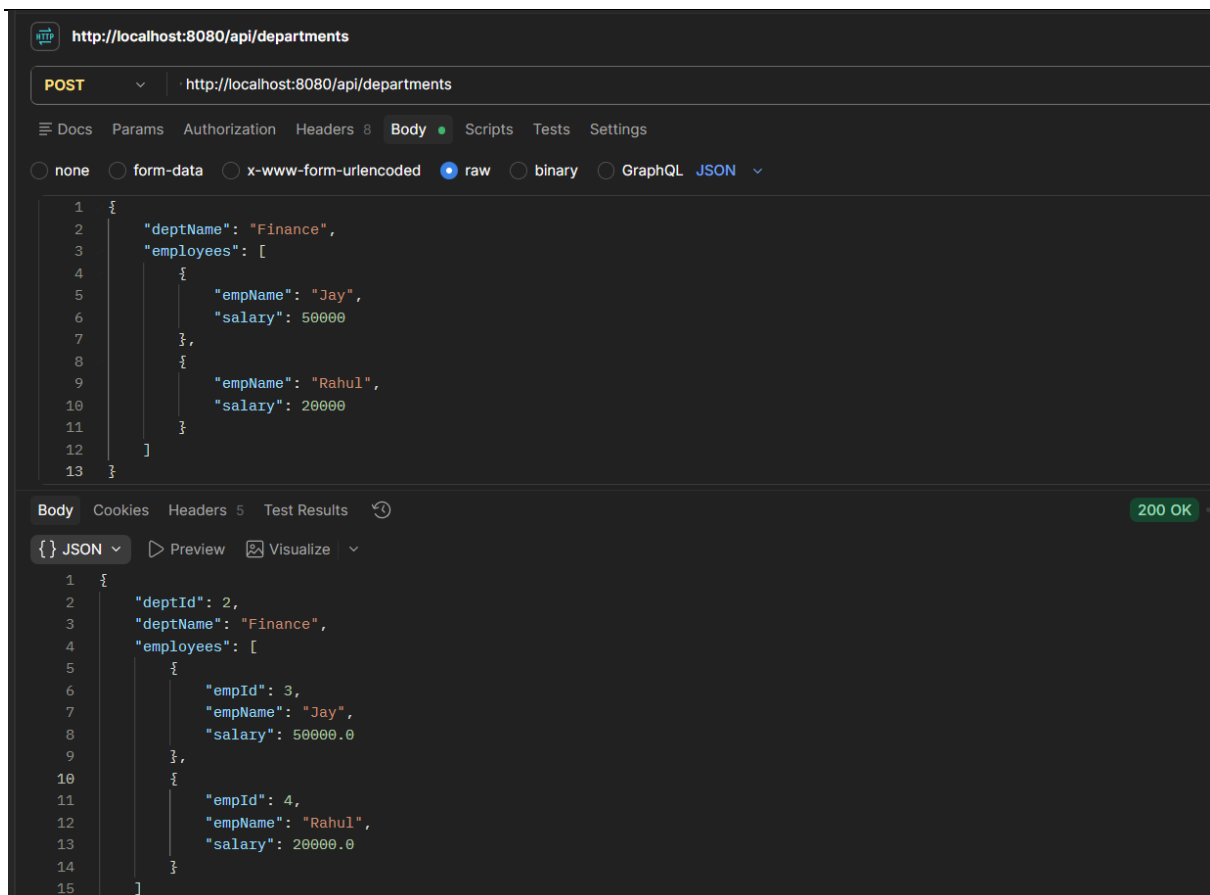
API'S TESTING USING POSTMAN

1. **POST API – Add Department:** - The POST API inserts a new Department along with multiple Employee records into the database. Because of the One-to-Many relationship, all Employees are saved automatically with the Department.

POST <http://localhost:8080/api/departments>



Added one more record:-



Verify in Database:-

Both tables are created and data inserted..

Server: MySQL:3306 » Database: fifth » Table: department

[Browse](#) [Structure](#) [SQL](#) [Search](#) [Insert](#) [Export](#) [Import](#) [Privileges](#)

Showing rows 0 - 1 (2 total, Query took 0.0009 seconds.)

```
SELECT * FROM `department`
```

☐ Profiling [\[Edit inline \]](#) [\[Edit \]](#) [\[Explain SQL \]](#) [\[Create PHP code \]](#) [\[Refresh \]](#)

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

				dept_id	dept_name
☐				1	Computer Science
☐				2	Finance

Server: MySQL:3306 » Database: fifth » Table: employee

[Browse](#) [Structure](#) [SQL](#) [Search](#) [Insert](#) [Export](#) [Import](#) [Privileges](#) [Operations](#) [Triggers](#)

Showing rows 0 - 3 (4 total, Query took 0.0004 seconds.)

```
SELECT * FROM `employee`
```

☐ Profiling [\[Edit inline \]](#) [\[Edit \]](#) [\[Explain SQL \]](#) [\[Create PHP code \]](#) [\[Refresh \]](#)

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

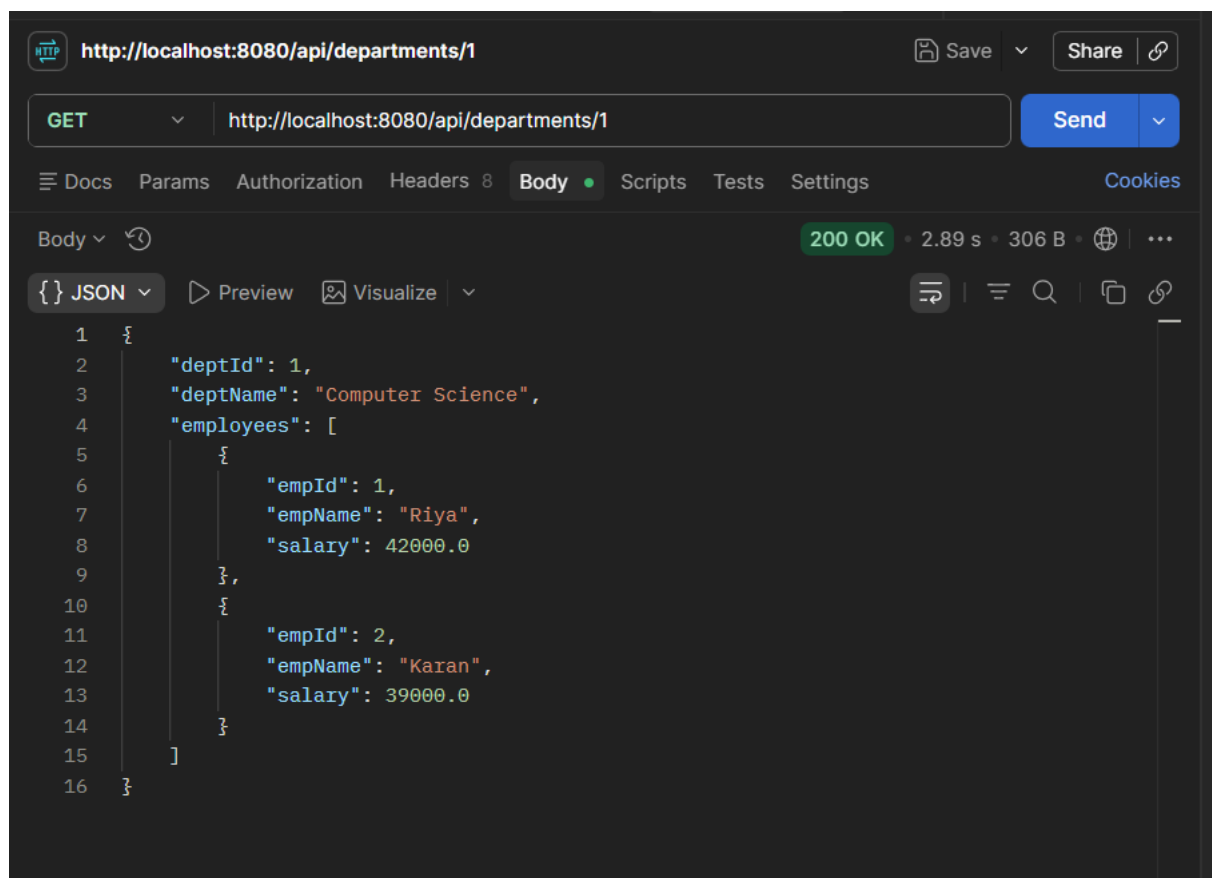
Extra options

				emp_id	emp_name	salary	dept_id
☐				1	Riya	42000	1
☐				2	Karan	39000	1
☐				3	Jay	50000	2
☐				4	Rahul	20000	2

-
2. **GET API – Retrieve Department by ID:-** The GET API retrieves a specific Department using the Department ID.

The associated Employee records are displayed because of the One-to-Many relationship.

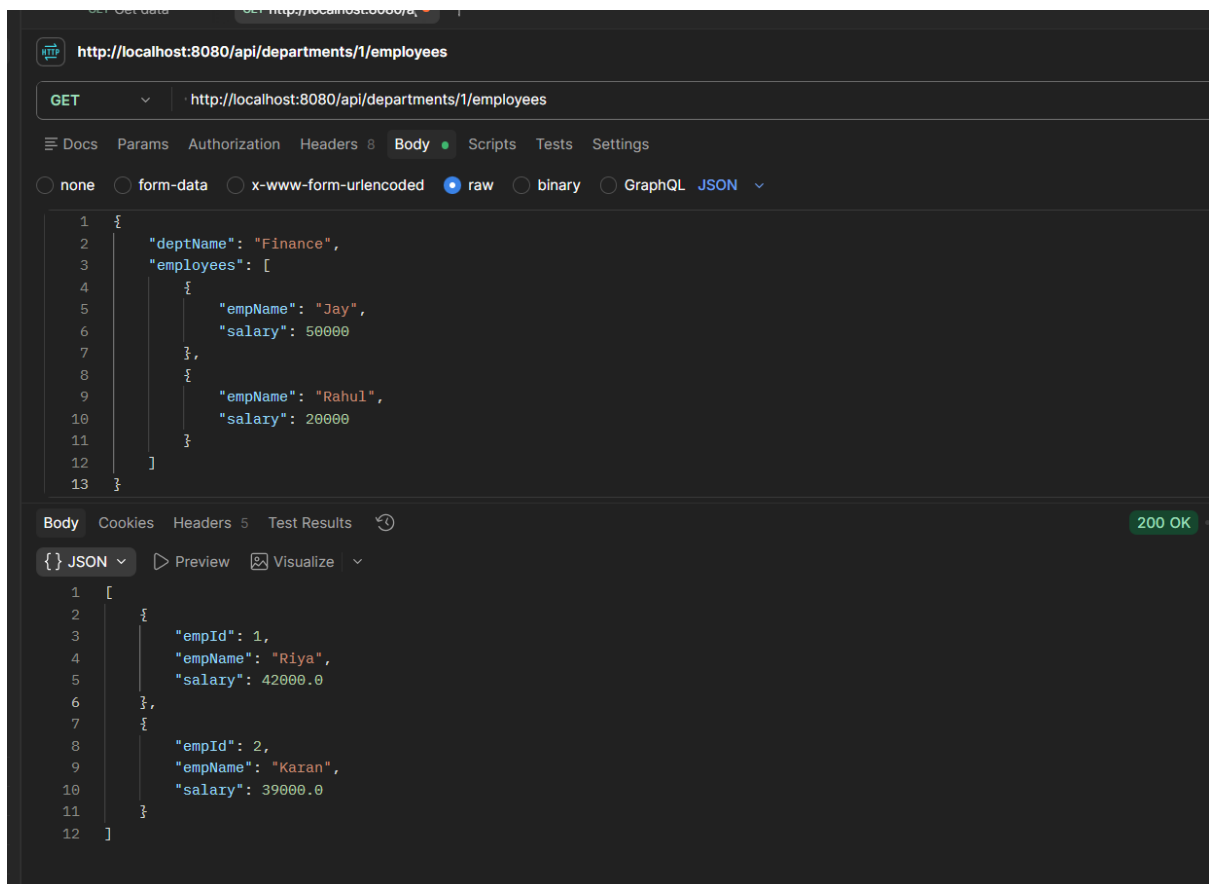
GET <http://localhost:8080/api/departments/1>



3 . GET API - Retrieve Employees of a Department:-

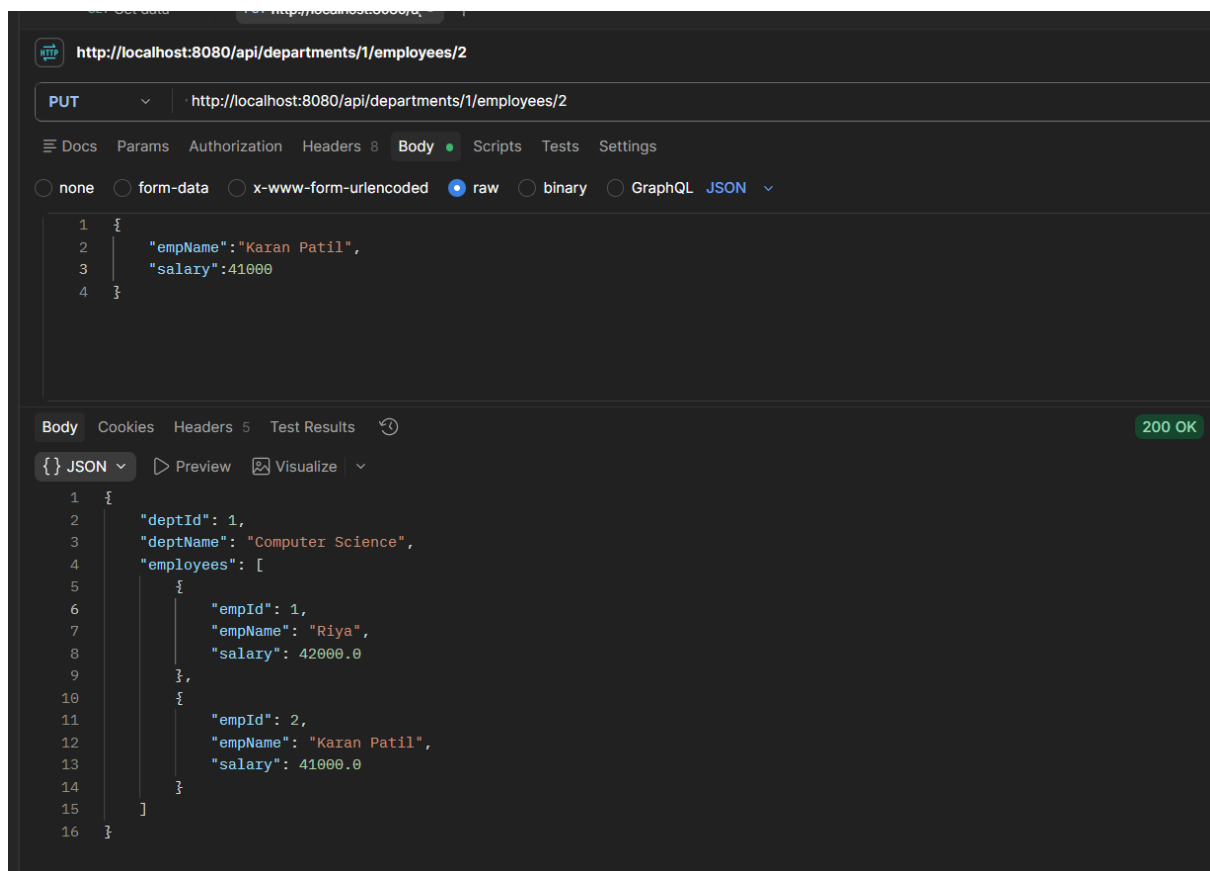
The GET API retrieves only the Employees belonging to a particular Department.

GET <http://localhost:8080/api/departments/1/employees>



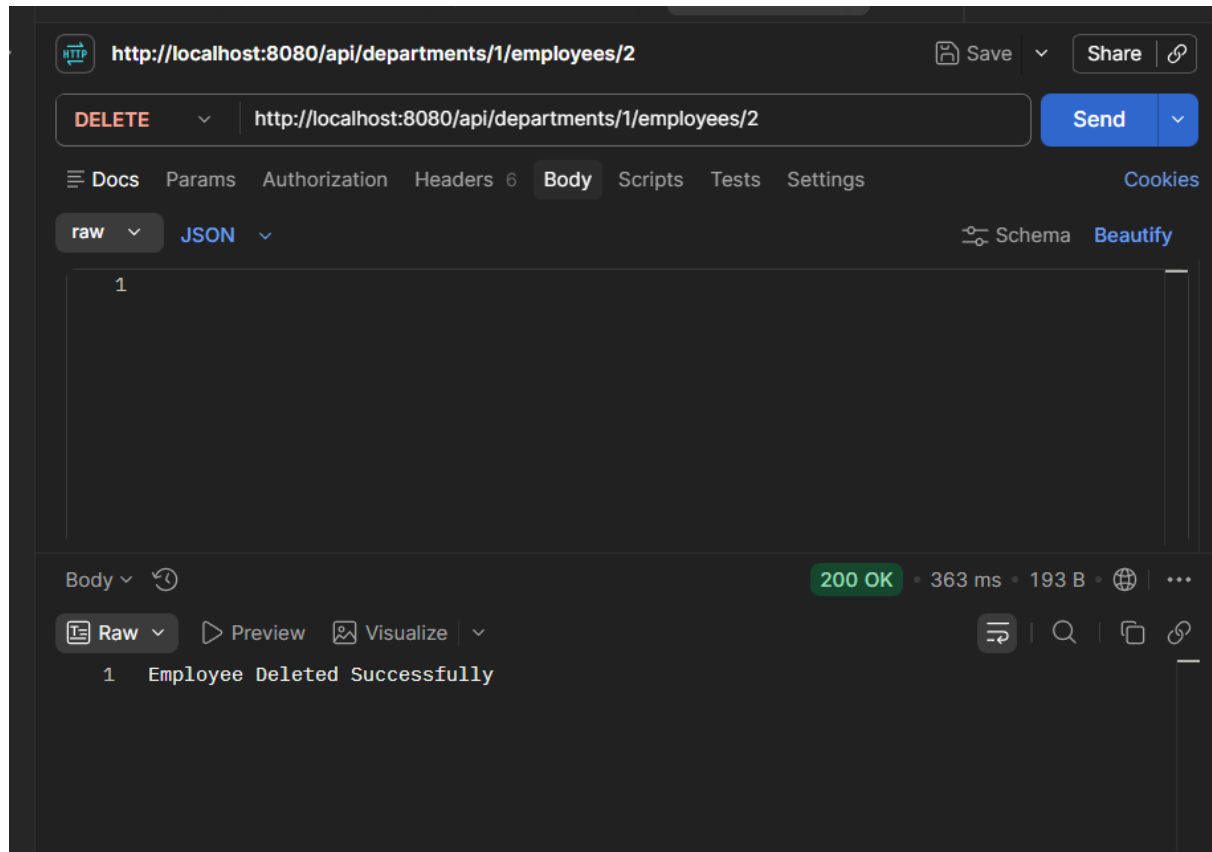
4. PUT API – Update Department :- The PUT API updates the Employee name and salary using the employee ID.

PUT <http://localhost:8080/api/departments/1/employees/2>



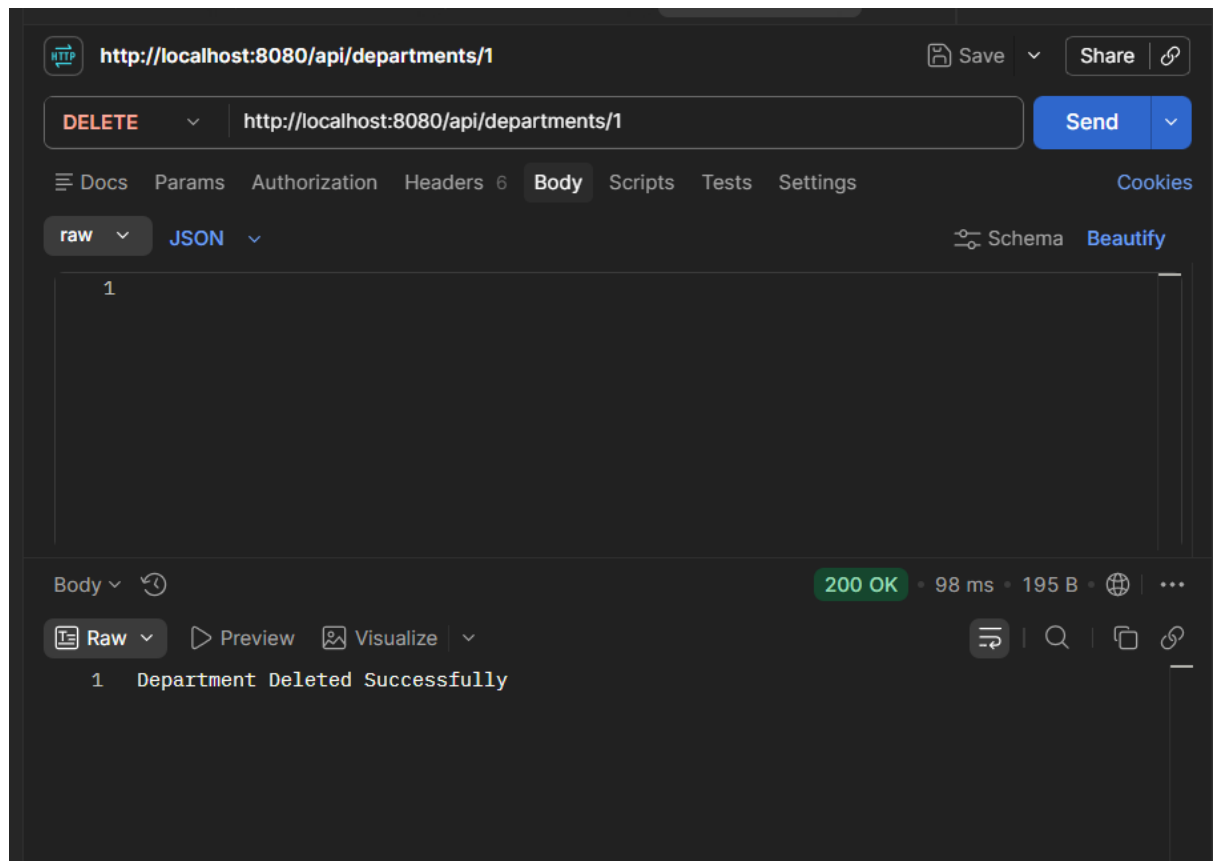
5. DELETE API –DELETE Employee:- The DELETE API deletes a specific Employee from the Department.

DELETE <http://localhost:8080/api/departments/1/employees/2>



6. DELETE API –DELETE department:- The DELETE API deletes the Department and all associated Employees because of **CascadeType.ALL**.

DELETE <http://localhost:8080/api/departments/1>

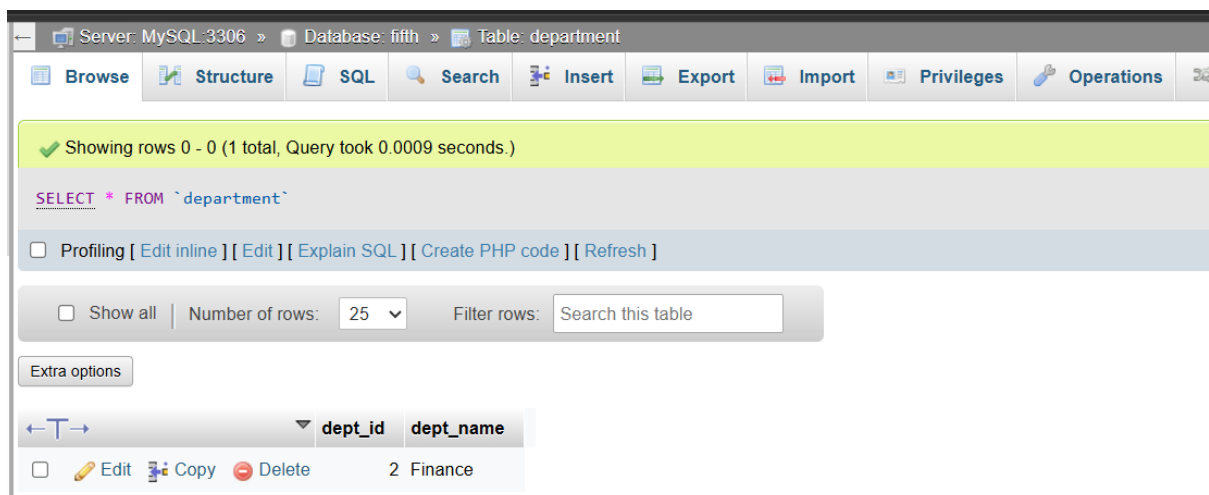


DATABASE OUTPUT

WE JUST NEED TO CREATE THE DATABASE. HIBERNATE ITSELF WILL CREATE THE TABLE FROM THE ENTITY CLASS.

After deleted the department show both tables:

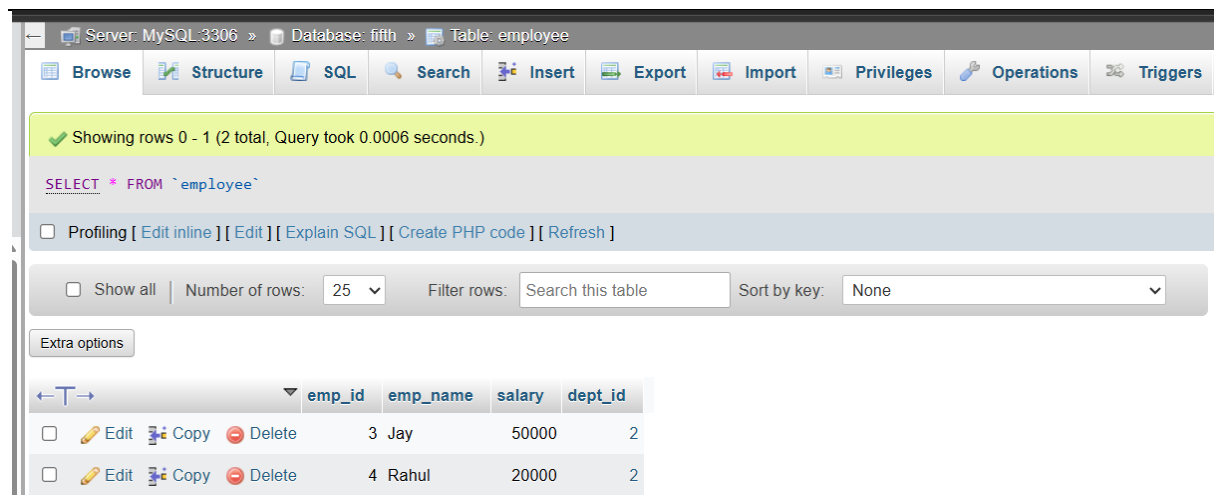
Department Table:-



The screenshot shows the MySQL Workbench interface for a table named 'department' in a database named 'fifth'. The table has two columns: 'dept_id' and 'dept_name'. There is one row of data with 'dept_id' as 2 and 'dept_name' as 'Finance'. The interface includes a toolbar with options like Browse, Structure, SQL, Search, Insert, Export, Import, Privileges, and Operations. A status bar at the bottom indicates 'Showing rows 0 - 0 (1 total, Query took 0.0009 seconds.)'.

dept_id	dept_name
2	Finance

Employee table: -



The screenshot shows the MySQL Workbench interface. At the top, the breadcrumb navigation indicates the path: Server: MySQL:3306 » Database: fifth » Table: employee. Below this is a toolbar with icons for Browse, Structure, SQL, Search, Insert, Export, Import, Privileges, Operations, and Triggers. A green status bar reports: 'Showing rows 0 - 1 (2 total, Query took 0.0006 seconds.)'. The SQL editor contains the query: `SELECT * FROM `employee``. Below the editor are links for Profiling, Edit inline, Edit, Explain SQL, Create PHP code, and Refresh. A control bar includes a 'Show all' checkbox, a 'Number of rows' dropdown set to 25, a 'Filter rows' search box, and a 'Sort by key' dropdown set to None. An 'Extra options' button is also present. The table data is displayed in a grid with columns: emp_id, emp_name, salary, and dept_id. Two rows are visible: one for Jay (emp_id 3, salary 50000) and one for Rahul (emp_id 4, salary 20000), both in department 2. Each row has checkboxes and icons for Edit, Copy, and Delete.

	emp_id	emp_name	salary	dept_id
☐ Edit Copy Delete	3	Jay	50000	2
☐ Edit Copy Delete	4	Rahul	20000	2

Practical No.6 Many-to-Many Mapping - Student and Course

Objective:-

To implement a Many-to-Many relationship between Student and Course using @ManyToMany and a join table

via @JoinTable, and to avoid infinite JSON recursion using @JsonManagedReference / @JsonBackReference.

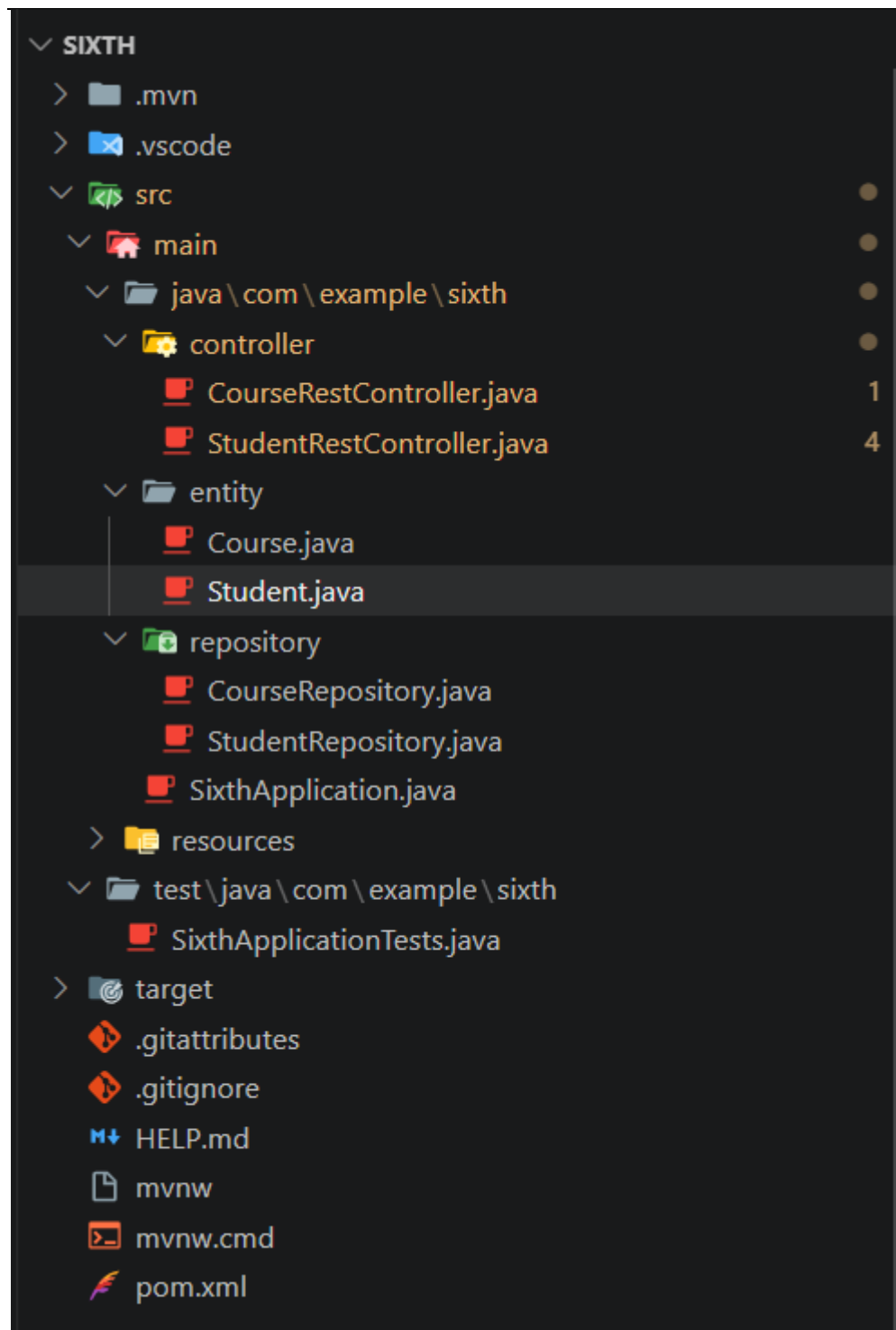
Class Name(s) with Attributes:

Student (@Entity)

- studentId : int (@Id @GeneratedValue)
- studentName : String
- courses : Set<Course> (@ManyToMany, @JoinTable(name="student_course"))

Course (@Entity)

- courseId : int (@Id @GeneratedValue)
- courseName : String
- students : Set<Student> (@ManyToMany(mappedBy="courses"))



Dependencies

- Spring Web
- Spring Data JPA
- MySQL Driver Database

Database Name:

employee_db

Table:

- 1.Student(created automatically by Hibernate)
- 2.Course (created automatically by Hibernate)
- 3.course_details(created automatically by Hibernate)

Student.java(ENTITY model class)

```
package com.example.sixth.entity;  
  
import java.util.HashSet;  
import java.util.Set;  
import com.fasterxml.jackson.annotation.JsonManagedReference;  
import jakarta.persistence.*;
```

```
@Entity

public class Student {

    @Id

    @GeneratedValue(strategy = GenerationType.IDENTITY)

    private Integer studentId;

    private String studentName;

    @ManyToMany

    @JoinTable(name = "student_course", joinColumns = @JoinColumn(name = "student_id"), inverseJoinColumns = @JoinColumn(name = "course_id"))

    @JsonManagedReference

    private Set<Course> courses = new HashSet<>();

    public Student() {

    }

    public Integer getStudentId() {

        return studentId;

    }

    public void setStudentId(Integer studentId) {

        this.studentId = studentId;

    }

    public String getStudentName() {

        return studentName;

    }

}
```

```
public void setStudentName(String studentName) {  
    this.studentName = studentName;  
}  
  
public Set<Course> getCourses() {  
    return courses;  
}  
  
public void setCourses(Set<Course> courses) {  
    this.courses = courses;  
}  
}
```

Course.java(Entity MODEL CLASS)

```
package com.example.sixth.entity;  
  
import java.util.HashSet;  
import java.util.Set;  
  
import com.fasterxml.jackson.annotation.JsonBackReference;  
  
import jakarta.persistence.Entity;  
import jakarta.persistence.GeneratedValue;  
import jakarta.persistence.GenerationType;
```

```
import jakarta.persistence.Id;

import jakarta.persistence.ManyToMany;

@Entity

public class Course {

    @Id

    @GeneratedValue(strategy = GenerationType.IDENTITY)

    private Integer courseId;

    private String courseName;

    @ManyToMany(mappedBy = "courses")

    @JsonBackReference

    private Set<Student> students = new HashSet<>();

    public Course() {

    }

    public Integer getCourseId() {

        return courseId;

    }

    public void setCourseId(Integer courseId) {

        this.courseId = courseId;

    }

}
```

```
public String getCourseName() {  
    return courseName;  
}  
  
public void setCourseName(String courseName) {  
    this.courseName = courseName;  
}  
  
public Set<Student> getStudents() {  
    return students;  
}  
  
public void setStudents(Set<Student> students) {  
    this.students = students;  
}  
}
```

StudentRestController.java(Controller file)

```
package com.example.sixth.controller;  
  
import java.util.Set;  
import org.springframework.beans.factory.annotation.Autowired;  
import org.springframework.web.bind.annotation.*;  
import com.example.sixth.entity.Course;
```

```
import com.example.sixth.entity.Student;

import com.example.sixth.repository.CourseRepository;
import com.example.sixth.repository.StudentRepository;

@RestController
@RequestMapping("/api/students")
public class StudentRestController {

    @Autowired
    private StudentRepository studentRepository;

    @Autowired
    private CourseRepository courseRepository;

    @PostMapping
    public Student addStudent(@RequestBody Student student) {
        return studentRepository.save(student);
    }

    @PostMapping("/{id}/enroll/{courseId}")
    public Student enrollStudent(@PathVariable Integer id, @PathVariable Integer courseId)
    {
        Student student = studentRepository.findById(id).orElse(null);
        Course course = courseRepository.findById(courseId).orElse(null);
        if (student == null || course == null) {
            return null;
        }
    }
}
```

```
    }

    student.getCourses().add(course);

    return studentRepository.save(student);
}

// Get Student
//
@GetMapping("/{id}")
public Student getStudent(@PathVariable Integer id) {
    return studentRepository.findById(id).orElse(null);
}

// Remove Course

@DeleteMapping("/{id}/unenroll/{courseId}")
public String unenrollStudent(@PathVariable Integer id, @PathVariable Integer
courseId) {

    Student student = studentRepository.findById(id).orElse(null);

    Course course = courseRepository.findById(courseId).orElse(null);

    if (student == null || course == null) {
        return "Not Found";

    }

    student.getCourses().remove(course);

    studentRepository.save(student);

    return "Course Removed Successfully";
}
}
```

CourseRestController.java(Controller file)

```
package com.example.sixth.controller;

import java.util.Set;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

import com.example.sixth.entity.Course;
import com.example.sixth.entity.Student;
import com.example.sixth.repository.CourseRepository;

@RestController
@RequestMapping("/api/courses")
public class CourseRestController {

    @Autowired
    private CourseRepository repository;

    // Add Course
    @PostMapping
    public Course addCourse(@RequestBody Course course) {
        return repository.save(course);
    }
}
```

```
// Get Students of Course

@GetMapping("/{id}/students")
public Set<Student> getStudents(@PathVariable Integer id) {

    Course course = repository.findById(id).orElse(null);

    if (course == null)
        return null;

    return course.getStudents();
}

// Update Course

@PutMapping("/{id}")
public Course updateCourse(@PathVariable Integer id,
                           @RequestBody Course course) {

    course.setCourseId(id);
    return repository.save(course);
}

// Delete Course

@DeleteMapping("/{id}")
public String deleteCourse(@PathVariable Integer id) {

    repository.deleteById(id);
    return "Course Deleted Successfully";
}
}
```


StudentRepository.java(Repository file)

```
package com.example.sixth.repository;

import org.springframework.data.jpa.repository.JpaRepository;

import com.example.sixth.entity.Student;

public interface StudentRepository extends JpaRepository<Student, Integer> {

}
```

CourseRepository.java(Repository file)

```
package com.example.sixth.repository;

import org.springframework.data.jpa.repository.JpaRepository;

import com.example.sixth.entity.Course;

public interface CourseRepository extends JpaRepository<Course, Integer> {

}
```

Application.properties(all about url of databases and hibernate)

```
spring.application.name=sixth

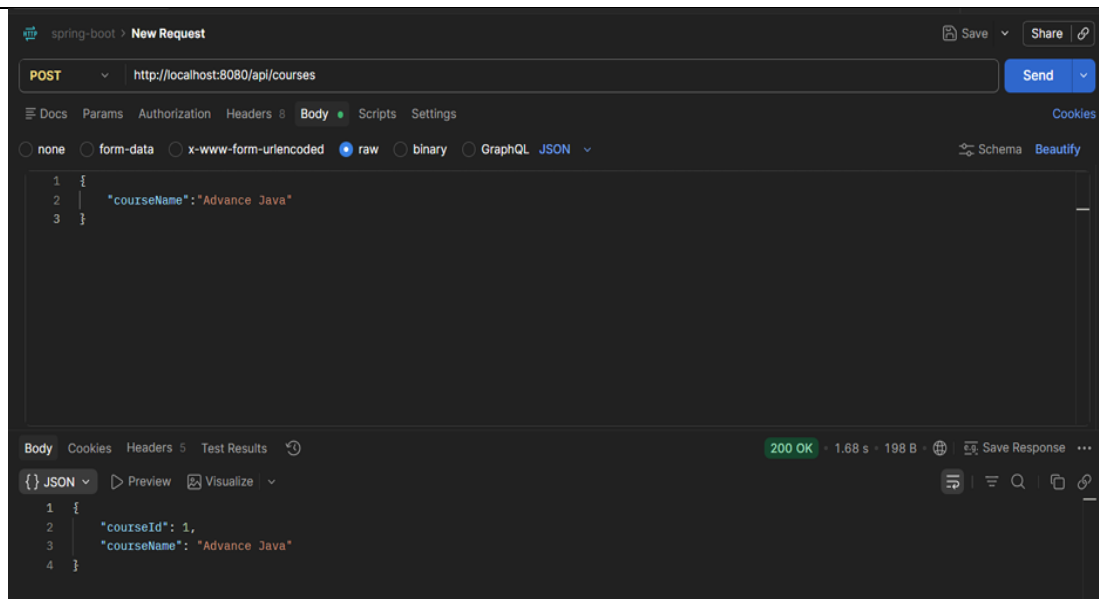
spring.datasource.url=jdbc:mysql://localhost:3306/sixth
spring.datasource.username=root
spring.datasource.password=

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
```

API'S TESTING USING POSTMAN

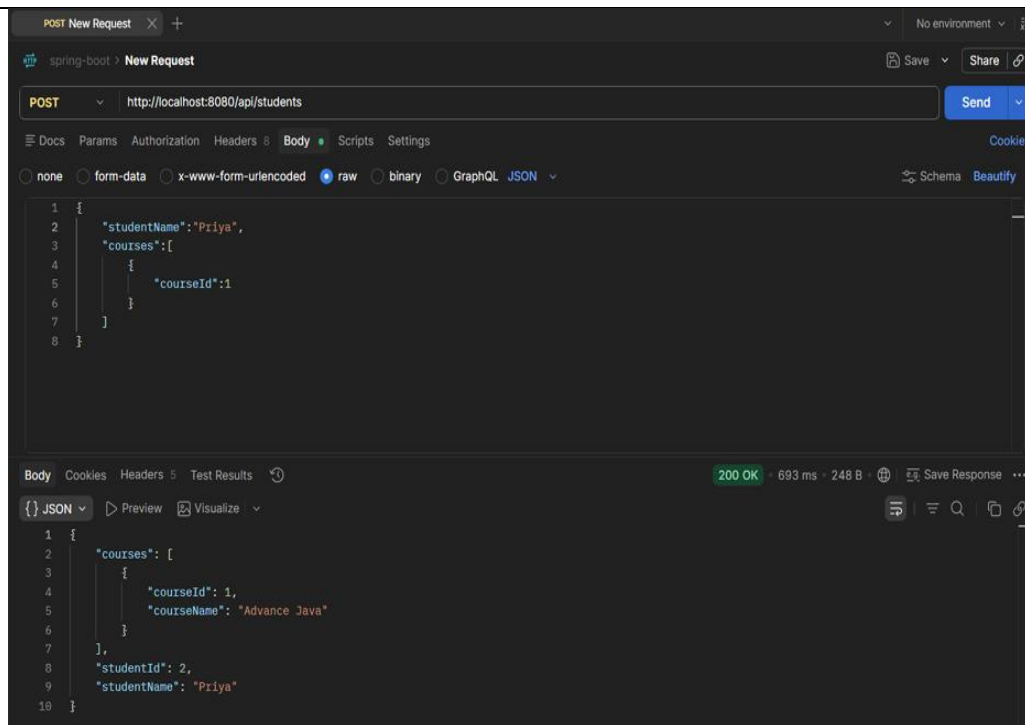
1. **POST API** – Add Course: The POST API is used to create a new course in the database. The course details are sent in JSON format, and Spring Boot stores the record using the save() method of CourseRepository.

POST <http://localhost:8080/api/courses>



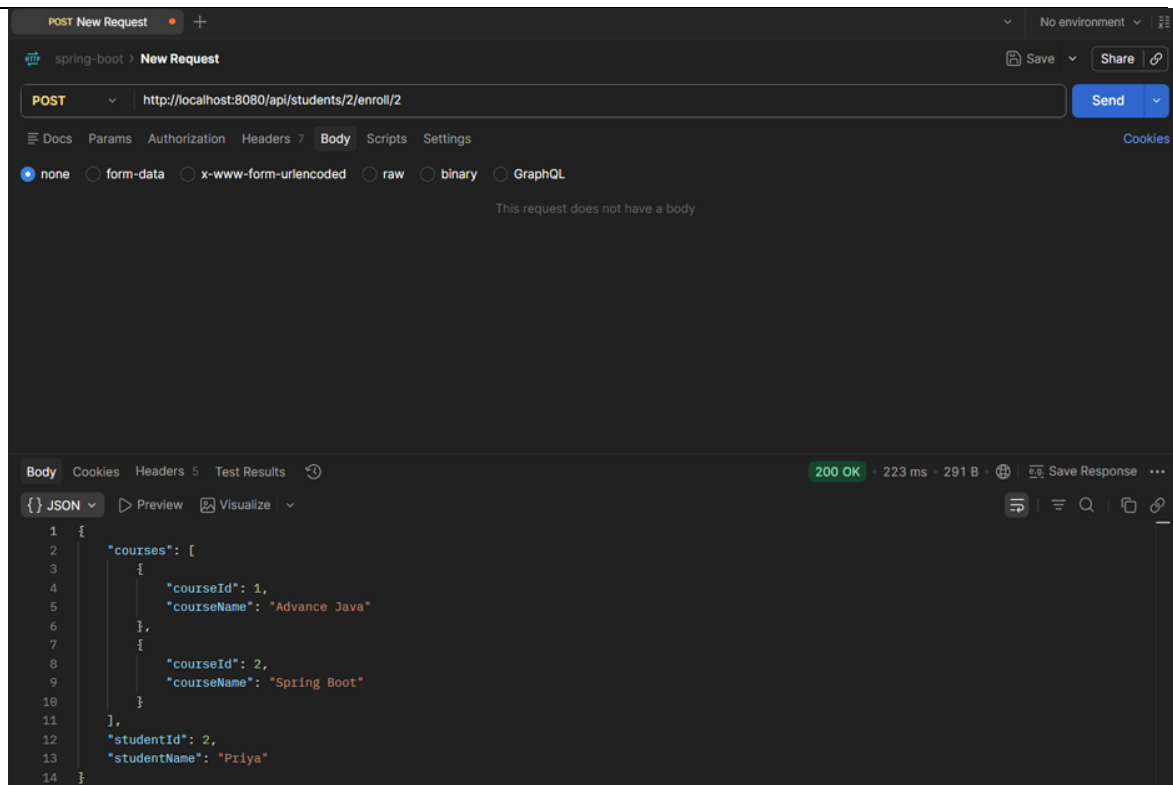
2. **POST API – Add Student:** The POST API is used to create a new student and enroll the student into one or more existing courses. The student details along with course IDs are sent in the request body, and the relationship is stored in the automatically generated join table (student_course)

POST <http://localhost:8080/api/students>



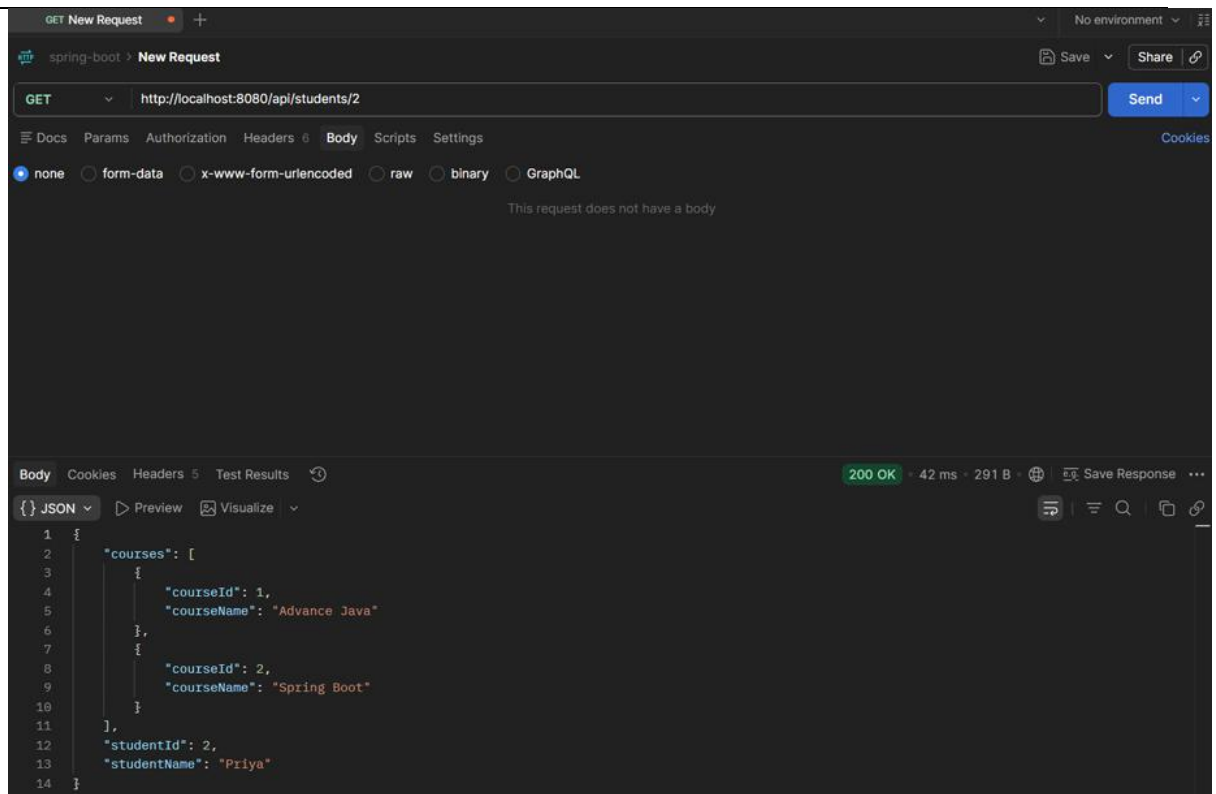
3. **POST API – Enroll Student into Course:** This API is used to enroll an existing student into another existing course. It creates a new relationship between the student and the course by inserting a record into the join table.

POST <http://localhost:8080/api/students/2/enroll/2>



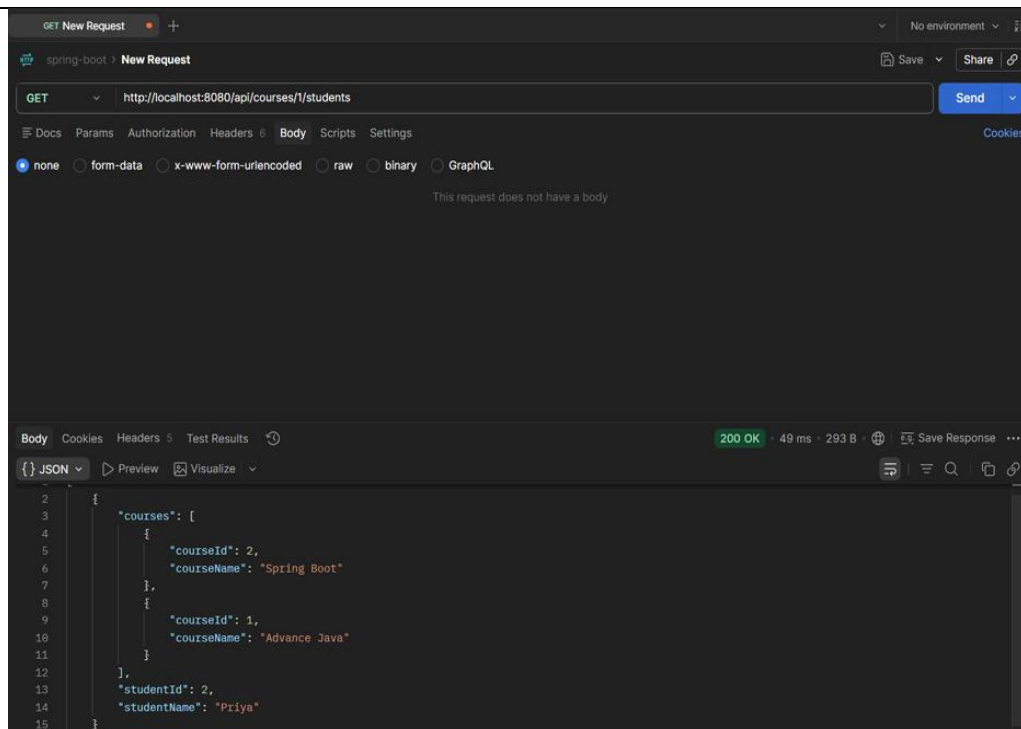
4. **GET API – Retrieve Student by ID:** The GET API retrieves the details of a specific student along with all the courses in which the student is enrolled

GET <http://localhost:8080/api/students/2>



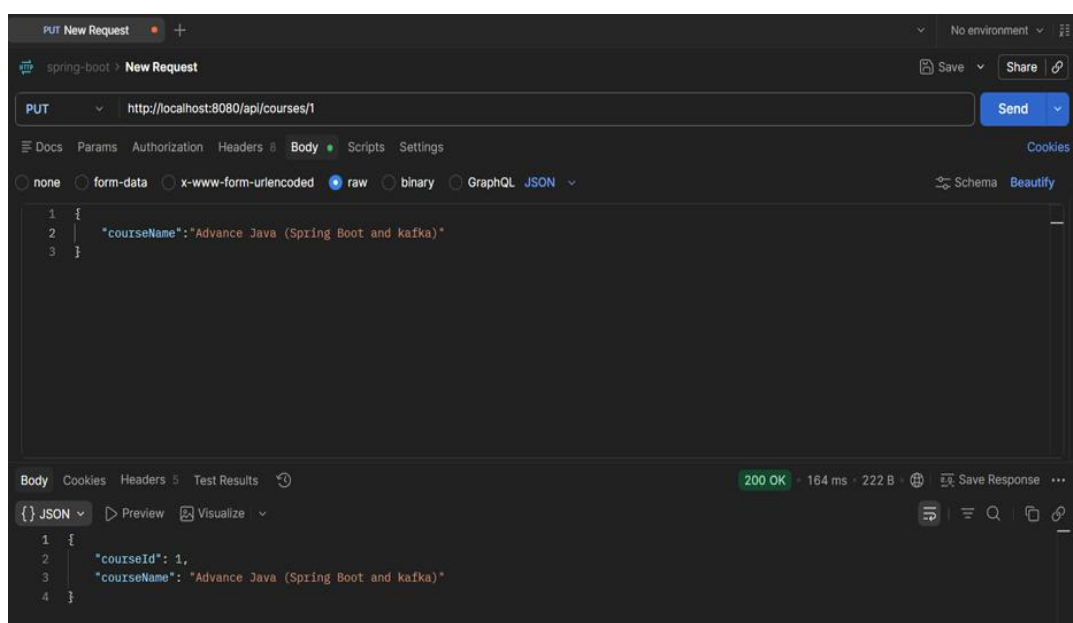
5. **GET API – Retrieve Students of a Course:** This API retrieves all students enrolled in a particular course. It fetches the course using its ID and returns the list of associated students.

GET <http://localhost:8080/api/courses/1/students>



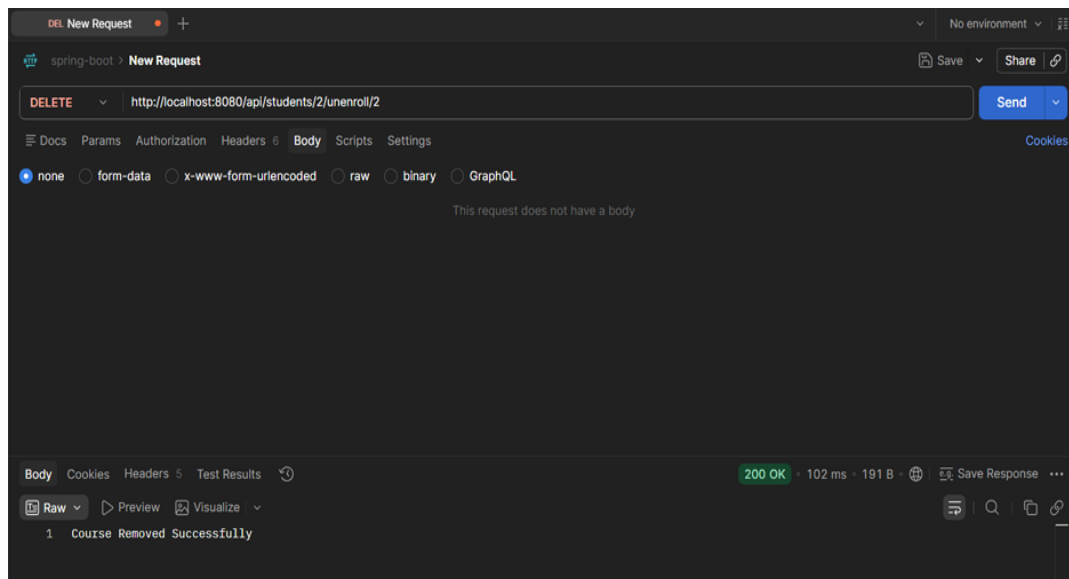
6. **PUT API – Update Course:** The PUT API is used to update the details of an existing course. The updated information is sent in the request body, and the record is updated in the database using the `save()` method..

PUT <http://localhost:8080/api/courses/2>



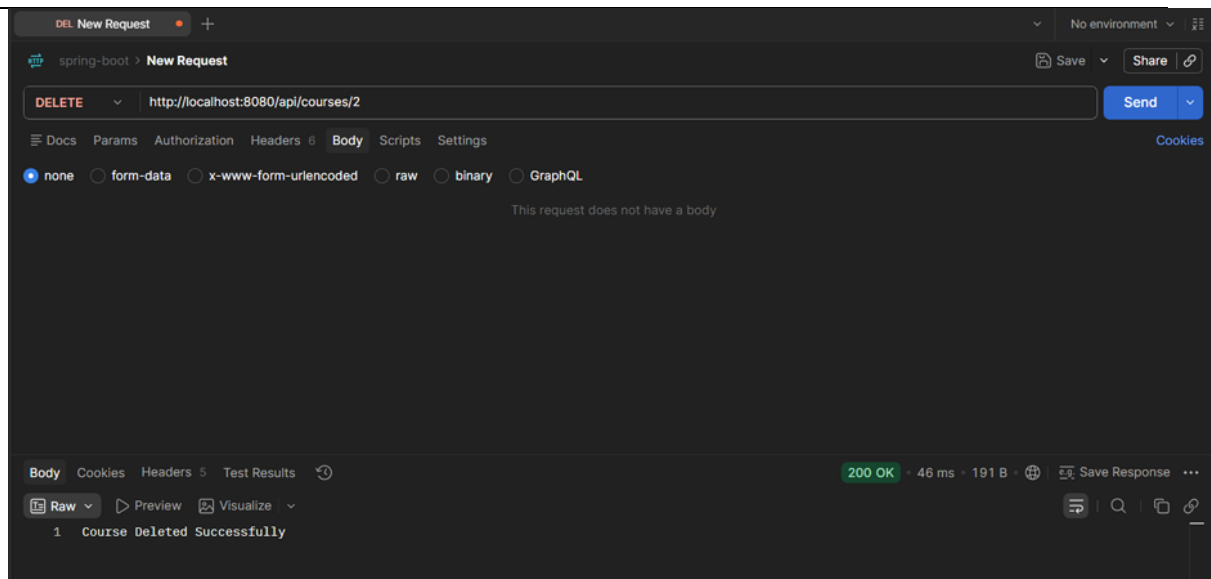
7. **DELETE API – Unenroll Student from Course:** This DELETE API removes the relationship between a student and a course. Only the entry from the student_course join table is deleted, while both the student and the course records remain in the database

DELETE <http://localhost:8080/api/students/2/unenroll/2>

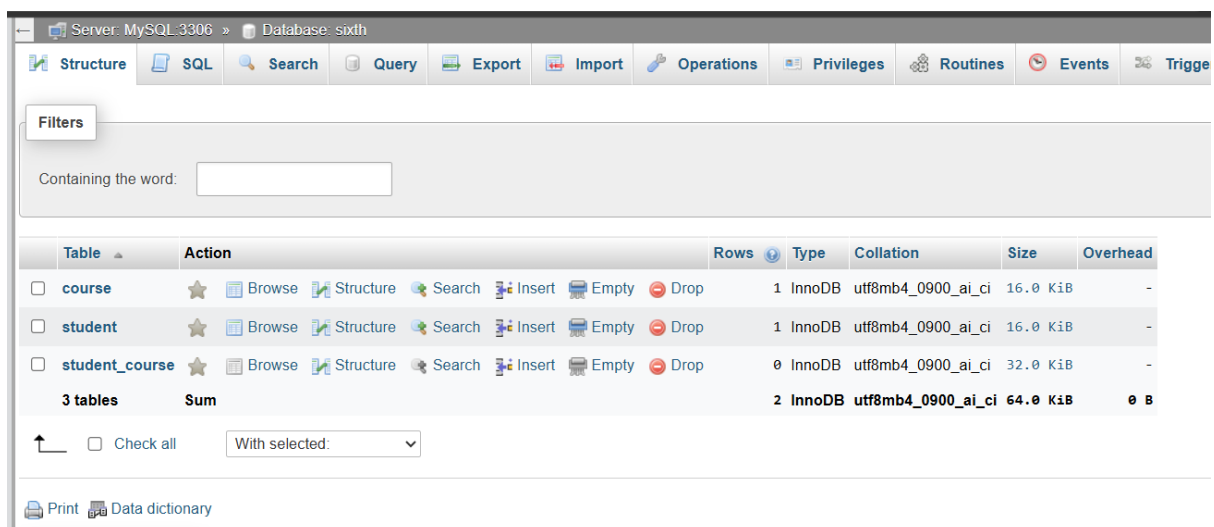


8. **DELETE API – Delete Course:** The DELETE API is used to remove a course from the database. The corresponding entries in the join table are also removed automatically, while student records remain unaffected.

DELETE <http://localhost:8080/api/courses/2>



Database : and all 3 tables created and data also added successfully



Practical No. 7: Transaction Management with @Transactional – Bank Fund Transfer

Objective:

To understand declarative transaction management in Spring Data JPA, and to ensure atomicity across multiple

repository operations (debit + credit) using @Transactional, including automatic rollback when a runtime

exception occurs.

Class Name(s) with Attributes:

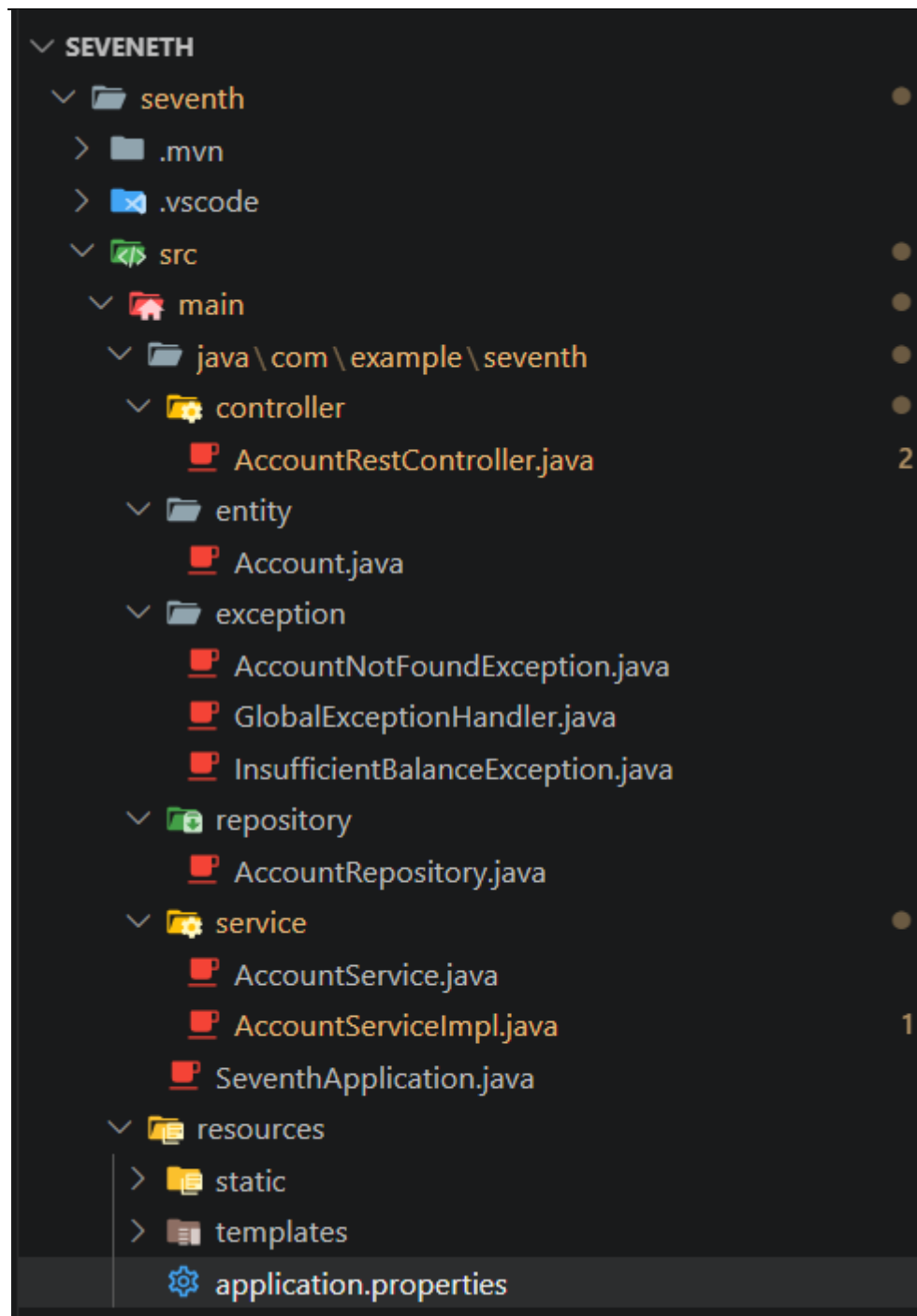
Account (@Entity)

- accountId : int (@Id @GeneratedValue)
- accountHolder : String
- balance : double

InsufficientBalanceException (custom RuntimeException)

AccountService / AccountServiceImpl (transferFunds is @Transactional)

- accountRepository : AccountRepository



Dependencies

- Spring Web
- Spring Data JPA
- MySQL Driver Database

Database Name:

seventh

Table: bank(created automatically by Hibernate)

Account.java(Entity Model class)

```
package com.example.seventh.entity;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;

@Entity
public class Account {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer accountId;
```

```
private String accountHolder;

private double balance;

public Account() {
}

public Integer getAccountId() {
    return accountId;
}

public void setAccountId(Integer accountId) {
    this.accountId = accountId;
}

public String getAccountHolder() {
    return accountHolder;
}

public void setAccountHolder(String accountHolder) {
    this.accountHolder = accountHolder;
}

public double getBalance() {
    return balance;
}
```

```
public void setBalance(double balance) {  
    this.balance = balance;  
}  
}
```

CUSTOM EXCEPTION FILE'S AND GLOBALEXCEPTION

- 1.AccountNotFoundException.java(if user entered account which didn't exist while transferring money)
- 2.InsufficientBalanceException.java(if user want to transfer insufficient amount)
- 3.GlobalExceptionHandler.java(to run this both exception we have to make it global so we also don't need to use try catch)

AccountNotFoundException.java (custom exception)

```
package com.example.seventh.exception;  
  
public class AccountNotFoundException extends RuntimeException {  
  
    public AccountNotFoundException(String message) {  
        super(message);  
    }  
}
```

InsufficientBalanceException.java(Custom Exception)

```
package com.example.seventh.exception;

public class InsufficientBalanceException extends RuntimeException {

    public InsufficientBalanceException(String msg) {

        super(msg);

    }

}
```

GlobalExceptionHandler.java(Exception Handler)

This will used to call the custom exception without getting 500 error

```
package com.example.seventh.exception;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

@RestControllerAdvice

public class GlobalExceptionHandler {

    @ExceptionHandler(InsufficientBalanceException.class)

    public ResponseEntity<String> handleInsufficientBalance(
```

```
InsufficientBalanceException ex) {

return new ResponseEntity<>{

    ex.getMessage(),

    HttpStatus.CONFLICT); // 409

}

@Override
public ResponseEntity<String> handleAccountNotFound(
    AccountNotFoundException ex) {

return new ResponseEntity<>{

    ex.getMessage(),

    HttpStatus.NOT_FOUND); // 404

}

}
```

SERVICE LAYER FILE'S WHICH INCLUDE THE BUSINESS LOGIC IN OUR EXAMPLE TO TRANSFER THE FUND IS LOGIC

- 1.AccountService.java(interface which will implement in main file)
- 2.AccountServiceImpl.java(override the interface and main logic in this file)

AccountService.java(Service layer file)

```
package com.example.seventh.service;

public interface AccountService {

    void transferFunds(int fromId, int toId, double amount);

}
```

AccountServiceImpl.java(Service layer file)

```
package com.example.seventh.service;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import com.example.seventh.entity.Account;
import com.example.seventh.exception.AccountNotFoundException;
import com.example.seventh.exception.InsufficientBalanceException;
import com.example.seventh.repository.AccountRepository;

@Service
public class AccountServiceImpl implements AccountService {
```

```
@Autowired
private AccountRepository repository;

@Override
@Transactional
public void transferFunds(int fromId,
                          int toId,
                          double amount) {

    Account fromAccount = repository.findById(fromId).orElse(null);

    Account toAccount = repository.findById(toId).orElse(null);

    if (fromAccount == null || toAccount == null) {
        throw new AccountNotFoundException("Account Not Found");
    }

    if (fromAccount.getBalance() < amount) {
        throw new InsufficientBalanceException("Insufficient Balance");
    }

    fromAccount.setBalance(
        fromAccount.getBalance() - amount);

    toAccount.setBalance(
        toAccount.getBalance() + amount);
}
```

```
        repository.save(fromAccount);  
        repository.save(toAccount);  
    }  
}
```

AccountRestController.java(main controller file)

```
package com.example.seventh.controller;  
  
import java.util.List;  
  
import org.springframework.beans.factory.annotation.Autowired;  
import org.springframework.web.bind.*;  
  
import com.example.seventh.entity.Account;  
import com.example.seventh.repository.AccountRepository;  
import com.example.seventh.service.AccountService;  
  
@RestController  
@RequestMapping("/api/accounts")  
public class AccountRestController {  
  
    @Autowired  
    private AccountRepository repository;  
  
    @Autowired  
    private AccountService service;
```

```
// Create Account
@PostMapping
public Account addAccount(@RequestBody Account account) {

    return repository.save(account);
}

// Get Account by ID
@GetMapping("/{id}")
public Account getAccount(@PathVariable Integer id) {

    return repository.findById(id).orElse(null);
}

// Get All Accounts
@GetMapping
public List<Account> getAll() {

    return repository.findAll();
}

// Transfer Money
@PostMapping("/transfer")
public String transferMoney(
    @RequestParam Integer fromId,
    @RequestParam Integer toId,
```

```
@RequestParam double amount) {

    service.transferFunds(fromId, toId, amount);

    return "Amount Transferred Successfully";
}

// Update Account
@PutMapping("/{id}")
public Account updateAccount(
    @PathVariable Integer id,
    @RequestBody Account account) {

    Account existing = repository.findById(id).orElse(null);

    if (existing == null)
        return null;

    existing.setAccountHolder(account.getAccountHolder());

    return repository.save(existing);
}

// Delete Account
@DeleteMapping("/{id}")
public String deleteAccount(@PathVariable Integer id) {
```

```
Account account = repository.findById(id).orElse(null);

if (account == null)
    return "Account Not Found";

if (account.getBalance() != 0)
    return "Account Cannot Be Deleted. Balance Must Be Zero.";

repository.delete(account);

return "Account Deleted Successfully";
}
}
```

AccountRepository.java(Repository file)

```
package com.example.seventh.repository;

import org.springframework.data.jpa.repository.JpaRepository;

import com.example.seventh.entity.Account;

public interface AccountRepository extends JpaRepository<Account, Integer> {

}
```

Application.properties

```
spring.application.name=seventh

# MySQL Database
spring.datasource.url=jdbc:mysql://localhost:3306/seventh
spring.datasource.username=root
spring.datasource.password=

# JPA
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true

# MySQL Dialect
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQLDialect
```

WHAT'S NEW IN THIS PRACTICAL

1.SERVICE LAYER FOR BUSINESS LOGIN IF WE WANT TO TRANSFER THE MONEY IT WOULD NEED TO IMPLEMENT THE LOGIC SO IT SHOULD BE COVERD IN SERVICE LAYER

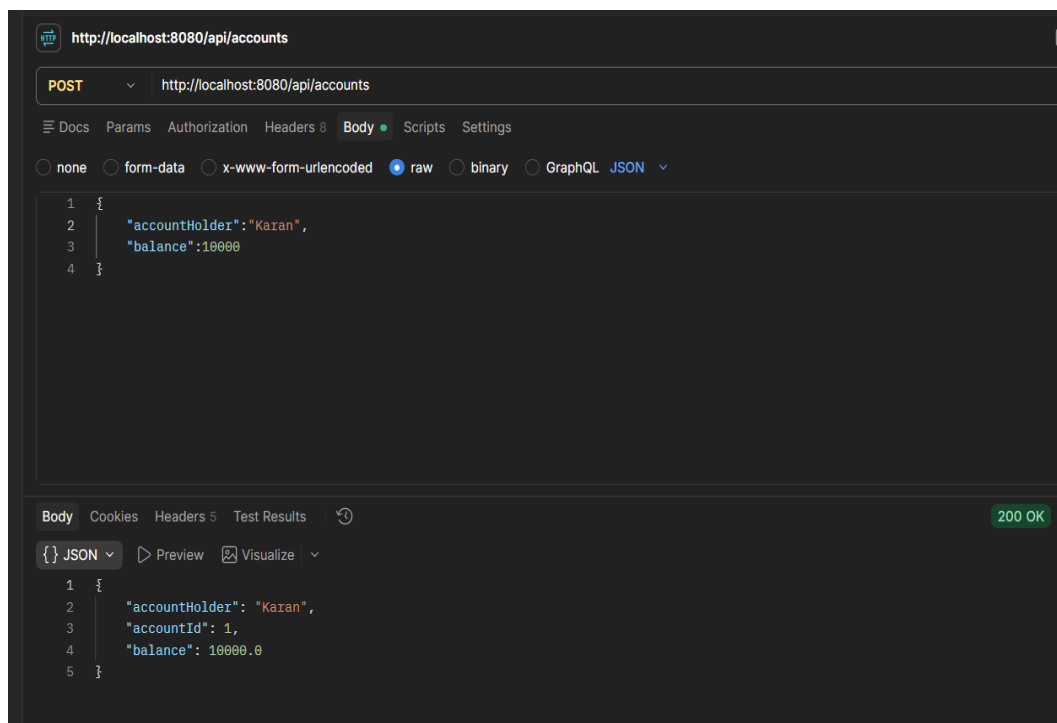
2.@Transactional ANNOTATION SUPPOSE WE ARE TRANSFERING MONEY OF 5000 FROM A TO B IT'S DEBITED FROM A AND NOT CREDITED IN B SOMETHING WRONG HAPPENED IN MIDDLE SO TRANSACTIONAL WILL HANDLE THIS AND ROLLBACK A S MONEY.

API'S TESTING USING POSTMAN

1. POST API – Create First Account

The **POST API** is used to create a new bank account. The account holder name and initial balance are sent in the request body. Spring Boot stores the account in the database and automatically generates the account ID.

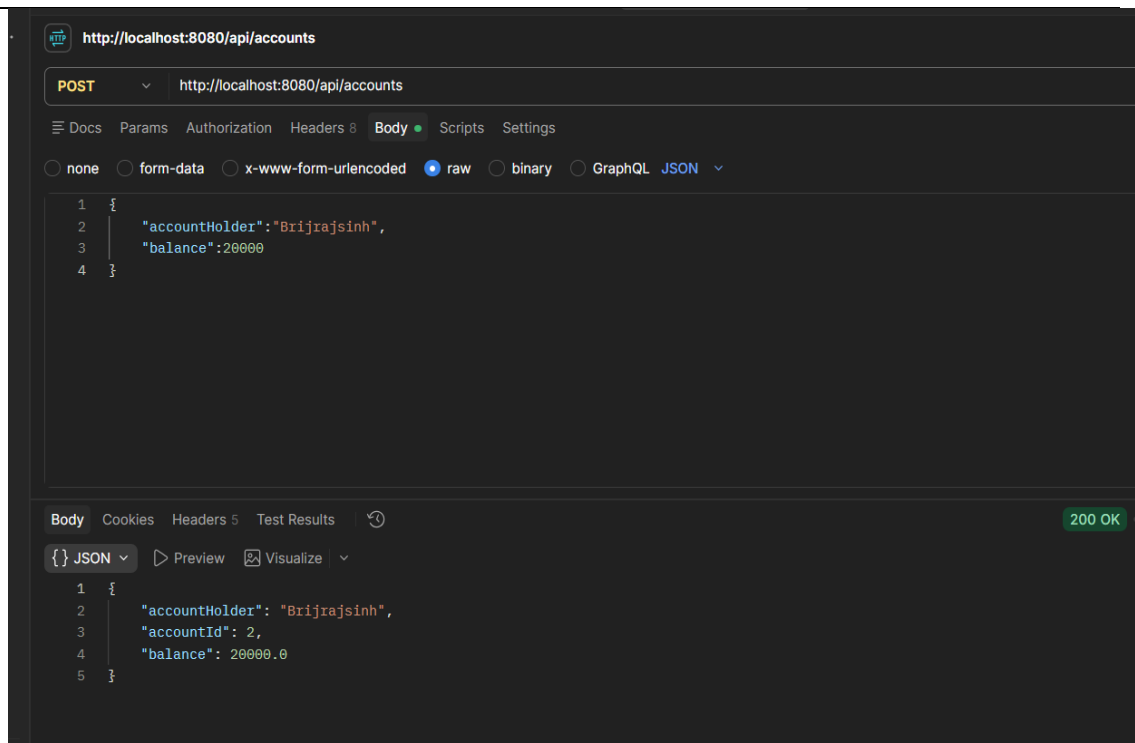
POST <http://localhost:8080/api/accounts>



2: POST API – Create Second Account

The **POST API** is used to create another bank account. The account holder details and initial balance are stored in the database.

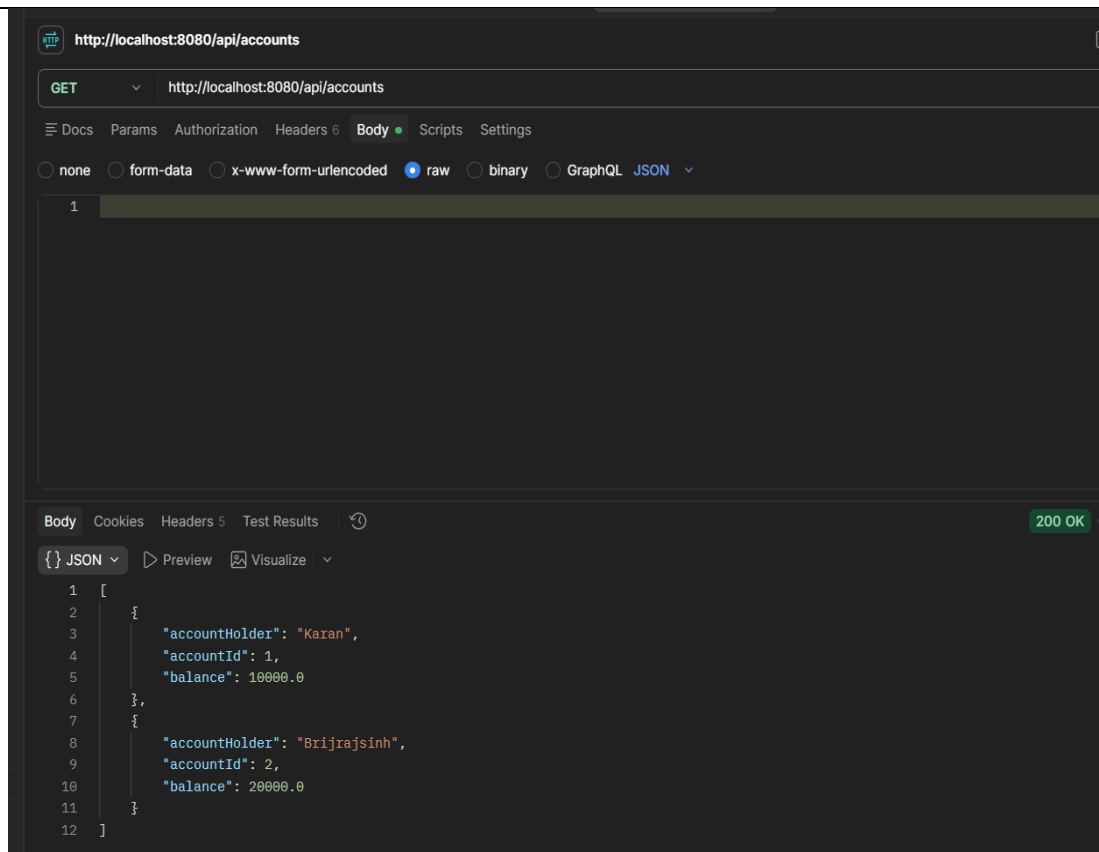
POST <http://localhost:8080/api/accounts>



3: GET API – Get All Accounts

The **GET API** is used to retrieve all account details available in the database.

GET <http://localhost:8080/api/accounts>



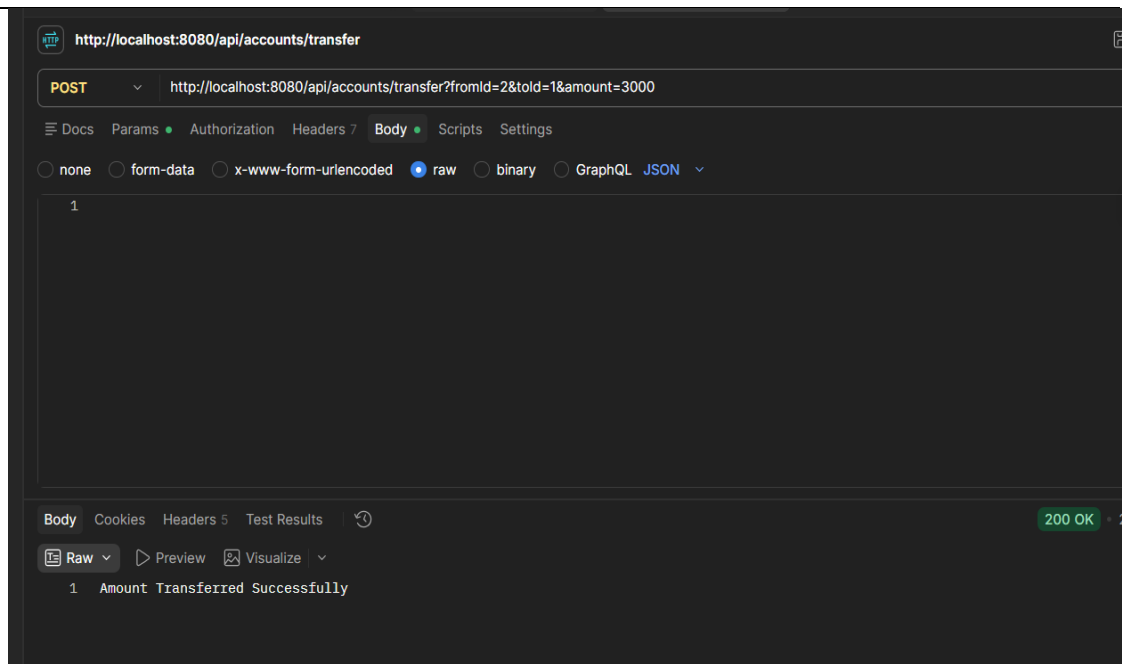
4: POST API – Transfer Funds

The **POST API** is used to transfer money from one account to another. The sender account ID (fromId), receiver account ID (told), and transfer amount (amount) are passed as request parameters. The service validates both accounts, checks the sender's balance, deducts the amount from the sender, credits the receiver, and saves both accounts within a single transaction using the **@Transactional** annotation. If the transfer is successful, the changes are permanently stored in the database.

(BRIJRAJSINH HAS ₹20000 AND KARAN HAS ₹10000)

POST

<http://localhost:8080/api/accounts/transfer?fromId=2&told=1&amount=3000>

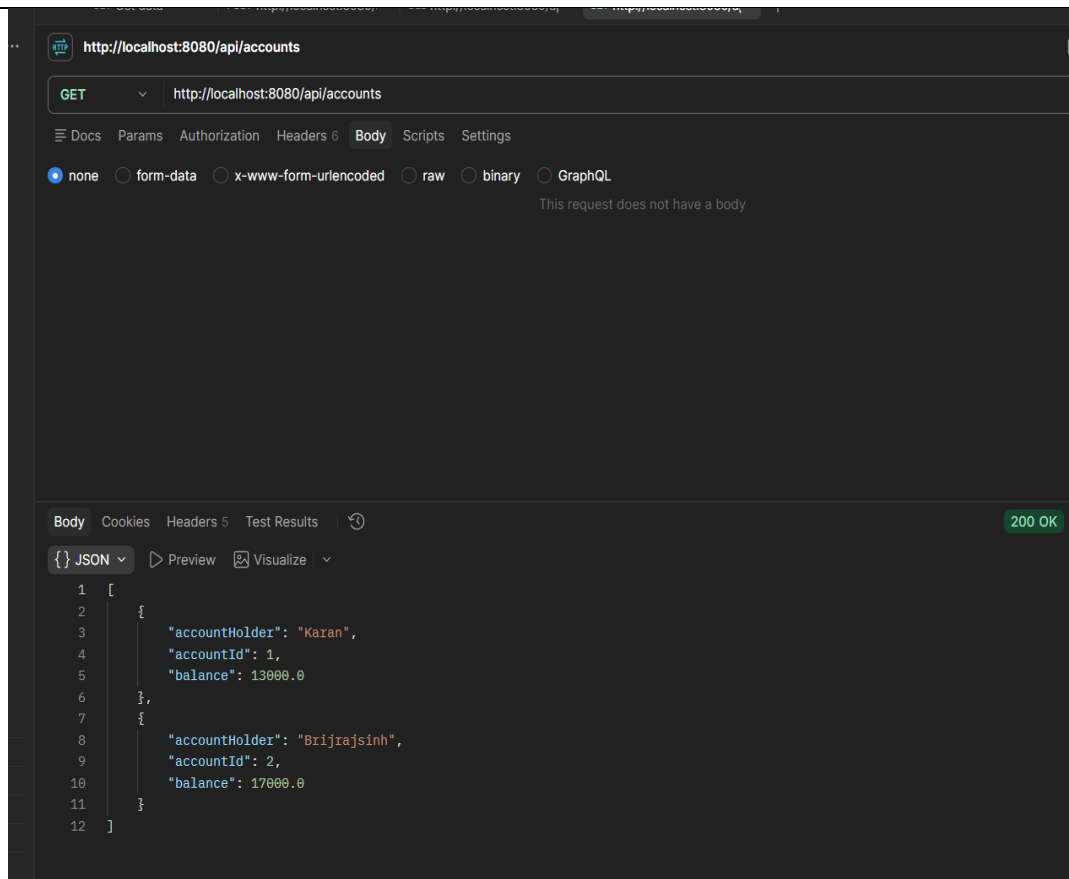


5: GET API – Verify Updated Balance

The **GET API** is used to verify that the money transfer was completed successfully.

GET <http://localhost:8080/api/accounts>

(SHOW BALANCE OF KARAN 3000 CREDITED AND BRIJRAJSINH DEBITED 3000)



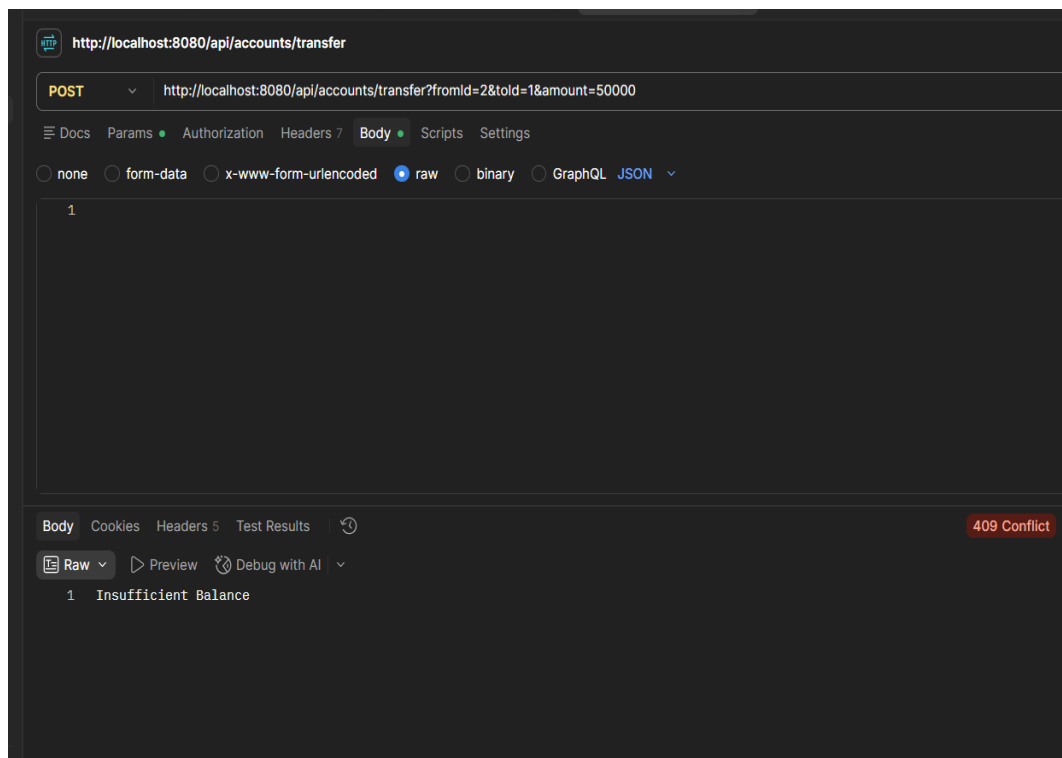
6: POST API – Transfer Funds (Insufficient Balance)

This API demonstrates transaction rollback using Spring Boot's **@Transactional** feature. If the sender's account balance is less than the requested transfer amount, the service throws an **InsufficientBalanceException**. The **GlobalExceptionHandler** catches the exception and returns an appropriate error response. Since the transaction is rolled back automatically, no changes are made to either account, ensuring data consistency.

POST

<http://localhost:8080/api/accounts/transfer?fromId=2&toid=1&amount=9999999>

(BALANCE IS NOT ENOUGH OF BRIJRAJSINH 17000 NOT 50000 show exception when transferred..)



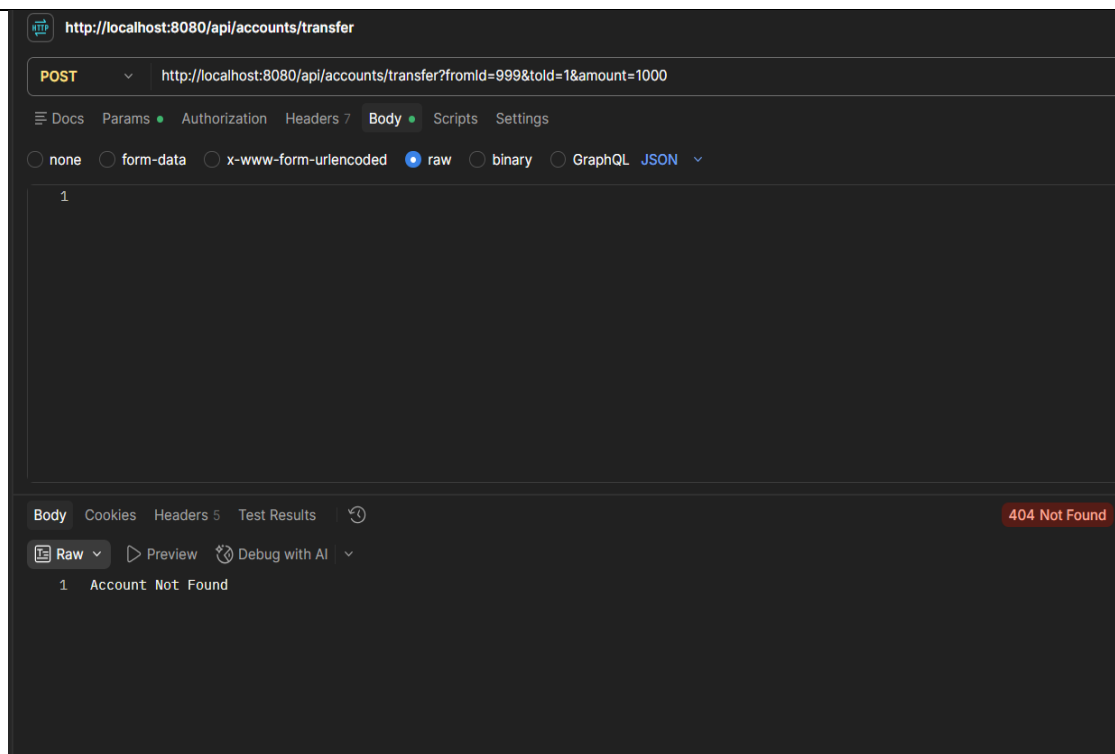
7: POST API – Account Not Found

This API demonstrates the **AccountNotFoundException**. If the sender or receiver account ID does not exist in the database, the application throws the exception and the **GlobalExceptionHandler** returns an appropriate error response.

POST

<http://localhost:8080/api/accounts/transfer?fromId=999&told=1&amount=1000>

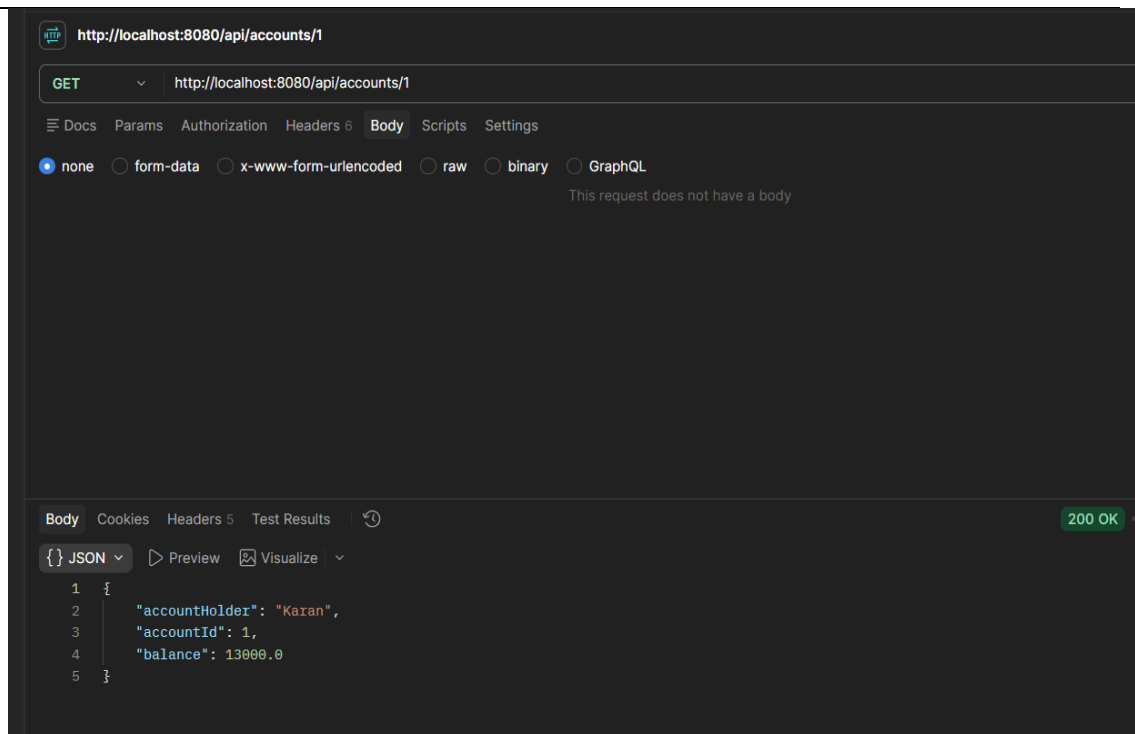
(ACCOUNT NOT FOUND BECAUSE WE HAVE ONLY 2 ACCOUNTS NOT 999)



8: GET API – Get Account by ID

The **GET API** is used to retrieve the details of a specific account using its unique account ID. The application searches the database using the **findById()** method and returns the account information in JSON format.

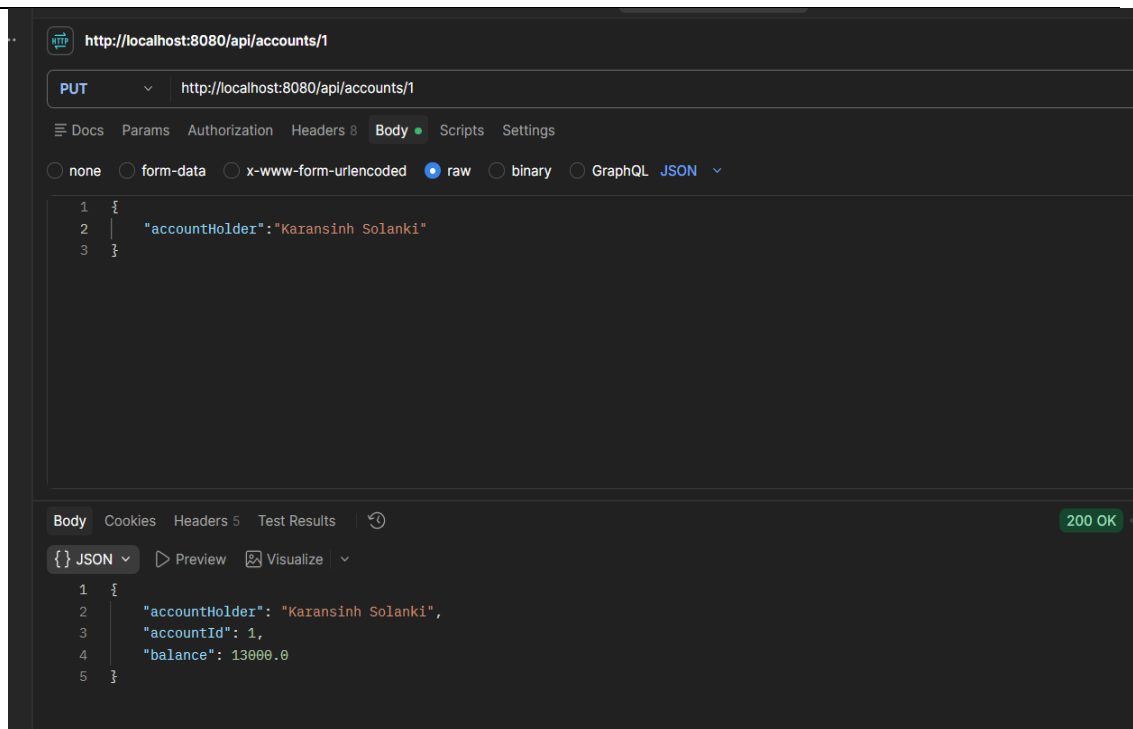
GET <http://localhost:8080/api/accounts/1>



9: PUT API – Update Account

The **PUT API** is used to update the details of an existing account. The account ID is provided in the URL, while the updated account information is sent in the request body. Spring Boot updates the existing account record and saves the modified details in the database.

PUT <http://localhost:8080/api/accounts/1>

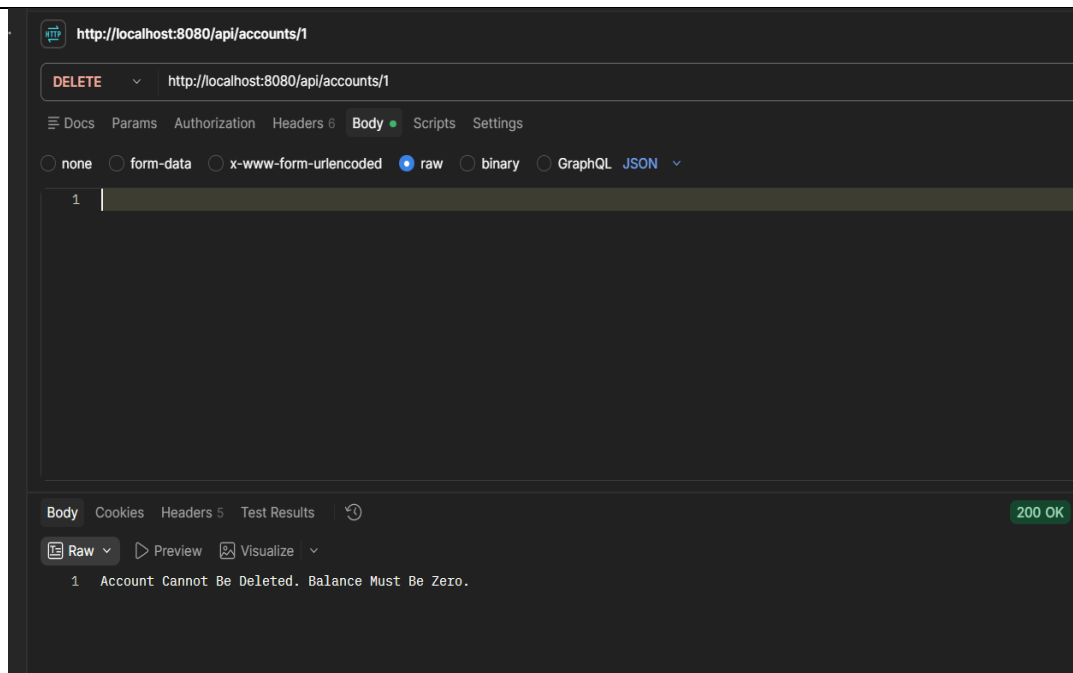


10: DELETE API – Delete Account

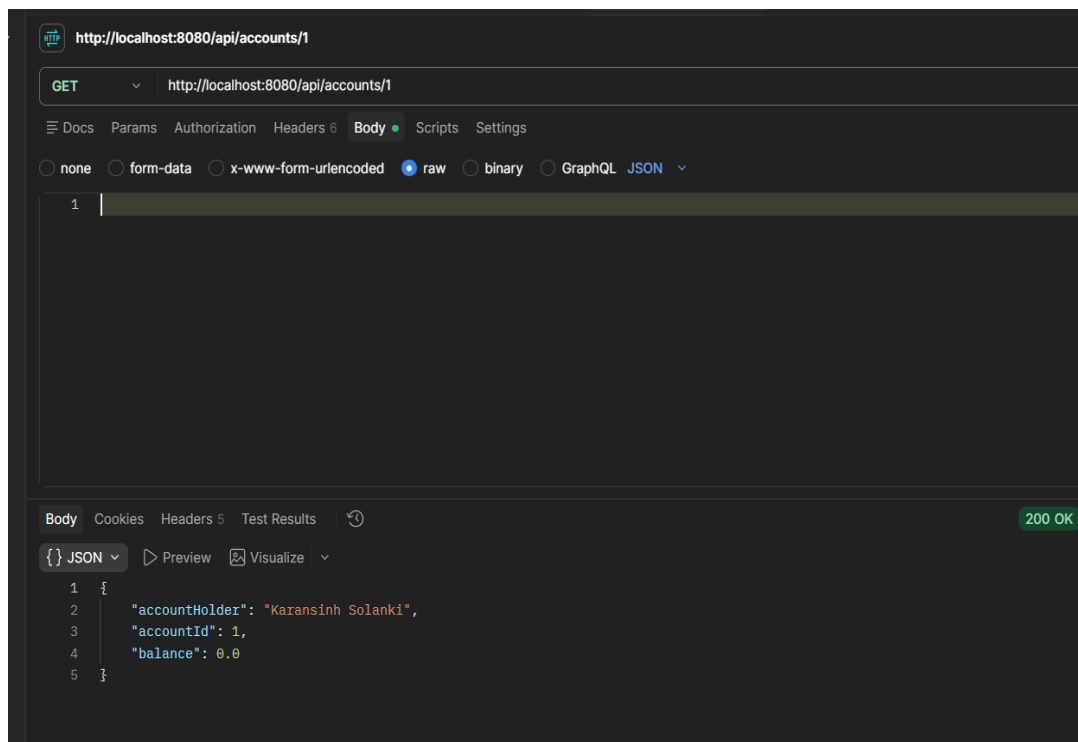
The **DELETE API** is used to remove an account from the database using its account ID. Before deletion, the application verifies that the account balance is zero. If the balance is zero, the account is deleted successfully. Otherwise, the application prevents deletion and returns an appropriate error message.

DELETE <http://localhost:8080/api/accounts/1>

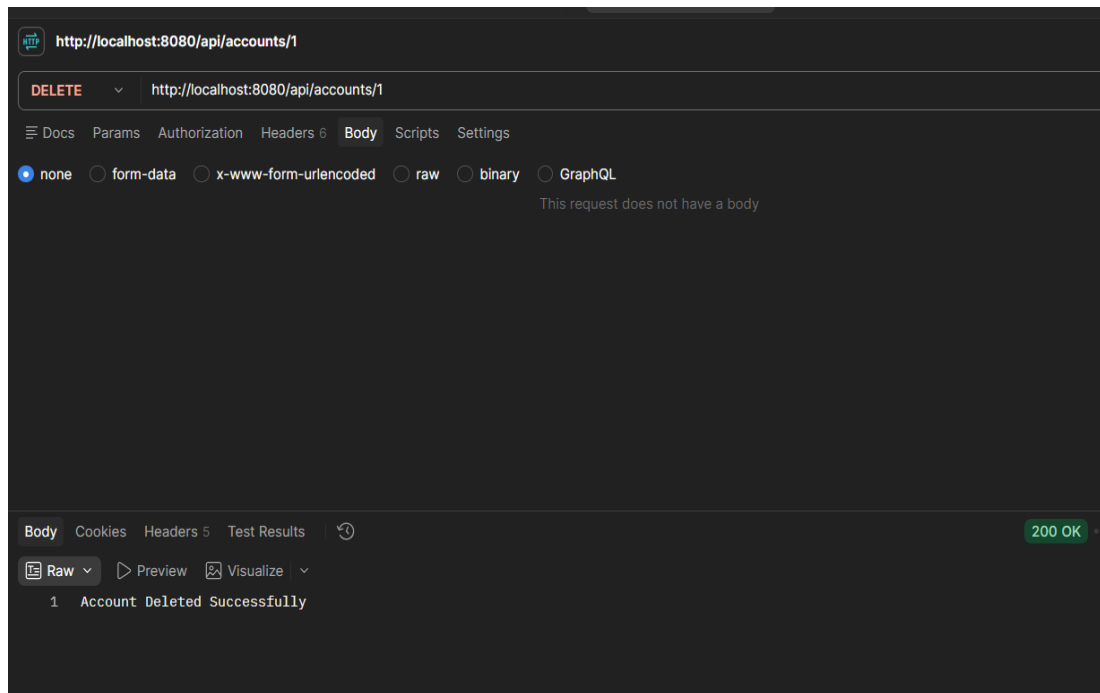
(ACCOUNT HAVE NOT ZERO BALANCE SHOW EXCEPTION FIRST NEED TO AMOUNT SET 0 AFTER DELETED..)



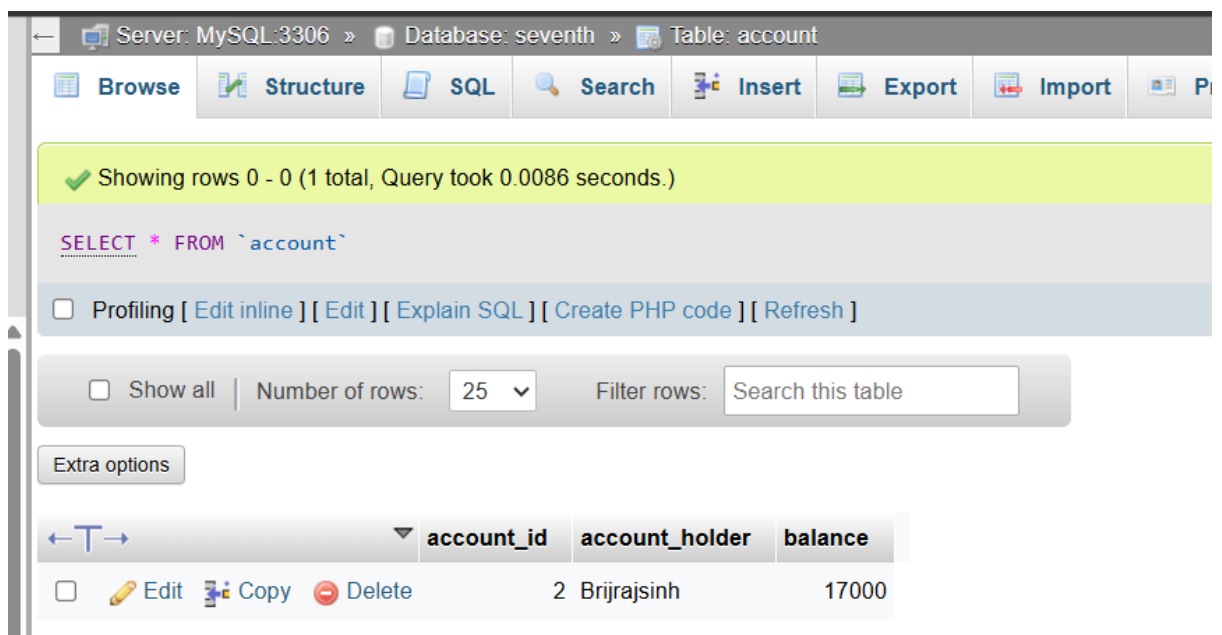
AFTER SET ZERO..



DELETED ACCOUNT SUCCESSFULLY



SHOW THE FINAL OUTPUT OF TABLE REMOVED ID 1



Practical No. 8: Pagination and Sorting with Spring Data JPA

Objective:

To implement server-side pagination and sorting for a large result set using Spring Data's Pageable/PageRequest

abstraction (PagingAndSortingRepository capability), exposed through query parameters on a REST endpoint.

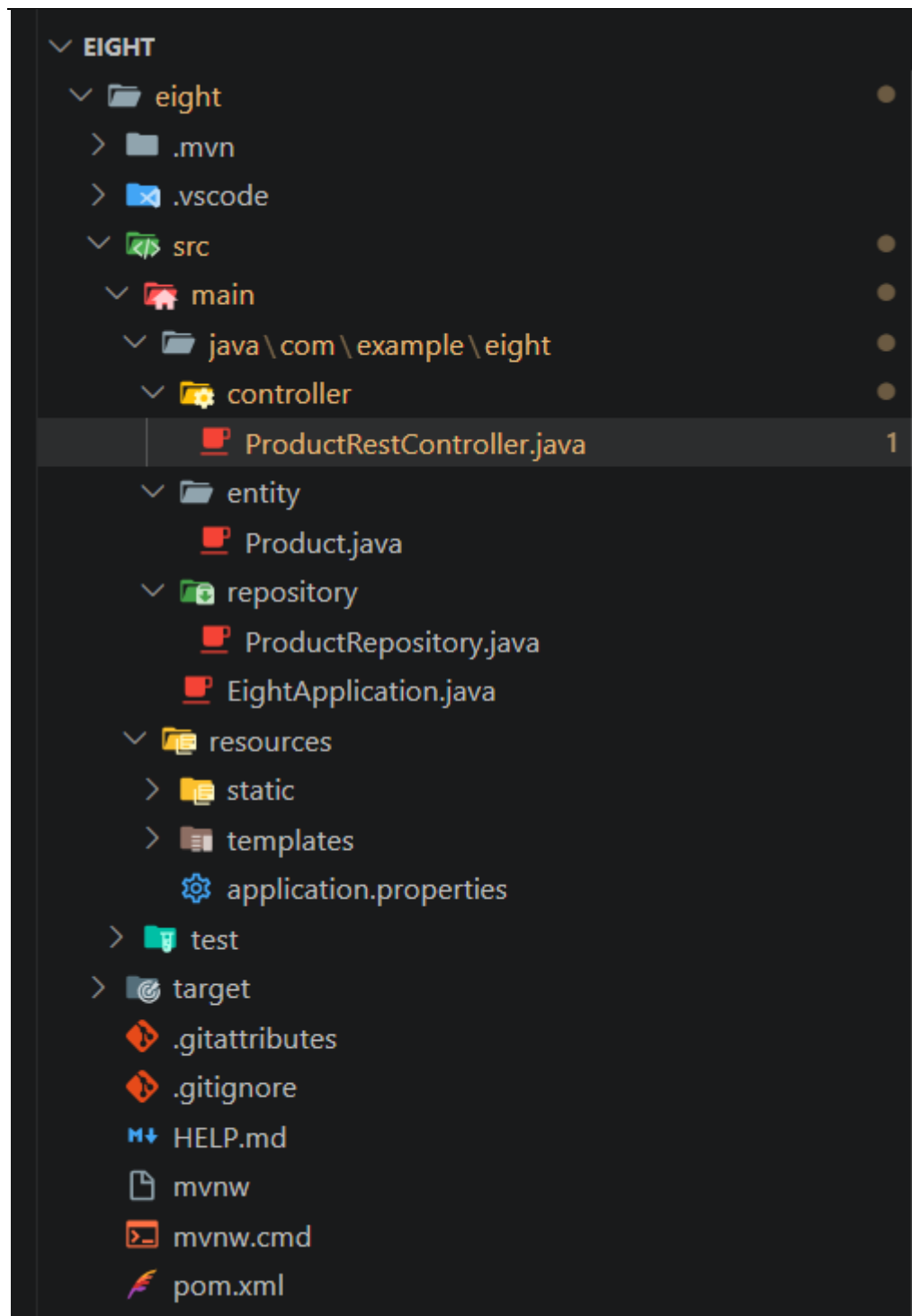
Class Name(s) with Attributes:

Product (reuse from Practical 2)

- productId, productName, category, price, quantity

ProductRestController (extended)

- Method: getProducts(page,size,sortBy) : Page<Product>



Dependencies

- Spring Web
- Spring Data JPA
- MySQL Driver

Database

Database Name: product_db

Table: product (created automatically by Hibernate)

Product.java (Entity file)

```
package com.example.eight.entity;

import jakarta.persistence.*;

@Entity
@Table(name = "product")
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer productId;

    private String productName;

    private String category;
```

```
private double price;

private int quantity;


public Product() {
}


public Integer getProductId() {
    return productId;
}


public void setProductId(Integer productId) {
    this.productId = productId;
}


public String getProductName() {
    return productName;
}


public void setProductName(String productName) {
    this.productName = productName;
}


public String getCategory() {
    return category;
}


public void setCategory(String category) {
```

```
        this.category = category;
    }

    public double getPrice() {
        return price;
    }

    public void setPrice(double price) {
        this.price = price;
    }

    public int getQuantity() {
        return quantity;
    }

    public void setQuantity(int quantity) {
        this.quantity = quantity;
    }
}
```

ProductRepository.java(Repository file)

```
package com.example.eight.repository;

import org.springframework.data.jpa.repository.JpaRepository;

import com.example.eight.entity.Product;

public interface ProductRepository
    extends JpaRepository<Product, Integer> {

}
```

ProductRestController.java(Controller file)

```
package com.example.eight.controller;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.PageRequest;
import org.springframework.data.domain.Pageable;
import org.springframework.data.domain.Sort;
import org.springframework.web.bind.annotation.*;

import com.example.eight.entity.Product;
import com.example.eight.repository.ProductRepository;
```



```
@RestController

@RequestMapping("/api/products")

public class ProductRestController {

    @Autowired

    private ProductRepository repository;

    // Add Product

    @PostMapping

    public Product addProduct(@RequestBody Product product) {

        return repository.save(product);

    }

    // Update Product

    @PutMapping("/{id}")

    public Product updateProduct(@PathVariable Integer id,

        @RequestBody Product product) {

        product.setProductId(id);

        return repository.save(product);

    }

    // Delete Product

    @DeleteMapping("/{id}")

    public String deleteProduct(@PathVariable Integer id) {

        repository.deleteById(id);

    }

}
```

```
        return "Product Deleted Successfully";
    }

    // Pagination and Sorting
    @GetMapping("/page")
    public Page<Product> getProducts(
        @RequestParam(defaultValue = "0") int page,
        @RequestParam(defaultValue = "5") int size,
        @RequestParam(defaultValue = "price") String sortBy) {

        System.out.println("sortBy = " + sortBy);

        Pageable pageable = PageRequest.of(
            page,
            size,
            Sort.by(sortBy));

        return repository.findAll(pageable);
    }
}
```

Application.properties

```
spring.application.name=eight
spring.datasource.url=jdbc:mysql://localhost:3306/productdb
spring.datasource.username=root
spring.datasource.password=
```

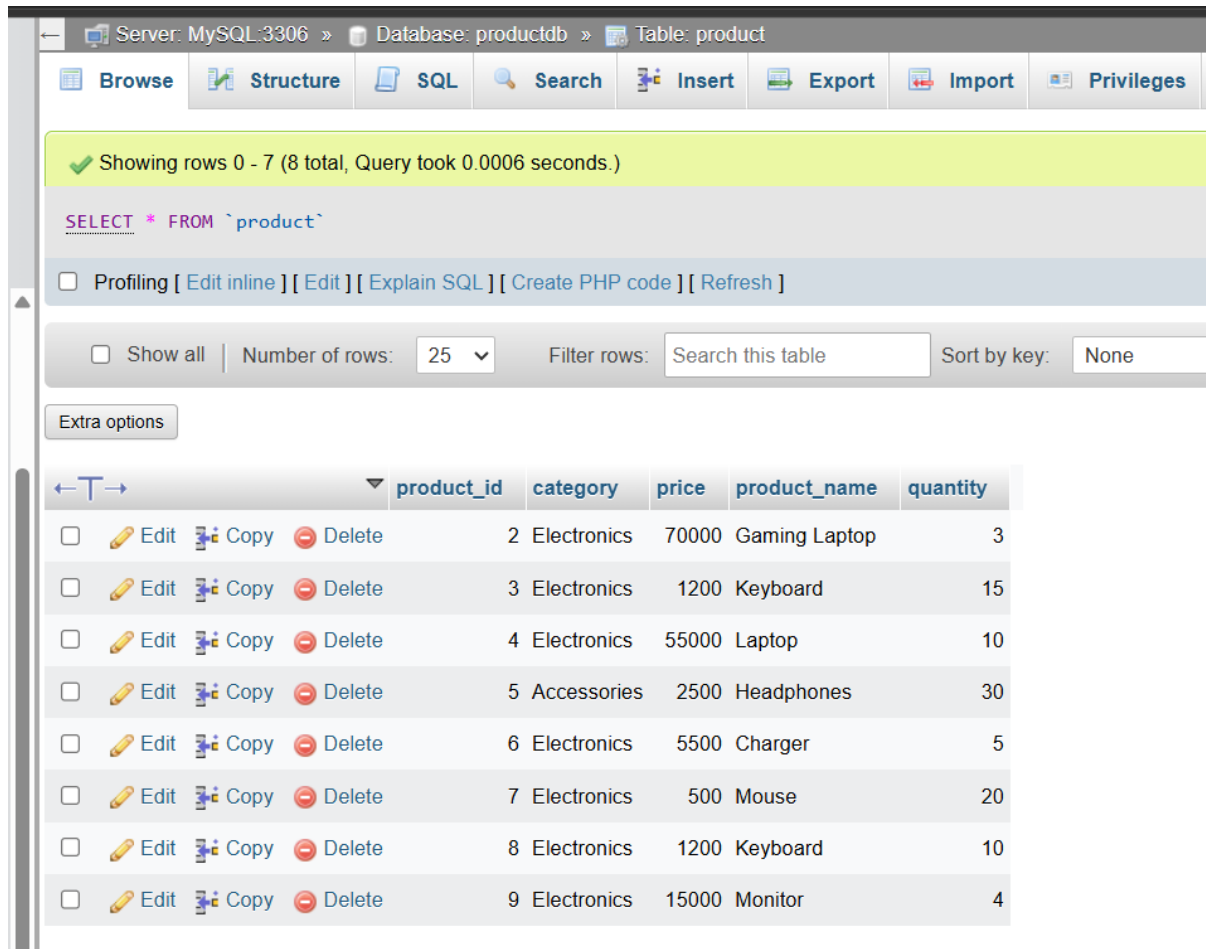
```
spring.jpa.hibernate.ddl-auto=update  
spring.jpa.show-sql=true  
spring.jpa.properties.hibernate.format_sql=true
```

What is difference from this and second one

This practical extends the Product CRUD application by implementing Pagination and Sorting using Spring Data JPA. Unlike Practical 2, where all product records are retrieved at once using `findAll()`, this practical fetches products page by page, reducing memory usage and improving performance. It also allows products to be sorted dynamically by fields such as price, productName, or category. These features are especially useful when working with large datasets in real-world applications.

API'S TESTING USING POSTMAN

SAME DATABASE AND TABLES ARE USED AS SECOND PRACTICAL SO ALREDY WE HAVE BELOW DATA OF PRODUCT TABLE.



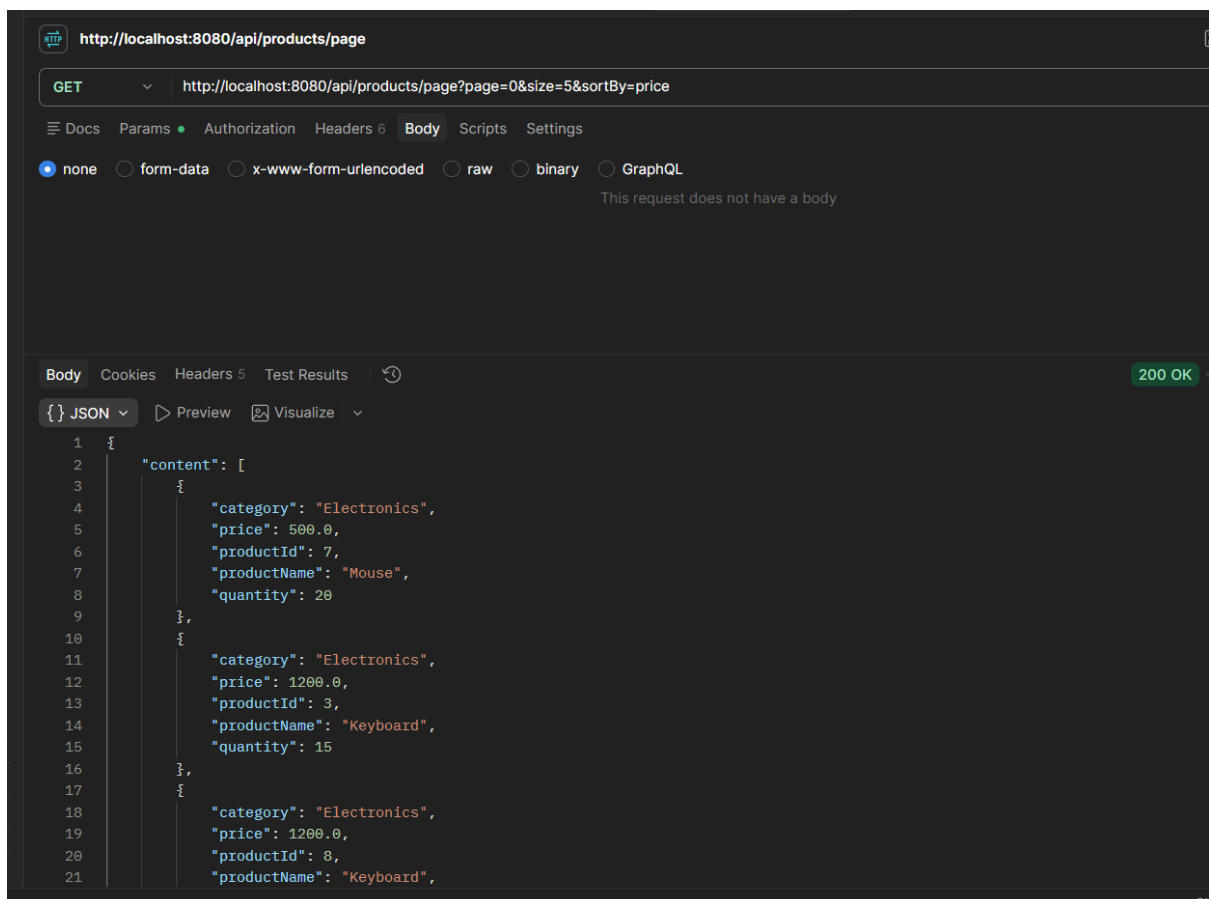
The screenshot shows the MySQL Workbench interface. At the top, the breadcrumb navigation indicates 'Server: MySQL:3306 » Database: productdb » Table: product'. Below this is a toolbar with buttons for Browse, Structure, SQL, Search, Insert, Export, Import, and Privileges. A status bar shows 'Showing rows 0 - 7 (8 total, Query took 0.0006 seconds.)'. The SQL editor contains the query 'SELECT * FROM `product`'. Below the editor are options for Profiling, Edit inline, Edit, Explain SQL, Create PHP code, and Refresh. A control bar shows 'Show all', 'Number of rows: 25', 'Filter rows: Search this table', and 'Sort by key: None'. An 'Extra options' button is also present. The main area displays a table with 8 rows of product data. Each row has a checkbox, an 'Edit' button, a 'Copy' button, and a 'Delete' button. The table columns are product_id, category, price, product_name, and quantity.

	product_id	category	price	product_name	quantity
☐ Edit Copy Delete	2	Electronics	70000	Gaming Laptop	3
☐ Edit Copy Delete	3	Electronics	1200	Keyboard	15
☐ Edit Copy Delete	4	Electronics	55000	Laptop	10
☐ Edit Copy Delete	5	Accessories	2500	Headphones	30
☐ Edit Copy Delete	6	Electronics	5500	Charger	5
☐ Edit Copy Delete	7	Electronics	500	Mouse	20
☐ Edit Copy Delete	8	Electronics	1200	Keyboard	10
☐ Edit Copy Delete	9	Electronics	15000	Monitor	4

1. GET API – Display First Page of Products Sorted by Price

The **GET API** is used to retrieve the first page of product records from the database. The API fetches only **5 products** from the first page and sorts them in **ascending order of price** using Spring Data JPA Pagination and Sorting. This improves application performance by loading only the required records instead of fetching all products.

GET <http://localhost:8080/api/products/page?page=0&size=5&sortBy=price>

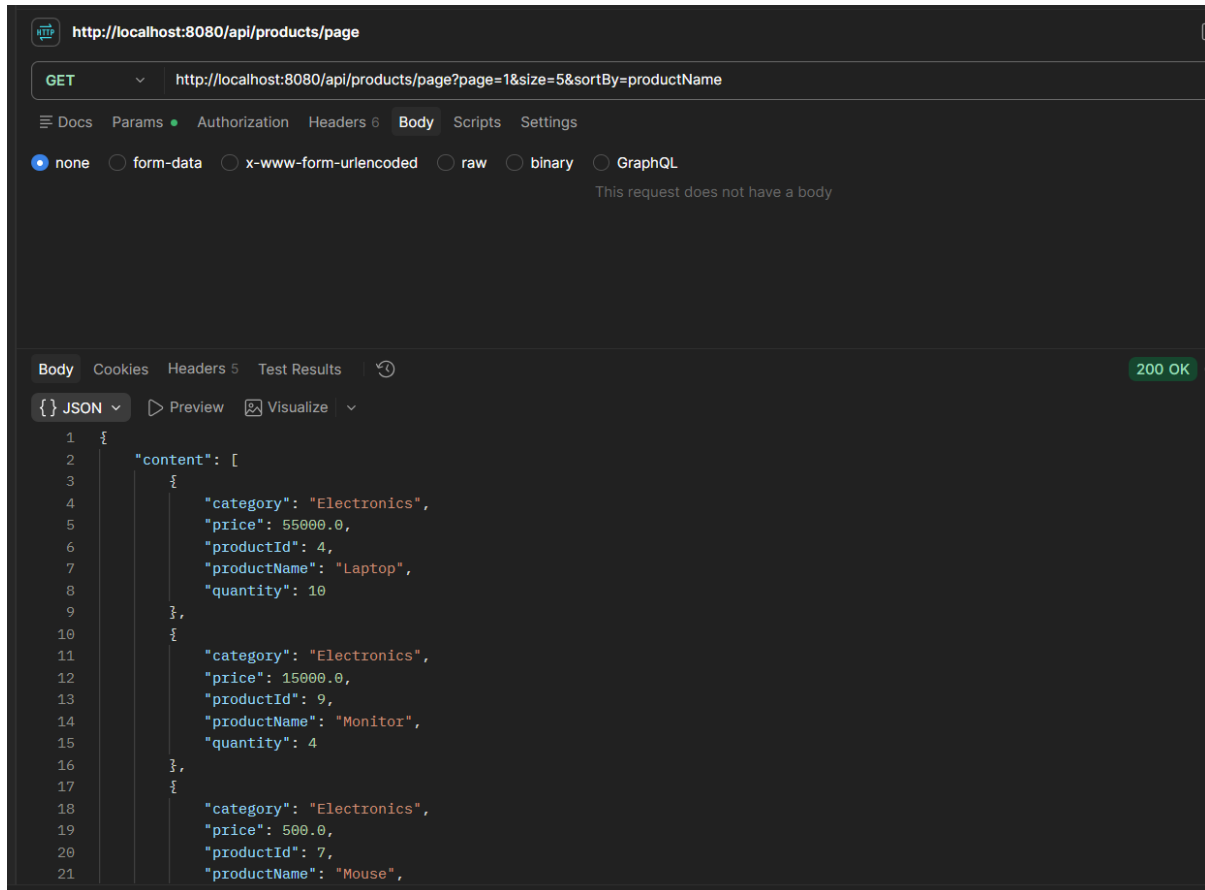


SEE THE FIRST PAGE OF PRODUCTS IS DISPLAYED SUCCESSFULLY IN ASCENDING ORDER OF PRICE.

2. GET API – Display Second Page of Products Sorted by Product Name

The **GET API** retrieves the second page of products sorted alphabetically according to the **productName** field. The API displays the second page while maintaining alphabetical order.

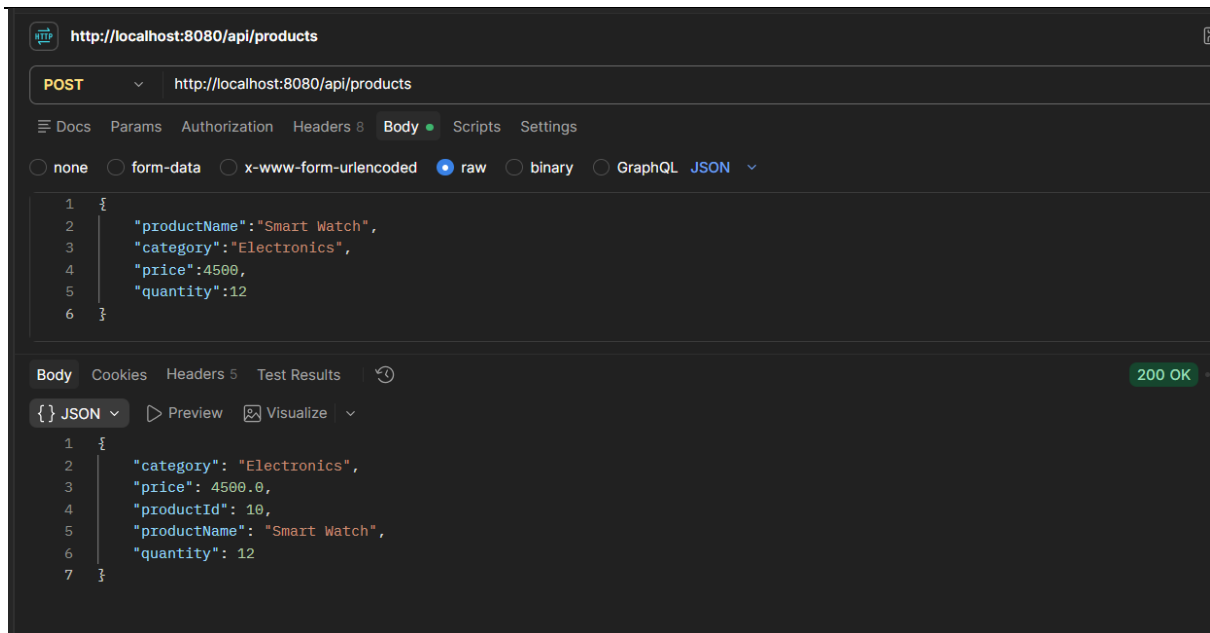
GET <http://localhost:8080/api/products/page?page=1&size=5&sortBy=productName>



3. POST API – Add Product

The **POST API** is used to insert a new product into the MySQL database. The product details are sent in the request body as JSON, and Spring Boot stores the record using the **save()** method.

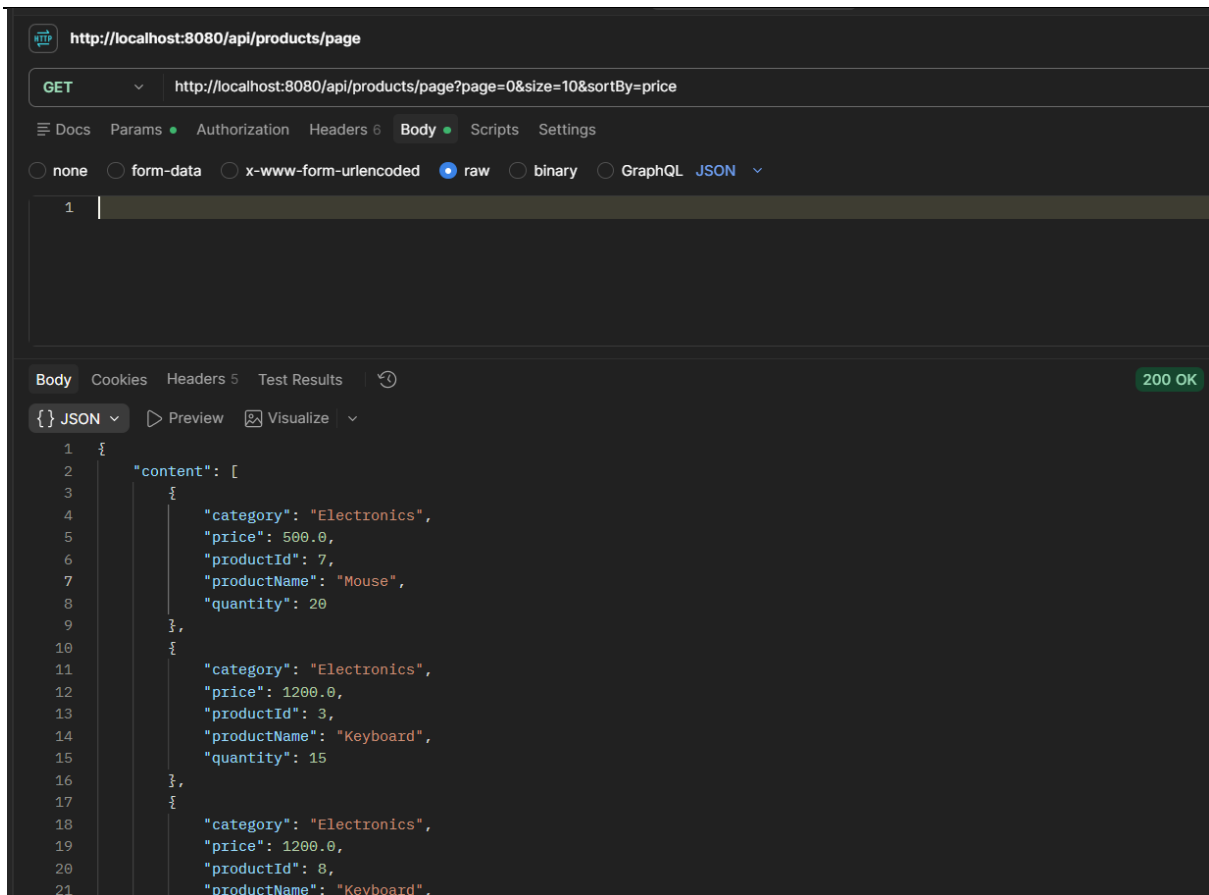
POST <http://localhost:8080/api/products>



4. GET API – Verify Added Product

The **GET API** is used to verify that the newly added product has been successfully stored in the database.

GET <http://localhost:8080/api/products/page?page=0&size=10&sortBy=price>



http://localhost:8080/api/products/page

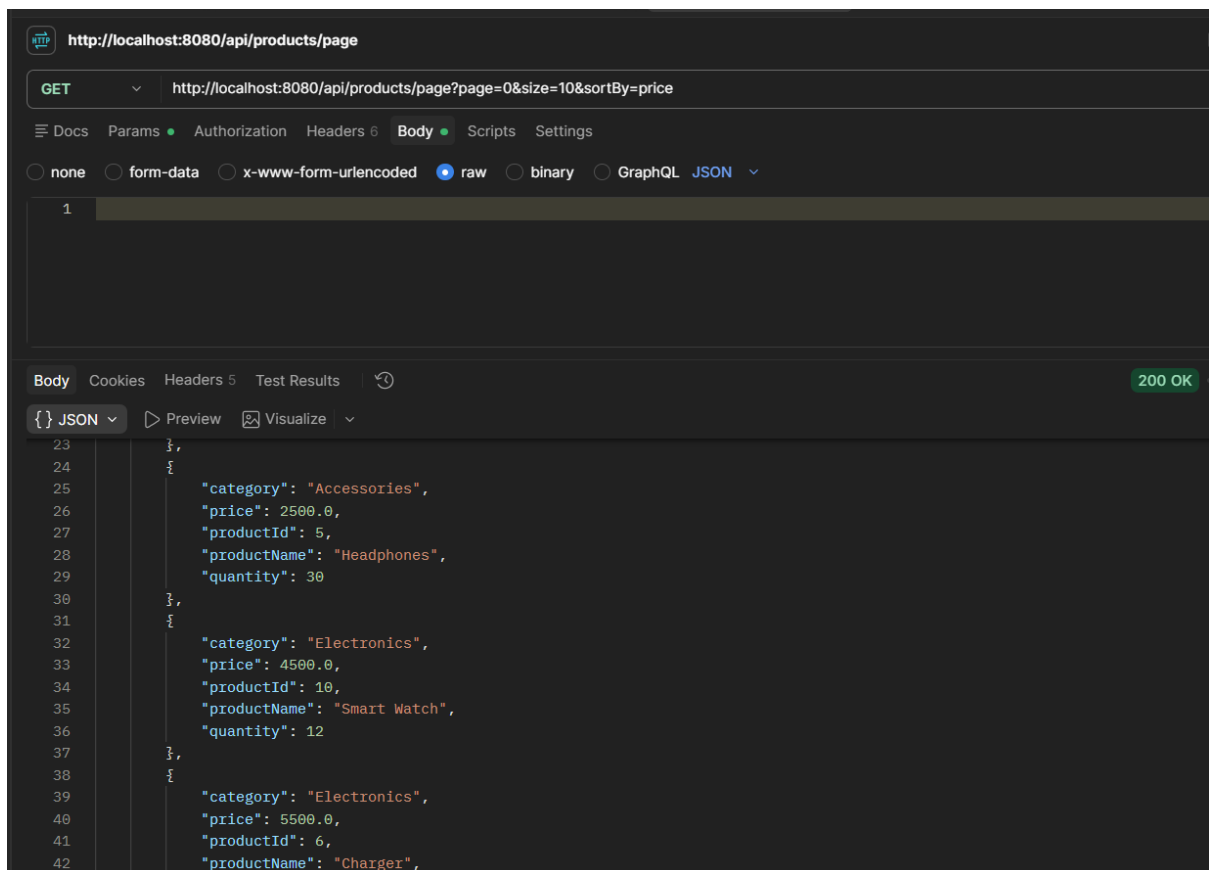
GET http://localhost:8080/api/products/page?page=0&size=10&sortBy=price

Docs Params Authorization Headers 6 Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

Body Cookies Headers 5 Test Results 200 OK

```
{
  "content": [
    {
      "category": "Electronics",
      "price": 500.0,
      "productId": 7,
      "productName": "Mouse",
      "quantity": 20
    },
    {
      "category": "Electronics",
      "price": 1200.0,
      "productId": 3,
      "productName": "Keyboard",
      "quantity": 15
    },
    {
      "category": "Electronics",
      "price": 1200.0,
      "productId": 8,
      "productName": "Keyboard",
      "quantity": 15
    }
  ]
}
```



http://localhost:8080/api/products/page

GET http://localhost:8080/api/products/page?page=0&size=10&sortBy=price

Docs Params Authorization Headers 6 Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

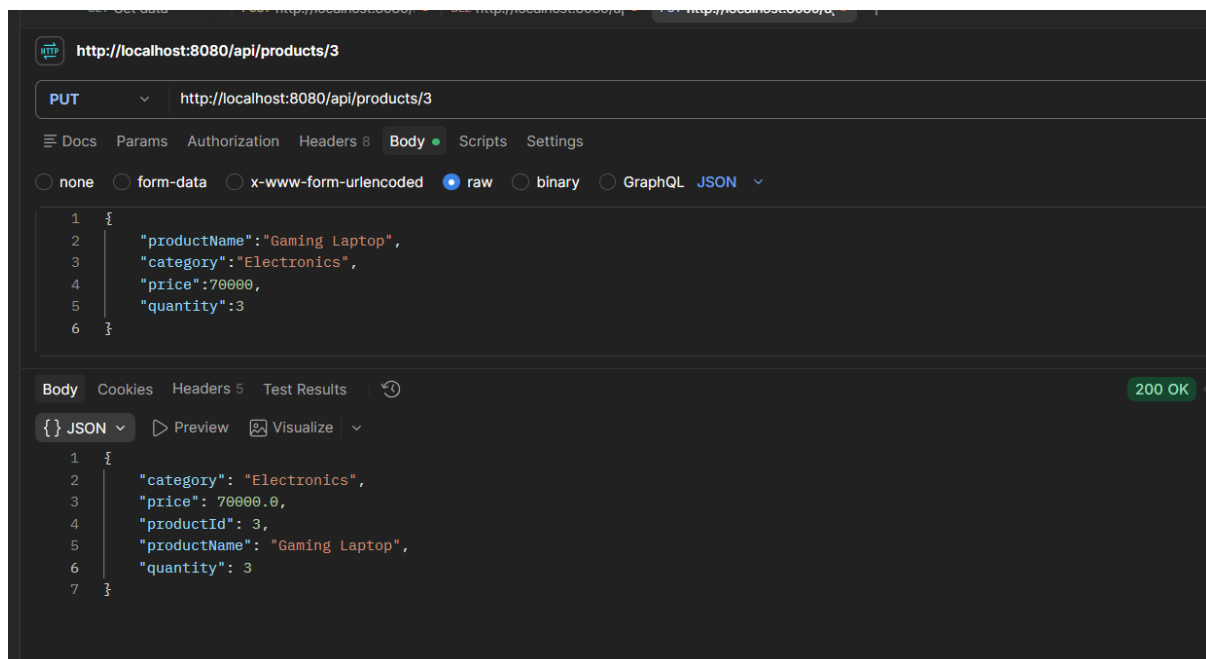
Body Cookies Headers 5 Test Results 200 OK

```
{
  "category": "Accessories",
  "price": 2500.0,
  "productId": 5,
  "productName": "Headphones",
  "quantity": 30
},
{
  "category": "Electronics",
  "price": 4500.0,
  "productId": 10,
  "productName": "Smart Watch",
  "quantity": 12
},
{
  "category": "Electronics",
  "price": 5500.0,
  "productId": 6,
  "productName": "Charger",
  "quantity": 10
}
```


5. PUT API – Update Product

The **PUT API** is used to modify the details of an existing product. The Product ID is specified in the URL, while the updated values are provided in the request body.

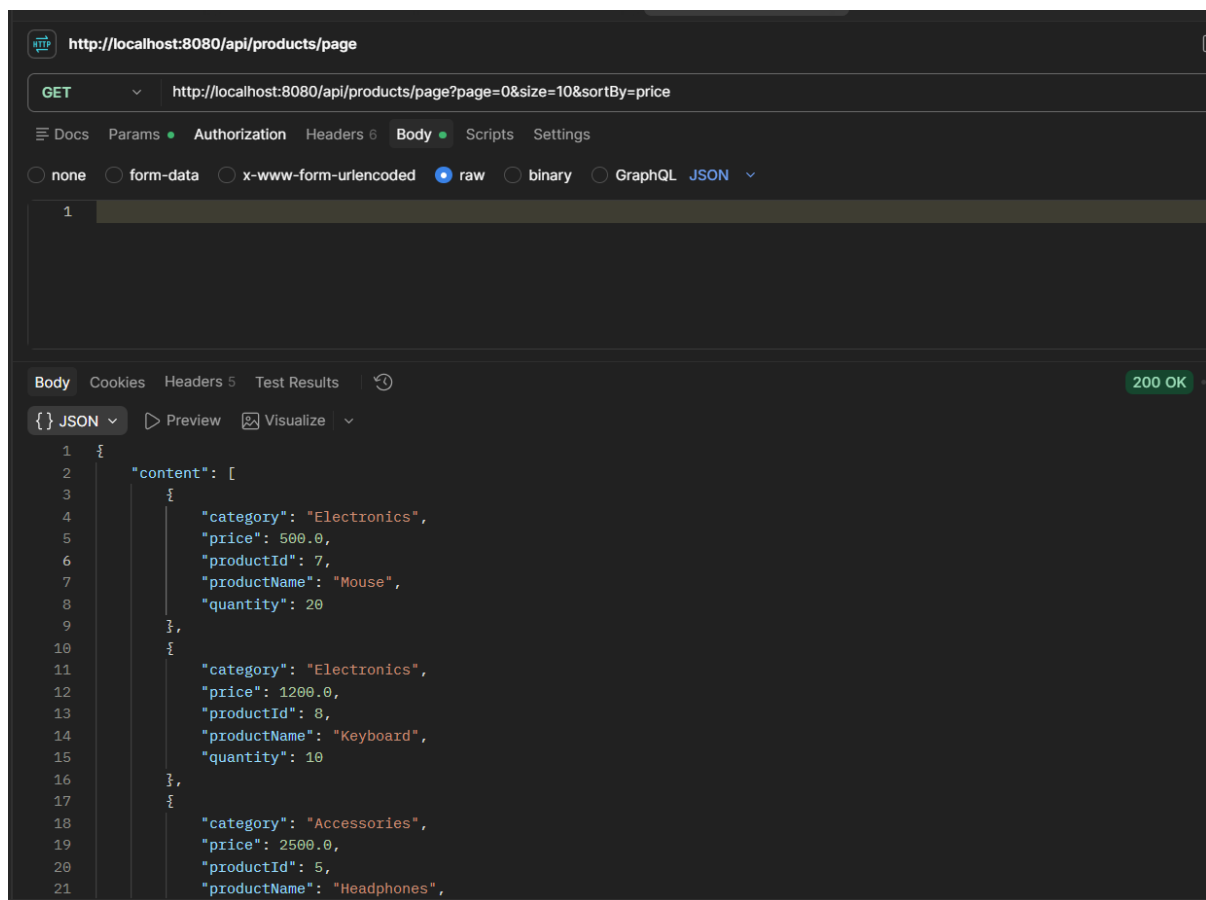
PUT <http://localhost:8080/api/products/3>



6. GET API – Verify Updated Product

The **GET API** is used to verify that the updated product details are reflected correctly in the database.

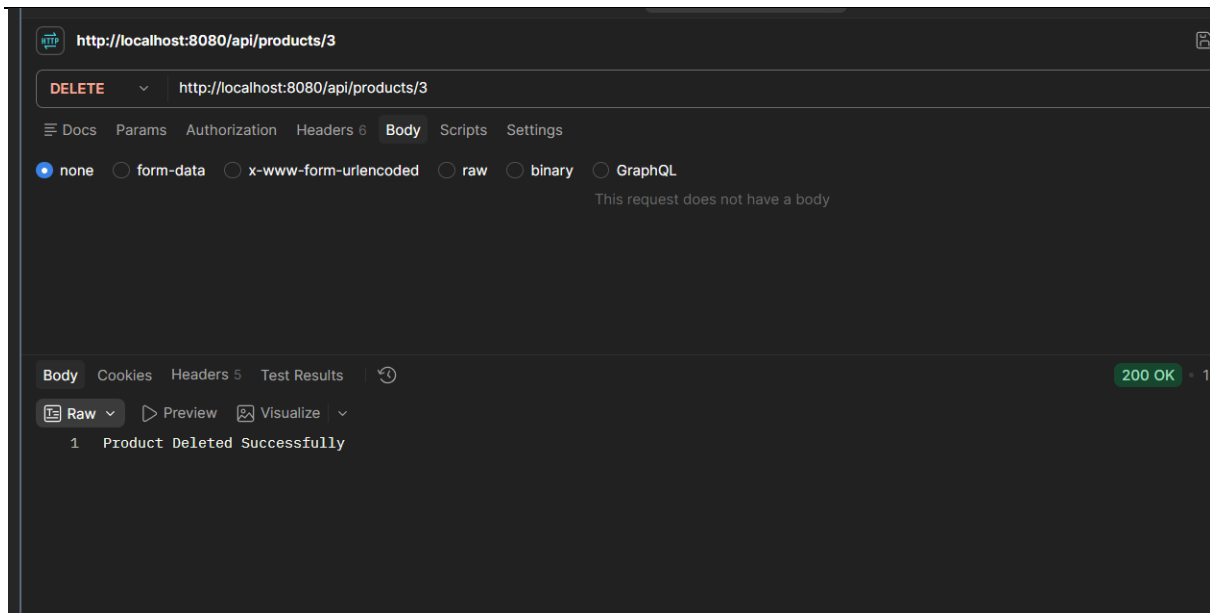
GET <http://localhost:8080/api/products/page?page=0&size=10&sortBy=price>



7. DELETE API – Delete Product

The **DELETE API** removes the selected product from the database using its Product ID.

DELETE <http://localhost:8080/api/products/3>

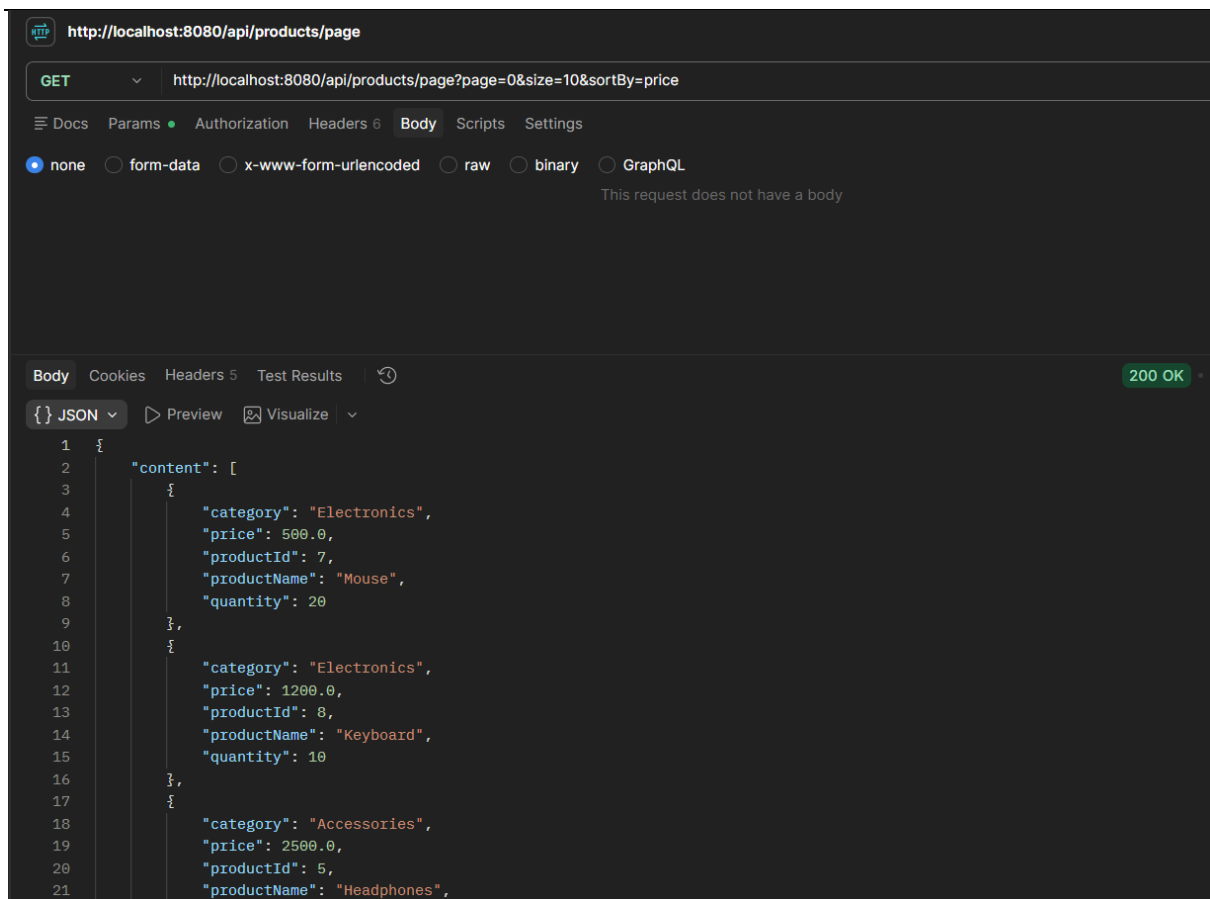


8. GET API – Verify Deleted Product

The **GET API** is used to verify that the deleted product no longer exists in the database. The total number of products decreases after deletion.

GET <http://localhost:8080/api/products/page?page=0&size=10&sortBy=price>

product ID 3 removed...



DATABASE OUTPUT:-

Server: MySQL:3306 » Database: productdb » Table: product

[Browse](#) [Structure](#) [SQL](#) [Search](#) [Insert](#) [Export](#) [Import](#) [Privileges](#)

✓ Showing rows 0 - 7 (8 total, Query took 0.0139 seconds.)

`SELECT * FROM `product``

☐ Profiling [\[Edit inline \]](#) [\[Edit \]](#) [\[Explain SQL \]](#) [\[Create PHP code \]](#) [\[Refresh \]](#)

☐ Show all | Number of rows: Filter rows: Sort by key:

Extra options

		product_id	category	price	product_name	quantity
☐	Edit Copy Delete	2	Electronics	70000	Gaming Laptop	3
☐	Edit Copy Delete	4	Electronics	55000	Laptop	10
☐	Edit Copy Delete	5	Accessories	2500	Headphones	30
☐	Edit Copy Delete	6	Electronics	5500	Charger	5
☐	Edit Copy Delete	7	Electronics	500	Mouse	20
☐	Edit Copy Delete	8	Electronics	1200	Keyboard	10
☐	Edit Copy Delete	9	Electronics	15000	Monitor	4
☐	Edit Copy Delete	10	Electronics	4500	Smart Watch	12

CH 6 BUILDING RESTFUL MICROSERVICES

Practical No. 9: Building a RESTful Microservice using @RestController and ResponseEntity

Objective:

To design and build a RESTful microservice endpoint using @RestController, mapping HTTP verbs to CRUD

operations, and using ResponseEntity to control the HTTP status code and headers returned to the client — the

core REST-building technique introduced in this chapter.

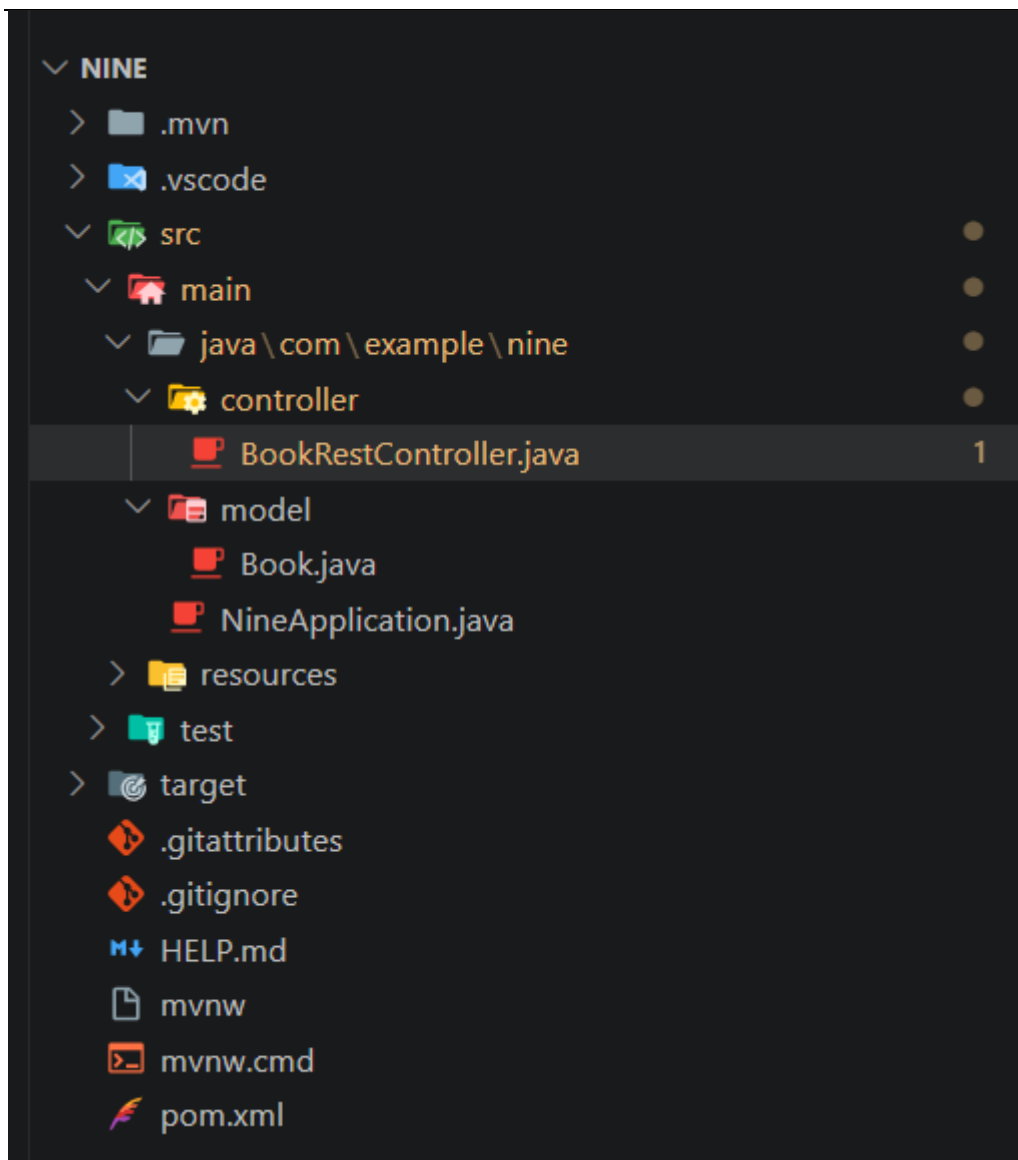
Class Name(s) with Attributes:

Book (POJO)

- id : int
- title : String
- author : String
- price : double

BookRestController (@RestController @RequestMapping("/api/books"))

- Methods return ResponseEntity<Book> / ResponseEntity<List<Book>>



In this practical we don't use MySql And JPA.

Book.java(Model Folder)

```
package com.example.nine.model;

public class Book {

    private int id;
    private String title;
    private String author;
    private double price;

    public Book() {
    }

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getTitle() {
        return title;
    }

    public void setTitle(String title) {
        this.title = title;
    }
}
```



```
}

public String getAuthor() {
    return author;
}

public void setAuthor(String author) {
    this.author = author;
}

public double getPrice() {
    return price;
}

public void setPrice(double price) {
    this.price = price;
}
}
```

BookRestController.java(Rest Controller)

```
package com.example.nine.controller;

import java.util.ArrayList;
import java.util.List;

import org.springframework.http.HttpStatus;
```

```
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import com.example.nine.model.Book;

@RestController
@RequestMapping("/api/books")
public class BookRestController {

    private List<Book> books = new ArrayList<>();

    // POST
    @PostMapping
    public ResponseEntity<Book> addBook(@RequestBody Book book) {
        book.setId(books.size() + 1);
        books.add(book);
        return ResponseEntity
            .status(HttpStatus.CREATED)
            .body(book);
    }

    // GET ALL
    @GetMapping
    public ResponseEntity<List<Book>> getAllBooks() {
        return ResponseEntity.ok(books);
    }

    // GET BY ID
```

```
@GetMapping("/{id}")

public ResponseEntity<Book> getBook(@PathVariable int id) {

    for (Book book : books) {

        if (book.getId() == id) {

            return ResponseEntity.ok(book);

        }

    }

    return ResponseEntity.notFound().build();

}


// UPDATE

@PutMapping("/{id}")

public ResponseEntity<Book> updateBook(

    @PathVariable int id,

    @RequestBody Book newBook) {

    for (Book book : books) {

        if (book.getId() == id) {

            book.setTitle(newBook.getTitle());

            book.setAuthor(newBook.getAuthor());

            book.setPrice(newBook.getPrice());

            return ResponseEntity.ok(book);

        }

    }

    return ResponseEntity.notFound().build();

}
```

```
}

// DELETE
@DeleteMapping("/{id}")
public ResponseEntity<String> deleteBook(@PathVariable int id) {

    for (Book book : books) {

        if (book.getId() == id) {
            books.remove(book);
            return ResponseEntity.ok("Book Deleted Successfully");
        }
    }

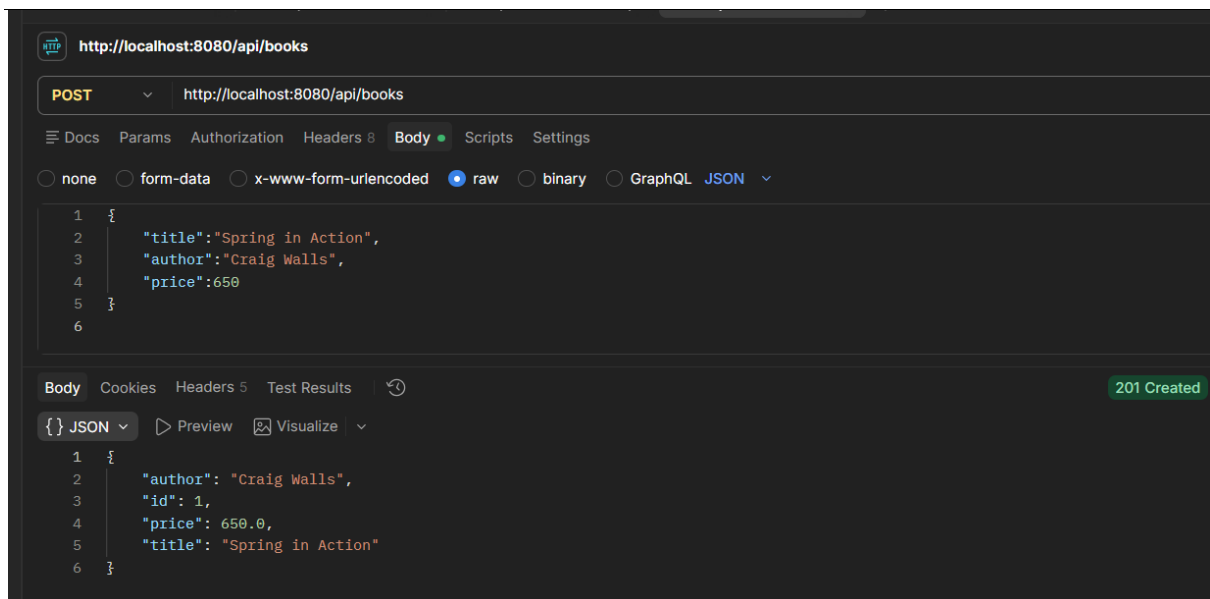
    return ResponseEntity.notFound().build();
}
}
```

API'S TESTING USING POSTMAN

1: POST API – Add Book

The **POST API** is used to add a new book into the application. The book details are sent in the request body as JSON. The application automatically generates the Book ID and stores the book in memory.

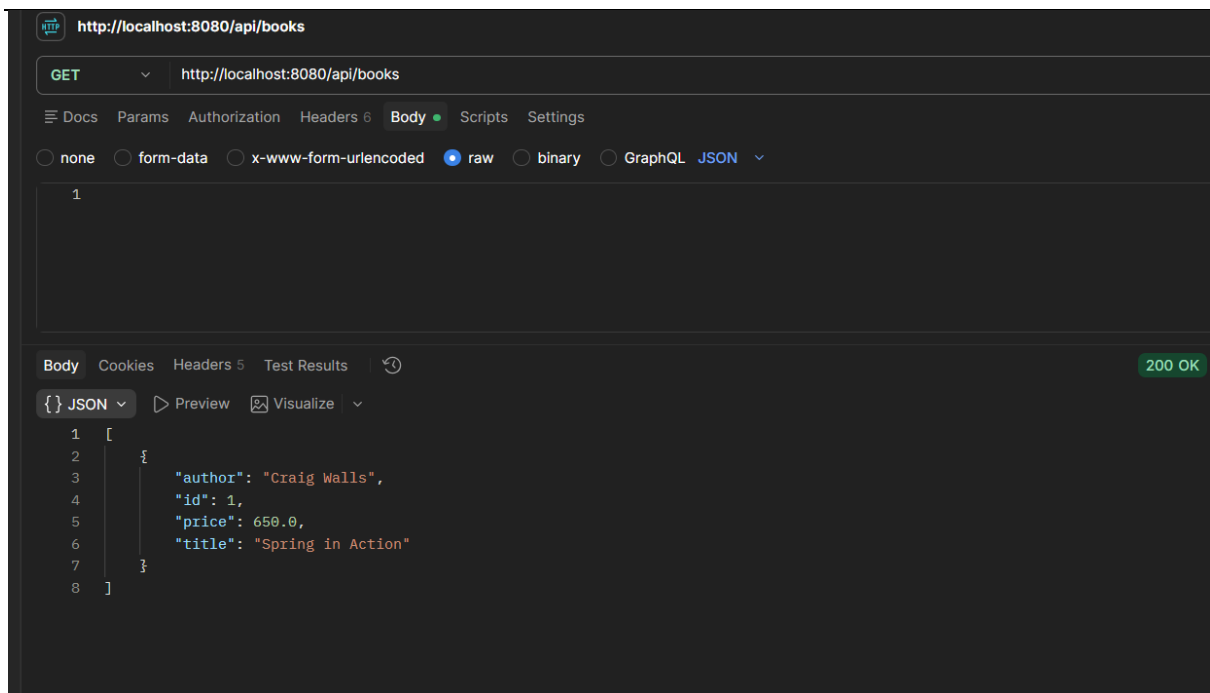
POST <http://localhost:8080/api/books>



2: GET API – Get All Books

The **GET API** is used to retrieve all books stored in the application.

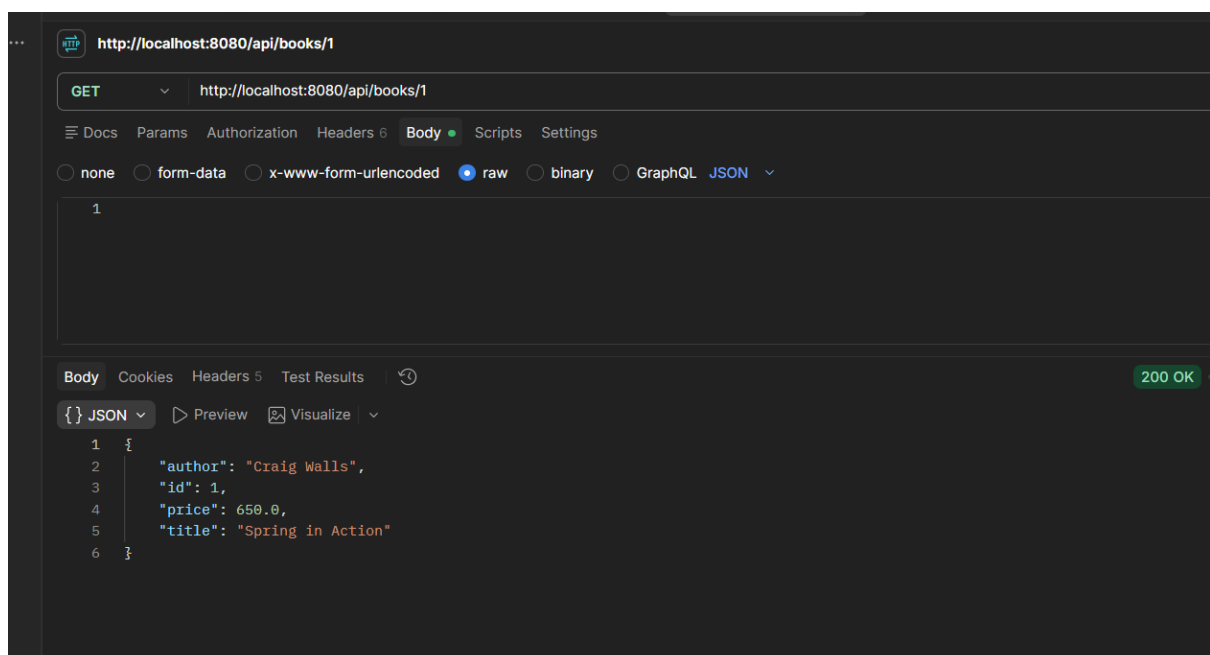
GET <http://localhost:8080/api/books>



3: GET API – Get Book by ID

The **GET API** is used to retrieve the details of a specific book using its unique Book ID.

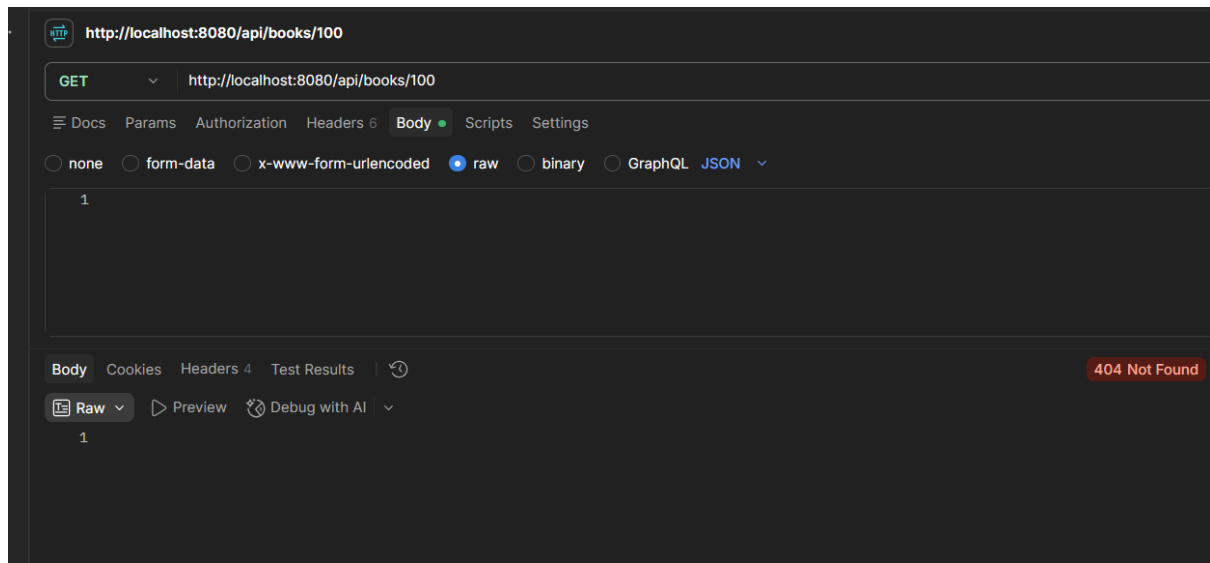
GET <http://localhost:8080/api/books/1>



4: GET API – Book Not Found

This API demonstrates **404 Not Found**. If the Book ID does not exist, the application returns **404 Not Found**.

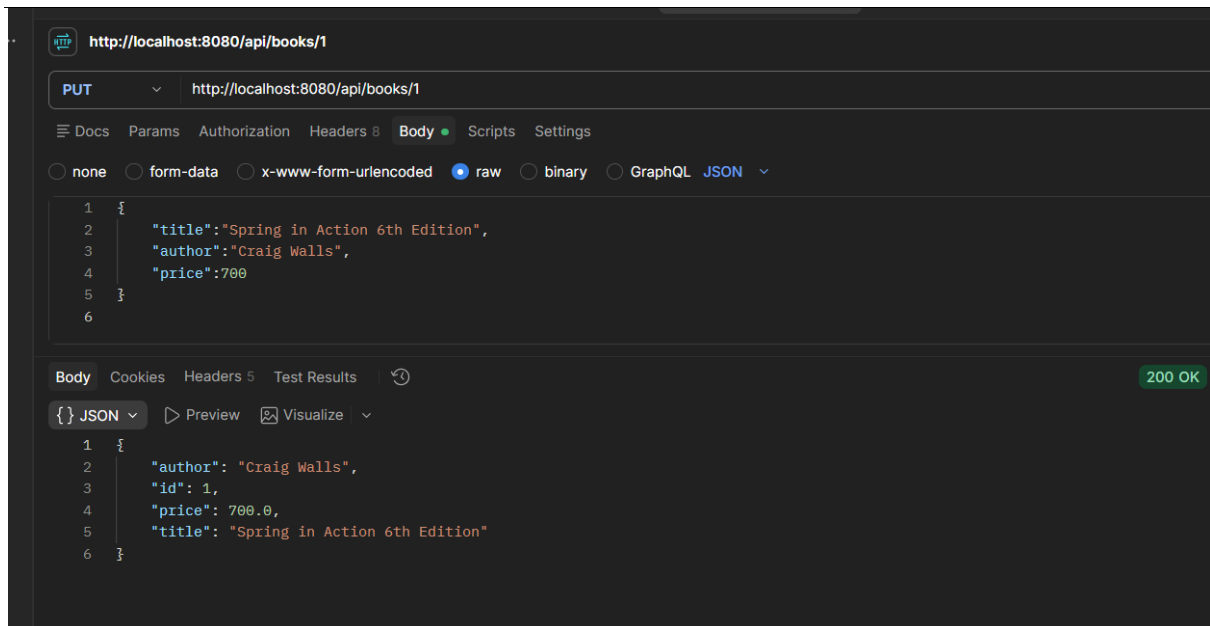
GET <http://localhost:8080/api/books/100>



5: PUT API – Update Book

The **PUT API** is used to update an existing book. The Book ID is provided in the URL, while the updated details are sent in the request body.

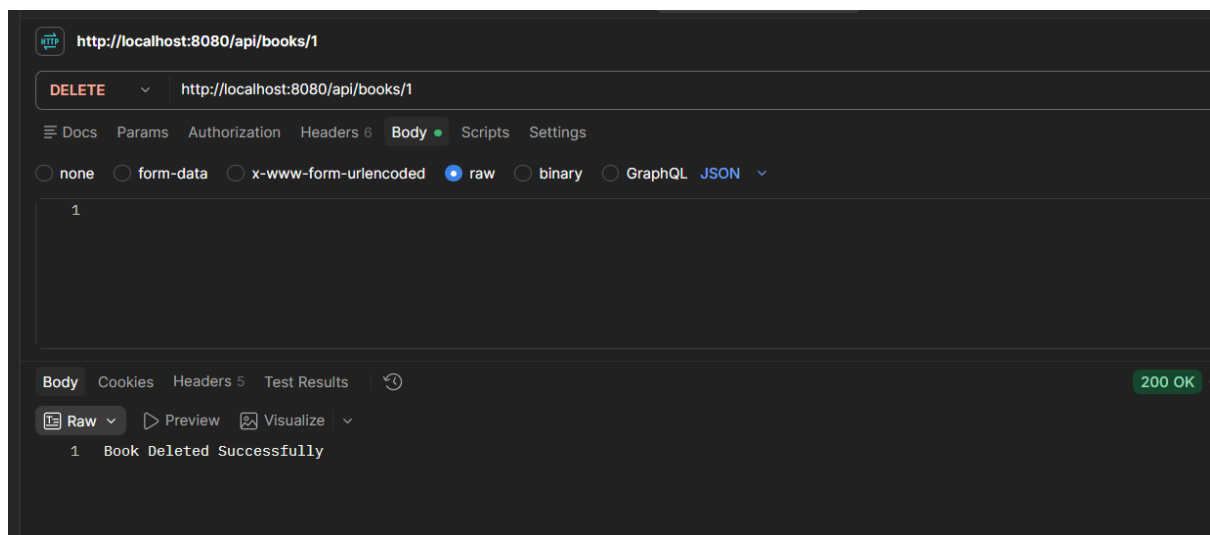
PUT <http://localhost:8080/api/books/1>



6: DELETE API – Delete Book

The **DELETE API** is used to remove a book from the application using its Book ID.

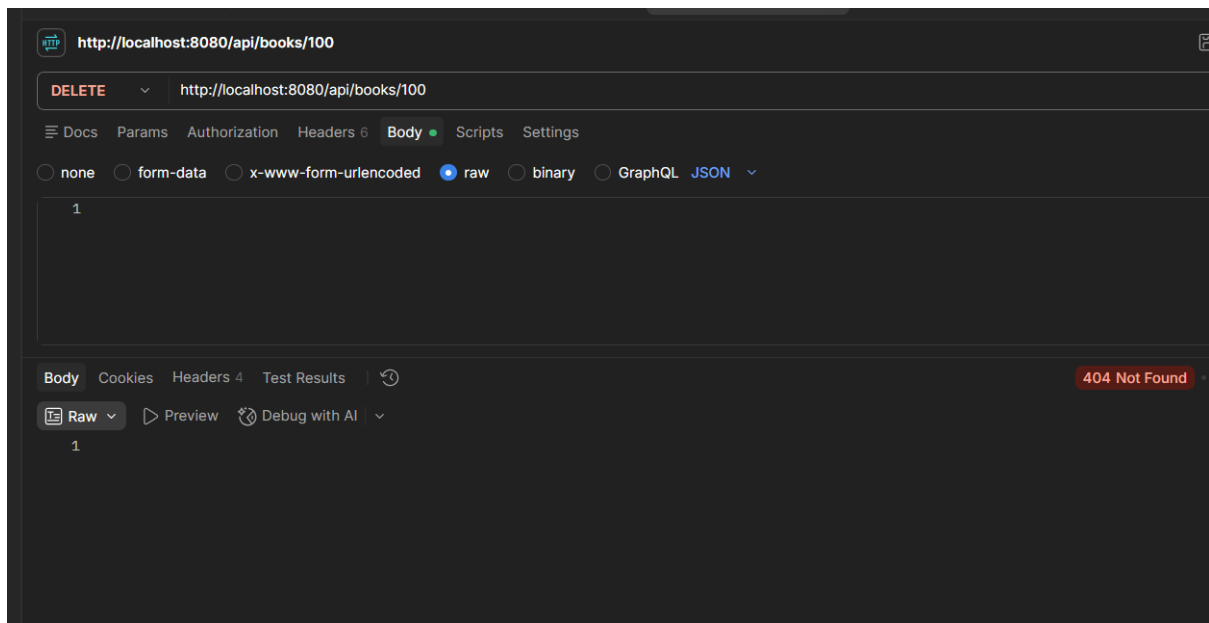
DELETE <http://localhost:8080/api/books/1>



7: DELETE API – Book Not Found

This API demonstrates **404 Not Found** when attempting to delete a Book ID that does not exist.

DELETE <http://localhost:8080/api/books/100>



Practical No. 10: Layered Microservice Architecture **- RestController + Service + JPA Repository**

Objective:

To structure a RESTful microservice into the standard Controller–Service–Repository layers recommended in this

chapter, keeping business logic out of the controller and persistence access out of the service.

Class Name(s) with Attributes:

Product (@Entity, reused)

- productId, productName, category, price, quantity

ProductService (interface)

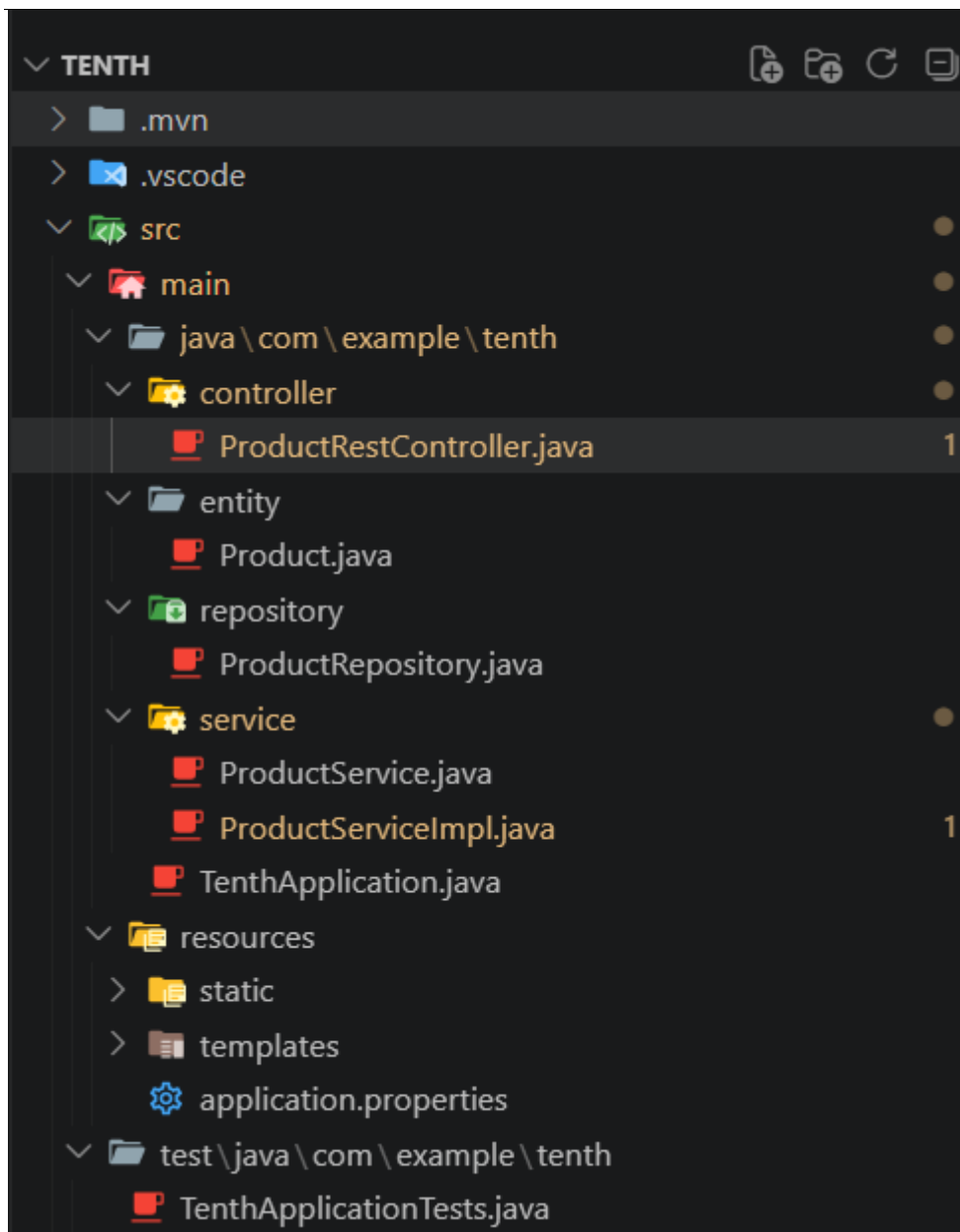
- CRUD method signatures

ProductServiceImpl (@Service)

- productRepository : ProductRepository

ProductRestController (@RestController)

- productService : ProductService



Dependencies

- Spring Web
- Spring Data JPA
- MySQL Driver

Database

Database Name: tenth

Table: product

Product.java (Entity)

```
package com.example.tenth.entity;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;

@Entity
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer productId;

    private String productName;
    private String category;
    private double price;
    private int quantity;

    public Product() {
    }
}
```

```
public Integer getProductId() {  
    return productId;  
}  
  
public void setProductId(Integer productId) {  
    this.productId = productId;  
}  
  
public String getProductName() {  
    return productName;  
}  
  
public void setProductName(String productName) {  
    this.productName = productName;  
}  
  
public String getCategory() {  
    return category;  
}  
  
public void setCategory(String category) {  
    this.category = category;  
}  
  
public double getPrice() {  
    return price;  
}
```

```
public void setPrice(double price) {  
    this.price = price;  
}  
  
public int getQuantity() {  
    return quantity;  
}  
  
public void setQuantity(int quantity) {  
    this.quantity = quantity;  
}  
}
```

ProductRepository.java(Repository File)

```
package com.example.tenth.repository;  
  
import org.springframework.data.jpa.repository.JpaRepository;  
  
import com.example.tenth.entity.Product;  
  
public interface ProductRepository extends JpaRepository<Product, Integer> {  
  
}
```

ProductService.java (Service File)

```
package com.example.tenth.service;

import java.util.List;

import com.example.tenth.entity.Product;

public interface ProductService {

    Product save(Product product);

    List<Product> getAll();

    Product getByld(Integer id);

    Product update(Integer id, Product product);

    String delete(Integer id);

}
```

ProductServiceImpl.java(Service file)

```
package com.example.tenth.service;

import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

import com.example.tenth.entity.Product;
import com.example.tenth.repository.ProductRepository;

@Service
public class ProductServiceImpl implements ProductService {

    @Autowired
    ProductRepository repository;

    @Override
    public Product save(Product product) {
        return repository.save(product);
    }

    @Override
    public List<Product> getAll() {
        return repository.findAll();
    }
}
```



```
@Override
public Product getById(Integer id) {
    return repository.findById(id).orElse(null);
}

@Override
public Product update(Integer id, Product product) {

    product.setProductId(id);

    return repository.save(product);
}

@Override
public String delete(Integer id) {

    repository.deleteById(id);

    return "Product Deleted Successfully";
}
}
```

ProductRestController.java(Controller File)

```
package com.example.tenth.controller;

import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

import com.example.tenth.entity.Product;
import com.example.tenth.service.ProductService;

@RestController
@RequestMapping("/api/products")
public class ProductRestController {

    @Autowired
    ProductService service;

    @PostMapping
    public Product save(@RequestBody Product product) {

        return service.save(product);
    }

    @GetMapping
    public List<Product> getAll() {
```

```
        return service.getAll();
    }

    @GetMapping("/{id}")
    public Product getById(@PathVariable Integer id) {

        return service.getById(id);
    }

    @PutMapping("/{id}")
    public Product update(@PathVariable Integer id,
                          @RequestBody Product product) {
        return service.update(id, product);
    }

    @DeleteMapping("/{id}")
    public String delete(@PathVariable Integer id) {

        return service.delete(id);
    }
}
```

Application.properties

```
spring.application.name=tenth
spring.datasource.url=jdbc:mysql://localhost:3306/ten
spring.datasource.username=root
spring.datasource.password=

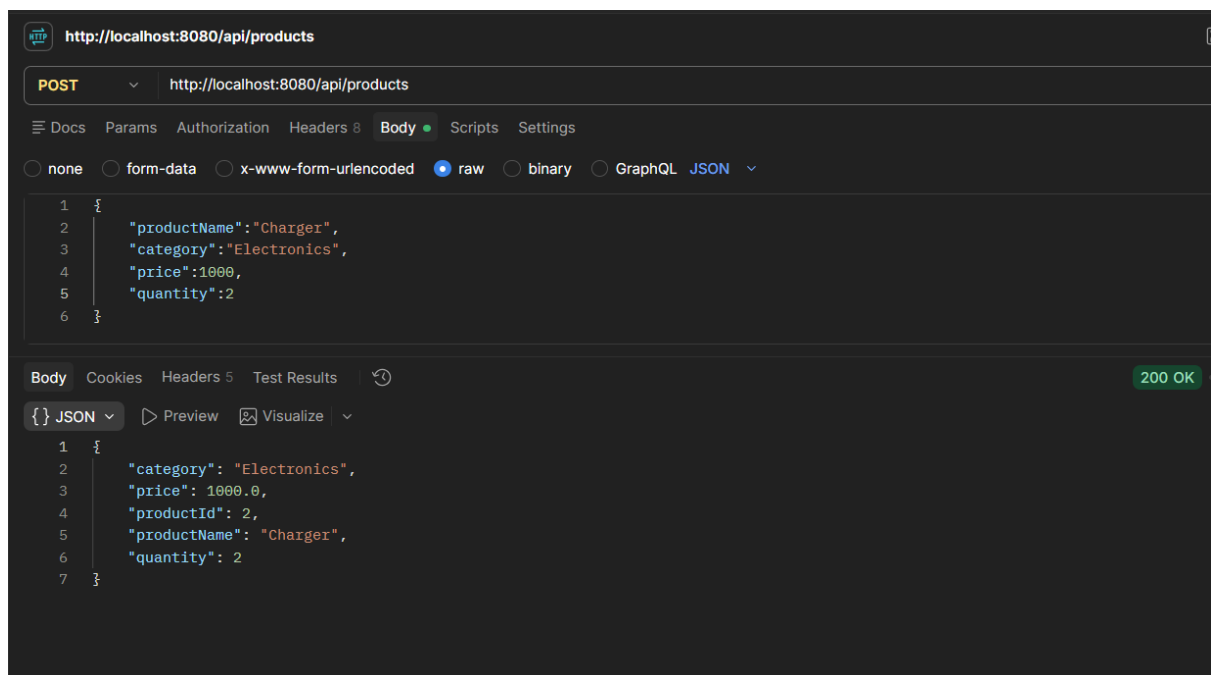
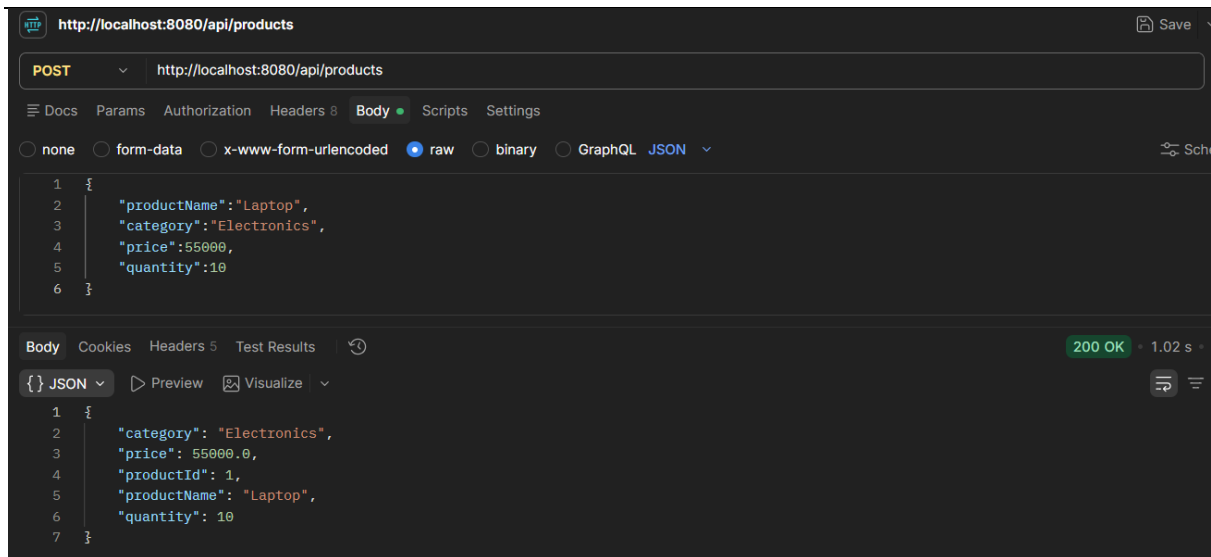
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

API'S TESTING USING POSTMAN

1: POST API – Add Product

The **POST API** is used to add a new product into the MySQL database. The product details are sent in JSON format. The request is received by the **ProductRestController**, processed by the **ProductService**, and finally stored in the database using the **ProductRepository**.

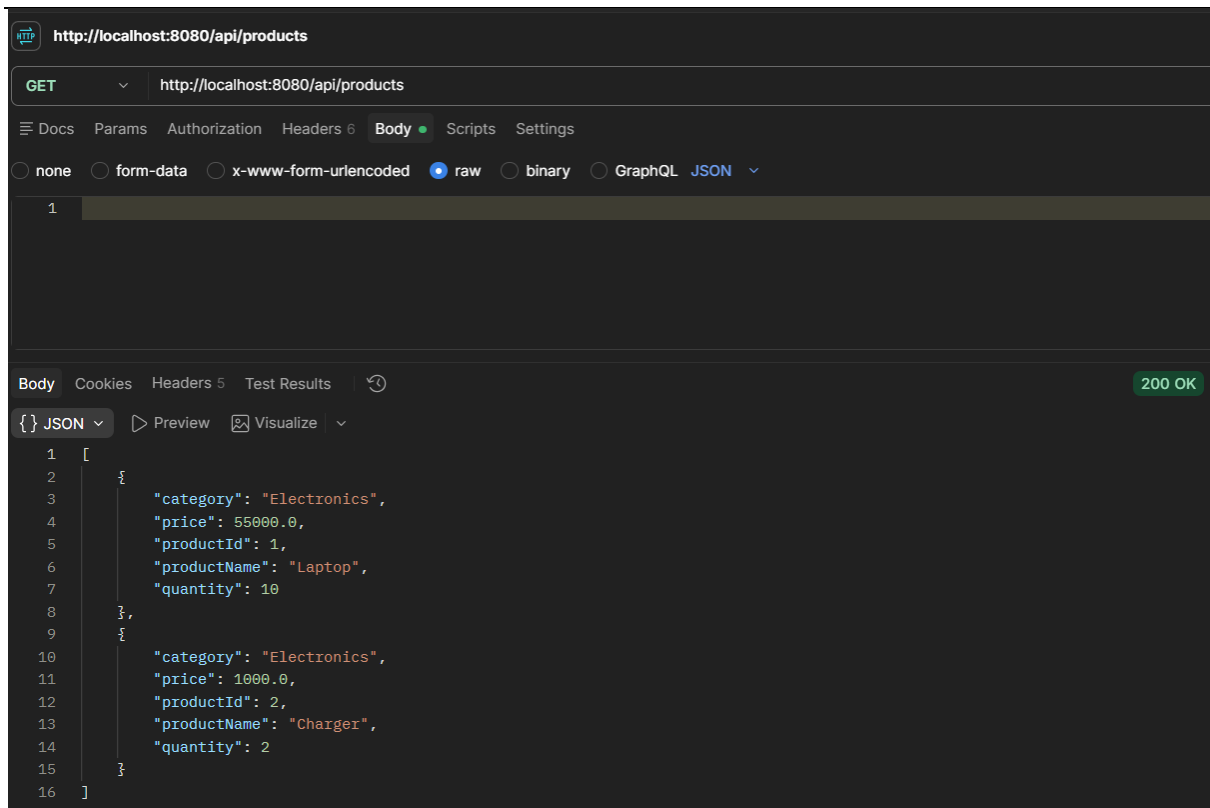
POST <http://localhost:8080/api/products>



2: GET API – Display All Products

The **GET API** is used to retrieve all product records stored in the database. The controller calls the service layer, and the service fetches all records from the repository.

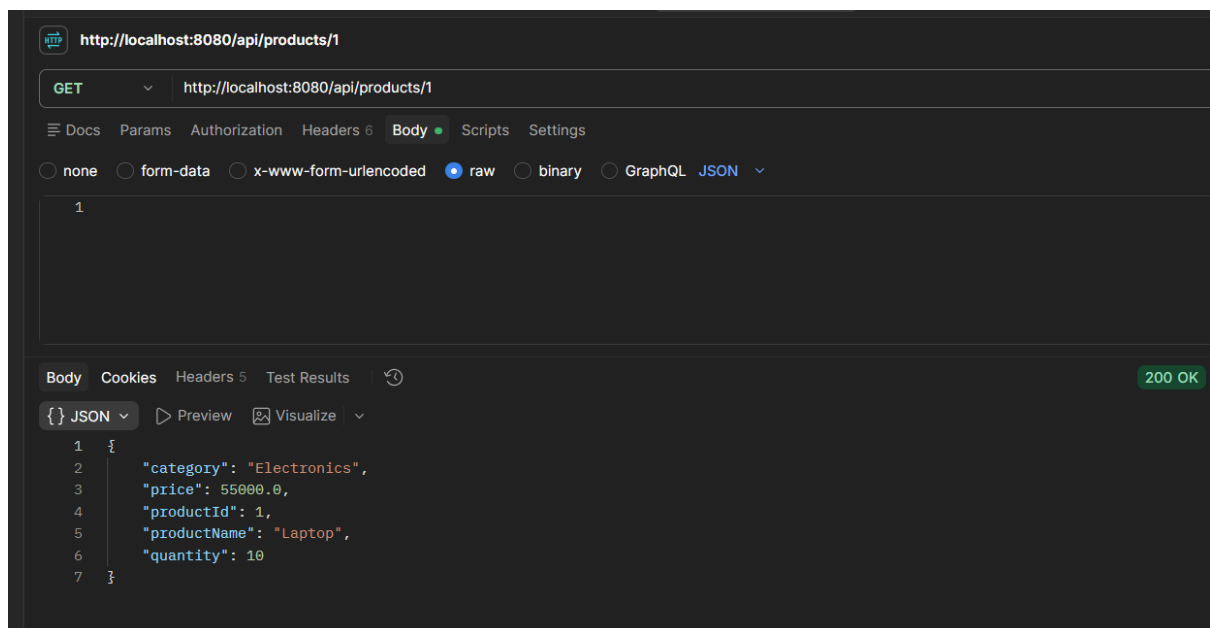
GET <http://localhost:8080/api/products>



3: GET API – Get Product by ID

The **GET API** is used to retrieve a specific product using its Product ID. The service searches the database and returns the matching product.

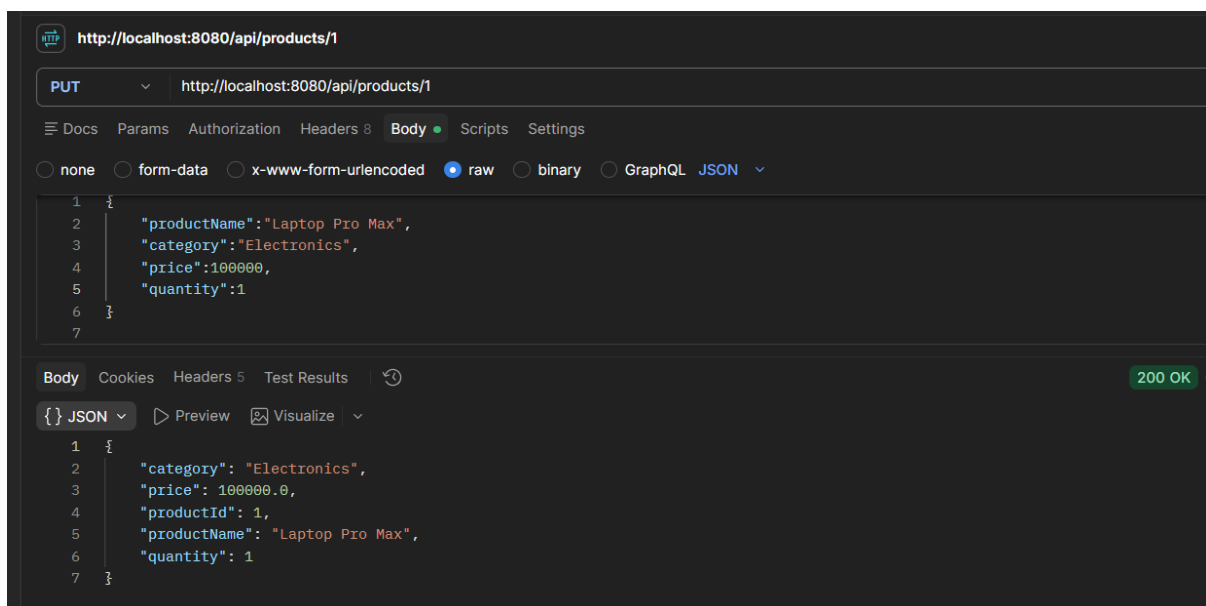
GET <http://localhost:8080/api/products/1>



4: PUT API – Update Product

The **PUT API** is used to update an existing product. The Product ID is passed in the URL while the updated product details are provided in the request body. The service updates the record and saves it back into the database.

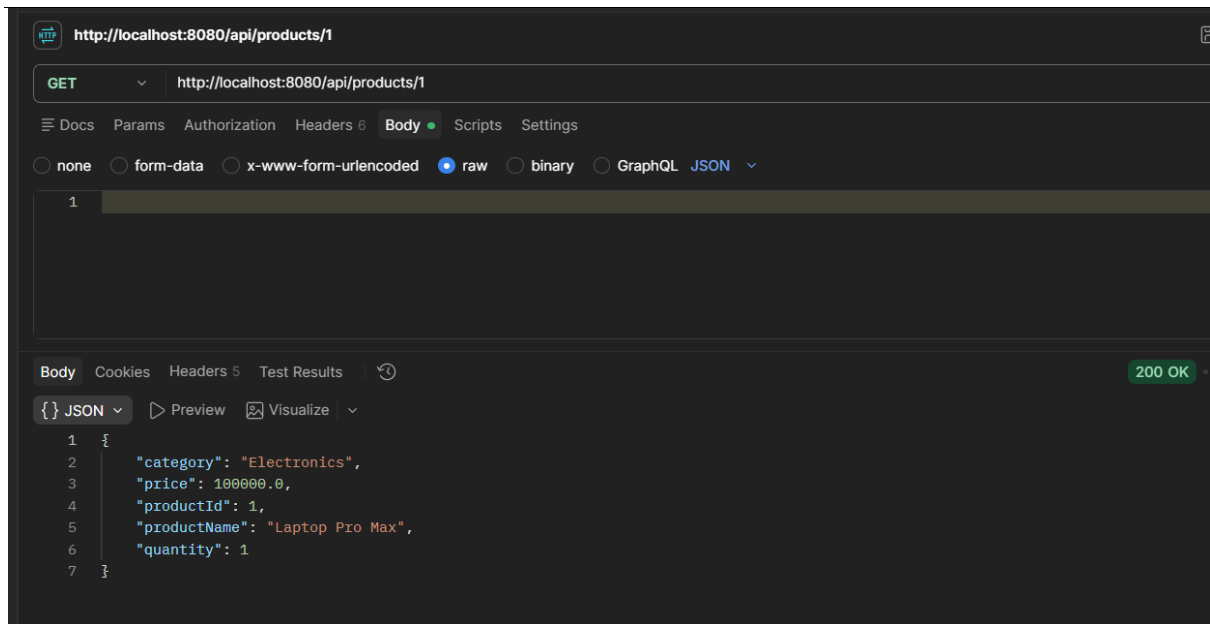
PUT <http://localhost:8080/api/products/1>



5: GET API – Verify Updated Product

The **GET API** is used to verify that the product details have been updated successfully.

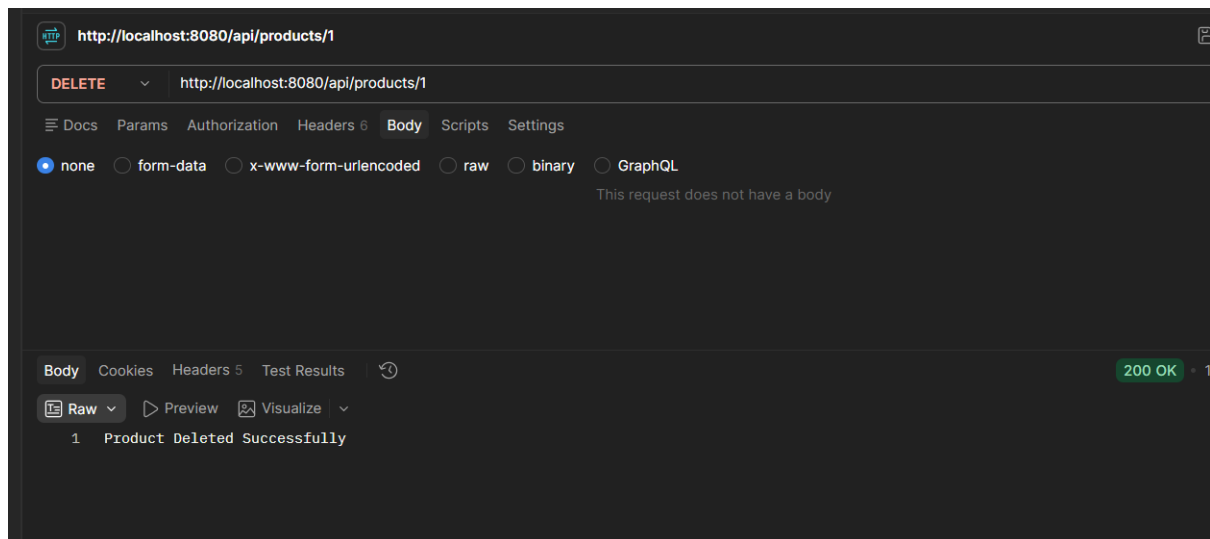
GET <http://localhost:8080/api/products/1>



6: DELETE API – Delete Product

The **DELETE API** is used to remove a product from the database using its Product ID. The request is handled through the Controller, Service, and Repository layers.

DELETE <http://localhost:8080/api/products/1>

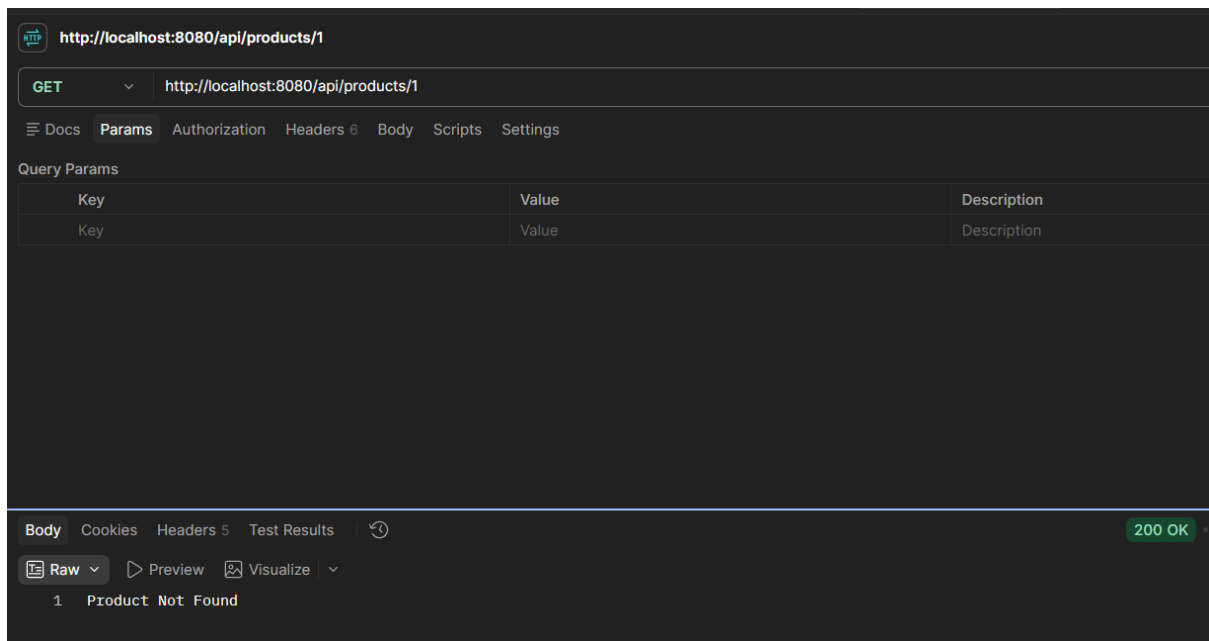


7: GET API – Verify Product Deletion

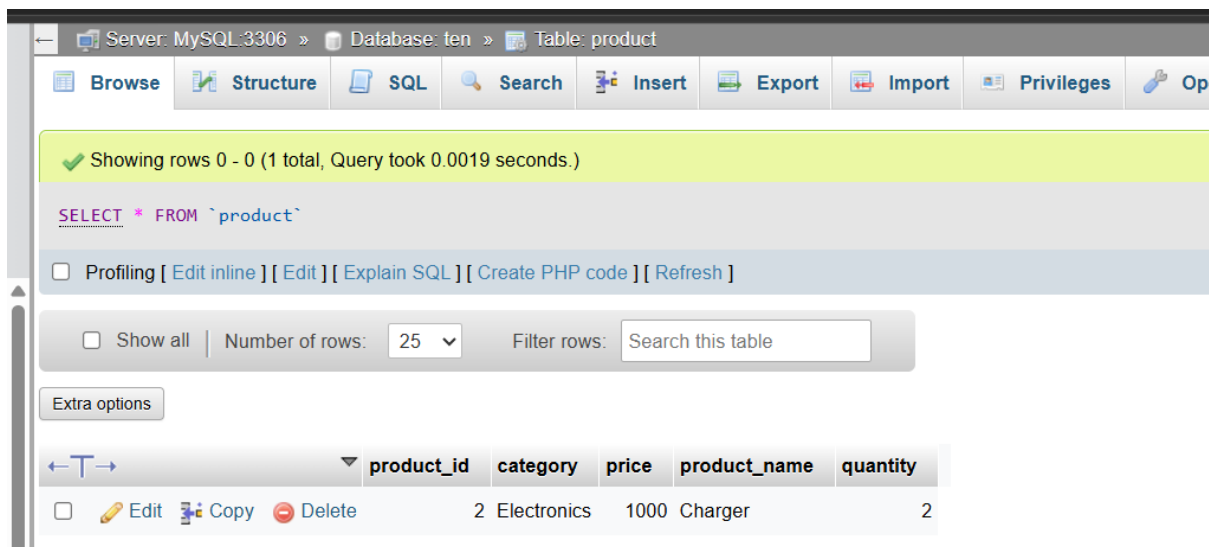
Purpose:

The **GET API** is used to verify that the product has been deleted successfully. Since the product no longer exists, the application returns **null** or **Product Not Found**, depending on your service implementation.

GET <http://localhost:8080/api/products/1>



Database Output:- (First Id removed)



**Practical No. 11: Handling Request Data -
@PathVariable, @RequestParam and @RequestBody**

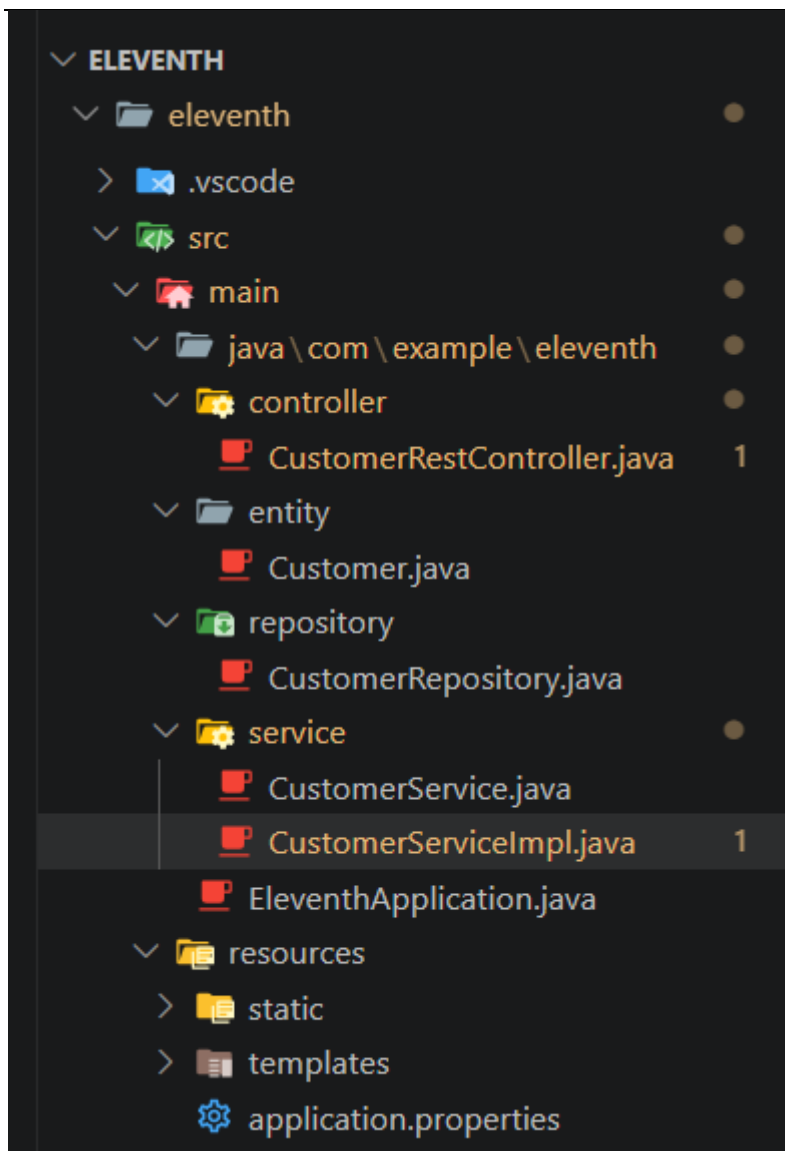
Objective:

To correctly use the three main ways a Spring REST controller receives client input — @PathVariable for identifying a resource, @RequestParam for optional query filters, and @RequestBody for JSON payloads — as covered in the request-handling section of this chapter.

Class Name(s) with Attributes:

Customer (@Entity)

- customerId : int (@Id @GeneratedValue)
- name : String
- city : String
- age : int CustomerRestController (@RestController)
- Method: getById(@PathVariable id)
- Method: search(@RequestParam city, @RequestParam(required=false) minAge)
- Method: create(@RequestBody CustomerDTO)



Dependencies –

Spring Web

Mysql driver

jpa

Customer.java(Entity)

```
package com.example.eleventh.entity;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;

@Entity
public class Customer {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer customerId;

    private String name;

    private String city;

    private int age;

    public Customer() {
    }

    public Integer getCustomerId() {
        return customerId;
    }
}
```

```
}

public void setCustomerId(Integer customerId) {
    this.customerId = customerId;
}

public String getName() {
    return name;
}

public void setName(String name) {
    this.name = name;
}

public String getCity() {
    return city;
}

public void setCity(String city) {
    this.city = city;
}

public int getAge() {
    return age;
}

public void setAge(int age) {
```

```
        this.age = age;
    }
}
```

CustomerService.java(SERVICE LAYER FOR BUSINESS)

```
package com.example.eleventh.service;

import java.util.List;

import com.example.eleventh.entity.Customer;

public interface CustomerService {

    Customer create(Customer customer);

    Customer getByld(Integer id);

    List<Customer> search(String city, int minAge);

    Customer update(Integer id, Customer customer);

    void delete(Integer id);
}
```

CustomerServiceImpl.java(SERVICE LAYER)

```
package com.example.eleventh.service;

import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

import com.example.eleventh.entity.Customer;
import com.example.eleventh.repository.CustomerRepository;

@Service
public class CustomerServiceImpl implements CustomerService {

    @Autowired
    private CustomerRepository repository;

    @Override
    public Customer create(Customer customer) {
        return repository.save(customer);
    }

    @Override
    public Customer getByld(Integer id) {
```

```
        return repository.findById(id).orElse(null);
    }

    @Override
    public List<Customer> search(String city, int minAge) {
        return repository.findByCityAndAgeGreaterThanOrEqualTo(city, minAge);
    }

    @Override
    public Customer update(Integer id, Customer customer) {
        customer.setCustomerId(id);
        return repository.save(customer);
    }

    @Override
    public void delete(Integer id) {
        repository.deleteById(id);
    }
}
```

CustomerRestController.java(Controller File)

```
package com.example.eleventh.controller;

import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
```



```
import org.springframework.web.bind.annotation.*;

import com.example.eleventh.entity.Customer;
import com.example.eleventh.service.CustomerService;

@RestController
@RequestMapping("/api/customers")
public class CustomerRestController {

    @Autowired
    private CustomerService service;

    // POST - @RequestBody
    @PostMapping
    public Customer create(@RequestBody Customer customer) {

        return service.create(customer);
    }

    // GET - @PathVariable
    @GetMapping("/{id}")
    public Customer getByld(@PathVariable Integer id) {

        return service.getByld(id);
    }

    // GET - @RequestParam
```

```
@GetMapping("/search")
public List<Customer> search(
    @RequestParam String city,
    @RequestParam(defaultValue = "0") int minAge) {

    return service.search(city, minAge);
}

// PUT - @PathVariable + @RequestBody
@PutMapping("/{id}")
public Customer update(
    @PathVariable Integer id,
    @RequestBody Customer customer) {

    return service.update(id, customer);
}

// DELETE - @PathVariable
@DeleteMapping("/{id}")
public String delete(@PathVariable Integer id) {

    service.delete(id);

    return "Customer Deleted Successfully";
}
}
```

CustomerRepository.java

```
package com.example.eleventh.repository;

import java.util.List;

import org.springframework.data.jpa.repository.JpaRepository;

import com.example.eleventh.entity.Customer;

public interface CustomerRepository extends JpaRepository<Customer, Integer> {

    List<Customer> findByCityAndAgeGreaterThanOrEqual(

        String city,

        int age

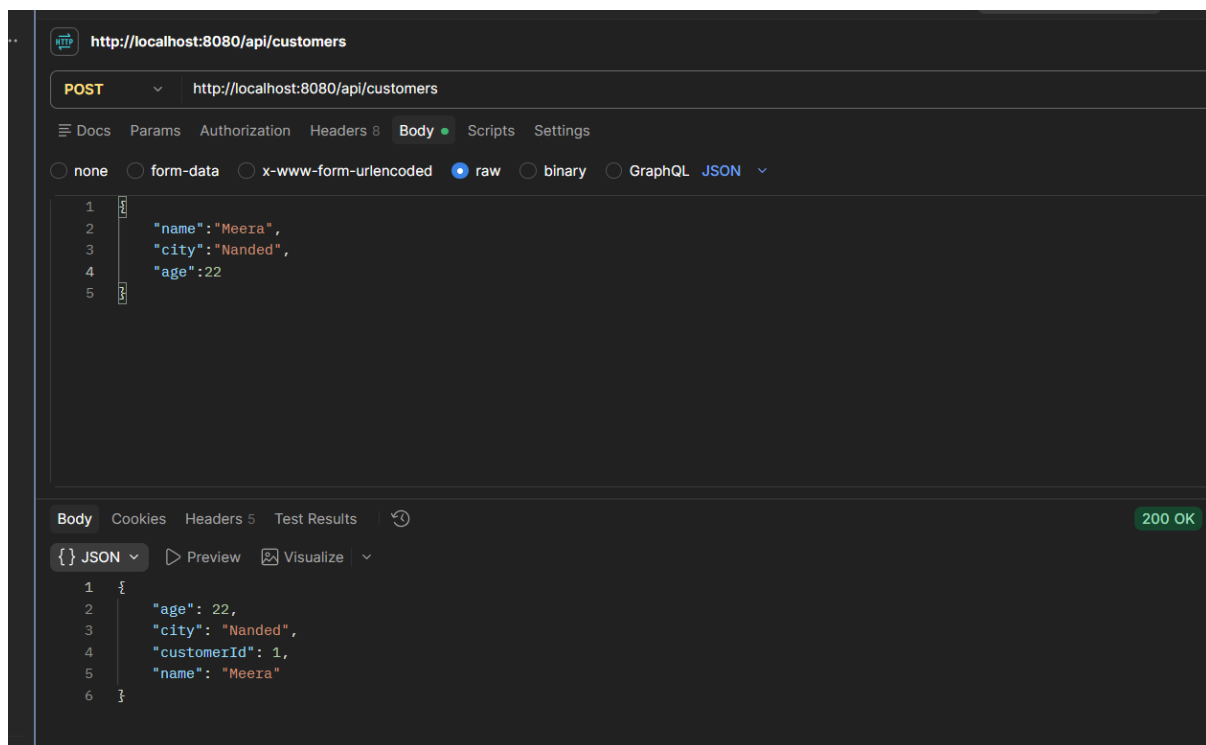
    );

}
```

API'S TESTING USING POSTMAN

1. **POST API – Add Customer (@RequestBody)** :- The POST API is used to create a new customer in the database. The customer details are sent in JSON format using the request body. Spring Boot converts the JSON into a Customer object using @RequestBody, and the service layer stores it in the database.

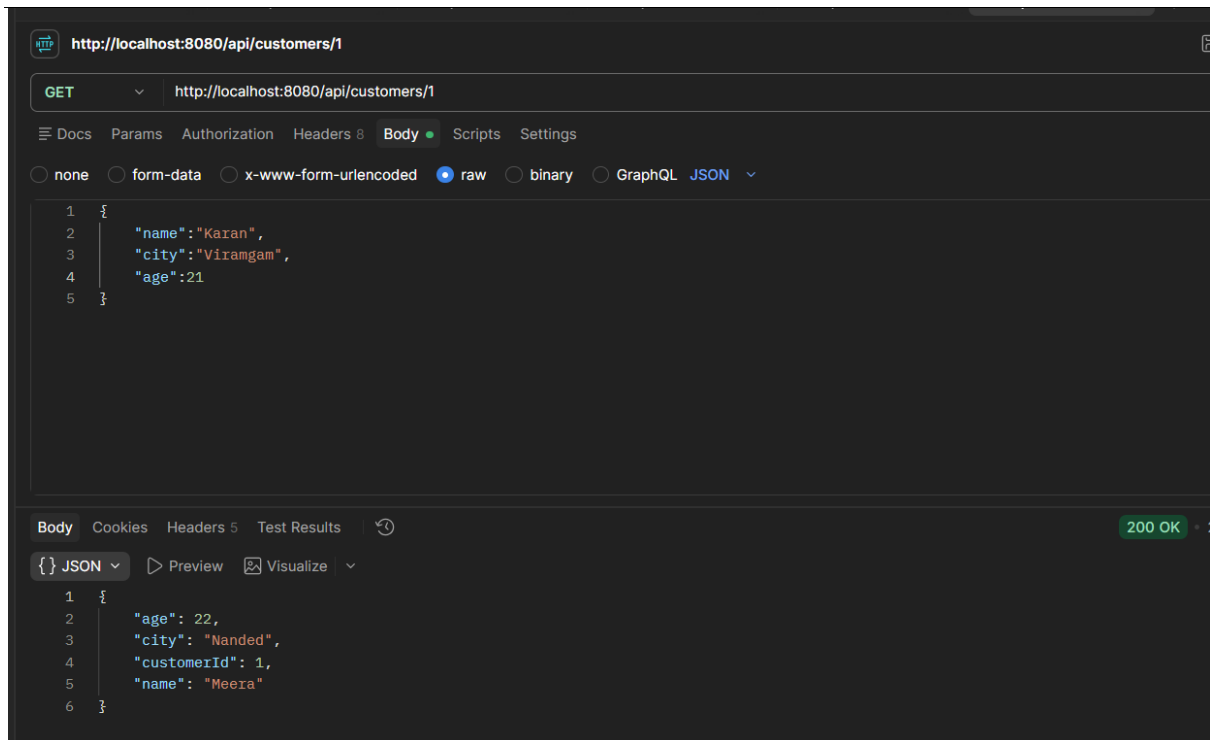
POST <http://localhost:8080/api/customers>



After add one more record..

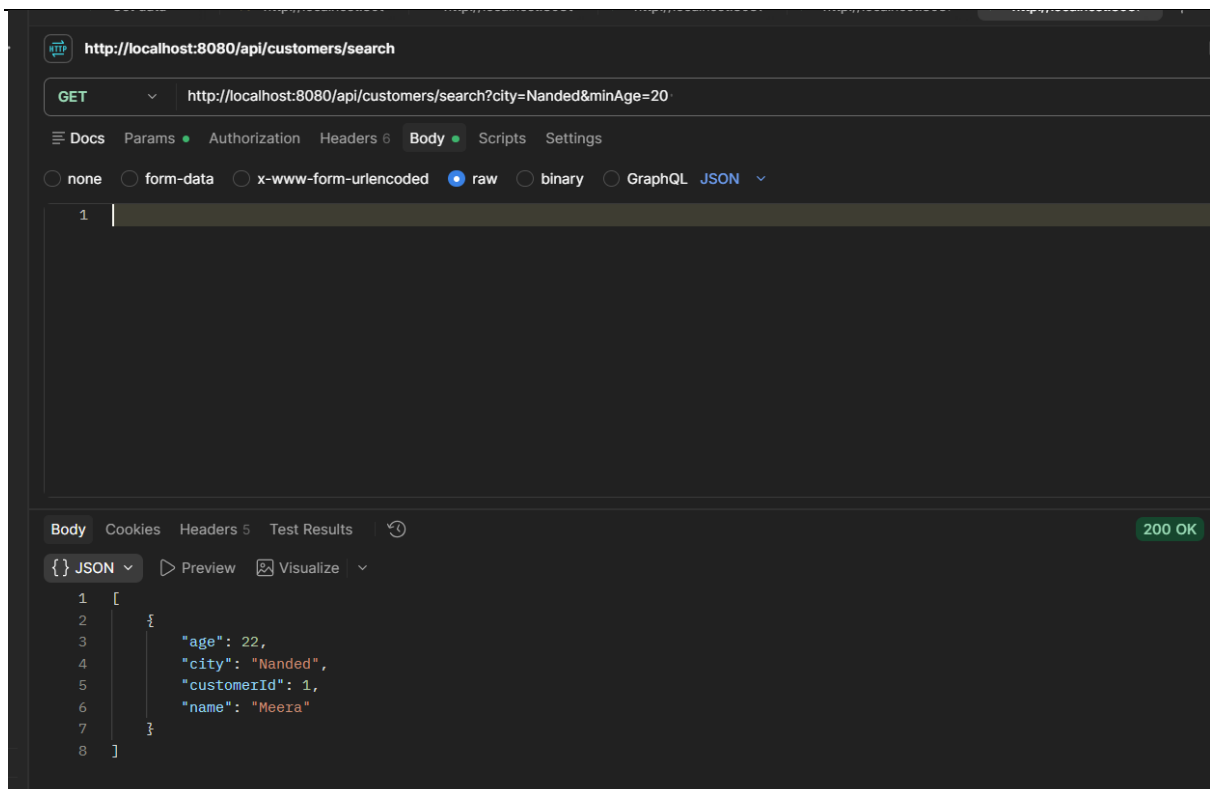
2. GET API – Display Customer by ID (@PathVariable) The GET API is used to retrieve a specific customer using the customer ID. The ID is passed in the URL using @PathVariable. The controller receives the ID, forwards it to the service layer, and the customer details are returned.

GET <http://localhost:8080/api/customers/1>



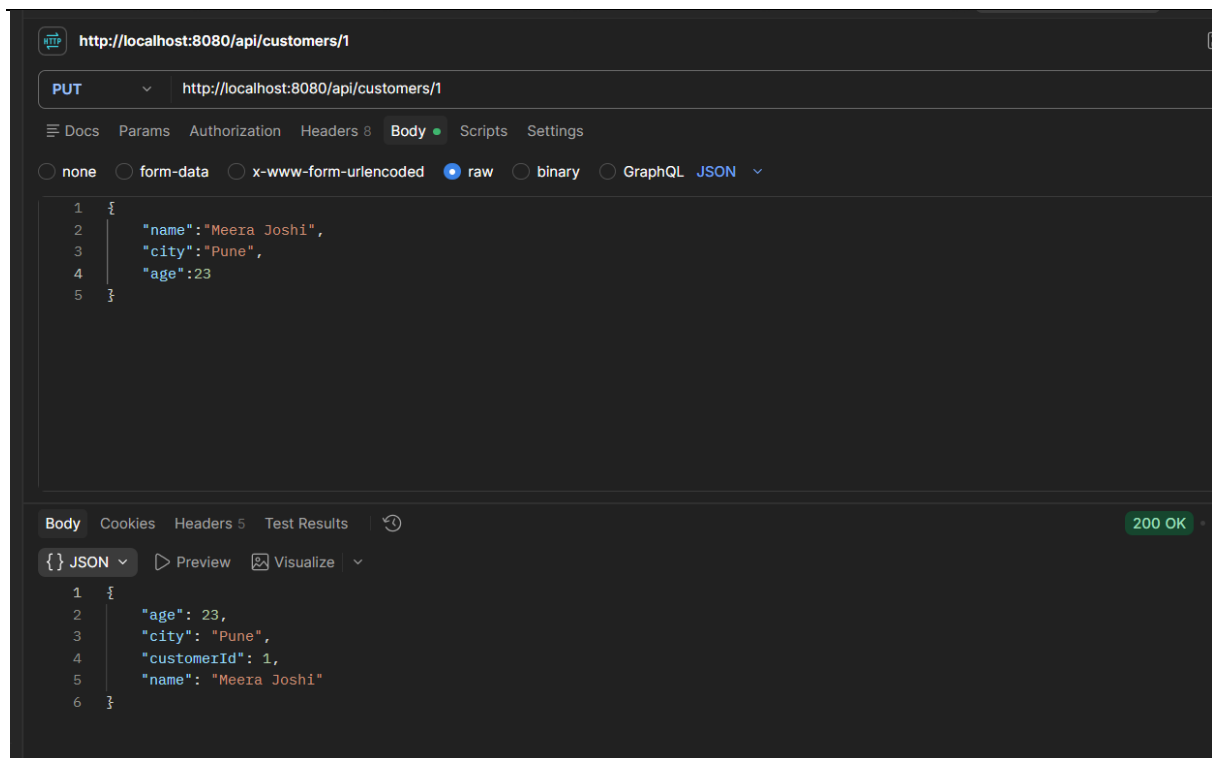
3.GET API – Search Customers (@RequestParam) The GET API is used to search customers based on the city and minimum age. Both values are passed as query parameters using @RequestParam. The service layer filters the records and returns the matching customers.

GET <http://localhost:8080/api/customers/search?city=Nanded&minAge=20>



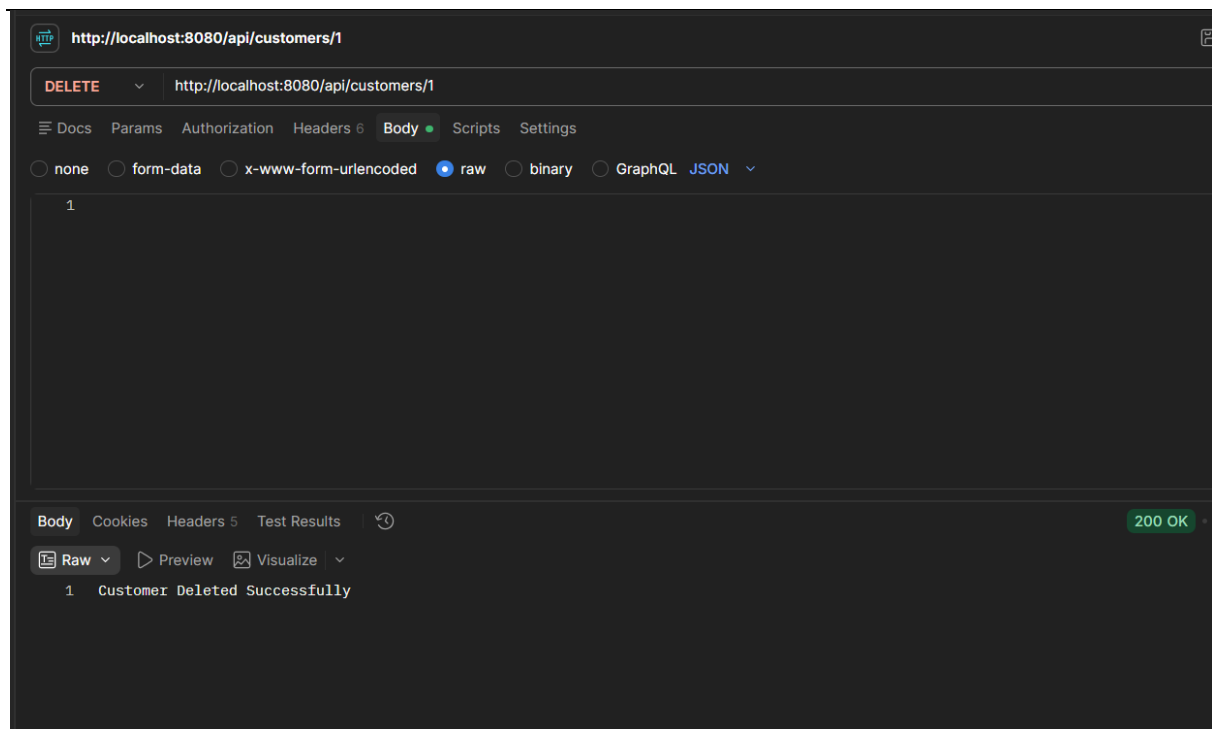
4. PUT API – Update Customer (@PathVariable + @RequestBody) The PUT API is used to update an existing customer's details. The customer ID is passed through @PathVariable, while the updated customer information is sent in the request body using @RequestBody. The service layer updates the record in the database.

PUT <http://localhost:8080/api/customers/1>



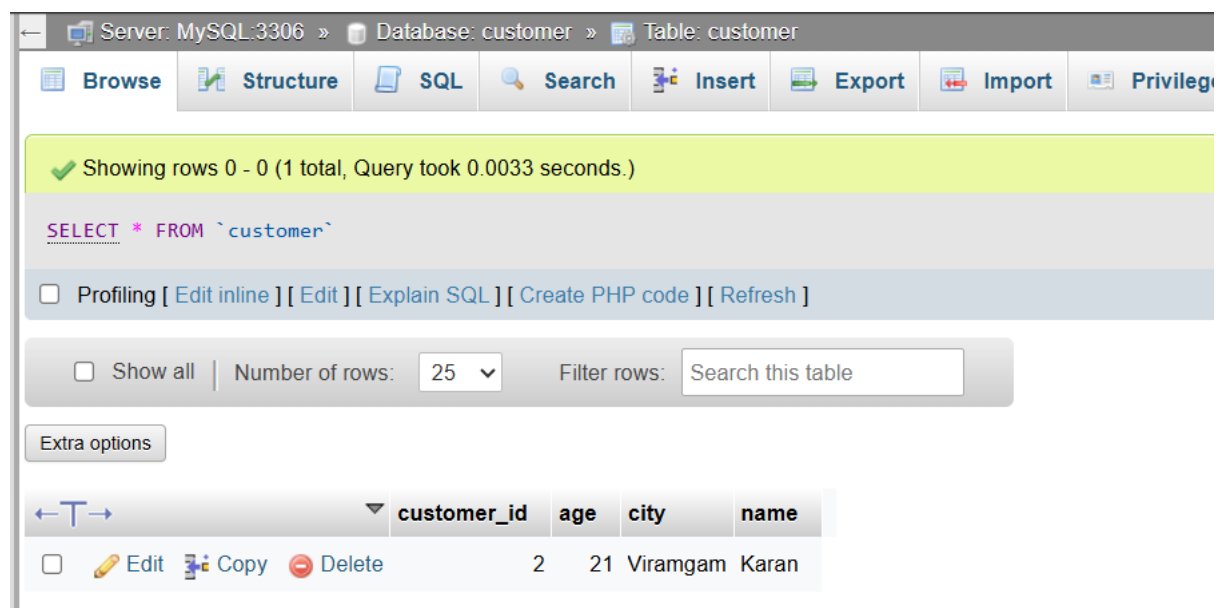
5. DELETE API – Delete Customer (@PathVariable) The DELETE API is used to remove a customer from the database using the customer ID. The ID is passed through the URL using @PathVariable. After successful deletion, a confirmation message is returned.

DELETE <http://localhost:8080/api/customers/1>



Database Output

WE JUST NEED TO CREATE THE DATABASE HIBERNATE IT SELF WILL CREATE TABLE FROM MODULE OR ENTITY CLASS



Practical No. 12: DTO Pattern - Separating the Entity from the API Contract

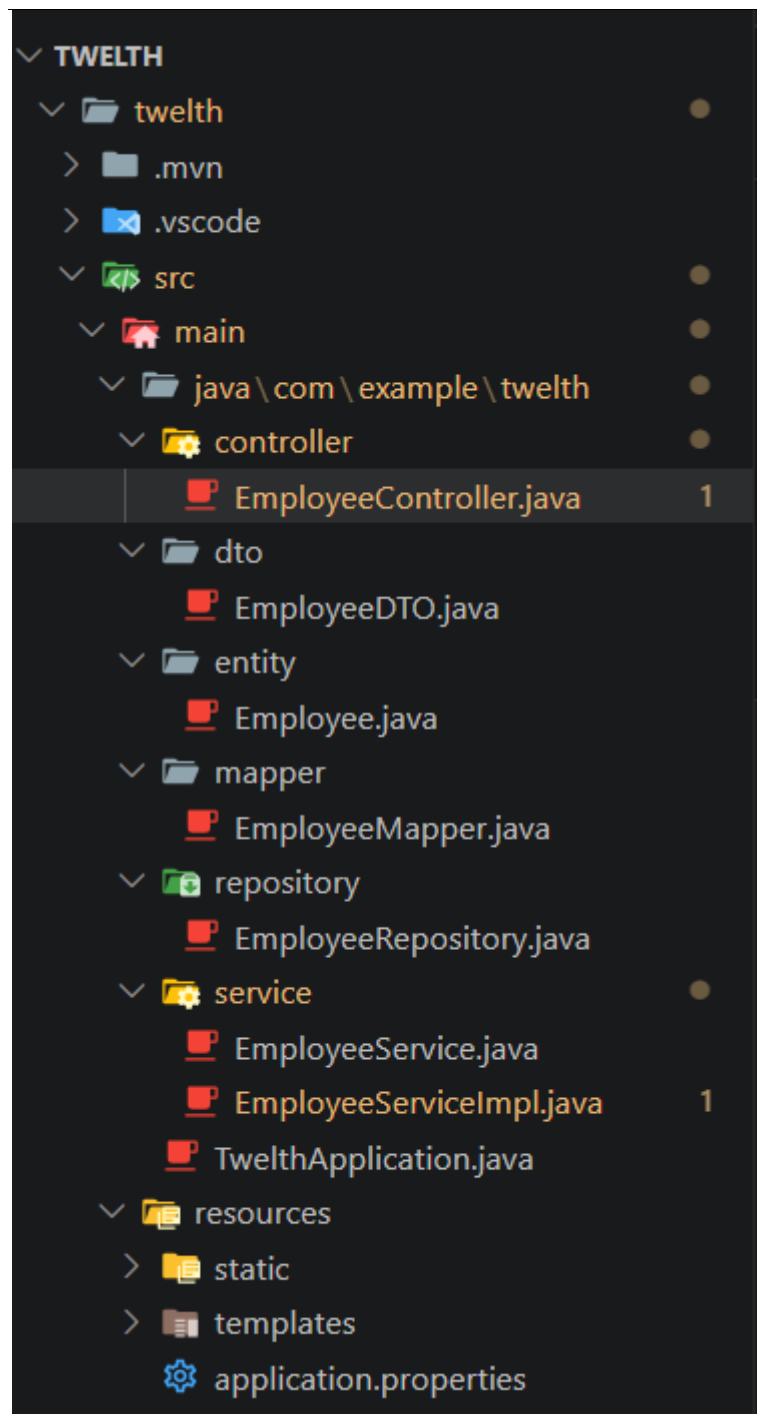
Objective:

To decouple the persistent JPA entity from the data actually exposed by the REST API using the Data Transfer Object (DTO) pattern discussed in this chapter, hiding sensitive/internal fields and mapping between Entity and DTO.

Class Name(s) with Attributes:

Employee (@Entity — add sensitive field)

- empId, empName, department, salary
- password : String (sensitive — never exposed) EmployeeDTO (POJO)
- empId : int
- empName : String
- department : String EmployeeMapper (utility class)
- static EmployeeDTO toDto(Employee e)
- static Employee toEntity(EmployeeDTO d)



Dependencies –

Spring Web

Mysql driver

Jpa

EmployeeController.java(Controller)

```
package com.example.twelfth.controller;

import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import com.example.twelfth.dto.EmployeeDTO;
import com.example.twelfth.service.EmployeeService;

@RestController
@RequestMapping("/api/employees")
public class EmployeeController {

    @Autowired
    private EmployeeService service;

    // POST - Add Employee
```

```
@PostMapping
public ResponseEntity<EmployeeDTO> save(
    @RequestBody EmployeeDTO dto) {

    EmployeeDTO savedEmployee = service.save(dto);

    return new ResponseEntity<>(
        savedEmployee,
        HttpStatus.OK
    );
}

// GET - Get All Employees
@GetMapping
public ResponseEntity<List<EmployeeDTO>> findAll() {

    List<EmployeeDTO> employees = service.findAll();

    return ResponseEntity.ok(employees);
}

// PUT - Update Employee
@PutMapping("/{id}")
public ResponseEntity<EmployeeDTO> update(
    @PathVariable Integer id,
    @RequestBody EmployeeDTO dto) {
```

```
EmployeeDTO updatedEmployee
    = service.update(id, dto);

    if (updatedEmployee == null) {

        return ResponseEntity.notFound().build();
    }

    return ResponseEntity.ok(updatedEmployee);
}

// DELETE - Delete Employee
@DeleteMapping("/{id}")
public ResponseEntity<String> delete(@PathVariable Integer id) {
    service.delete(id);
    return ResponseEntity.ok("Employee deleted successfully");
}
}
```

EmployeeDTO.java(DTO)

```
package com.example.twelfth.dto;

public class EmployeeDTO {

    private Integer empId;

    private String empName;

    private String department;

    public EmployeeDTO() {
    }

    public Integer getEmpId() {
        return empId;
    }

    public void setEmpId(Integer empId) {
        this.empId = empId;
    }

    public String getEmpName() {
        return empName;
    }
}
```

```
public void setEmpName(String empName) {  
    this.empName = empName;  
}  
  
public String getDepartment() {  
    return department;  
}  
  
public void setDepartment(String department) {  
    this.department = department;  
}  
}
```

Employee.java(Entity)

```
package com.example.twelfth.entity;  
  
import jakarta.persistence.Entity;  
import jakarta.persistence.GeneratedValue;  
import jakarta.persistence.GenerationType;  
import jakarta.persistence.Id;  
  
@Entity  
public class Employee {  
  
    @Id  
    @GeneratedValue(strategy = GenerationType.IDENTITY)  
    private Integer empId;
```

```
private String empName;

private String department;

private double salary;

private String password;

public Employee() {
}

public Integer getEmpId() {
    return empId;
}

public void setEmpId(Integer empId) {
    this.empId = empId;
}

public String getEmpName() {
    return empName;
}

public void setEmpName(String empName) {
    this.empName = empName;
}
```



```
public String getDepartment() {  
    return department;  
}  
  
public void setDepartment(String department) {  
    this.department = department;  
}  
  
public double getSalary() {  
    return salary;  
}  
  
public void setSalary(double salary) {  
    this.salary = salary;  
}  
  
public String getPassword() {  
    return password;  
}  
  
public void setPassword(String password) {  
    this.password = password;  
}  
}
```

EmployeeMapper.java(Mapper)

```
package com.example.twelfth.mapper;

import com.example.twelfth.dto.EmployeeDTO;
import com.example.twelfth.entity.Employee;

public class EmployeeMapper {

    // Entity -> DTO
    public static EmployeeDTO toDTO(Employee employee) {

        EmployeeDTO dto = new EmployeeDTO();

        dto.setEmpId(employee.getEmpId());
        dto.setEmpName(employee.getEmpName());
        dto.setDepartment(employee.getDepartment());

        return dto;
    }

    // DTO -> Entity
    public static Employee toEntity(EmployeeDTO dto) {

        Employee employee = new Employee();
```

```
        employee.setEmpId(dto.getEmpId());  
        employee.setEmpName(dto.getEmpName());  
        employee.setDepartment(dto.getDepartment());  
  
        return employee;  
    }  
}
```

EmployeeRepository.java(Repository)

```
package com.example.twelfth.repository;  
  
import org.springframework.data.jpa.repository.JpaRepository;  
  
import com.example.twelfth.entity.Employee;  
  
public interface EmployeeRepository  
    extends JpaRepository<Employee, Integer> {  
  
}
```

EmployeeService.java(service)

```
package com.example.twelfth.service;

import java.util.List;

import com.example.twelfth.dto.EmployeeDTO;

public interface EmployeeService {

    EmployeeDTO save(EmployeeDTO dto);

    List<EmployeeDTO> findAll();

    EmployeeDTO update(Integer id, EmployeeDTO dto);

    void delete(Integer id);
}
```

EmployeeServiceImpl.java(Service)

```
package com.example.twelfth.service;

import java.util.ArrayList;
import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

import com.example.twelfth.dto.EmployeeDTO;
import com.example.twelfth.entity.Employee;
import com.example.twelfth.mapper.EmployeeMapper;
import com.example.twelfth.repository.EmployeeRepository;

@Service
public class EmployeeServiceImpl implements EmployeeService {

    @Autowired
    private EmployeeRepository repository;

    @Override
    public EmployeeDTO save(EmployeeDTO dto) {

        Employee employee = EmployeeMapper.toEntity(dto);

        // Sensitive/internal fields
    }
}
```

```
employee.setSalary(50000);

employee.setPassword("admin123");


Employee savedEmployee = repository.save(employee);


return EmployeeMapper.toDTO(savedEmployee);
}


@Override

public List<EmployeeDTO> findAll() {

    List<Employee> employees = repository.findAll();

    List<EmployeeDTO> dtoList = new ArrayList<>();

    for (Employee employee : employees) {

        dtoList.add(EmployeeMapper.toDTO(employee));
    }

    return dtoList;
}


@Override

public EmployeeDTO update(Integer id, EmployeeDTO dto) {

    Employee employee = repository.findById(id).orElse(null);
```

```
    if (employee == null) {  
        return null;  
    }  
  
    employee.setEmpName(dto.getEmpName());  
    employee.setDepartment(dto.getDepartment());  
    Employee updatedEmployee = repository.save(employee);  
  
    return EmployeeMapper.toDTO(updatedEmployee);  
}  
  
@Override  
public void delete(Integer id) {  
  
    repository.deleteById(id);  
}  
}
```

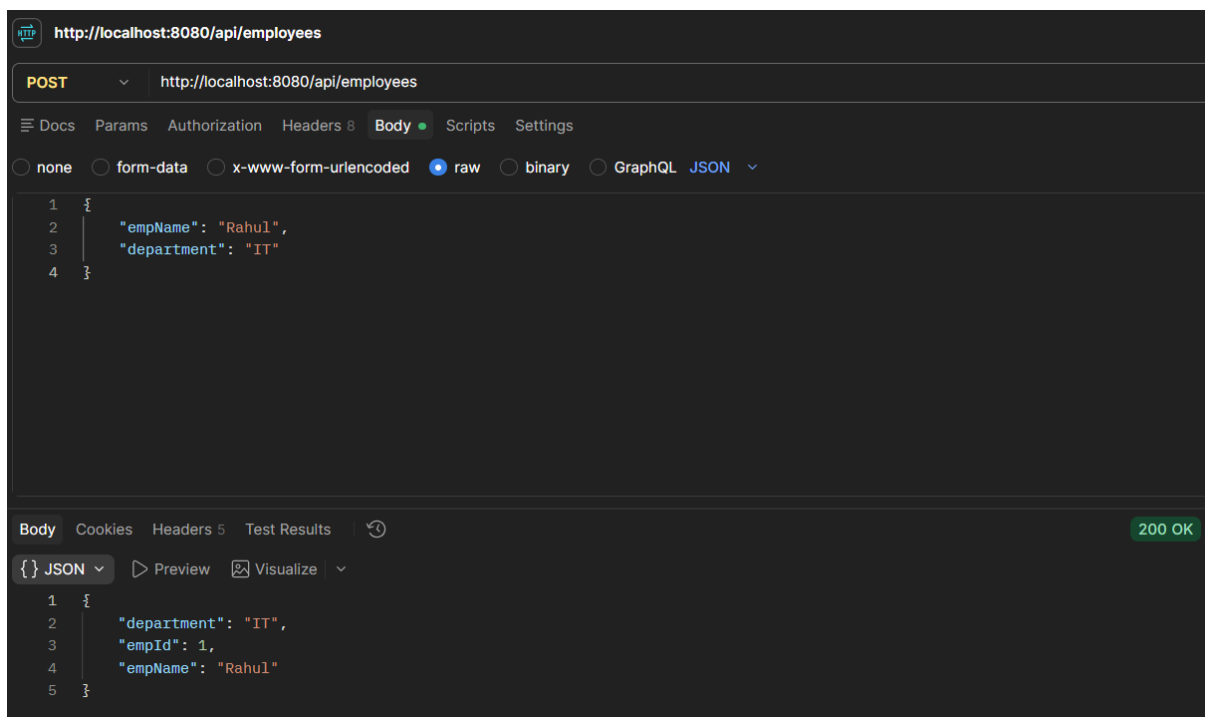
Application.properties

```
spring.application.name=twelth  
spring.datasource.url=jdbc:mysql://localhost:3306/twelth  
spring.datasource.username=root  
spring.datasource.password=  
  
spring.jpa.hibernate.ddl-auto=update  
spring.jpa.show-sql=true
```

API'S TESTING USING POSTMAN

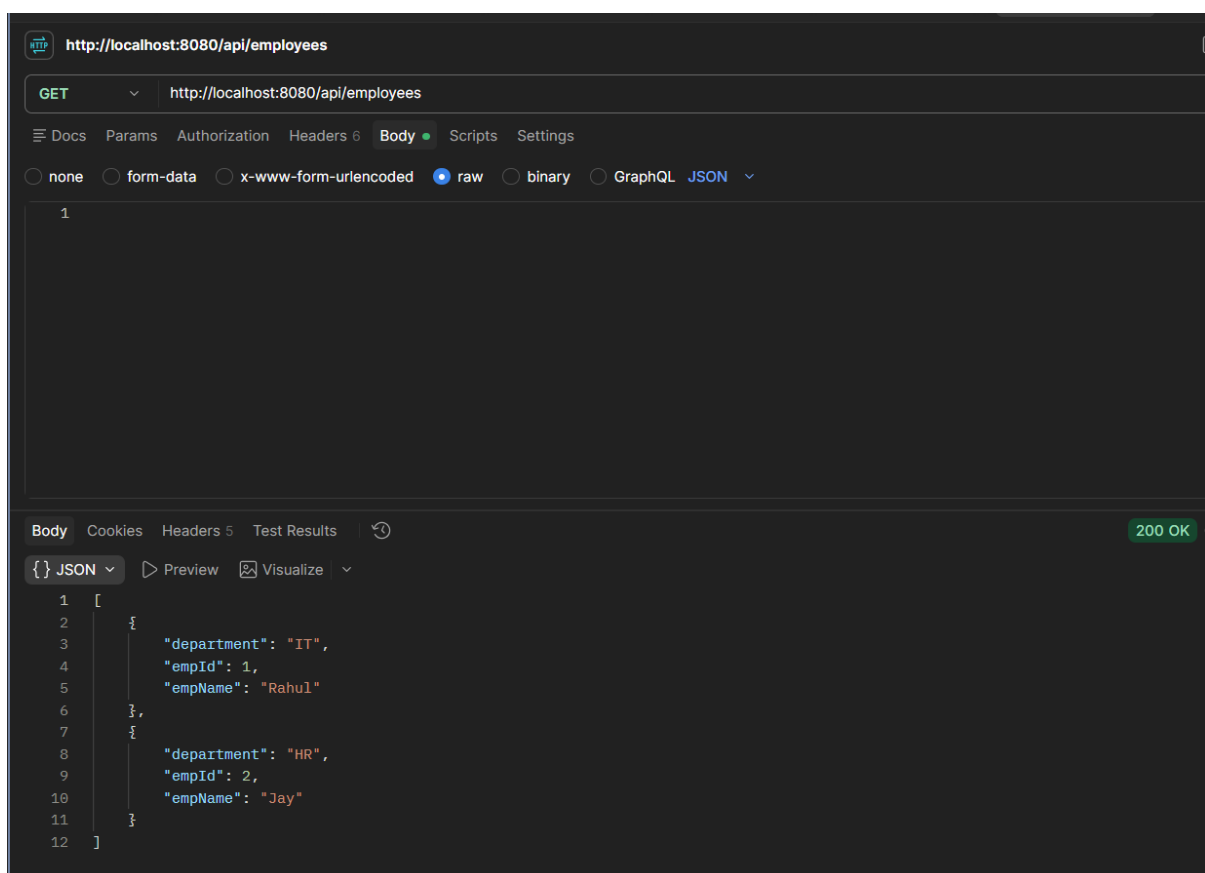
1. **POST API –Add Employee (Using DTO)** The POST API is used to add a new employee to the database using the EmployeeDTO object. The employee details are received in JSON format through @RequestBody. The DTO is converted into an Entity using the EmployeeMapper, stored in the database, and the response is returned as a DTO without exposing sensitive

POST <http://localhost:8080/api/employees>



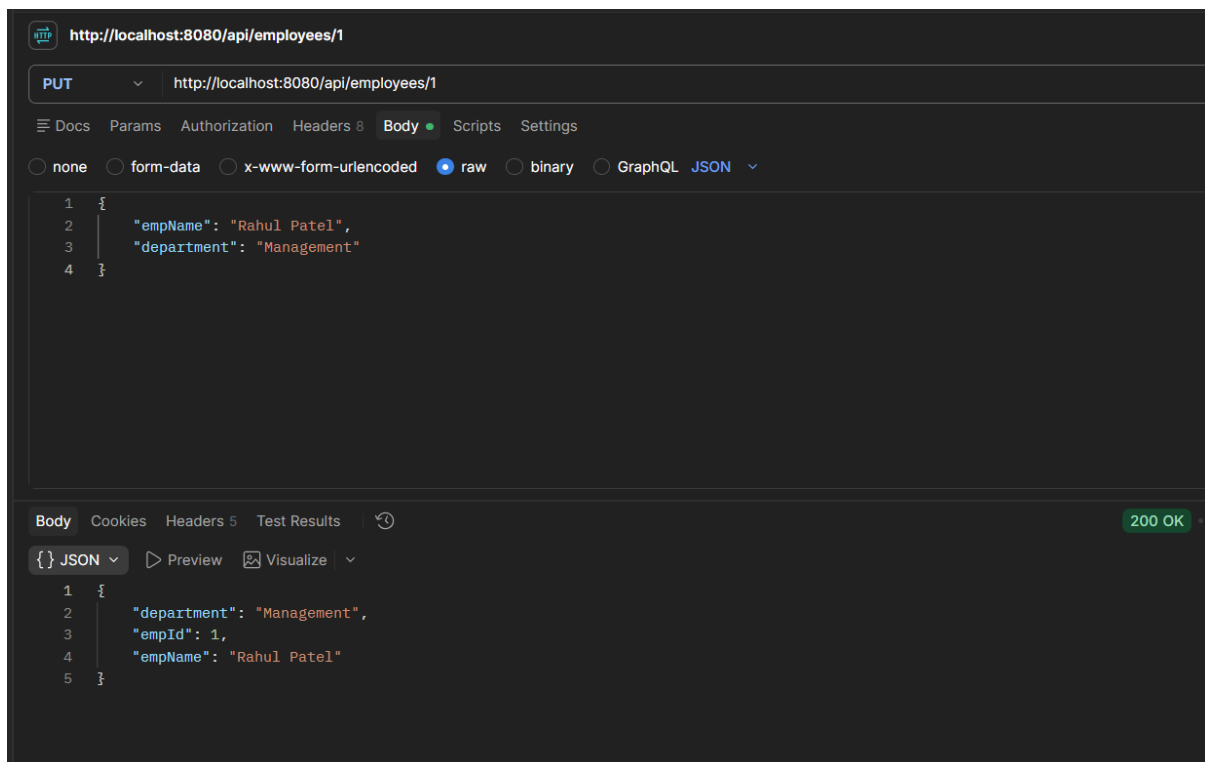
2. **GET API :-** Display All Employees (Using DTO) The GET API is used to retrieve all employee records from the database. The service layer fetches all Employee entities, converts them into EmployeeDTO objects using the EmployeeMapper, and returns only the required information. Sensitive fields such as salary and password are not included in the response.

GET <http://localhost:8080/api/employees>



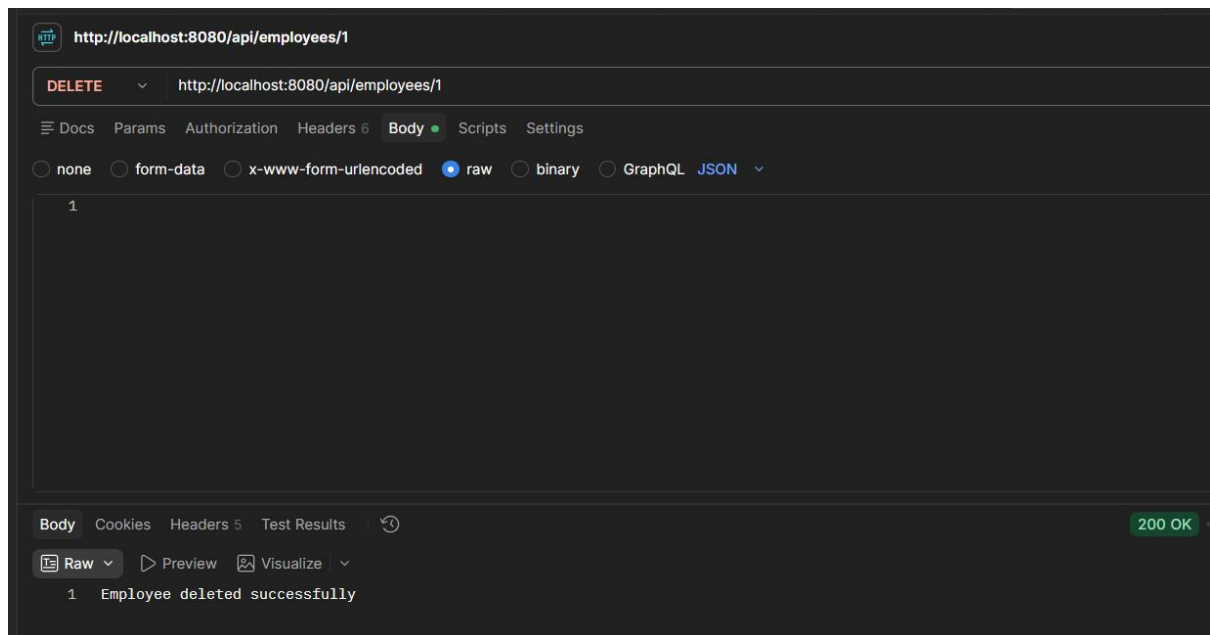
3. PUT API – Update Employee (Using DTO) The PUT API is used to update an existing employee's information. The employee ID is passed using `@PathVariable`, while the updated details are received through `@RequestBody` as an `EmployeeDTO`. The service layer updates the record and returns the updated DTO.

PUT <https://localhost:8080/api/employees/1>



4. DELETE API – Delete Employee The DELETE API is used to remove an employee record from the database using the employee ID. The ID is received through `@PathVariable`, and the service layer deletes the corresponding record. A success message is returned after successful deletion.

DELETE <https://localhost:8080/api/employees/1>



Database Output

WE JUST NEED TO CREATE THE DATABASE HIBERNATE IT SELF WILL CREATE TABLE FROM MODULE OR ENTITY CLASS

The screenshot displays the MySQL Workbench interface for a table named 'employee' in the 'twelfth' database. The top navigation bar shows the path: Server: MySQL:3306 » Database: twelfth » Table: employee. Below this, a toolbar contains icons for Browse, Structure, SQL, Search, Insert, Export, Import, Privileges, and Options. A green status bar indicates 'Showing rows 0 - 0 (1 total, Query took 0.0029 seconds.)'. The SQL editor contains the query: `SELECT * FROM `employee``. Below the query, there are links for Profiling, Edit inline, Edit, Explain SQL, Create PHP code, and Refresh. A control bar shows 'Show all', 'Number of rows: 25', and a 'Filter rows' search box. An 'Extra options' button is also present. The table data is displayed in a grid with columns: emp_id, department, emp_name, password, and salary. The first row shows emp_id 2, department HR, emp_name Jay, password admin123, and salary 50000. Action icons (Edit, Copy, Delete) are visible for each row.

	emp_id	department	emp_name	password	salary
☐ Edit Copy Delete	2	HR	Jay	admin123	50000

Practical No. 13: Request Validation using Bean Validation (@Valid) with RestController

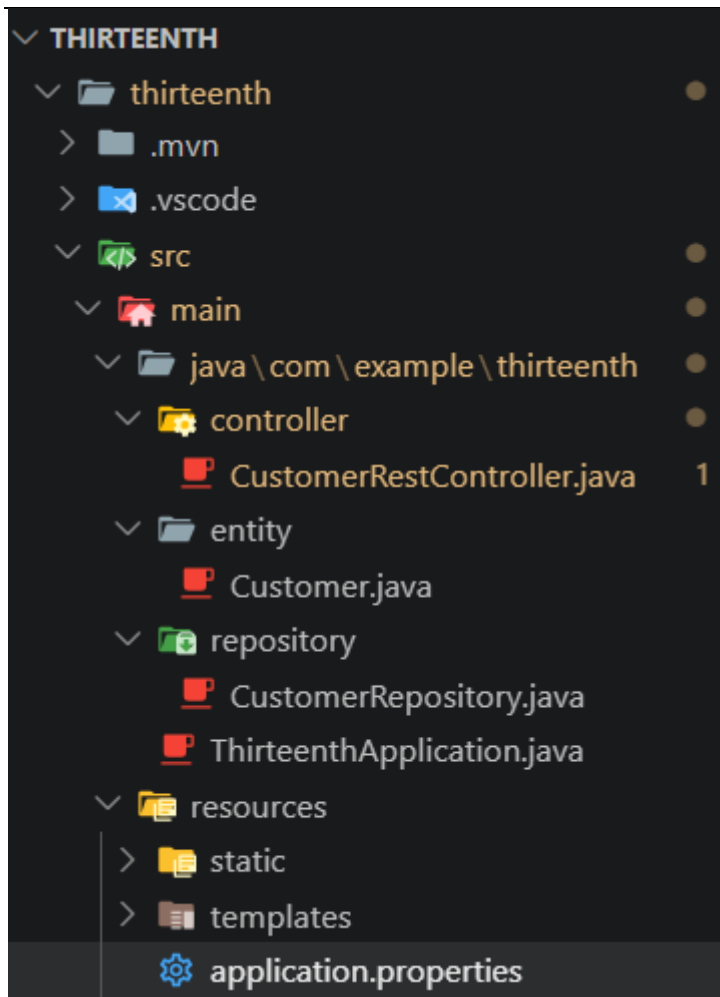
Objective:

To validate incoming JSON payloads declaratively using Jakarta Bean Validation annotations (@NotBlank, @Email, @Min) combined with @Valid on the controller method, as covered in this chapter's discussion of building robust REST endpoints.

Class Name(s) with Attributes:

Customer (@Entity / DTO, reused)

- customerId : int (@Id @GeneratedValue)
- name : String (@NotBlank)
- email : String (@Email)
- age : int (@Min(18))



Dependencies

- Spring Web
- Spring Data JPA
- MySQL Driver
- starter validation Database

Database Name:

thirteenth

Table: customer(created automatically by Hibernate)

CustomerRestController.java(Controller)

```
package com.example.thirteenth.controller;

import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

import com.example.thirteenth.entity.Customer;
import com.example.thirteenth.repository.CustomerRepository;

import jakarta.validation.Valid;

@RestController
@RequestMapping("/api/customers")
public class CustomerRestController {

    @Autowired
    private CustomerRepository repository;

    // Add Customer
    @PostMapping
    public Customer addCustomer(
        @Valid @RequestBody Customer customer) {

        return repository.save(customer);
    }
}
```

```
}

// Display All Customers
@GetMapping
public List<Customer> getAllCustomers() {

    return repository.findAll();
}

// Display Customer By Id
@GetMapping("/{id}")
public Customer getCustomer(
    @PathVariable Integer id) {

    return repository.findById(id).orElse(null);
}

// Update Customer
@PutMapping("/{id}")
public Customer updateCustomer(
    @PathVariable Integer id,
    @Valid @RequestBody Customer customer) {

    customer.setCustomerId(id);

    return repository.save(customer);
}
```



```
// Delete Customer
@DeleteMapping("/{id}")
public String deleteCustomer(
    @PathVariable Integer id) {

    repository.deleteById(id);

    return "Customer Deleted Successfully";
}
}
```

Customer.java(Entity)

```
package com.example.thirteenth.entity;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.Min;
import jakarta.validation.constraints.NotBlank;

@Entity
public class Customer {

    @Id
```

```
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Integer customerId;

@NotBlank(message = "Name cannot be empty")
private String name;

>Email(message = "Enter a valid email")
@NotBlank(message = "Email cannot be empty")
private String email;

@Min(value = 18, message = "Age must be at least 18")
private int age;

public Customer() {
}

public Integer getCustomerId() {
    return customerId;
}

public void setCustomerId(Integer customerId) {
    this.customerId = customerId;
}

public String getName() {
    return name;
}
```

```
public void setName(String name) {  
    this.name = name;  
}  
  
public String getEmail() {  
    return email;  
}  
  
public void setEmail(String email) {  
    this.email = email;  
}  
  
public int getAge() {  
    return age;  
}  
  
public void setAge(int age) {  
    this.age = age;  
}  
}
```

CustomerRespository.java(Repository)

```
package com.example.thirteenth.repository;  
  
import org.springframework.data.jpa.repository.JpaRepository;
```

```
import com.example.thirteenth.entity.Customer;

public interface CustomerRepository
    extends JpaRepository<Customer, Integer> {

}
```

Application.properties

```
spring.application.name=thirteenth

spring.datasource.url=jdbc:mysql://localhost:3306/thirteenth
spring.datasource.username=root
spring.datasource.password=

spring.jpa.hibernate.ddl-auto=update

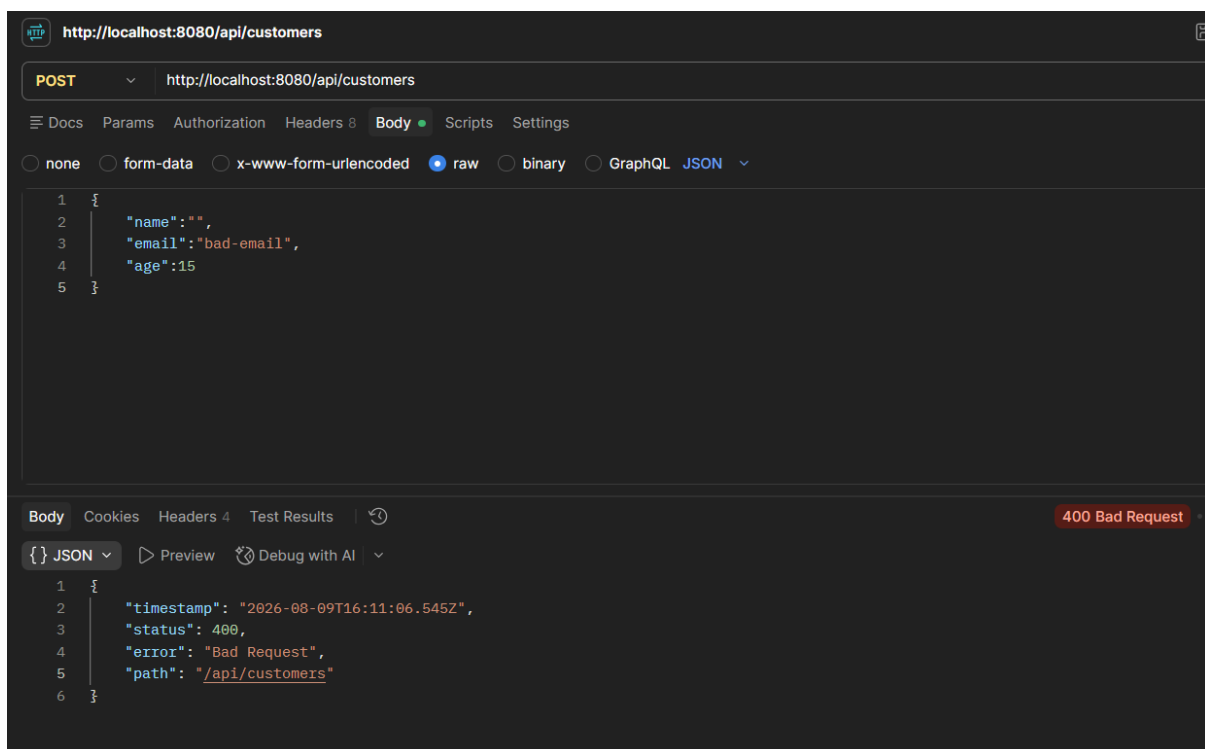
spring.jpa.show-sql=true

spring.jpa.properties.hibernate.format_sql=true
```

API'S TESTING USING POSTMAN

1. POST API – Add Customer with Validation The POST API is used to create a new customer in the database. Before saving the customer details, Spring Boot validates the input using the `@Valid` annotation along with validation constraints such as `@NotBlank`, `@Email`, and `@Min`. If all validations are satisfied, the customer record is stored successfully; otherwise, Spring Boot returns a validation error.

POST <http://localhost:8080/api/customers>



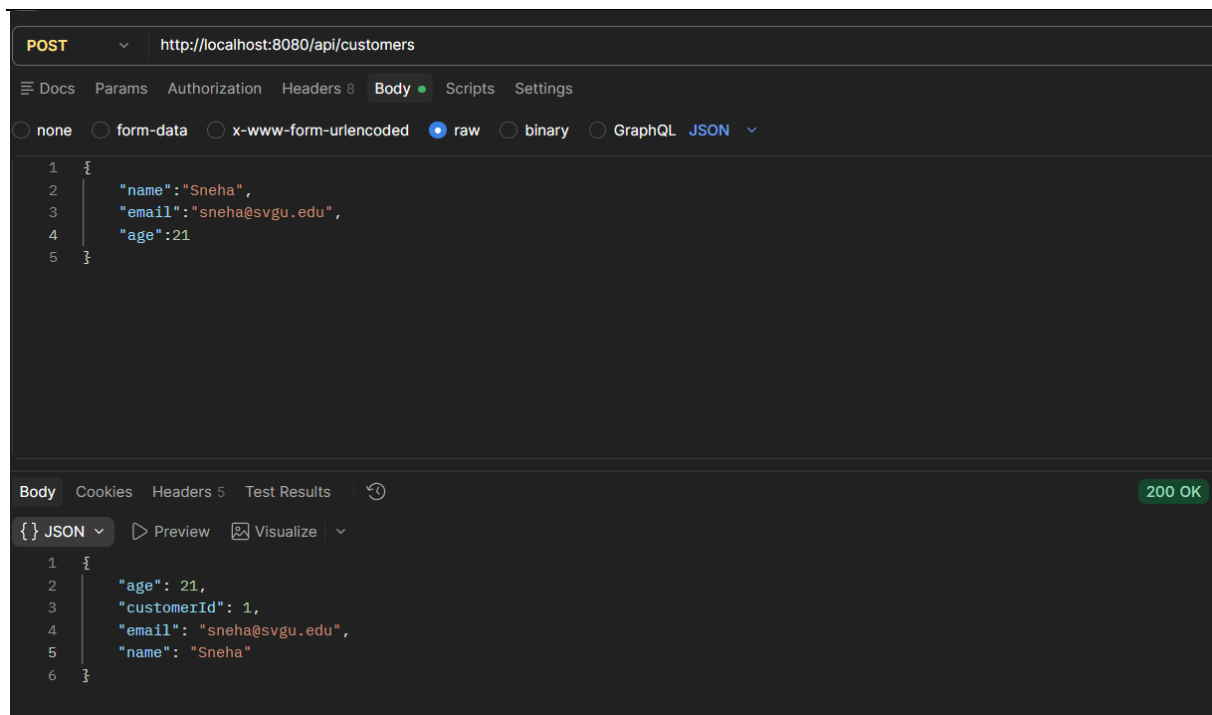
Show's error cause it's not valid value's

Like name :""

Email : "bad-email"

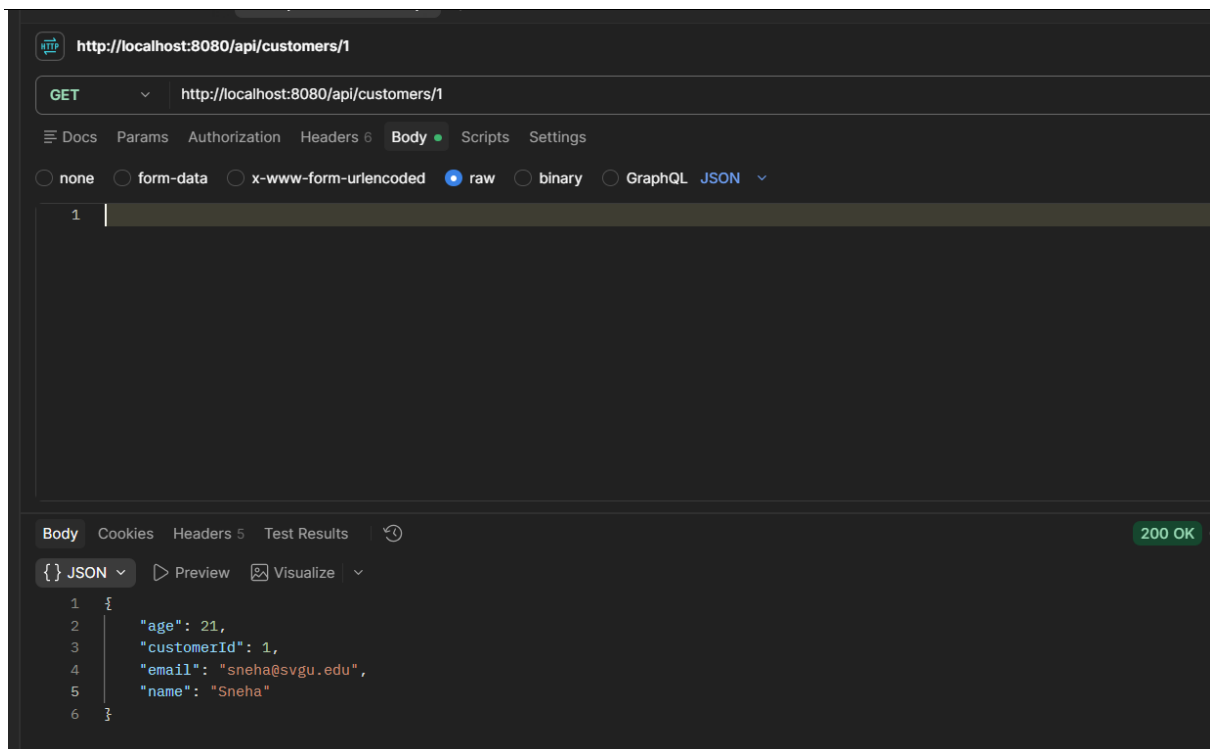
Age:15

Now we ll see correct data output



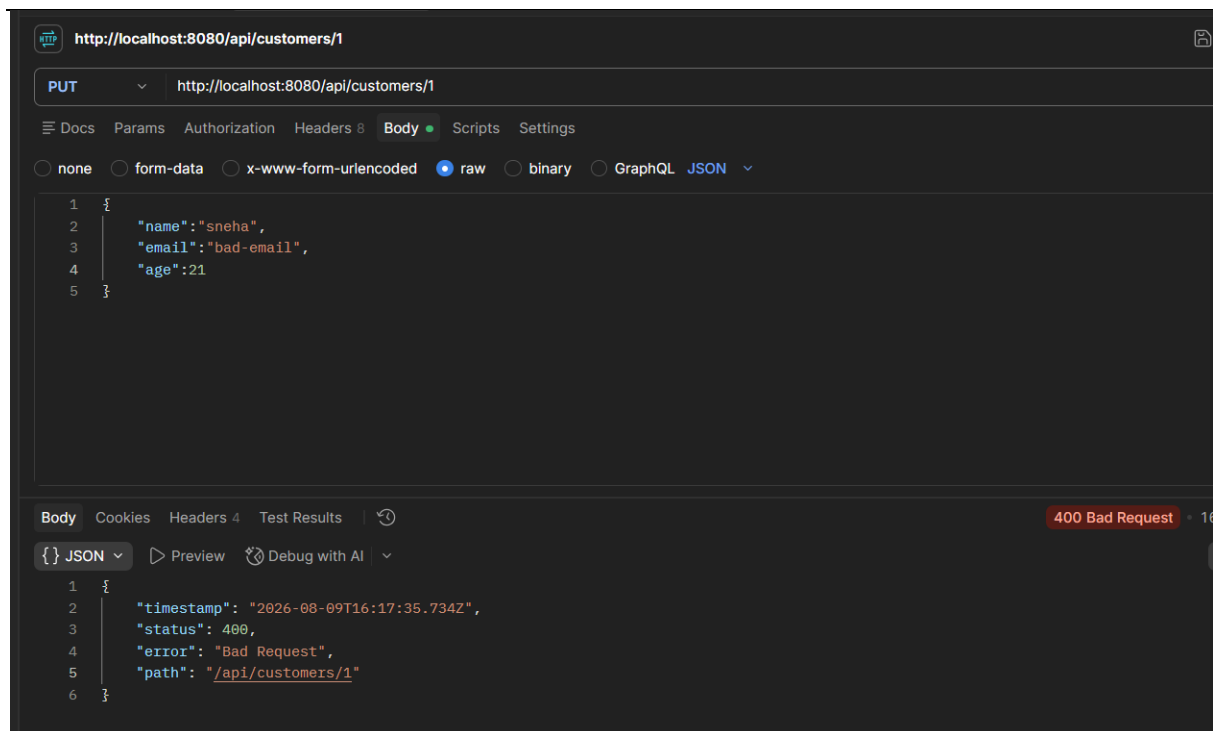
2 GET API – Display Customer by ID The GET API retrieves a single customer record using the customer ID. The ID is passed as a path variable, and Spring Boot returns the corresponding customer information if it exists.

GET <http://localhost:8080/api/customers/1>



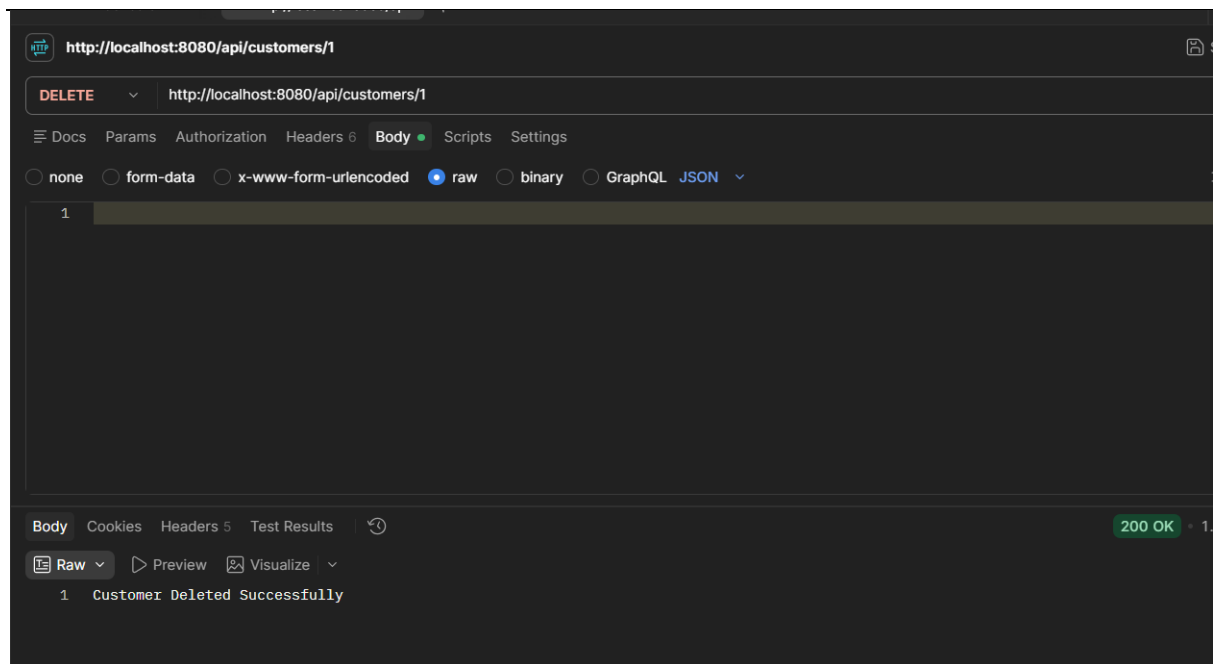
3. PUT API – Update Customer The PUT API is used to update an existing customer's information. The customer ID is passed through the URL, while the updated customer details are provided in the request body. The input is validated before updating the record in the database.

PUT <http://localhost:8080/api/customers/1>

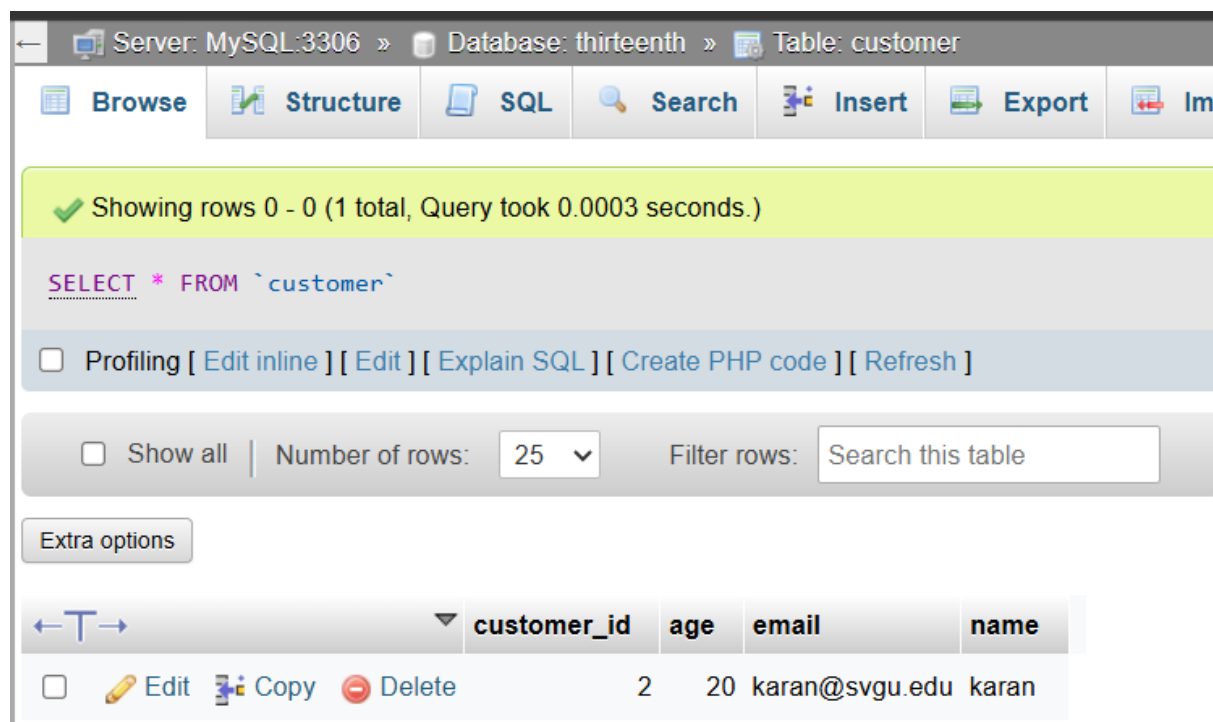


4. DELETE API – Delete Customer The DELETE API is used to remove an existing customer record from the database. The customer ID is passed through the URL, and the corresponding record is deleted successfully.

DELETE <http://localhost:8080/api/customers/1>



Database Output



Practical No. 14: Global Exception Handling using @ControllerAdvice and @ExceptionHandlerPersisting a Single Entity

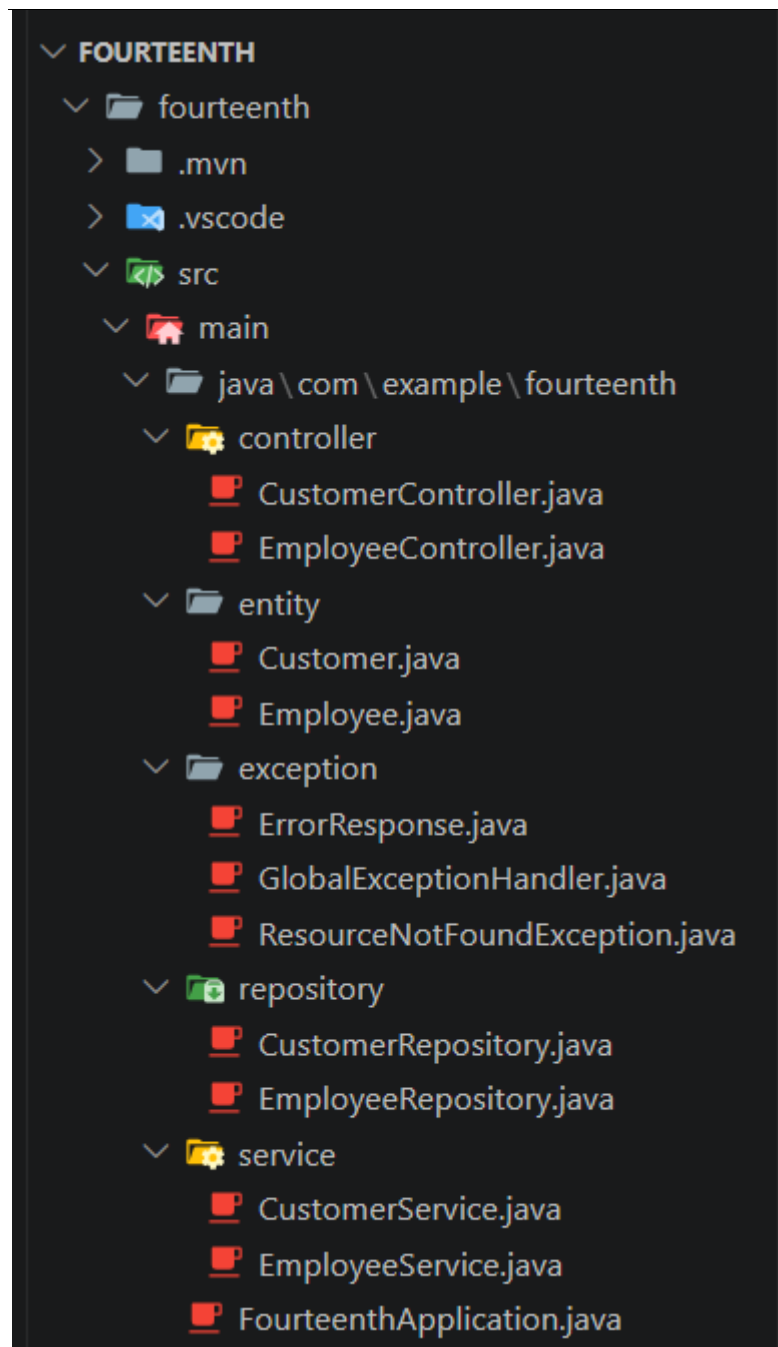
Objective:

To centralize exception handling across all REST controllers using @ControllerAdvice, returning a consistent, structured error response (timestamp, message, status, path) instead of default Spring stack-trace pages — the concluding topic of this chapter.

Class Name(s) with Attributes:

ResourceNotFoundException (custom RuntimeException) ErrorResponse (POJO)

- timestamp : LocalDateTime
- message : String
- status : int
- path : String GlobalExceptionHandler (@ControllerAdvice)
- Method: handleNotFound(ResourceNotFoundException)
- Method: handleValidation(MethodArgumentNotValidException)
- Method: handleGenericException(Exception)



Dependencies

- Spring Web
- Spring Data JPA
- MySQL Driver
- starter validation

Database Name: fourteenth

Table: customer(created automatically by Hibernate)

CustomerController.java(Controller)

```
package com.example.fourteenth.controller;

import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import com.example.fourteenth.entity.Customer;
import com.example.fourteenth.service.CustomerService;

import jakarta.validation.Valid;

@RestController
@RequestMapping("/api/customers")
public class CustomerController {

    private final CustomerService customerService;

    public CustomerController(CustomerService customerService) {
        this.customerService = customerService;
    }
}
```

```
}

@PostMapping
public ResponseEntity<Customer> createCustomer(
    @Valid @RequestBody Customer customer) {

    return ResponseEntity.ok(
        customerService.createCustomer(customer)
    );
}
}
```

EmployeeController.java(Controller)

```
package com.example.fourteenth.controller;

import java.util.List;

import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
```

```
import com.example.fourteenth.entity.Employee;
import com.example.fourteenth.service.EmployeeService;

@RestController
@RequestMapping("/api/employees")
public class EmployeeController {

    private final EmployeeService employeeService;

    public EmployeeController(EmployeeService employeeService) {
        this.employeeService = employeeService;
    }

    // GET ALL
    @GetMapping
    public ResponseEntity<List<Employee>> getAllEmployees() {

        return ResponseEntity.ok(
            employeeService.getAllEmployees()
        );
    }

    // GET BY ID
    @GetMapping("/{id}")
    public ResponseEntity<Employee> getEmployeeById(
        @PathVariable Integer id) {
```

```
        return ResponseEntity.ok(
            employeeService.getEmployeeById(id)
        );
    }

    // POST
    @PostMapping
    public ResponseEntity<Employee> createEmployee(
        @RequestBody Employee employee) {

        return ResponseEntity.ok(
            employeeService.createEmployee(employee)
        );
    }

    // PUT
    @PutMapping("/{id}")
    public ResponseEntity<Employee> updateEmployee(
        @PathVariable Integer id,
        @RequestBody Employee employee) {

        return ResponseEntity.ok(
            employeeService.updateEmployee(id, employee)
        );
    }
}
```

```
// DELETE

@DeleteMapping("/{id}")
public ResponseEntity<String> deleteEmployee(
    @PathVariable Integer id) {

    employeeService.deleteEmployee(id);

    return ResponseEntity.ok(
        "Employee deleted successfully"
    );
}
}
```

Customer.java(entity)

```
package com.example.fourteenth.entity;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.Min;
import jakarta.validation.constraints.NotBlank;
```


@Entity

```
public class Customer {
```

```
    @Id
```

```
    @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
    private Integer customerId;
```

```
    @NotBlank(message = "Name is required")
```

```
    private String name;
```

```
    @NotBlank(message = "Email is required")
```

```
    @Email(message = "Email must be valid")
```

```
    private String email;
```

```
    @Min(value = 18, message = "Age must be at least 18")
```

```
    private int age;
```

```
    public Customer() {
```

```
    }
```

```
    public Customer(Integer customerId, String name, String email, int age) {
```

```
        this.customerId = customerId;
```

```
        this.name = name;
```

```
        this.email = email;
```

```
        this.age = age;
```

```
    }
```

```
public Integer getCustomerId() {  
    return customerId;  
}  
  
public void setCustomerId(Integer customerId) {  
    this.customerId = customerId;  
}  
  
public String getName() {  
    return name;  
}  
  
public void setName(String name) {  
    this.name = name;  
}  
  
public String getEmail() {  
    return email;  
}  
  
public void setEmail(String email) {  
    this.email = email;  
}  
  
public int getAge() {  
    return age;  
}
```

```
public void setAge(int age) {  
    this.age = age;  
}  
}
```

Employee.java(Entity)

```
package com.example.fourteenth.entity;  
  
import jakarta.persistence.Entity;  
import jakarta.persistence.GeneratedValue;  
import jakarta.persistence.GenerationType;  
import jakarta.persistence.Id;  
  
@Entity  
public class Employee {  
  
    @Id  
    @GeneratedValue(strategy = GenerationType.IDENTITY)  
    private Integer id;  
  
    private String empName;  
  
    public Employee() {  
    }  
}
```

```
public Employee(Integer id, String empName) {  
    this.id = id;  
    this.empName = empName;  
}  
  
public Integer getId() {  
    return id;  
}  
  
public void setId(Integer id) {  
    this.id = id;  
}  
  
public String getEmpName() {  
    return empName;  
}  
  
public void setEmpName(String empName) {  
    this.empName = empName;  
}  
}
```

ErrorResponse.java(exception)

```
package com.example.fourteenth.exception;

import java.time.LocalDateTime;

public class ErrorResponse {

    private LocalDateTime timestamp;
    private String message;
    private int status;
    private String path;

    public ErrorResponse() {
    }

    public ErrorResponse(LocalDateTime timestamp, String message, int status, String path)
    {
        this.timestamp = timestamp;
        this.message = message;
        this.status = status;
        this.path = path;
    }

    public LocalDateTime getTimestamp() {
        return timestamp;
    }
}
```

```
}

public void setTimestamp(LocalDateTime timestamp) {
    this.timestamp = timestamp;
}

public String getMessage() {
    return message;
}

public void setMessage(String message) {
    this.message = message;
}

public int getStatus() {
    return status;
}

public void setStatus(int status) {
    this.status = status;
}

public String getPath() {
    return path;
}

public void setPath(String path) {
```

```
        this.path = path;
    }
}
```

GlobalException.java(Exception)

```
package com.example.fourteenth.exception;

import java.time.LocalDateTime;
import java.util.LinkedHashMap;
import java.util.Map;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ControllerAdvice;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.context.request.WebRequest;

@ControllerAdvice
public class GlobalExceptionHandler {

    // =====
    // 404 - Resource Not Found
    // =====
```

```
@ExceptionHandler(ResourceNotFoundException.class)
public ResponseEntity<ErrorResponse> handleNotFound(
    ResourceNotFoundException ex,
    WebRequest request) {

    ErrorResponse errorResponse = new ErrorResponse(
        LocalDateTime.now(),
        ex.getMessage(),
        HttpStatus.NOT_FOUND.value(),
        request.getDescription(false).replace("uri=", ""))
    );

    return new ResponseEntity<>(
        errorResponse,
        HttpStatus.NOT_FOUND
    );
}

// =====
// 400 - Validation Error
// =====

@ExceptionHandler(MethodArgumentNotValidException.class)
public ResponseEntity<Map<String, Object>> handleValidation(
    MethodArgumentNotValidException ex,
    WebRequest request) {
```



```
Map<String, String> fieldErrors = new LinkedHashMap<>();
```

```
ex.getBindingResult()
    .getFieldErrors()
    .forEach(error -> {
        fieldErrors.put(
            error.getField(),
            error.getDefaultMessage()
        );
    });
```

```
Map<String, Object> response = new LinkedHashMap<>();
```

```
response.put("timestamp", LocalDateTime.now());
response.put("message", "Validation failed");
response.put("status", HttpStatus.BAD_REQUEST.value());
response.put(
    "path",
    request.getDescription(false)
        .replace("uri=", "")
);
response.put("errors", fieldErrors);

return new ResponseEntity<>(
    response,
    HttpStatus.BAD_REQUEST
```

```
    );  
}  
  
// =====  
// 500 - Generic Exception  
// =====  
  
@ExceptionHandler(Exception.class)  
public ResponseEntity<ErrorResponse> handleGenericException(  
    Exception ex,  
    WebRequest request) {  
  
    ErrorResponse errorResponse = new ErrorResponse(  
        LocalDateTime.now(),  
        "Internal server error",  
        HttpStatus.INTERNAL_SERVER_ERROR.value(),  
        request.getDescription(false).replace("uri=", "")  
    );  
  
    return new ResponseEntity<>(  
        errorResponse,  
        HttpStatus.INTERNAL_SERVER_ERROR  
    );  
}  
}
```

ResultNotFoundException.java(Execption)

```
package com.example.fourteenth.exception;

public class ResourceNotFoundException extends RuntimeException {

    public ResourceNotFoundException(String message) {
        super(message);
    }
}
```

CustomerRepository.java(repository)

```
package com.example.fourteenth.repository;

import org.springframework.data.jpa.repository.JpaRepository;

import com.example.fourteenth.entity.Customer;

public interface CustomerRepository extends JpaRepository<Customer, Integer> {

}
```

EmployeeRepository.java(Repository)

```
package com.example.fourteenth.repository;

import org.springframework.data.jpa.repository.JpaRepository;

import com.example.fourteenth.entity.Employee;

public interface EmployeeRepository extends JpaRepository<Employee, Integer> {

}
```

CusotmerService.java(Service)

```
package com.example.fourteenth.service;

import org.springframework.stereotype.Service;

import com.example.fourteenth.entity.Customer;
import com.example.fourteenth.repository.CustomerRepository;

@Service
public class CustomerService {
```

```
private final CustomerRepository customerRepository;

public CustomerService(CustomerRepository customerRepository) {
    this.customerRepository = customerRepository;
}

public Customer createCustomer(Customer customer) {
    return customerRepository.save(customer);
}
}
```

EmployeeService.java(service)

```
package com.example.fourteenth.service;

import java.util.List;

import org.springframework.stereotype.Service;

import com.example.fourteenth.entity.Employee;
import com.example.fourteenth.exception.ResourceNotFoundException;
import com.example.fourteenth.repository.EmployeeRepository;

@Service
public class EmployeeService {
```

```
private final EmployeeRepository employeeRepository;

public EmployeeService(EmployeeRepository employeeRepository) {
    this.employeeRepository = employeeRepository;
}

// GET ALL EMPLOYEES
public List<Employee> getAllEmployees() {
    return employeeRepository.findAll();
}

// GET EMPLOYEE BY ID
public Employee getEmployeeById(Integer id) {

    return employeeRepository.findById(id)
        .orElseThrow(() ->
            new ResourceNotFoundException(
                "Employee not found with id: " + id
            )
        );
}

// CREATE EMPLOYEE
public Employee createEmployee(Employee employee) {
    return employeeRepository.save(employee);
}
```

```
// UPDATE EMPLOYEE

public Employee updateEmployee(Integer id, Employee employee) {

    Employee existingEmployee = employeeRepository.findById(id)
        .orElseThrow(() ->
            new ResourceNotFoundException(
                "Employee not found with id: " + id
            )
        );

    existingEmployee.setEmpName(employee.getEmpName());

    return employeeRepository.save(existingEmployee);
}

// DELETE EMPLOYEE

public void deleteEmployee(Integer id) {

    Employee existingEmployee = employeeRepository.findById(id)
        .orElseThrow(() ->
            new ResourceNotFoundException(
                "Employee not found with id: " + id
            )
        );

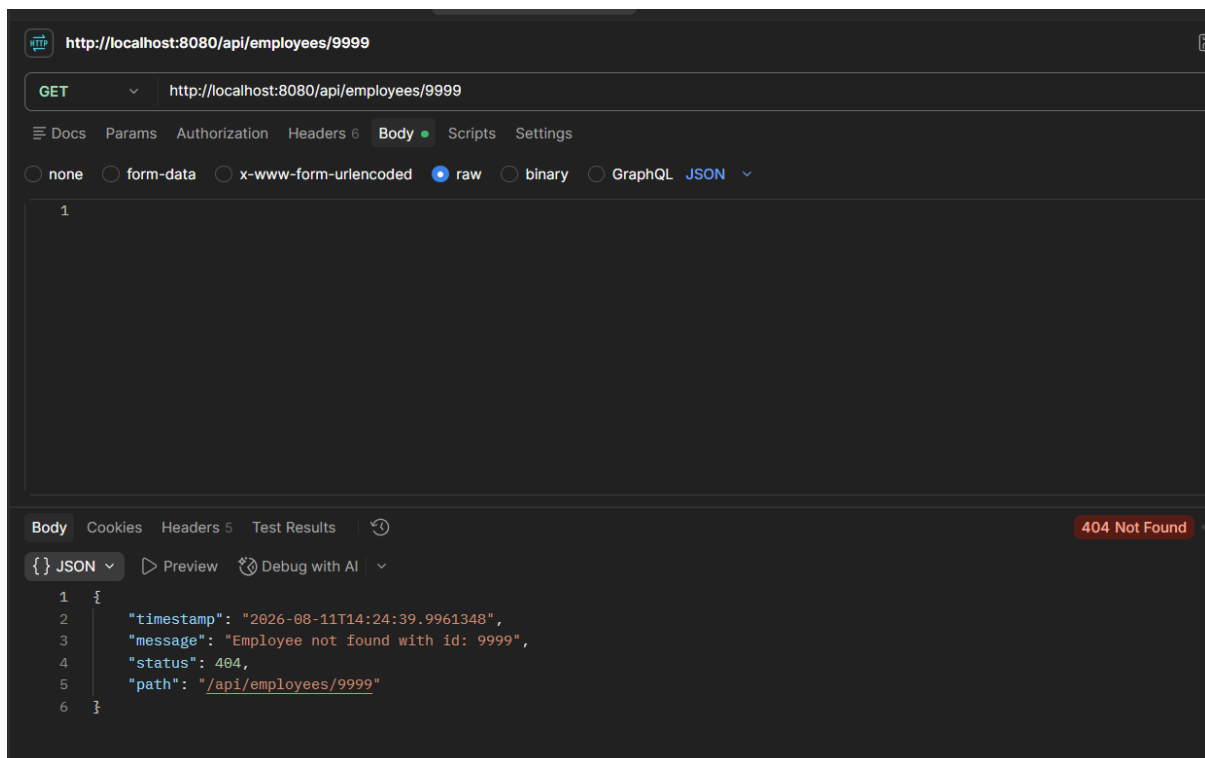
    employeeRepository.delete(existingEmployee);
}
}
```

API'S TESTING USING POSTMAN

1 GET API – Resource Not Found Exception

The GET API is used to retrieve an employee using the employee ID. When a nonexisting employee ID is requested, the application throws a `ResourceNotFoundException`. The `GlobalExceptionHandler` catches the exception and returns a custom JSON response containing the timestamp, message, status code, and request path instead of Spring Boot's default error page.

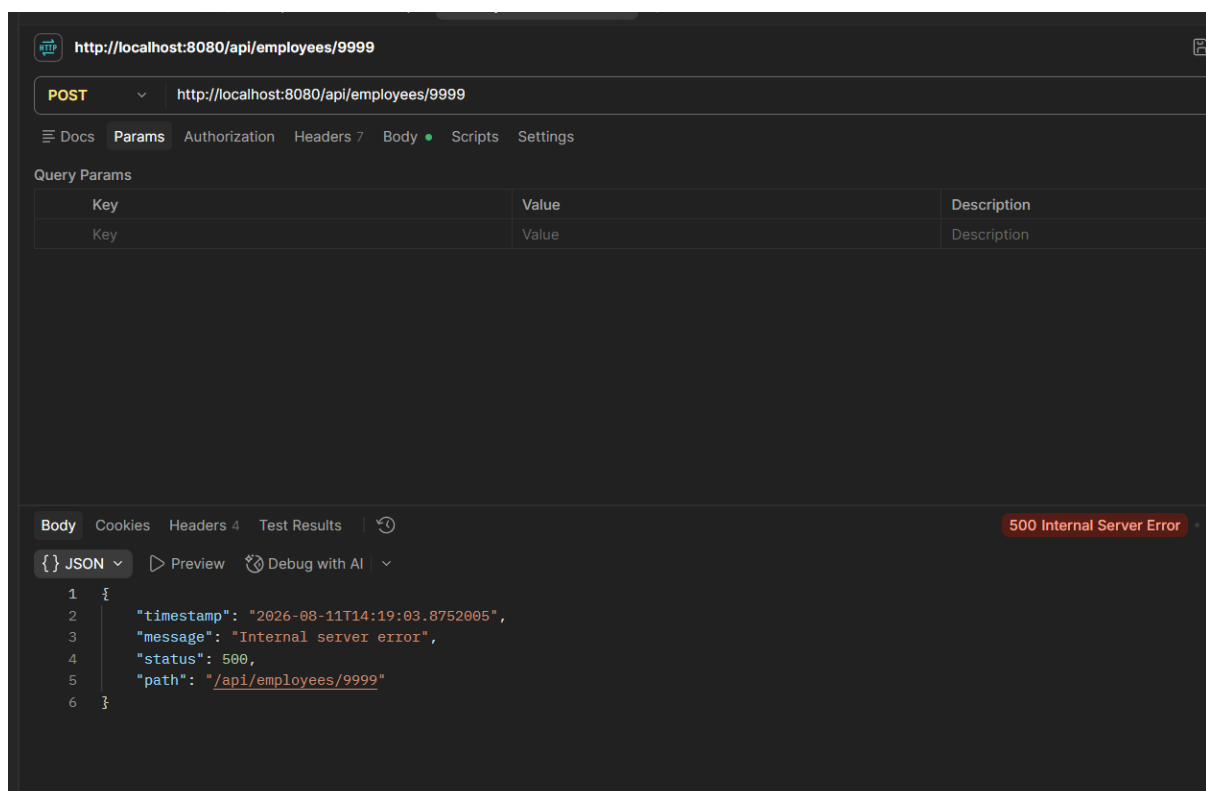
GET <http://localhost:8080/api/employees/9999>



2 POST API –

Validation Exception The POST API is used to demonstrate Bean Validation together with Global Exception Handling. When invalid customer details such as an empty name, invalid email, or age below 18 are submitted, Spring Boot throws a validation exception. The `GlobalExceptionHandler` captures the exception and returns a structured JSON response containing field-level validation errors.

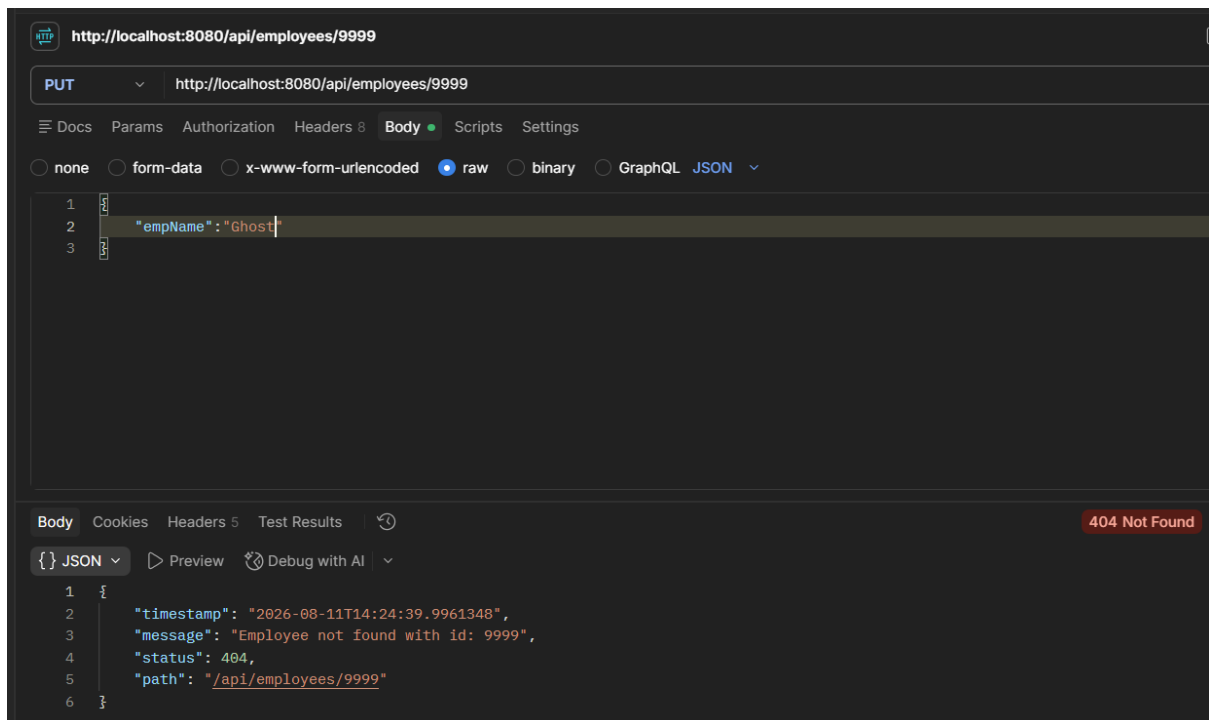
POST <http://localhost:8080/api/employees/9999>



3 PUT API –

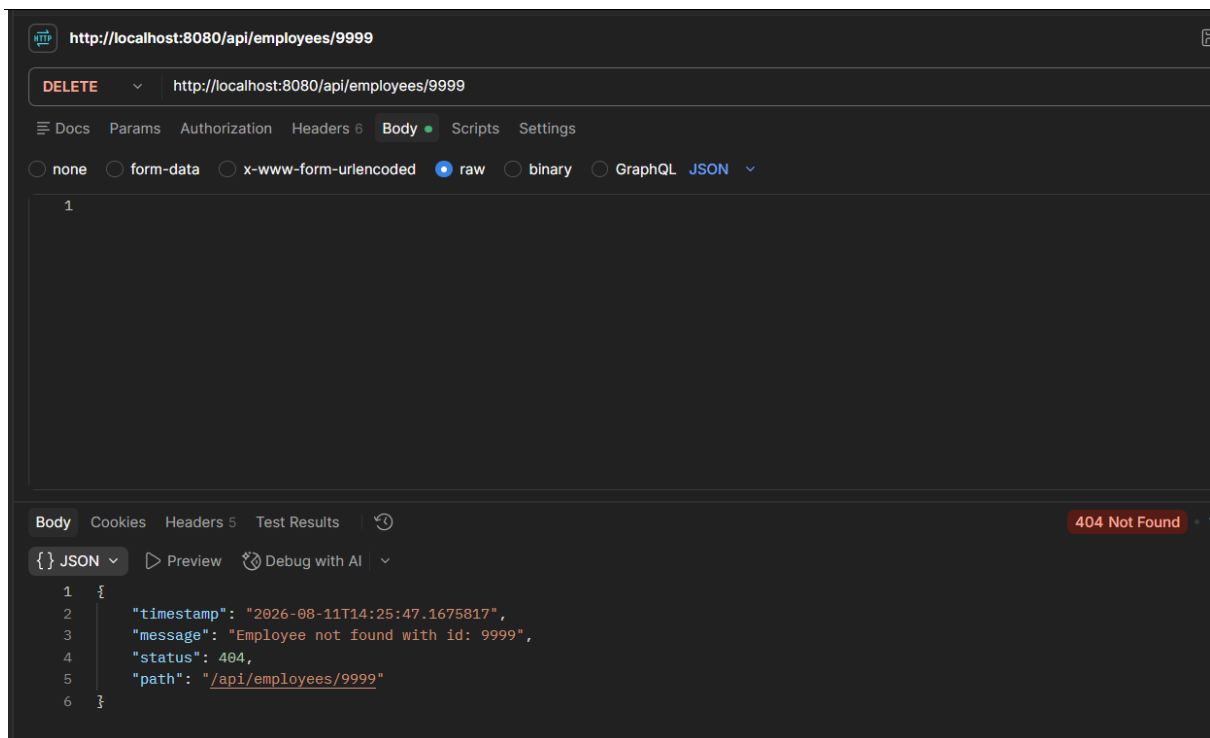
Update Non-Existing Employee The PUT API is used to update an employee record. If the specified employee ID does not exist, the application throws a `ResourceNotFoundException`. The Global Exception Handler catches the exception and returns a custom 404 Not Found response instead of displaying a stack trace

PUT <http://localhost:8080/api/employees/9999>



4.Delete API– Delete Non-Existing Employee The DELETE API is used to remove an employee record from the database. If the employee ID does not exist, a `ResourceNotFoundException` is thrown. The Global Exception Handler processes the exception and returns a structured JSON error response with the appropriate HTTP status code.

DELETE <http://localhost:8080/api/employees/9999>



CH 7 SECURING A WEB APPLICATION

Practical No. 15: Securing REST API with HTTP Basic Authentication (Spring Security)

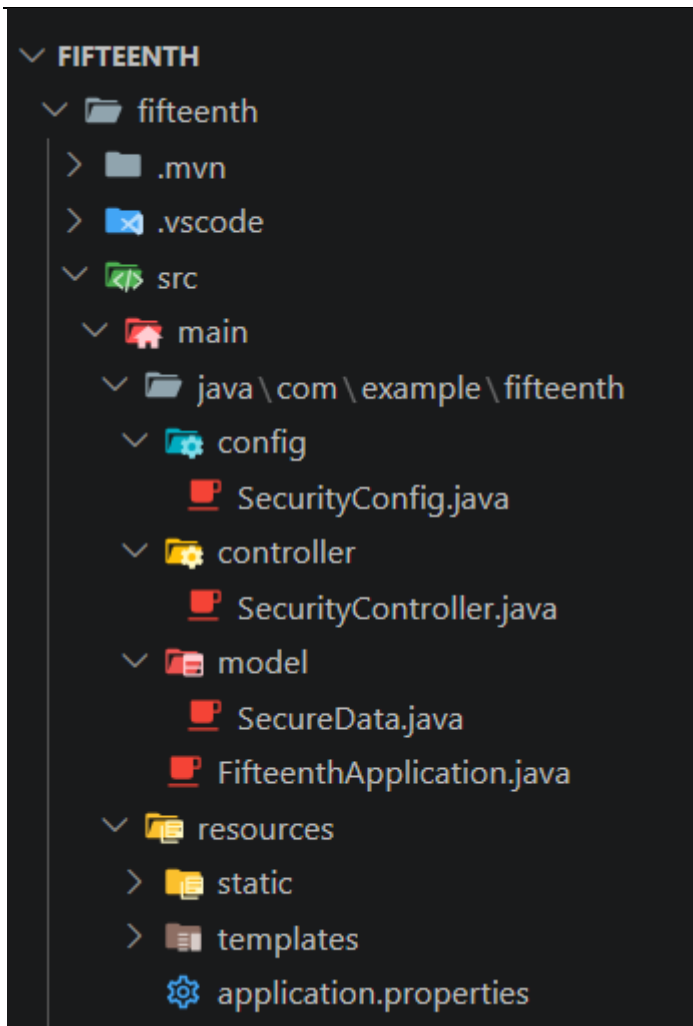
Objective:

To add Spring Security to a Spring Boot REST application, configure in-memory users with encoded passwords, and secure selected endpoints so that only authenticated requests can access them — the introductory security mechanism covered in this chapter.

Class Name(s) with Attributes:

SecurityConfig (@Configuration)

- Bean: SecurityFilterChain filterChain()
- Bean: PasswordEncoder passwordEncoder() — BCryptPasswordEncoder
- Bean: InMemoryUserDetailsManager userDetailsService()



Dependencies

-starter security

SecurityConfig.java(Config)

```
package com.example.fifteenth.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.Customizer;
```

```
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.core.userdetails.User;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.provisioning.InMemoryUserDetailsManager;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
public class SecurityConfig {

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Bean
    public UserDetailsService userDetailsService() {

        UserDetails user = User.builder()
            .username("user")
            .password(passwordEncoder().encode("password"))
            .roles("USER")
            .build();

        return new InMemoryUserDetailsManager(user);
    }
}
```

```
}

@Bean
public SecurityFilterChain filterChain(HttpSecurity http)
    throws Exception {

    http

        // Disable CSRF for REST API
        .csrf(csrf -> csrf.disable())

        .authorizeHttpRequests(auth -> auth

            // Public APIs
            .requestMatchers("/api/public/**")
            .permitAll()

            // Secure APIs
            .requestMatchers("/api/secure/**")
            .authenticated()

            // Everything else
            .anyRequest()
            .authenticated()
        )

        // Basic Authentication
        .httpBasic(Customizer.withDefaults());
}
```

```
        return http.build();
    }
}
```

SecurityController.java(Controller)

```
package com.example.fifteenth.controller;

import java.util.ArrayList;
import java.util.List;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import com.example.fifteenth.model.SecureData;

@RestController
@RequestMapping("/api")
public class SecurityController {

    private final List<SecureData> dataList = new ArrayList<>();

    // Used to automatically generate IDs
    private int nextId = 1;
```



```
// =====  
  
// PUBLIC API  
  
// =====  
  
@GetMapping("/public/data")  
public String publicData() {  
  
    return "This is Public Data";  
}  
  
  
// =====  
  
// SECURE GET - GET ALL DATA  
  
// =====  
  
@GetMapping("/secure/data")  
public ResponseEntity<List<SecureData>> getData() {  
  
    return ResponseEntity.ok(dataList);  
}  
  
  
// =====  
  
// SECURE GET - GET DATA BY ID  
  
// =====  
  
@GetMapping("/secure/data/{id}")  
public ResponseEntity<SecureData> getDataById(  
    @PathVariable Integer id) {  
  
    for (SecureData data : dataList) {
```

```
        if (data.getId().equals(id)) {

            return ResponseEntity.ok(data);
        }
    }

    return ResponseEntity.notFound().build();
}

// SECURE POST - ADD DATA

@PostMapping("/secure/data")
public ResponseEntity<SecureData> addData(
    @RequestBody SecureData data) {

    // Automatically generate ID
    data.setId(nextId++);

    // Add object to list
    dataList.add(data);

    return ResponseEntity
        .status(HttpStatus.CREATED)
        .body(data);
}

// SECURE PUT - UPDATE DATA
```

```
@PutMapping("/secure/data/{id}")

public ResponseEntity<SecureData> updateData(

    @PathVariable Integer id,

    @RequestBody SecureData data) {

    for (SecureData d : dataList) {

        if (d.getId().equals(id)) {

            d.setText(data.getText());

            return ResponseEntity.ok(d);

        }

    }

    return ResponseEntity.notFound().build();

}

// SECURE DELETE - DELETE DATA

@DeleteMapping("/secure/data/{id}")

public ResponseEntity<String> deleteData(

    @PathVariable Integer id) {

    boolean removed = dataList.removeIf(

        d -> d.getId().equals(id)
```

```
);

if (removed) {
    return ResponseEntity.ok("Data deleted successfully");
}

return ResponseEntity
    .status(HttpStatus.NOT_FOUND)
    .body("Data not found");
}
}
```

SecureData.java(model)

```
package com.example.fifteenth.model;

public class SecureData {

    private Integer id;

    private String text;

    // Default constructor
    public SecureData() {
    }
}
```

```
// Parameterized constructor
public SecureData(Integer id, String text) {

    this.id = id;

    this.text = text;
}

// Getter for ID
public Integer getId() {

    return id;
}

// Setter for ID
public void setId(Integer id) {

    this.id = id;
}

// Getter for text
public String getText() {

    return text;
}

// Setter for text
public void setText(String text) {

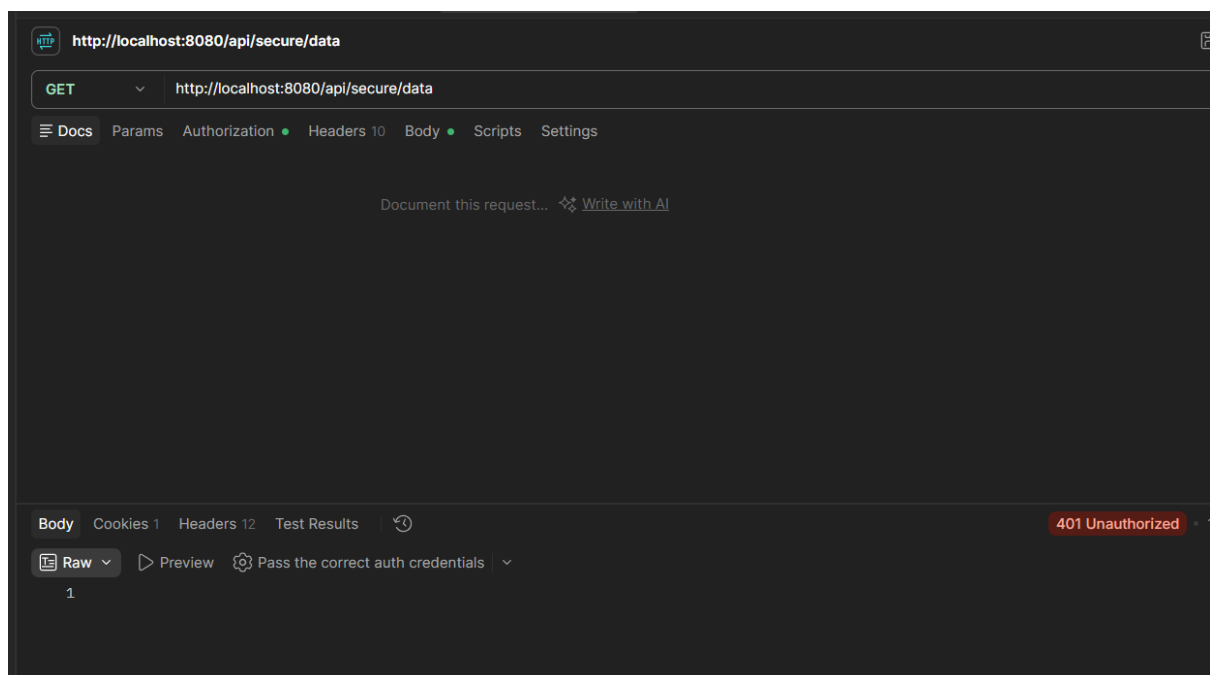
    this.text = text;
}
}
```

API'S TESTING USING POSTMAN

1. GET API – Access Secure Data Without Authentication

The GET API is used to access the secured REST endpoint without providing any authentication credentials. Since the endpoint is protected by Spring Security, the request is rejected and the server returns **401 Unauthorized**.

GET <http://localhost:8080/api/secure/data>

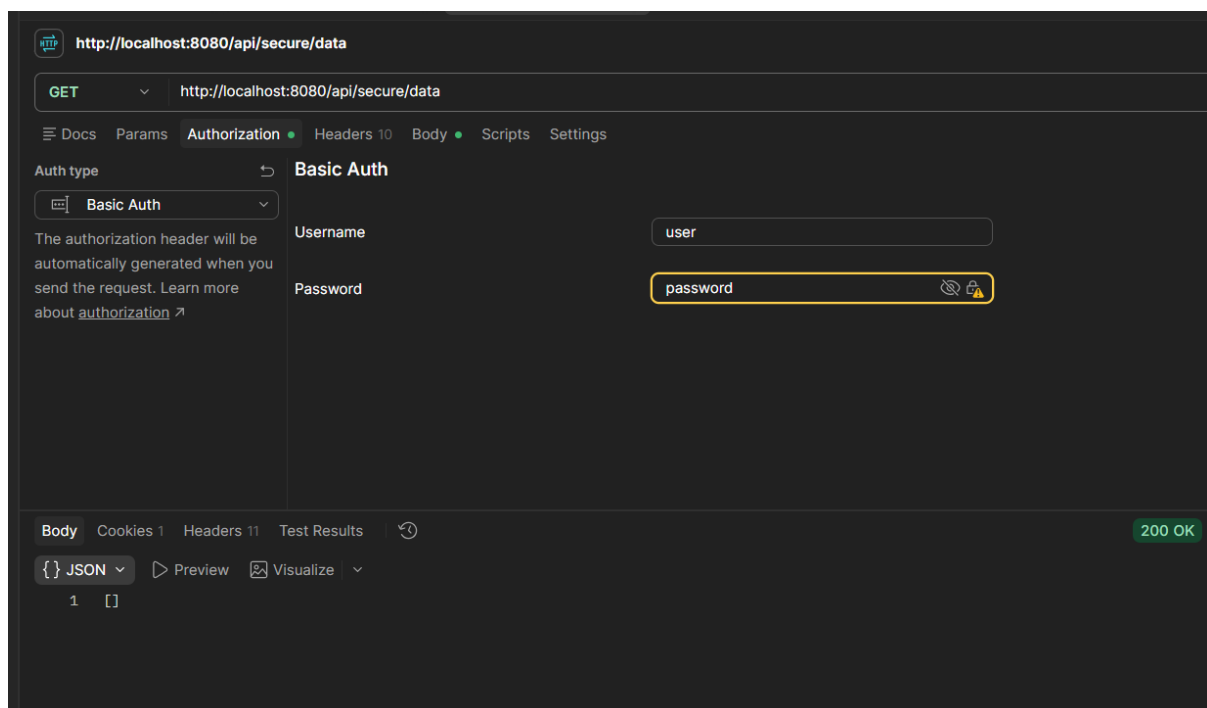


Shows' unauthorized

2. GET API – Access Secure Data With Basic Authentication

The GET API is used to access the secured REST endpoint using valid Basic Authentication credentials. Spring Security authenticates the user successfully and returns the protected data.

GET <http://localhost:8080/api/secure/data>

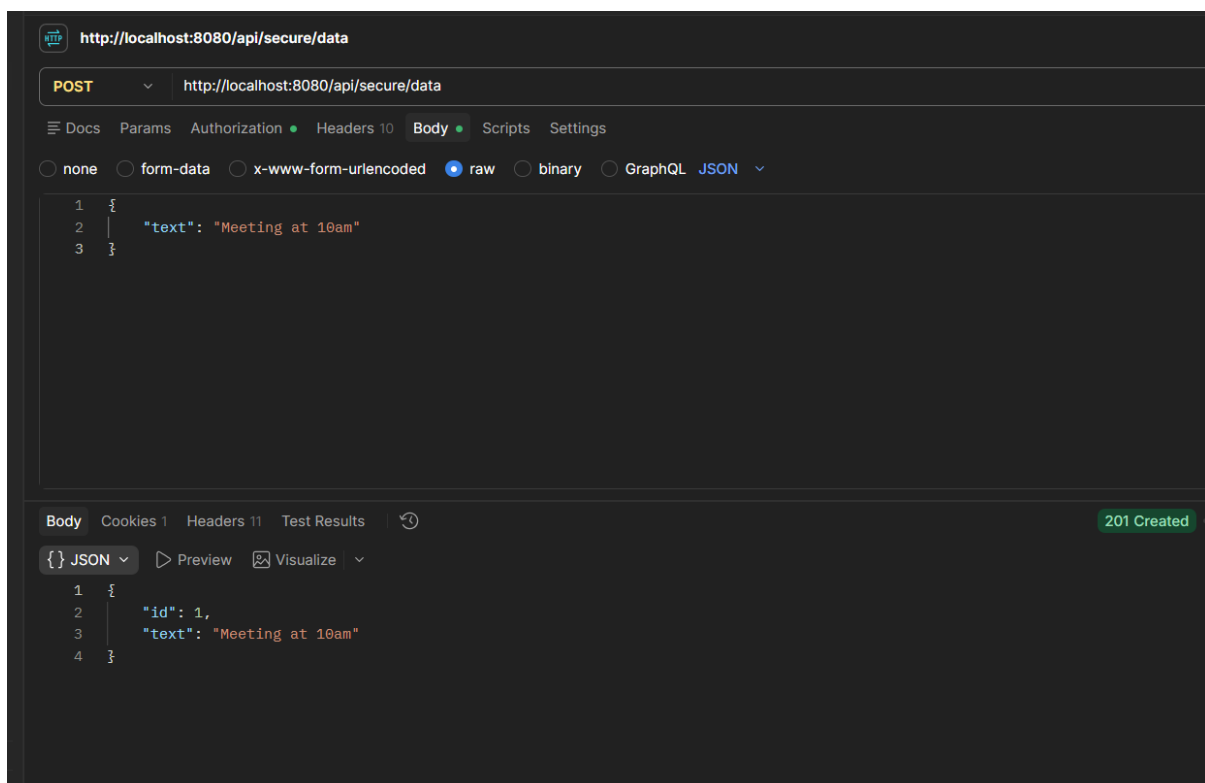


SEE THE SECURED DATA IS DISPLAYED SUCCESSFULLY AFTER SUCCESSFUL AUTHENTICATION.

3. POST API – Add Secure Data

The POST API is used to create a new secure data record. The data is sent in JSON format through the request body. Only authenticated users can access this endpoint.

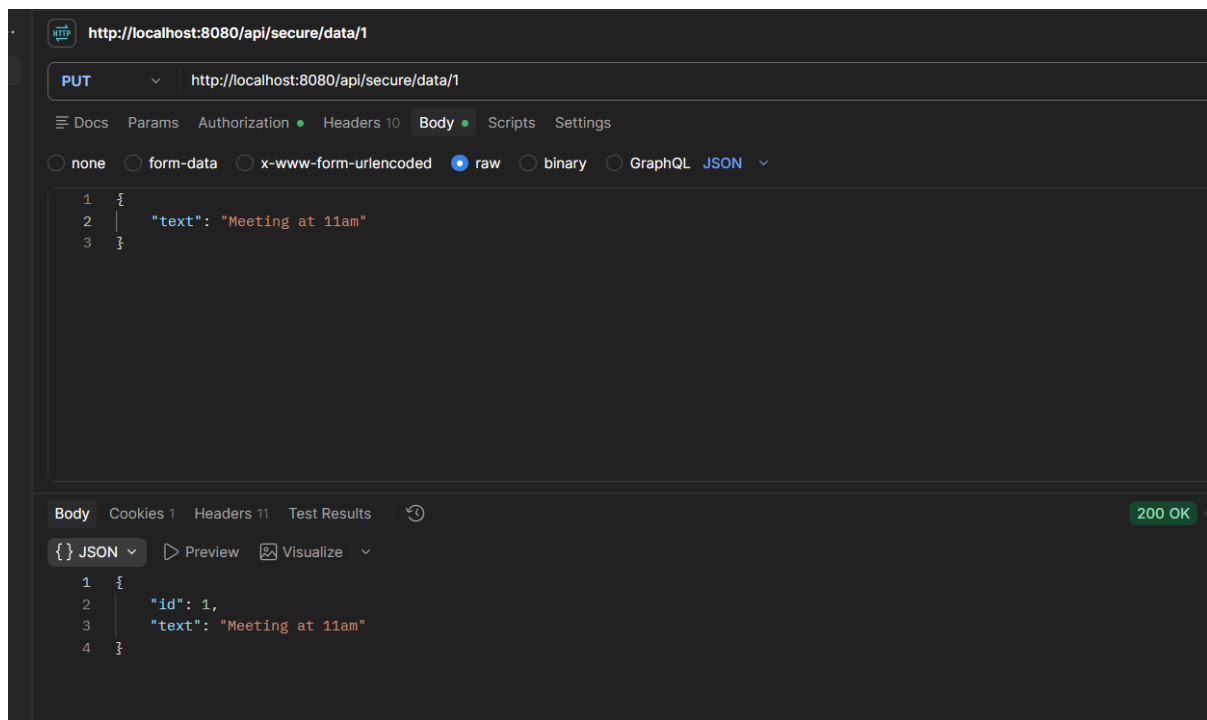
POST <http://localhost:8080/api/secure/data>



4. PUT API – Update Secure Data

The PUT API is used to update the existing secure data. The data ID is passed through the URL, while the updated information is sent in the request body. Authentication is required before updating the resource..

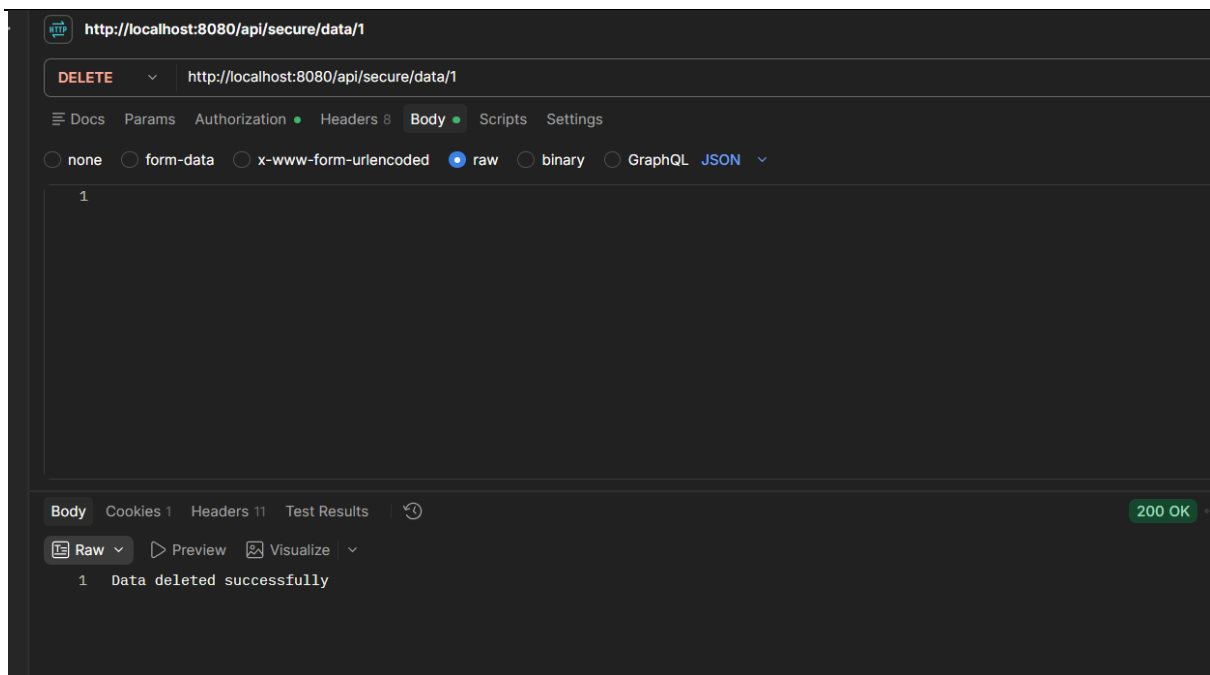
PUT <http://localhost:8080/api/secure/data/1>



5. DELETE API – Delete Secure Data

The DELETE API is used to remove the secure data having ID 1. Only authenticated users are allowed to delete the resource.

DELETE <http://localhost:8080/api/secure/data/1>

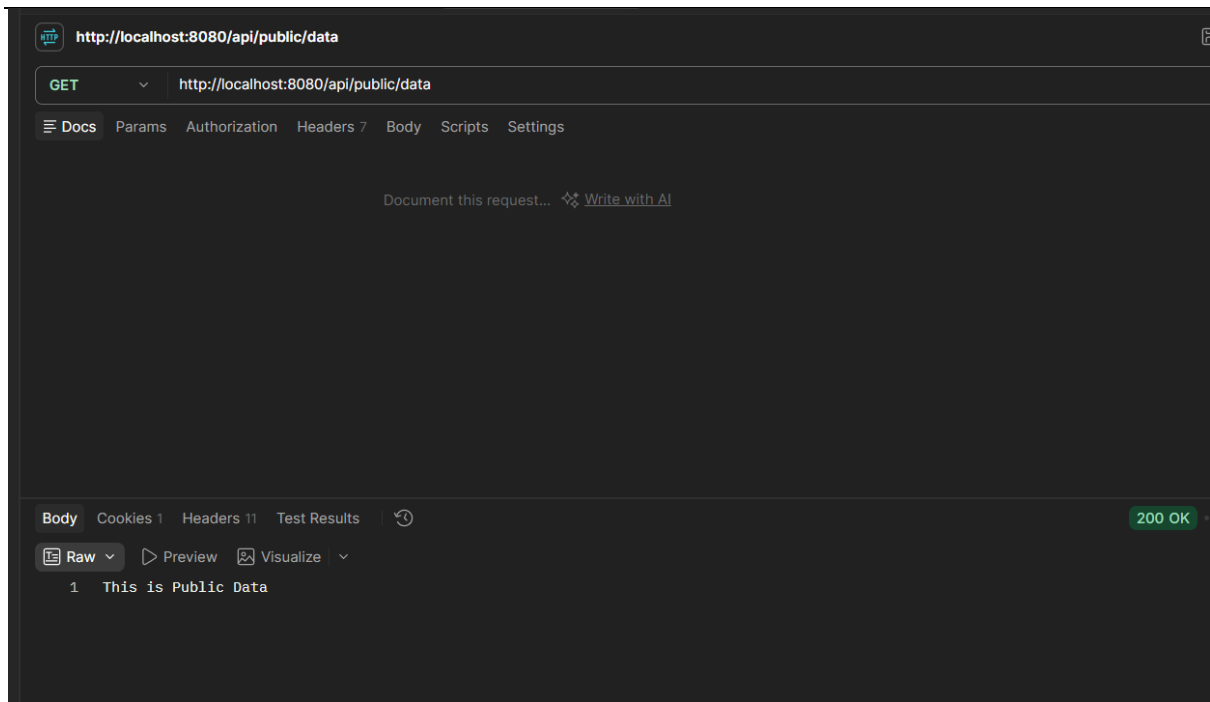


6 GET API – Access Public Data

➔ REMOVE PASSWORD AND USER NAME

The GET API is used to access the public REST endpoint. This endpoint is configured with `permitAll()` in Spring Security, so no authentication is required.

GET <http://localhost:8080/api/public/data>



SEE THE PUBLIC DATA IS DISPLAYED SUCCESSFULLY WITHOUT LOGIN OR AUTHENTICATION.

Practical No. 16: User Registration and Login with BCrypt Password Encoding

Objective:

To implement a database-backed user registration and login mechanism, storing passwords securely using BCryptPasswordEncoder instead of plain text, and authenticating against the database via a custom UserDetailsService — extending the password-encoding concept from this chapter to a persistent user store

Class Name(s) with Attributes:

AppUser (@Entity)

- userId : int (@Id @GeneratedValue)
- username : String (unique)
- password : String (BCrypt-encoded)
- role : String

AppUserRepository (extends JpaRepository)

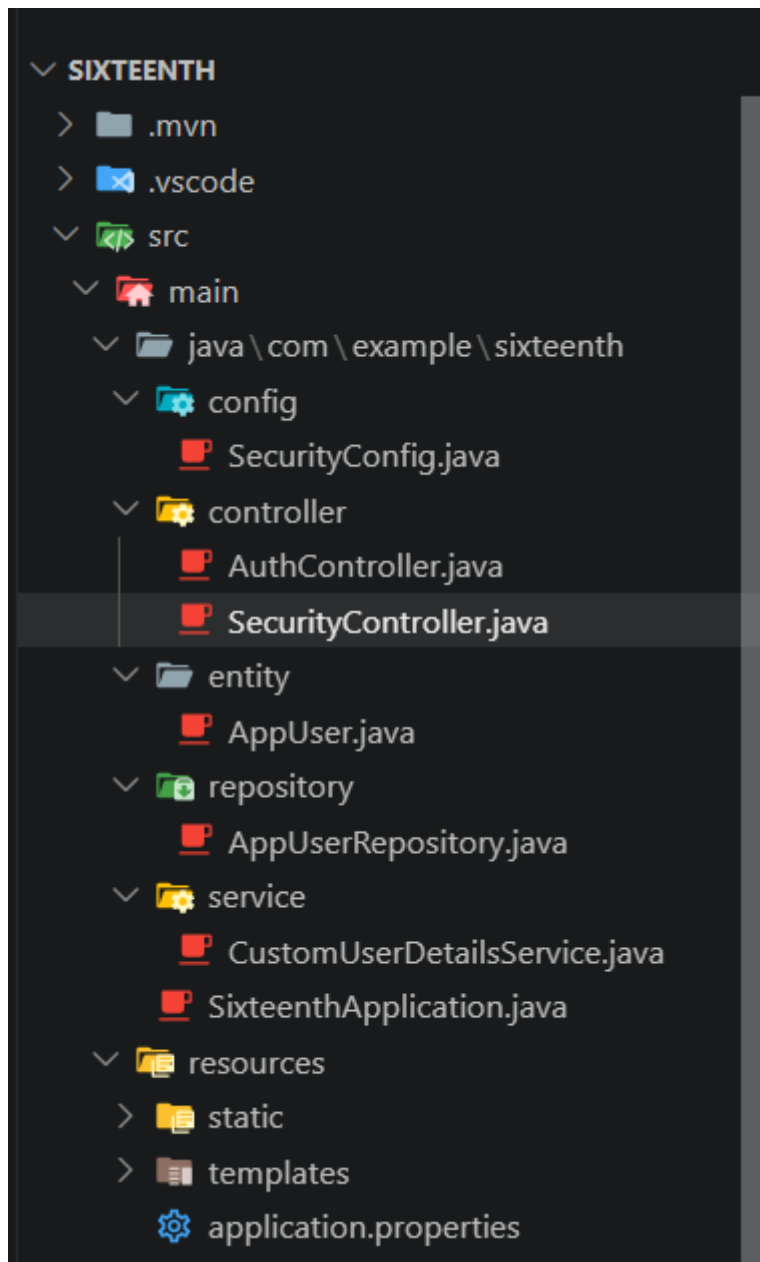
- Optional findByUsername(String username)

CustomUserDetailsService (implements UserDetailsService)

- appUserRepository : AppUserRepository

AuthController (@RestController)

- Method: register(@RequestBody AppUser) — encodes password before saving

**Dependencies:-**

1. Spring Web
 - 2 Spring Data JPA
 - 3 Spring Security
 - 4 MySQL Driver
-

Database:-

Sixteenth

Table:- App_user (which create automatic)

SecutrityConfig.java (Config)

```
package com.example.sixteenth.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.authentication.AuthenticationProvider;
import org.springframework.security.authentication.dao.DaoAuthenticationProvider;
import org.springframework.security.config.Customizer;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
public class SecurityConfig {

    // BCrypt Password Encoder

    @Bean
    public PasswordEncoder passwordEncoder() {
```

```
        return new BCryptPasswordEncoder();
    }

    // Authentication Provider
    @Bean
    public AuthenticationProvider authenticationProvider(
        UserDetailsService userDetailsService,
        PasswordEncoder passwordEncoder) {

        DaoAuthenticationProvider provider =
            new DaoAuthenticationProvider(userDetailsService);

        provider.setPasswordEncoder(passwordEncoder);
        return provider;
    }

    // Security Configuration
    @Bean
    public SecurityFilterChain filterChain(
        HttpSecurity http) throws Exception {

        http
            .csrf(csrf -> csrf.disable())

            .authorizeHttpRequests(auth -> auth
```

```
// Register is public
.requestMatchers(
    "/api/auth/register")
.permitAll()

// Other auth APIs are secure
.requestMatchers(
    "/api/auth/**")
.authenticated()

// Secure API
.requestMatchers(
    "/api/secure/**")
.authenticated()

.anyRequest()
.authenticated()
)

.httpBasic(Customizer.withDefaults());

return http.build();
}
}
```


AuthController.java(controller)

```
package com.example.sixteenth.controller;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.security.core.Authentication;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import com.example.sixteenth.entity.AppUser;
import com.example.sixteenth.repository.AppUserRepository;

@RestController
@RequestMapping("/api/auth")
public class AuthController {

    private final AppUserRepository appUserRepository;
    private final PasswordEncoder passwordEncoder;

    public AuthController(
```

```
AppUserRepository appUserRepository,
PasswordEncoder passwordEncoder) {

    this.appUserRepository = appUserRepository;
    this.passwordEncoder = passwordEncoder;
}

// Register new user
@PostMapping("/register")
public ResponseEntity<AppUser> register(
    @RequestBody AppUser appUser) {

    appUser.setPassword(
        passwordEncoder.encode(
            appUser.getPassword()));

    AppUser savedUser
        = appUserRepository.save(appUser);

    return ResponseEntity
        .status(HttpStatus.CREATED)
        .body(savedUser);
}

// Update authenticated user's profile
@PutMapping("/profile")
public ResponseEntity<AppUser> updateProfile(
```

```
@RequestBody AppUser updatedUser,
Authentication authentication) {

String currentUsername
    = authentication.getName();

AppUser existingUser
    = appUserRepository
        .findByUsername(currentUsername)
        .orElseThrow(()
            -> new RuntimeException(
                "User not found"));

if (updatedUser.getUsername() != null
    && !updatedUser.getUsername().isBlank()) {

    existingUser.setUsername(
        updatedUser.getUsername());
}

if (updatedUser.getRole() != null
    && !updatedUser.getRole().isBlank()) {

    existingUser.setRole(
        updatedUser.getRole());
}
```

```
AppUser savedUser
    = appUserRepository.save(existingUser);

return ResponseEntity.ok(savedUser);
}

// Delete authenticated user's account
// Delete user
@DeleteMapping("/register/{id}")
public ResponseEntity<String> deleteUser(
    @PathVariable Integer id,
    Authentication authentication) {

    AppUser user = appUserRepository
        .findById(id)
        .orElseThrow(()
            -> new RuntimeException("User not found"));

    if (!user.getUsername()
        .equals(authentication.getName())) {

        return ResponseEntity
            .status(HttpStatus.FORBIDDEN)
            .body("You can delete only your own account");
    }

    appUserRepository.delete(user);
}
```

```
        return ResponseEntity
            .ok("User deleted successfully");
    }
}
```

SecurityController.java(Controller)

```
package com.example.sixteenth.controller;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/api/secure")
public class SecurityController {

    @GetMapping("/data")
    public String getData() {

        return "This is Secure Data";
    }
}
```

AppUser.java(entity)

```
package com.example.sixteenth.entity;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;

@Entity
public class AppUser {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer userId;

    private String username;

    private String password;

    private String role;

    public AppUser() {
    }

    public Integer getUserId() {
        return userId;
    }
}
```

```
public void setUserId(Integer userId) {  
    this.userId = userId;  
}  
  
public String getUsername() {  
    return username;  
}  
  
public void setUsername(String username) {  
    this.username = username;  
}  
  
public String getPassword() {  
    return password;  
}  
  
public void setPassword(String password) {  
    this.password = password;  
}  
  
public String getRole() {  
    return role;  
}  
  
public void setRole(String role) {  
    this.role = role;  
}
```

```
}  
}
```

AppRepository.java(repository)

```
package com.example.sixteenth.repository;  
  
import java.util.Optional;  
  
import org.springframework.data.jpa.repository.JpaRepository;  
  
import com.example.sixteenth.entity.AppUser;  
  
public interface AppUserRepository  
    extends JpaRepository<AppUser, Integer> {  
  
    Optional<AppUser> findByUsername(String username);  
}
```

CustomUserDetailsService.java(Service)

```
package com.example.sixteenth.service;  
  
import org.springframework.security.core.userdetails.User;  
import org.springframework.security.core.userdetails.UserDetails;  
import org.springframework.security.core.userdetails.UserDetailsService;  
import org.springframework.security.core.userdetails.UsernameNotFoundException;  
import org.springframework.stereotype.Service;
```



```
import com.example.sixteenth.entity.AppUser;

import com.example.sixteenth.repository.AppUserRepository;

@Service

public class CustomUserDetailsService

    implements UserDetailsService {

    private final AppUserRepository appUserRepository;

    public CustomUserDetailsService(

        AppUserRepository appUserRepository) {

        this.appUserRepository = appUserRepository;
    }

    @Override

    public UserDetails loadUserByUsername(String username)

        throws UsernameNotFoundException {

        AppUser appUser = appUserRepository

            .findByUsername(username)

            .orElseThrow(() ->

                new UsernameNotFoundException(

                    "User not found"));

        return User.builder()

            .username(appUser.getUsername())
```

```
.password(appUser.getPassword())  
.roles(appUser.getRole())  
.build();  
}  
}
```

Application.properties

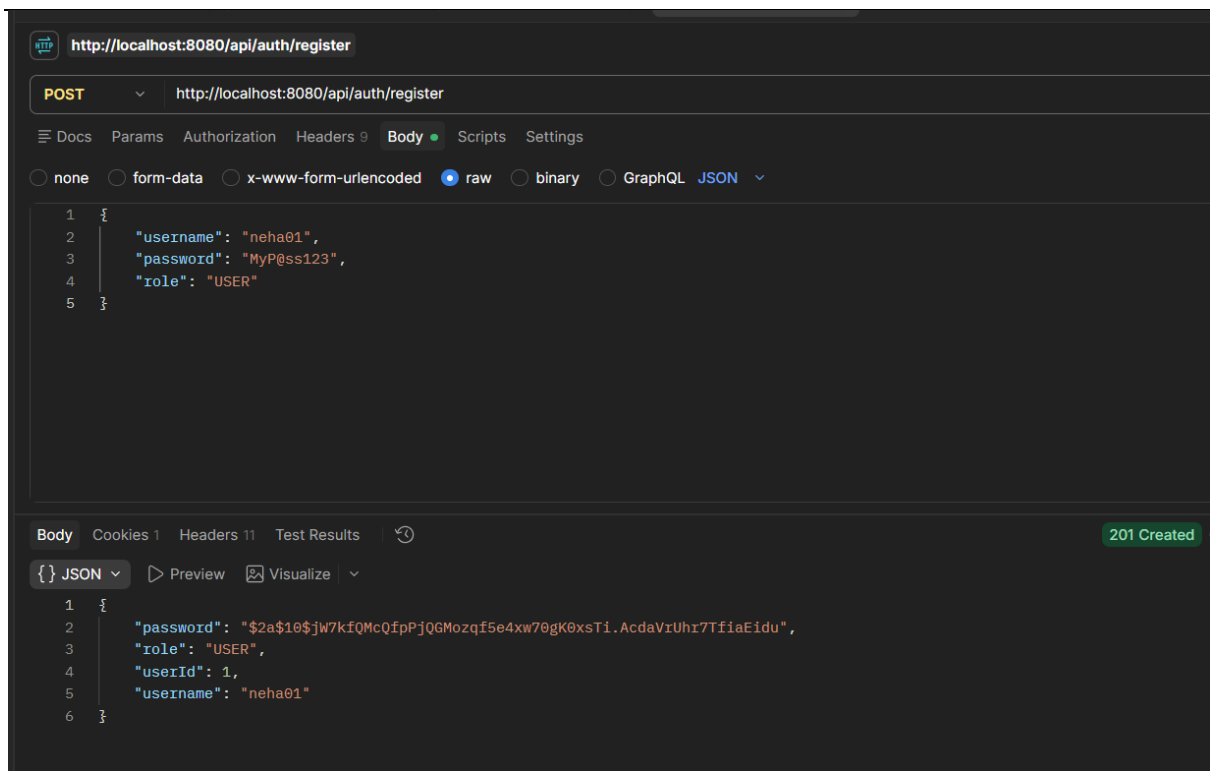
```
spring.application.name=sixteenth  
  
spring.datasource.url=jdbc:mysql://localhost:3306/sixteenth  
spring.datasource.username=root  
spring.datasource.password=  
  
spring.jpa.hibernate.ddl-auto=update  
spring.jpa.show-sql=true
```

API'S TESTING USING POSTMAN

1. POST API – User Registration

The POST API is used to register a new user. The username, plain-text password, and role are sent in JSON format. Before saving, BCryptPasswordEncoder converts the password into a BCrypt hash and stores the hashed password in the database.

POST <http://localhost:8080/api/auth/register>

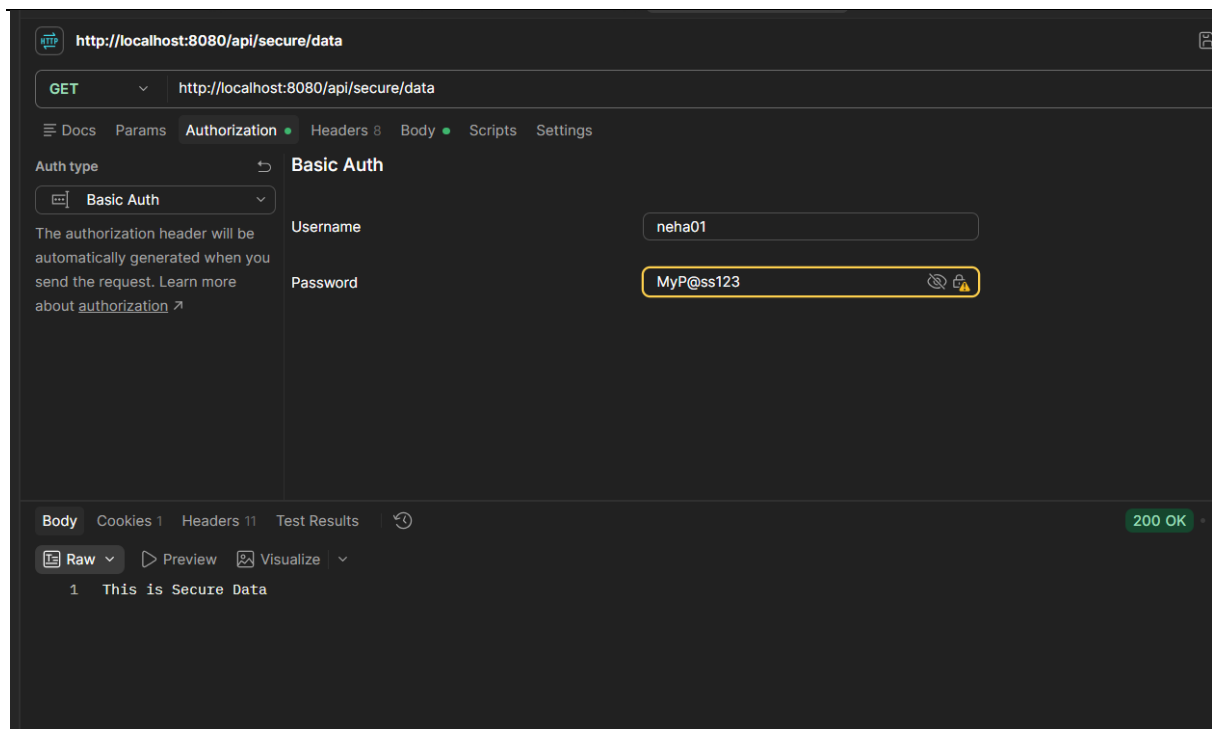


2. GET API – Access Secure Data

The GET API is used to access the secured data endpoint using the username and password of the newly registered user. Spring Security uses CustomUserDetailsService to find the username from the database and BCrypt to verify the password.

After fill the password and username

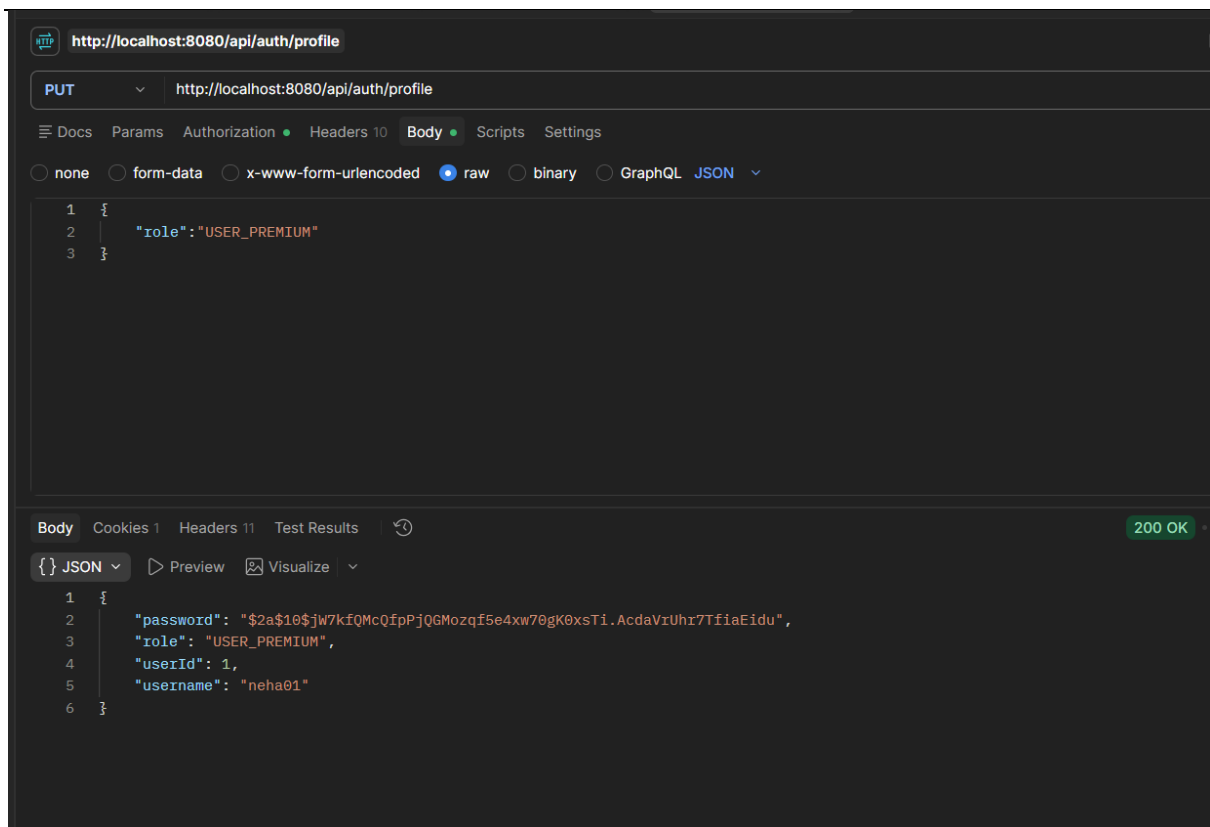
GET <http://localhost:8080/api/secure/data>



3 .PUT API – Update User Profile

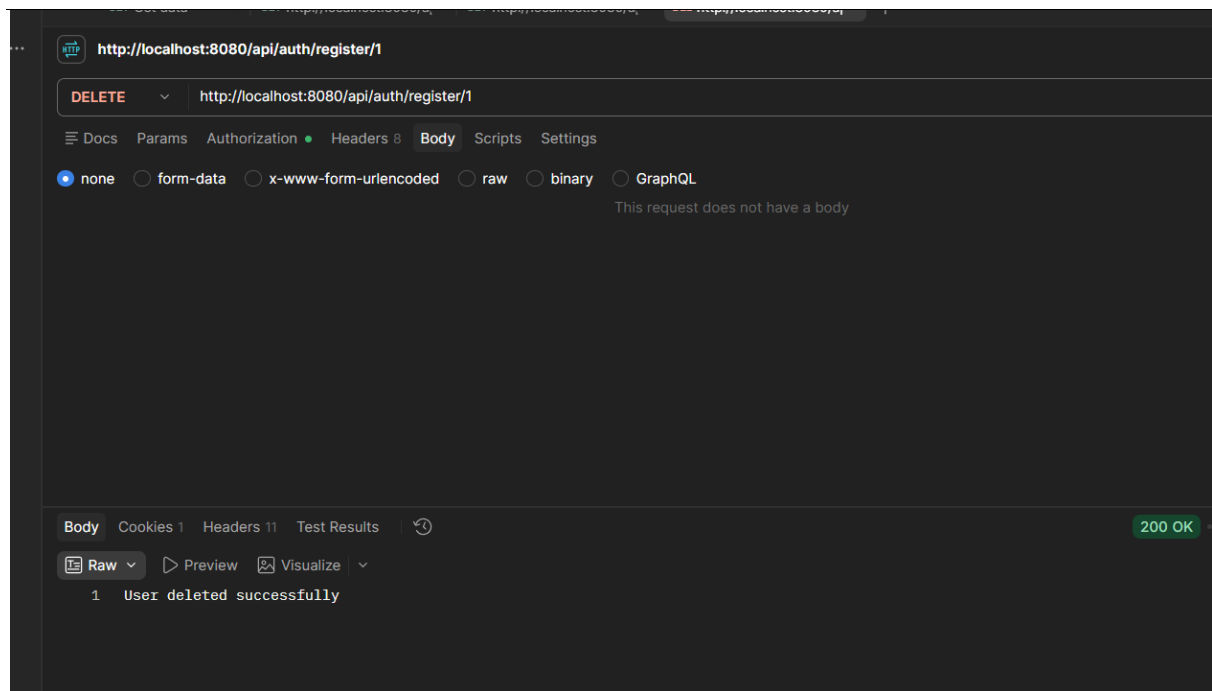
The PUT API is used to update the profile of the currently authenticated user. Basic Authentication is required, and the authenticated username is obtained from Spring Security.

PUT <http://localhost:8080/api/auth/profile>



4.DELETE :- to delete any user delete api key end point

DELETE <http://localhost:8080/api/auth/register/1>



BEFORE THE DELETE DATABASE OUTPUT:-



Practical No. 17: Role-Based Access Control (RBAC) **with Spring Security**

Objective:

To extend Spring Security configuration for role-based authorization, defining users with different roles (ADMIN, USER) and restricting specific endpoints using @PreAuthorize or role-based matchers, as covered in this chapter's authorization section

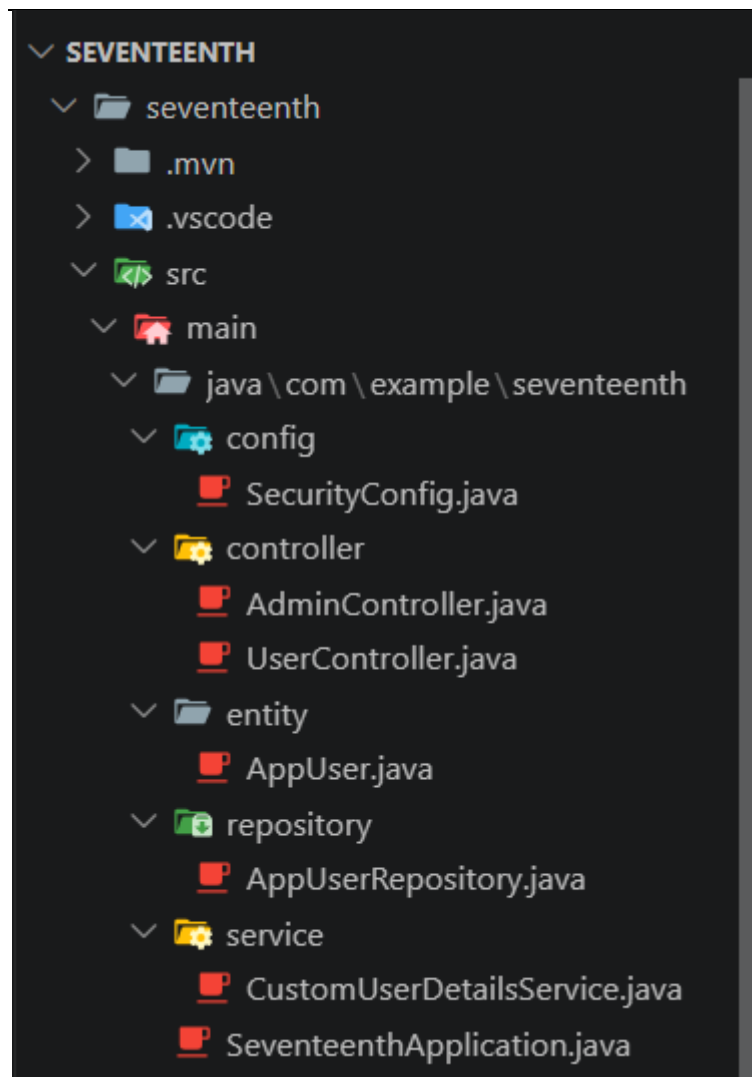
Class Name(s) with Attributes:

SecurityConfig (extended)

- Users carrying ROLE_ADMIN / ROLE_USER
- @EnableMethodSecurity enabled

AdminController (@RestController)

- Method: getAdminData() — @PreAuthorize("hasRole('ADMIN')")



Dependencies

- Spring Web
- Spring Data JPA
- MySQL Driver

-

Database Name: SevenTeenth

Table: User(created automatically by Hibernate)

SecurityConfig.java(config)

```
package com.example.seventeenth.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.Customizer;
import
org.springframework.security.config.annotation.method.configuration.EnableMethodSecurity;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
@EnableMethodSecurity
public class SecurityConfig {

    @Bean
    public PasswordEncoder passwordEncoder() {

        return new BCryptPasswordEncoder();
    }

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http)
```

```
throws Exception {

    http

        .csrf(csrf -> csrf.disable())

        .authorizeHttpRequests(auth -> auth

            // Registration is public
            .requestMatchers("/api/auth/register")
            .permitAll()

            // Public APIs
            .requestMatchers("/api/public/**")
            .permitAll()

            // Other APIs require login
            .anyRequest()
            .authenticated()

        .httpBasic(Customizer.withDefaults());

    return http.build();
}
}
```

AdminController.java(Controller)

```
package com.example.seventeenth.controller;

import org.springframework.http.ResponseEntity;
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/admin")
public class AdminController {

    // GET - ADMIN ONLY
    @GetMapping("/data")
    @PreAuthorize("hasRole('ADMIN')")
    public ResponseEntity<String> getAdminData() {

        return ResponseEntity.ok(
            "Admin data accessed successfully.");
    }

    // POST - ADMIN ONLY
    @PostMapping("/data")
    @PreAuthorize("hasRole('ADMIN')")
    public ResponseEntity<String> createAdminData(
        @RequestBody String data) {

        return ResponseEntity
```

```
.status(201)

.body("Admin data created: " + data);
}

// PUT - ADMIN ONLY
@PutMapping("/data/{id}")
@PreAuthorize("hasRole('ADMIN')")
public ResponseEntity<String> updateAdminData(

    @PathVariable Integer id,
    @RequestBody String data) {

    return ResponseEntity.ok(

        "Admin data " + id +
        " updated: " + data);
}

// DELETE - ADMIN ONLY
@DeleteMapping("/data/{id}")
@PreAuthorize("hasRole('ADMIN')")
public ResponseEntity<Void> deleteAdminData(

    @PathVariable Integer id) {

    return ResponseEntity.noContent().build();
}
}
```

UserController.java(Controller)

```
package com.example.seventeenth.controller;

import org.springframework.http.ResponseEntity;
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/api/user")
public class UserController {

    @GetMapping("/data")
    @PreAuthorize("hasAnyRole('ADMIN', 'USER')")
    public ResponseEntity<String> getUserData() {

        return ResponseEntity.ok(

            "User data accessed successfully.");
    }
}
```

AppUser.java(entity)

```
package com.example.seventeenth.entity;

import jakarta.persistence.*;

@Entity
public class AppUser {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer userId;

    @Column(unique = true, nullable = false)
    private String username;

    private String password;

    private String role;

    public AppUser() {
    }

    public Integer getUserId() {
        return userId;
    }
}
```

```
public void setUserId(Integer userId) {  
    this.userId = userId;  
}  
  
public String getUsername() {  
    return username;  
}  
  
public void setUsername(String username) {  
    this.username = username;  
}  
  
public String getPassword() {  
    return password;  
}  
  
public void setPassword(String password) {  
    this.password = password;  
}  
  
public String getRole() {  
    return role;  
}  
  
public void setRole(String role) {  
    this.role = role;  
}
```

```
}  
}
```

AppUserRepository.java(repository)

```
package com.example.seventeenth.repository;  
  
import java.util.Optional;  
  
import org.springframework.data.jpa.repository.JpaRepository;  
  
import com.example.seventeenth.entity.AppUser;  
  
public interface AppUserRepository  
    extends JpaRepository<AppUser, Integer> {  
  
    Optional<AppUser> findByUsername(String username);  
}
```

CustomUserDetailsService.java(service)

```
package com.example.seventeenth.service;  
  
import org.springframework.security.core.userdetails.User;  
import org.springframework.security.core.userdetails.UserDetails;  
import org.springframework.security.core.userdetails.UserDetailsService;  
import org.springframework.security.core.userdetails.UsernameNotFoundException;  
import org.springframework.stereotype.Service;
```



```
import com.example.seventeenth.entity.AppUser;
import com.example.seventeenth.repository.AppUserRepository;

@Service
public class CustomUserDetailsService
    implements UserDetailsService {

    private final AppUserRepository appUserRepository;

    public CustomUserDetailsService(
        AppUserRepository appUserRepository) {

        this.appUserRepository = appUserRepository;
    }

    @Override
    public UserDetails loadUserByUsername(String username)
        throws UsernameNotFoundException {

        AppUser appUser = appUserRepository
            .findByUsername(username)
            .orElseThrow(() ->
                new UsernameNotFoundException(
                    "User not found: " + username));

        return User.builder()
```

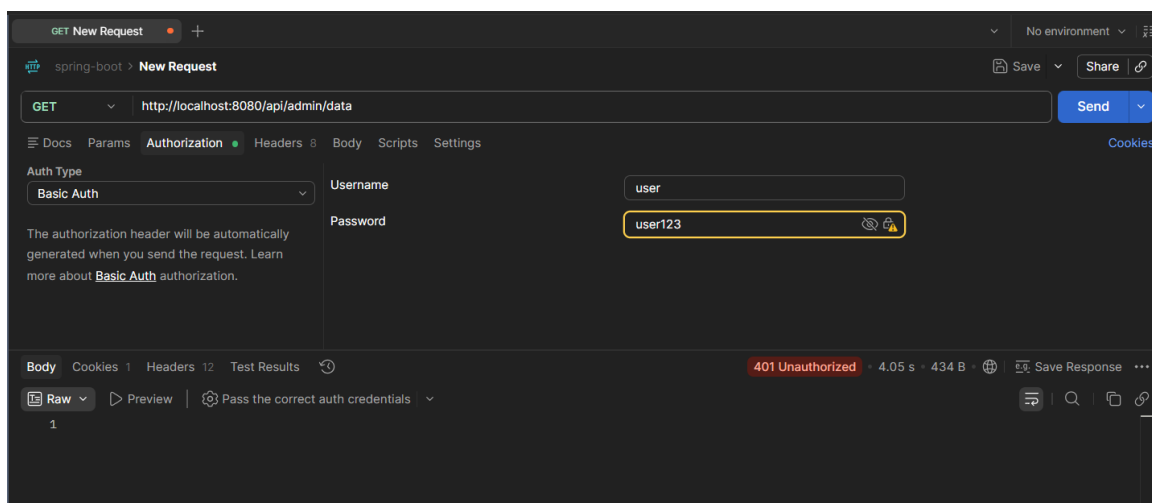
```
.username(appUser.getUsername())  
.password(appUser.getPassword())  
.roles(appUser.getRole())  
.build();  
}  
}
```

API'S TESTING USING POSTMAN

1. GET API – Admin Data with USER Credentials

The GET API is used to access the admin-specific resource. This endpoint is protected using `@PreAuthorize("hasRole('ADMIN')")`, so only users having the ADMIN role can access it.

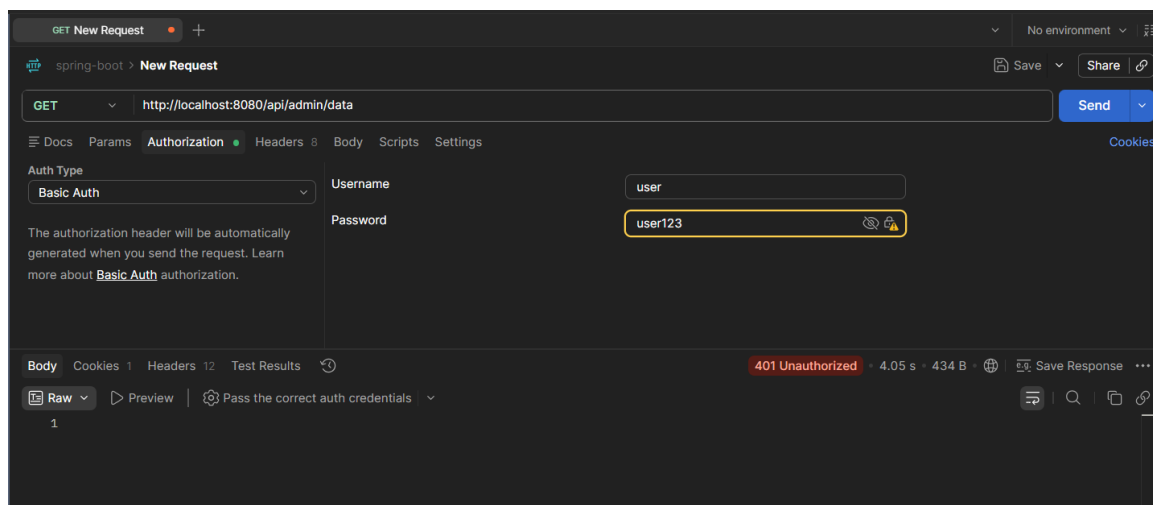
GET <http://localhost:8080/api/admin/data>



2. GET API – Admin Data with ADMIN Credentials

The same GET API is tested again using an ADMIN user's credentials. Since the authenticated user has the ADMIN role, Spring Security allows access to the endpoint.

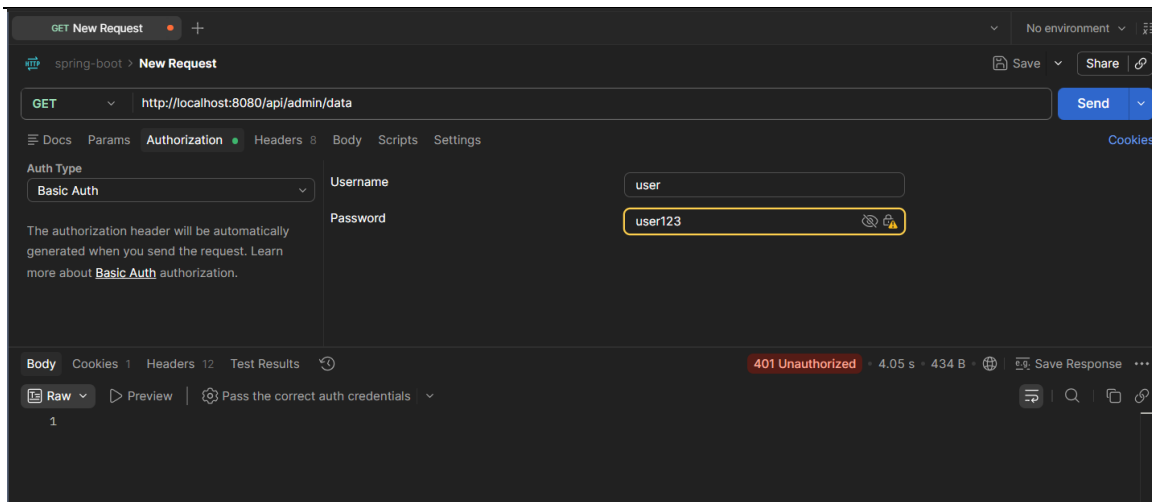
GET <http://localhost:8080/api/admin/data>



3 POST API – Create Admin Data

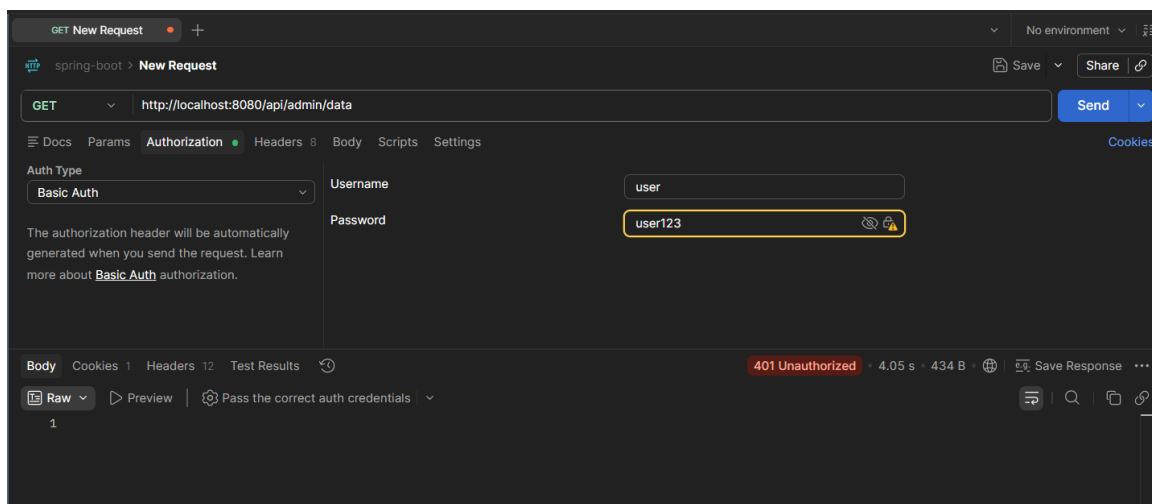
The POST API is used to create a new admin-only resource. The endpoint is protected with the ADMIN role, therefore only an ADMIN user can perform the operation.

POST <http://localhost:8080/api/admin/data>



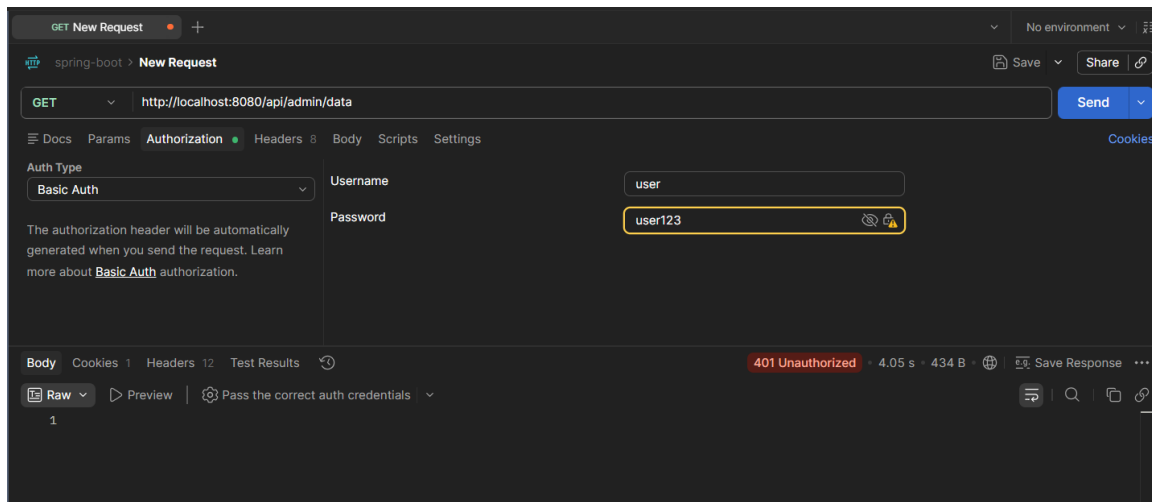
4 PUT API – Update Admin Data

The PUT API is used to update an existing admin resource. The resource ID is provided in the URL, while the updated data is sent in the request body. Only an ADMIN user can perform this operation.



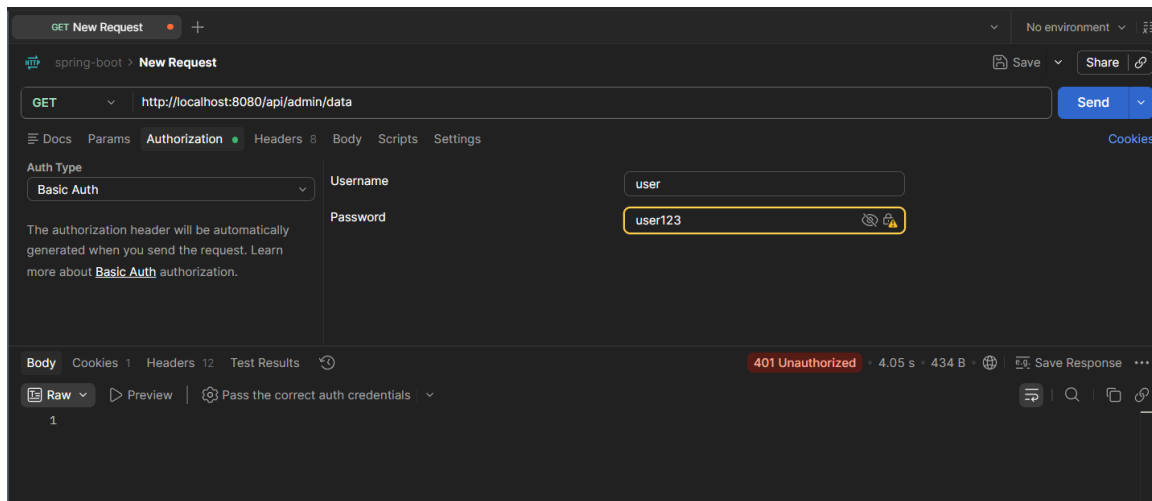
5 DELETE API – Delete Admin Data

The DELETE API is used to delete an admin resource using its ID. This endpoint is also restricted to users having the ADMIN role.



6 GET API – User Data

The GET API is used to access the user resource. It is configured using:



Practical No. 18: Securing REST API as an OAuth2 Resource Server with JWT

Objective:

To configure the Spring Boot application as an OAuth2 Resource Server that validates JSON Web Tokens (JWT) issued by an Authorization Server, protecting REST endpoints so that only requests carrying a valid Bearer token succeed — the token-based security mechanism covered in this chapter

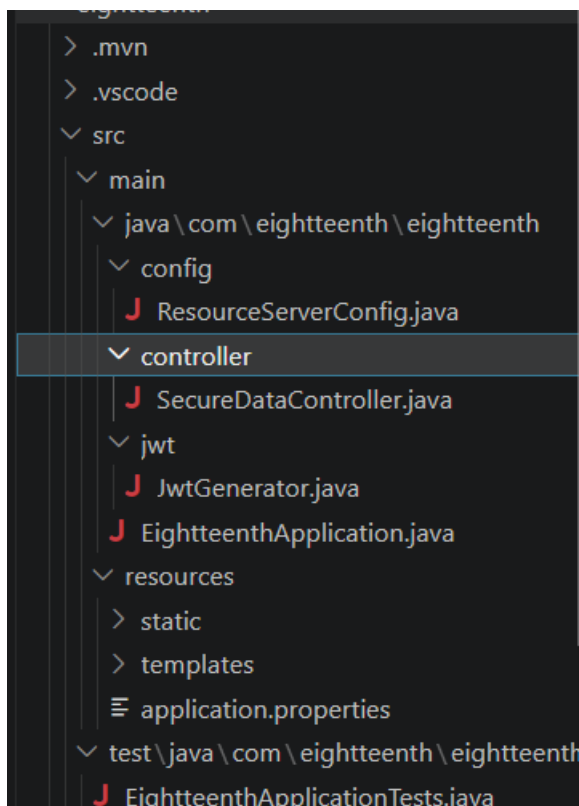
Class Name(s) with Attributes:

ResourceServerConfig (@Configuration — OAuth2 resource server)

- Bean: SecurityFilterChain configured with oauth2ResourceServer().jwt() •

SecureDataController (@RestController)

- Method: getSecureData() — requires valid JWT with configured scope



Dependencies

- oauth2 resource server for jwt
- Security
- jjwt for getting token

ResourceServerConfig.java(config)

```
package com.eightteenth.eightteenth.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.Customizer;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
public class ResourceServerConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(
        HttpSecurity http) throws Exception {

        http
            .csrf(csrf -> csrf.disable())

            .authorizeHttpRequests(auth -> auth

                .requestMatchers("/api/secure/**")
                .authenticated()

                .anyRequest()
                .permitAll()
            )

            .oauth2ResourceServer(
                oauth2 -> oauth2.jwt(Customizer.withDefaults())
            );
    }
}
```

```
    return http.build();  
  }  
}
```

SecureDataController.java(Controller file)

```
package com.eightteenth.eightteenth.controller;  
  
import org.springframework.http.ResponseEntity;  
import org.springframework.web.bind.annotation.*;  
  
@RestController  
@RequestMapping("/api/secure")  
public class SecureDataController {  
  
    @GetMapping("/data")  
    public ResponseEntity<String> getSecureData() {  
  
        return ResponseEntity.ok(  
            "Protected resource data");  
    }  
  
    @PostMapping("/data")  
    public ResponseEntity<String> createSecureData(  
        @RequestBody String data) {  
  
        return ResponseEntity  
            .status(201)  
            .body("Created: " + data);  
    }  
  
    @PutMapping("/data/{id}")  
    public ResponseEntity<String> updateSecureData(  
        @PathVariable Integer id,  
        @RequestBody String data) {  
  
        return ResponseEntity.ok(  
            "Updated " + id + ": " + data);  
    }  
  
    @DeleteMapping("/data/{id}")  
    public ResponseEntity<Void> deleteSecureData(  

```



```
@PathVariable Integer id) {  
  
    return ResponseEntity.noContent().build();  
}  
}
```

JwtGenrator.java(jwt)file

```
package com.eightteenth.eightteenth.jwt;  
  
import java.util.Date;  
  
import org.springframework.boot.CommandLineRunner;  
import org.springframework.stereotype.Component;  
  
import io.jsonwebtoken.Jwts;  
import io.jsonwebtoken.security.Keys;  
  
@Component  
public class JwtGenerator implements CommandLineRunner {  
  
    private static final String SECRET =  
        "01234567890123456789012345678901";  
  
    @Override  
    public void run(String... args) {  
  
        String token = Jwts.builder()  
  
            .subject("admin")  
  
            .claim("scope", "read write")  
  
            .issuedAt(new Date())  
  
            .expiration(  
                new Date(System.currentTimeMillis()  
                    + 3600000)  
            )  
  
            .signWith(  
                Keys.hmacShaKeyFor(  

```

```
        SECRET.getBytes()
    )
)

.compact();

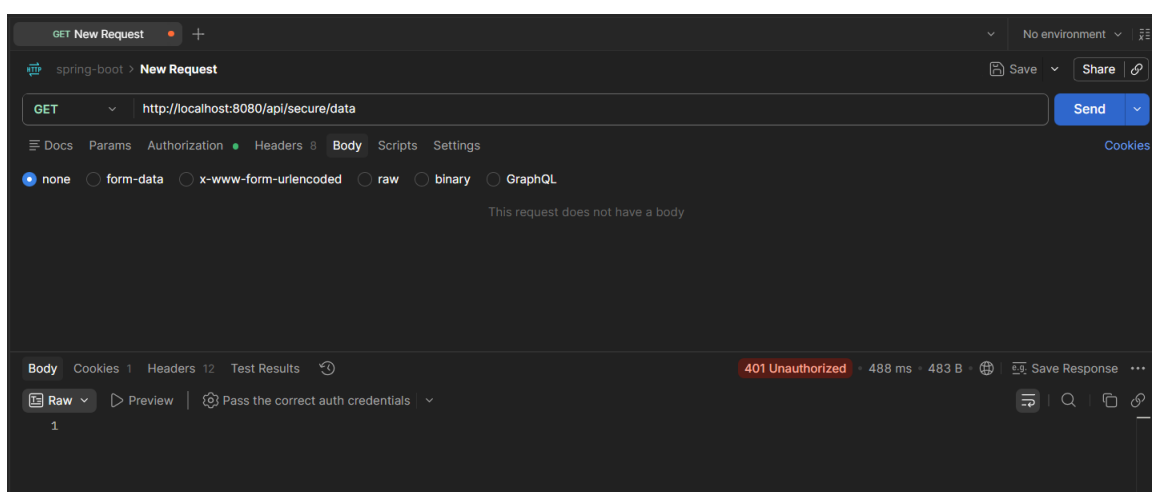
System.out.println();
System.out.println("=====");
System.out.println("        JWT TOKEN");
System.out.println("=====");
System.out.println(token);
System.out.println("=====");
System.out.println();
}
}
```

API'S TESTING USING POSTMAN

1. GET API – Access Secure Data Without JWT

The GET API is used to access a protected resource. Since the `/api/secure/**` endpoints are secured using OAuth2 Resource Server, a valid JWT Bearer token is required to access the API. In this test, no authentication token is provided.

GET <http://localhost:8080/api/secure/data>



The GET API is used to retrieve protected data. A valid JWT access token is sent through the Authorization header using the Bearer Token scheme. Spring Security validates the JWT before allowing the request to reach the controller.

<http://localhost:8080/api/secure/data>

```

PROBLEMS 34 OUTPUT DEBUG CONSOLE TERMINAL PORTS

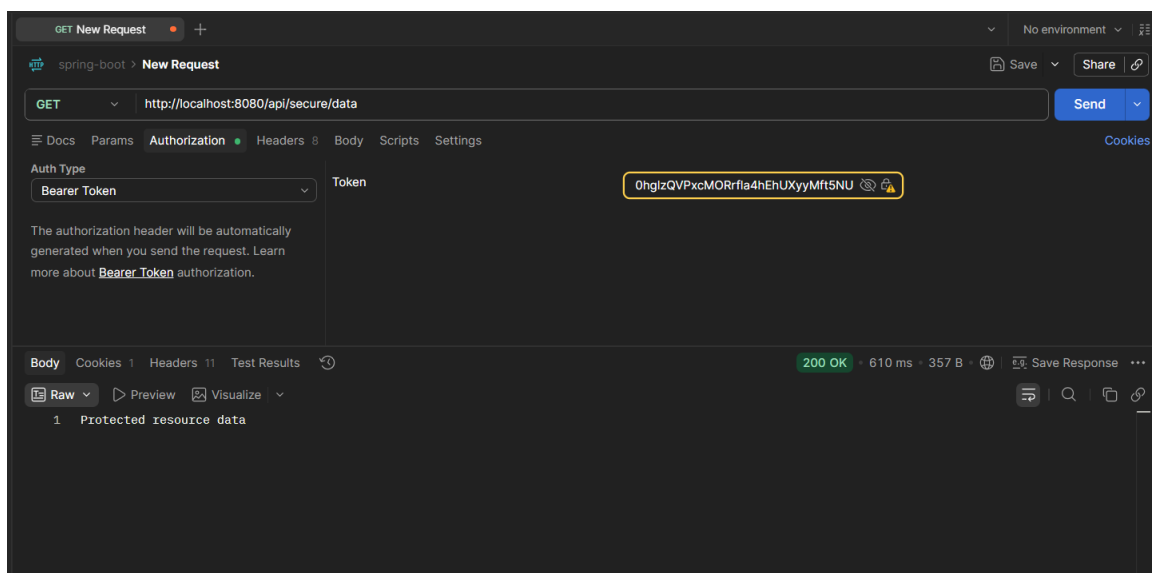
PS C:\Users\karansinh\Desktop\spring-practise> c:; cd 'c:\Users\karansinh\Desktop\spring-practise'; & 'C:\Users\karansinh\.vscode\extensions\vdhat.java-1.55.0-win32-x64\jre\21.0.11-win32-x86_64\bin\java.exe' '-agentlib:jdwp=transport=dt_socket,server=n,suspend=y,address=localhost:6452' '@C:\Users\KARANS~1\AppData\Local\Temp\cp_60mli9v1fkv93eovuee7xog4.argfile' 'com.eightteent.eightteent.EightteentApplication'
cation in 3.932 seconds (process running for 4.502)

=====
JWT TOKEN
=====
eyJhbGciOiJIUzI1Ni9y.eyJzdWIiOiJhZG1pb1IsInNjb3BlIjoicmVhZCB3cm0lZS1lmlhCIG6MTc4NjQ0Tg3NywiZiJwIjoxZg2NDc2NDc3fQ.CdM-gQficvA20hg1zQVPxcMORrf
a4hEhUxyyMft5NU

=====
2026-08-11T23:08:14.254+05:30 INFO 6816 --- [eightteent] [nio-8080-exec-1] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring Dis
atcherServlet 'dispatcherServlet'
2026-08-11T23:08:14.254+05:30 INFO 6816 --- [eightteent] [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet : Initializing Servlet 'd
spatcherServlet'
2026-08-11T23:08:14.263+05:30 INFO 6816 --- [eightteent] [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet : Completed initializatio
in 8 ms

```

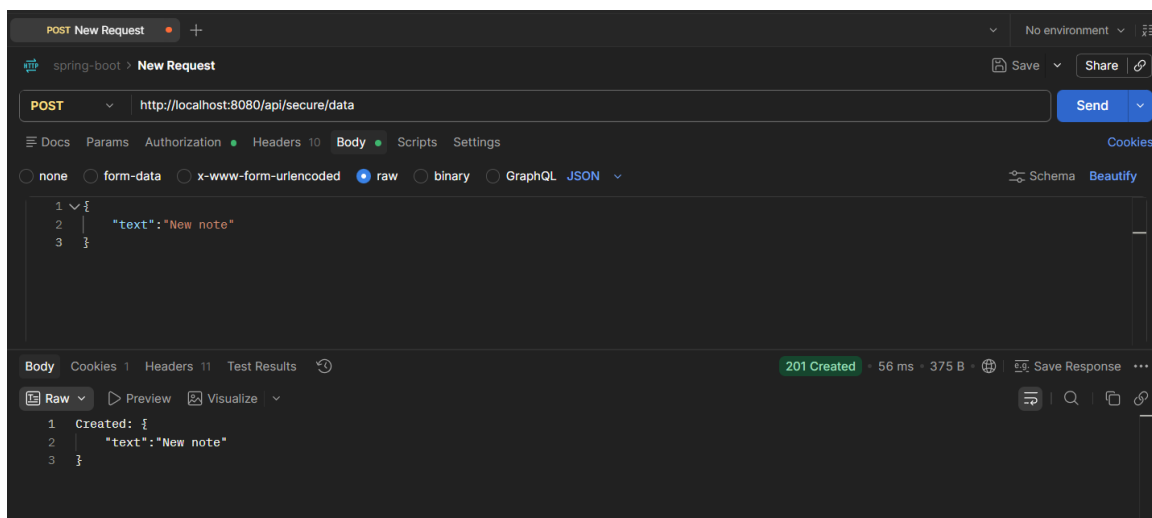
THEY GIVE TOKEN IN CONSOLE COPY AND PASTE IT IN THE POST MAN



3 POST API – Create Secure Data

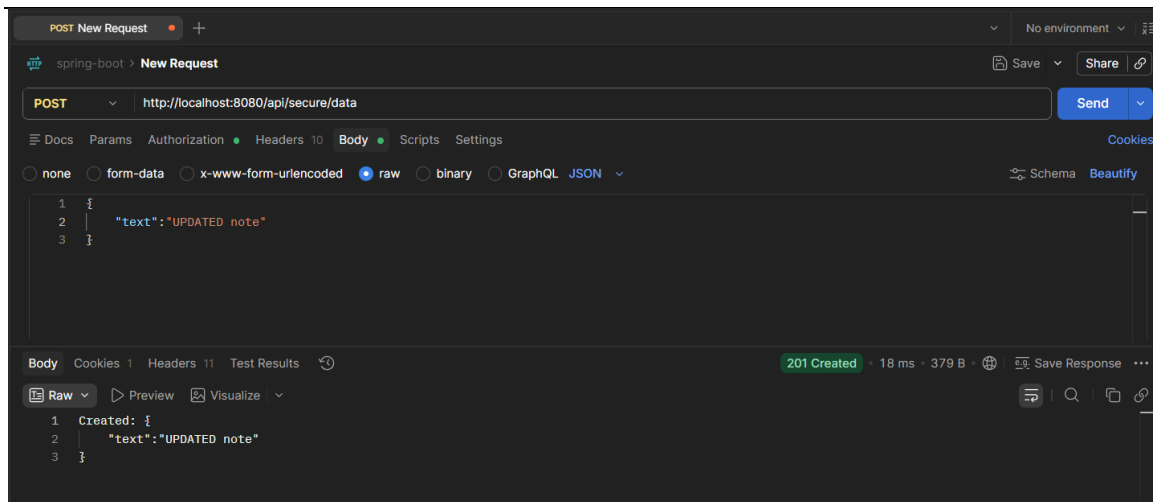
The POST API is used to create new protected data. The request requires a valid JWT access token with the required write scope. The data is sent in the request body.

<http://localhost:8080/api/secure/data>



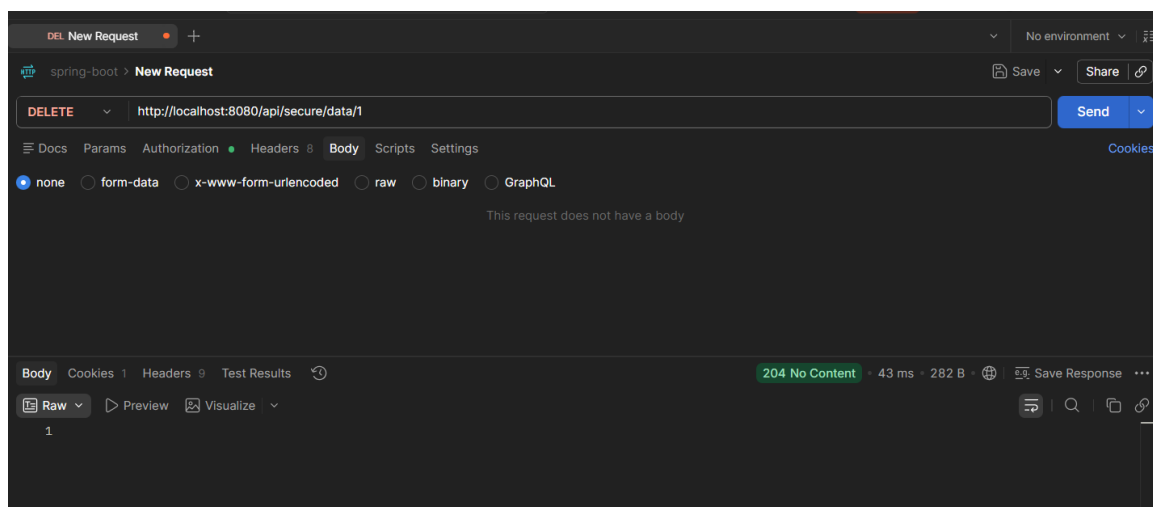
4 PUT API – Update Secure Data

The PUT API is used to update an existing protected resource. The resource ID is passed in the URL and the updated information is provided in the request body. A valid JWT with the required write scope is required.



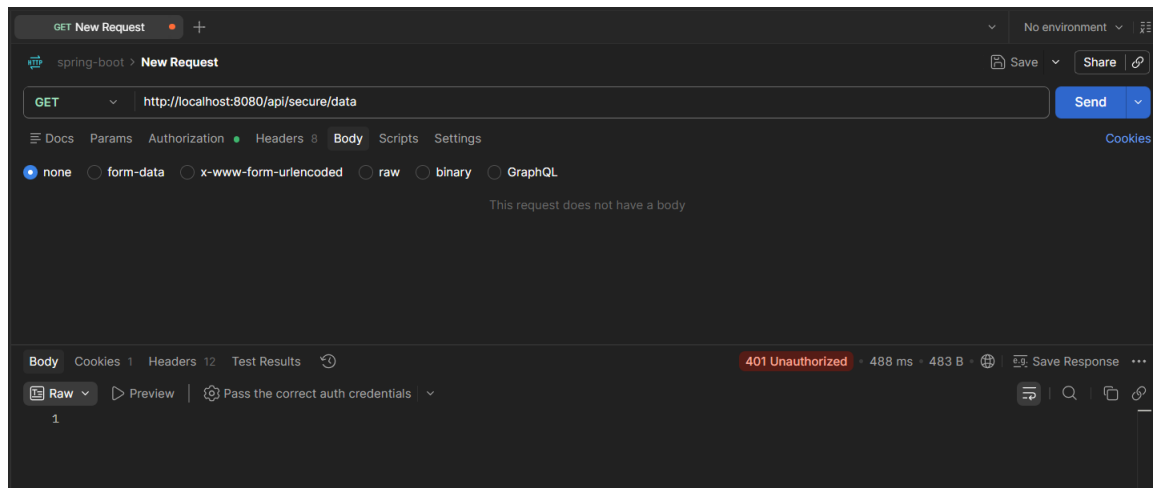
5. DELETE API – Delete Secure Data

The DELETE API is used to delete a protected resource using its ID. A valid JWT access token must be provided with the request.



6. GET API – Access Secure Data with Invalid JWT

The GET API is tested again using an invalid or expired JWT token. Spring Security validates the token before the request reaches the controller. Since the token is invalid, the request is rejected.



**Practical No. 19: OAuth2 Login with a Third-Party
Provider (Google/GitHub) and Accessing a Secured
REST API**

Objective:

To implement the OAuth2 Authorization Code grant using Spring Security's OAuth2 Client support with a real

provider (Google or GitHub), obtain an access token, and use that token to call a secured REST endpoint —

completing this chapter's end-to-end OAuth2 flow using Postman's built-in OAuth 2.0 authorization support.

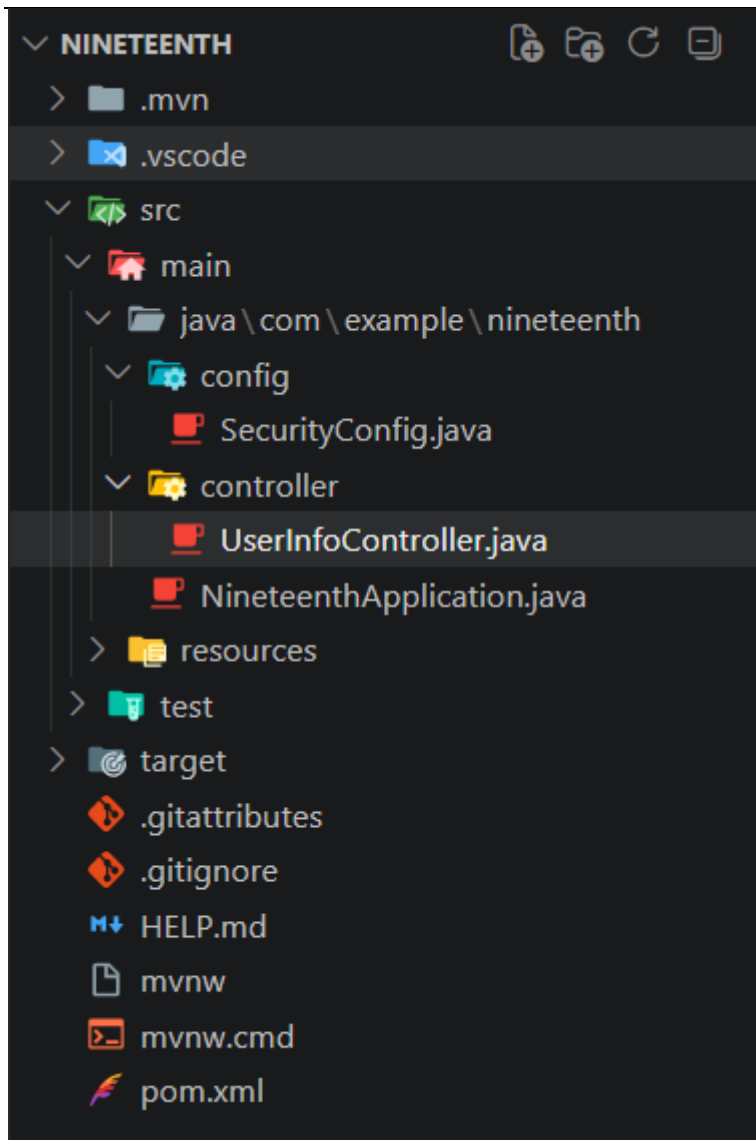
Class Name(s) with Attributes:

OAuth2ClientConfig (application.properties based client registration)

- spring.security.oauth2.client.registration.google.client-id
- spring.security.oauth2.client.registration.google.client-secret
- spring.security.oauth2.client.registration.google.scope

UserInfoController (@RestController)

- Method: getUserProfile(@AuthenticationPrincipal OAuth2User principal) :
Map<String,Object>

**Dependencies:-**

spring-boot-starter-web
spring-boot-starter-security
spring-boot-starter-oauth2-client

SecurityConfig.java(Config)

```
package com.example.nineteenth.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
public class SecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http)
        throws Exception {

        http

            // Disable CSRF for REST API testing
            .csrf(csrf -> csrf.disable())

            .authorizeHttpRequests(auth -> auth

                .requestMatchers("/oauth2/**")
                .permitAll()

                .requestMatchers("/login/**")
                .permitAll()

            );
    }
}
```

```
        .anyRequest()
        .authenticated()
    )

    .oauth2Login(oauth -> {});

    return http.build();
}
}
```

UserInfoController.java(Controller)

```
package com.example.nineteenth.controller;

import java.util.HashMap;
import java.util.Map;

import org.springframework.http.ResponseEntity;
import org.springframework.security.core.annotation.AuthenticationPrincipal;
import org.springframework.security.oauth2.core.user.OAuth2User;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/user")
public class UserInfoController {
```

```
private final Map<Long, Map<String, Object>> notes = new HashMap<>();

private long nextId = 1;


// GET User Profile
@GetMapping("/profile")
public Map<String, Object> getUserProfile(
    @AuthenticationPrincipal OAuth2User principal) {

    return Map.of(
        "name", principal.getAttribute("name"),
        "email", principal.getAttribute("email")
    );
}


// POST Create Note
@PostMapping("/notes")
public ResponseEntity<Map<String, Object>> createNote(
    @RequestBody Map<String, String> request,
    @AuthenticationPrincipal OAuth2User principal) {

    Map<String, Object> note = new HashMap<>();

    note.put("id", nextId);
    note.put("text", request.get("text"));
    note.put("user", principal.getAttribute("email"));

    notes.put(nextId, note);
}
```

```
        nextId++;

        return ResponseEntity.status(201).body(note);
    }

    // PUT Update Note
    @PutMapping("/notes/{id}")
    public ResponseEntity<Map<String, Object>> updateNote(
        @PathVariable Long id,
        @RequestBody Map<String, String> request) {

        Map<String, Object> note = notes.get(id);

        if (note == null) {
            return ResponseEntity.notFound().build();
        }

        note.put("text", request.get("text"));

        return ResponseEntity.ok(note);
    }

    // DELETE Delete Note
    @DeleteMapping("/notes/{id}")
    public ResponseEntity<Void> deleteNote(
        @PathVariable Long id) {
```

```
    if (!notes.containsKey(id)) {  
        return ResponseEntity.notFound().build();  
    }  
  
    notes.remove(id);  
  
    return ResponseEntity.noContent().build();  
}  
}
```

Application.properties

```
spring.application.name=ninteenth  
  
spring.security.oauth2.client.registration.google.client-id=196767098203-  
leb4dmec933m49e7n189idvtlhpim2f2.apps.googleusercontent.com  
spring.security.oauth2.client.registration.google.client-secret=GOCS PX-  
0RwdQN_86F5wgqSHhSNWd5nGiwil  
  
spring.security.oauth2.client.registration.google.scope=openid,profile,email
```

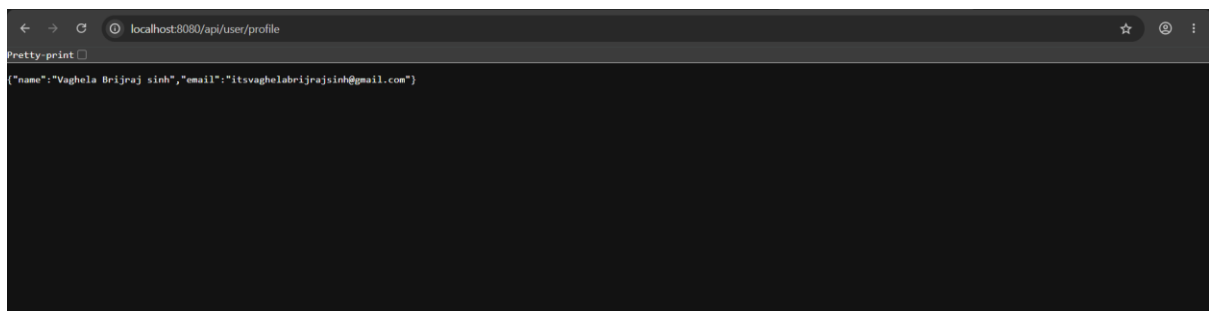
API TESTING USING POSTMAN

1.

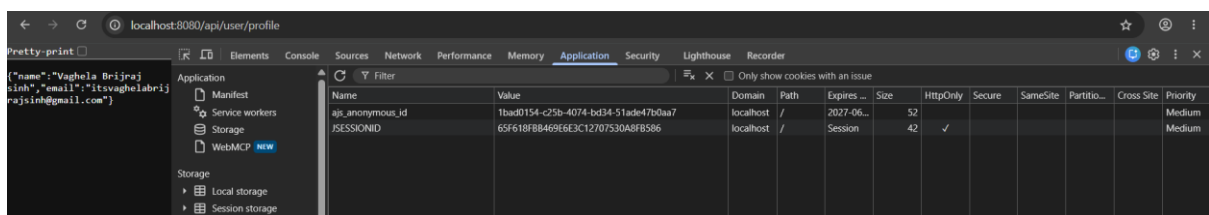
Open Below URL AND select the Email id to login

<http://localhost:8080/oauth2/authorization/google>

Then you will be redirect to this page where Show the name and email address when to use in login:



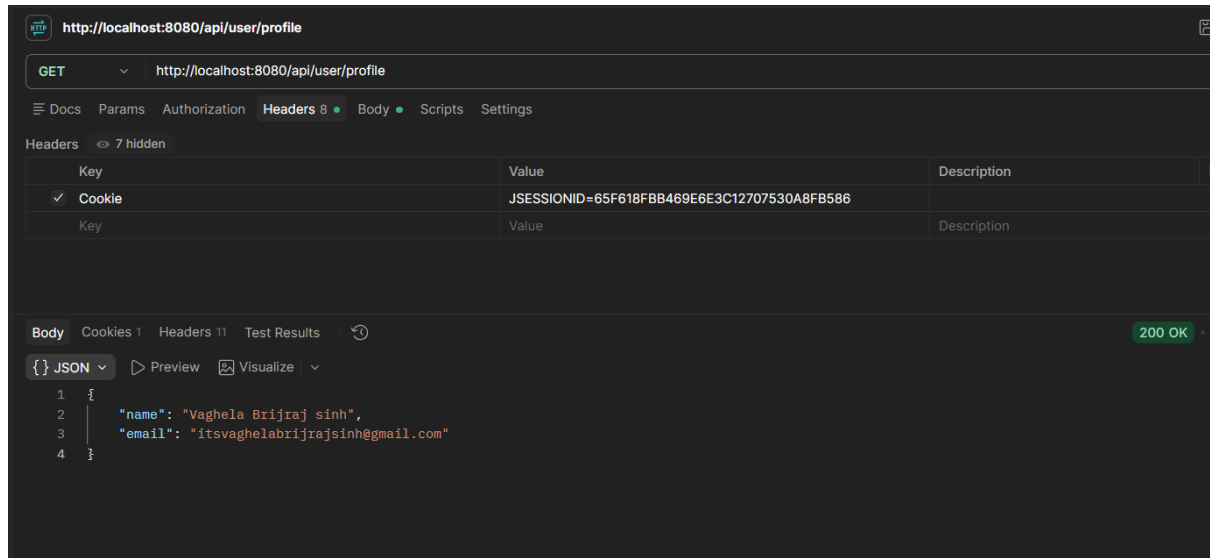
Then Press F12 and open in console Application under cookies show this type



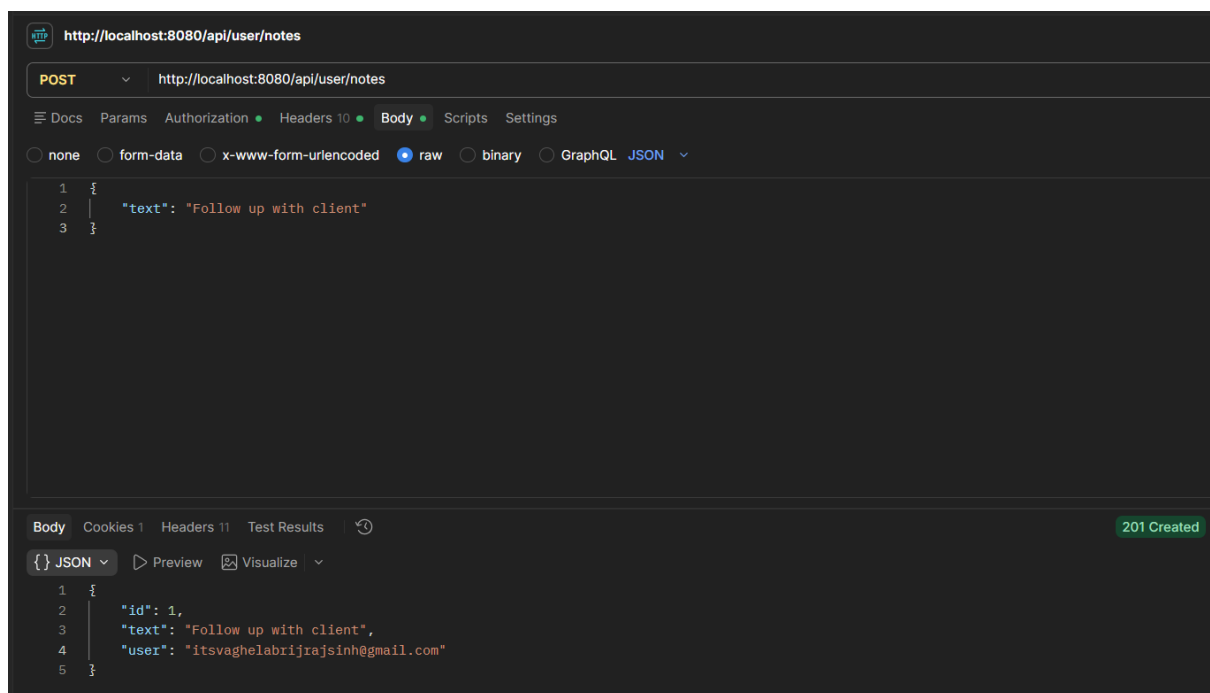
After fetch the data using cookies

Enter cookies and get the name and email:-

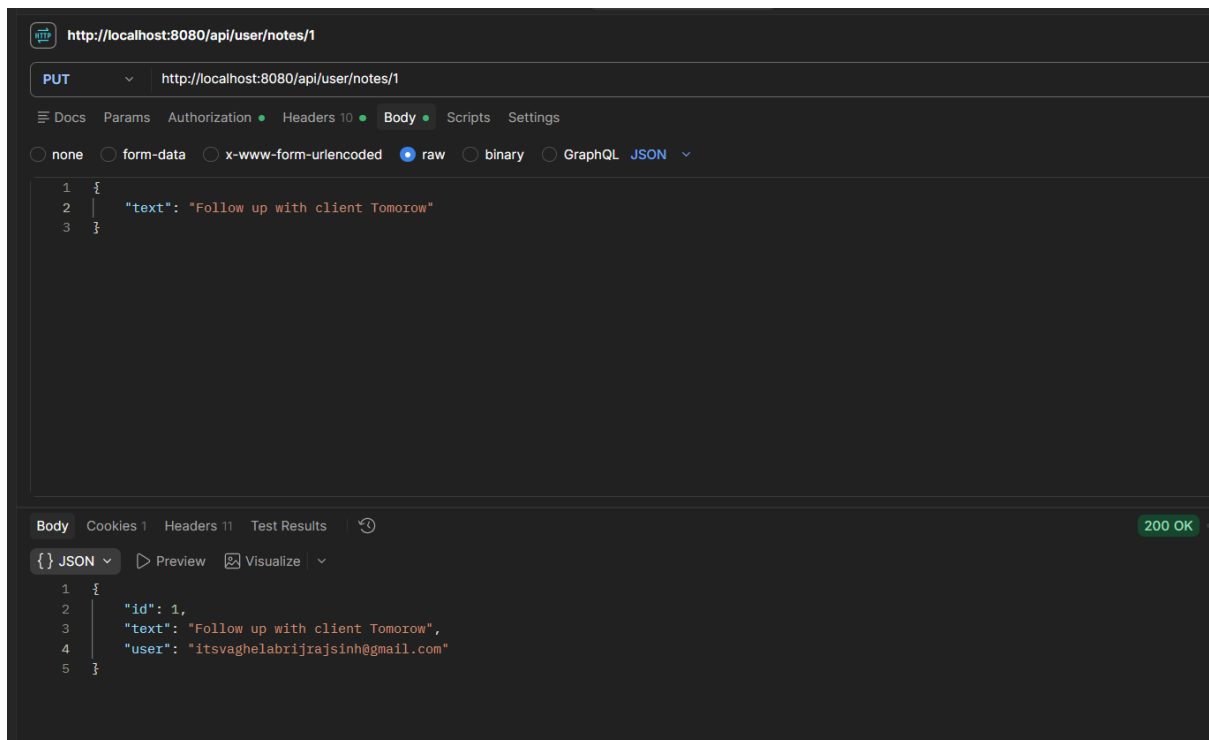
2. GET API:- RETRIVE EMAIL AND NAME



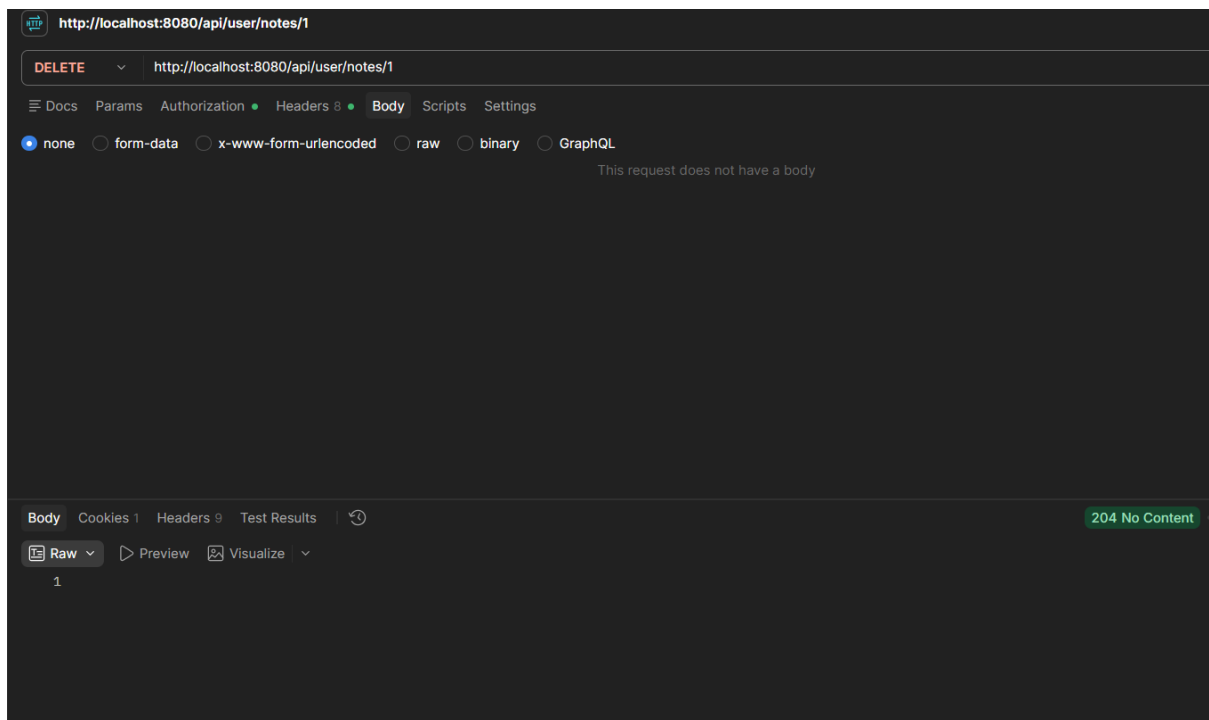
3. POST API:- ADDED NOTE



4.PUT API:- UPDATED NOTE



5.DELETE API:- NOTE DELETED Successfully....



Practical No. 20: Cross-Origin Resource Sharing (CORS) Configuration for Secured REST APIs

Objective:

To configure CORS at the application level so that a secured REST API can be safely consumed by a browser-

based frontend hosted on a different origin — a practical security configuration concern addressed alongside

authentication/authorization in this chapter.

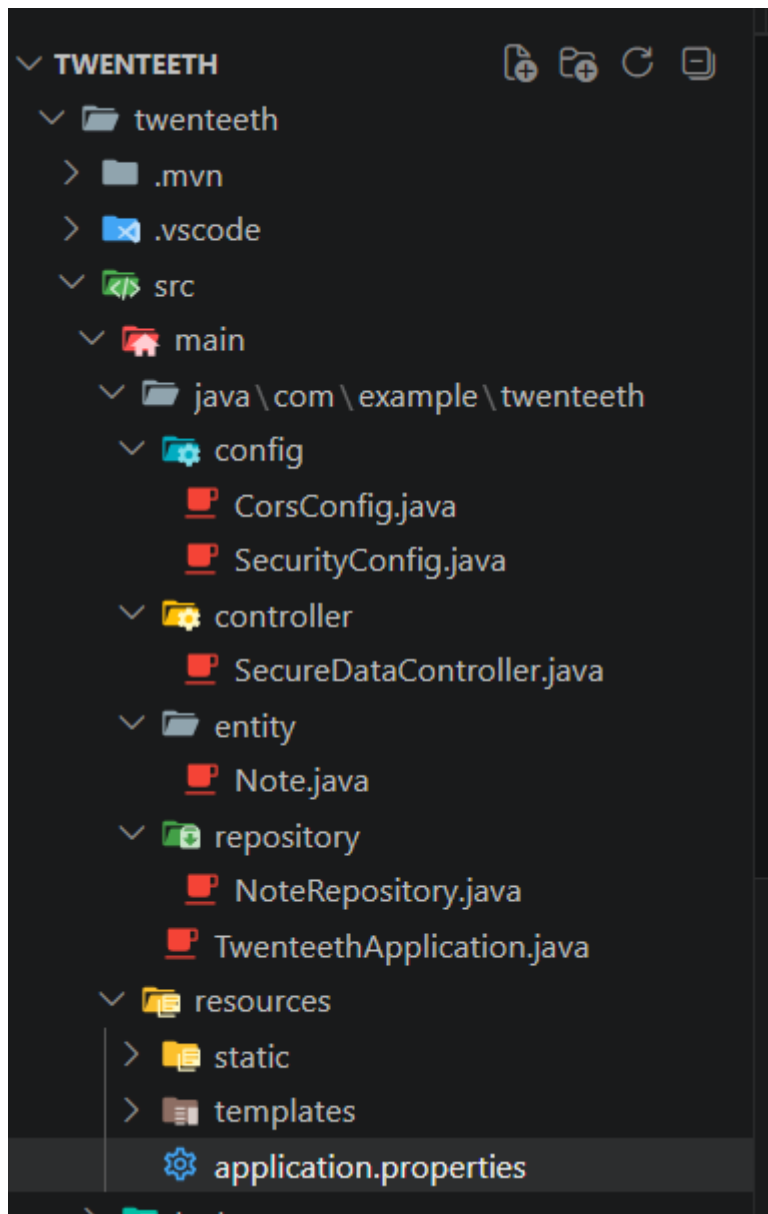
Class Name(s) with Attributes:

CorsConfig (@Configuration implements WebMvcConfigurer)

- Method: addCorsMappings(CorsRegistry registry)

SecurityConfig (extended)

- .cors(Customizer.withDefaults()) enabled in the SecurityFilterChain

**Dependencies:-**

- spring-boot-starter-web
 - spring-boot-starter-security
 - spring-boot-starter-data-jpa
 - h2
-

CorsConfig.java(config)

```
package com.example.twentieth.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.cors.CorsConfiguration;
import org.springframework.web.cors.CorsConfigurationSource;
import org.springframework.web.cors.UrlBasedCorsConfigurationSource;

import java.util.List;

@Configuration
public class CorsConfig {

    @Bean
    public CorsConfigurationSource corsConfigurationSource() {

        CorsConfiguration configuration = new CorsConfiguration();

        configuration.setAllowedOrigins(
            List.of("http://localhost:3000")
        );

        configuration.setAllowedMethods(
            List.of("GET", "POST", "PUT", "DELETE", "OPTIONS")
        );
    }
}
```

```
configuration.setAllowedHeaders(  
    List.of("*")  
);  
  
UrlBasedCorsConfigurationSource source =  
    new UrlBasedCorsConfigurationSource();  
  
source.registerCorsConfiguration(  
    "/api/**",  
    configuration  
);  
  
return source;  
}  
}
```

SecurityConfig.java(config)

```
package com.example.twentieth.config;  
  
import org.springframework.context.annotation.Bean;  
import org.springframework.context.annotation.Configuration;  
import org.springframework.security.config.Customizer;  
import org.springframework.security.config.annotation.web.builders.HttpSecurity;  
import org.springframework.security.core.userdetails.User;  
import org.springframework.security.core.userdetails.UserDetails;  
import org.springframework.security.provisioning.InMemoryUserDetailsManager;
```

```
import org.springframework.security.core.userdetails.UserDetailsService;

import org.springframework.security.web.SecurityFilterChain;

@Configuration

public class SecurityConfig {

    @Bean

    public SecurityFilterChain securityFilterChain(

        HttpSecurity http) throws Exception {

        http

            // Enable CORS

            .cors(Customizer.withDefaults())

            // Disable CSRF for REST API testing

            .csrf(csrf -> csrf.disable())

            // Authorization

            .authorizeHttpRequests(auth -> auth

                .requestMatchers("/api/secure/**")

                .authenticated()

                .anyRequest()

                .permitAll()

            )

            // Basic Authentication

            .httpBasic(Customizer.withDefaults());
```

```
        return http.build();
    }

    @Bean
    public UserDetailsService userDetailsService() {

        UserDetails user = User
            .withUsername("admin")
            .password("{noop}1234")
            .roles("USER")
            .build();

        return new InMemoryUserDetailsManager(user);
    }
}
```

SecureDataController.java(Controller)

```
package com.example.twentieth.controller;

import com.example.twentieth.entity.Note;
import com.example.twentieth.repository.NoteRepository;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
```

```
import java.util.List;

@RestController
@RequestMapping("/api/secure/data")
public class SecureDataController {

    private final NoteRepository noteRepository;

    public SecureDataController(NoteRepository noteRepository) {
        this.noteRepository = noteRepository;
    }

    // GET
    @GetMapping
    public ResponseEntity<List<Note>> getAllNotes() {

        return ResponseEntity.ok(noteRepository.findAll());
    }

    // POST
    @PostMapping
    public ResponseEntity<Note> createNote(@RequestBody Note note) {

        Note savedNote = noteRepository.save(note);

        return ResponseEntity.status(HttpStatus.CREATED).body(savedNote);
    }
}
```

```
// PUT
@PutMapping("/{id}")
public ResponseEntity<Note> updateNote(
    @PathVariable Long id,
    @RequestBody Note updatedNote) {

    return noteRepository.findById(id)
        .map(note -> {

            note.setText(updatedNote.getText());

            Note savedNote = noteRepository.save(note);

            return ResponseEntity.ok(savedNote);
        })
        .orElseGet(() ->
            ResponseEntity.notFound().build()
        );
}

// DELETE
@DeleteMapping("/{id}")
public ResponseEntity<Void> deleteNote(@PathVariable Long id) {

    if (!noteRepository.existsById(id)) {
        return ResponseEntity.notFound().build();
    }
}
```



```
}

    noteRepository.deleteById(id);

    return ResponseEntity.noContent().build();
}
}
```

Note.java(Entity)

```
package com.example.twentieth.entity;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;

@Entity
public class Note {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String text;

    public Note() {
```

```
}

public Note(String text) {
    this.text = text;
}

public Long getId() {
    return id;
}

public void setId(Long id) {
    this.id = id;
}

public String getText() {
    return text;
}

public void setText(String text) {
    this.text = text;
}
}
```

NoteRepository.java(Repository)

```
package com.example.twentieth.repository;

import com.example.twentieth.entity.Note;
import org.springframework.data.jpa.repository.JpaRepository;

public interface NoteRepository extends JpaRepository<Note, Long> {
}
```

Application.properties

```
spring.application.name=twentieth

# H2 Database
spring.datasource.url=jdbc:h2:mem:corsdb
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=

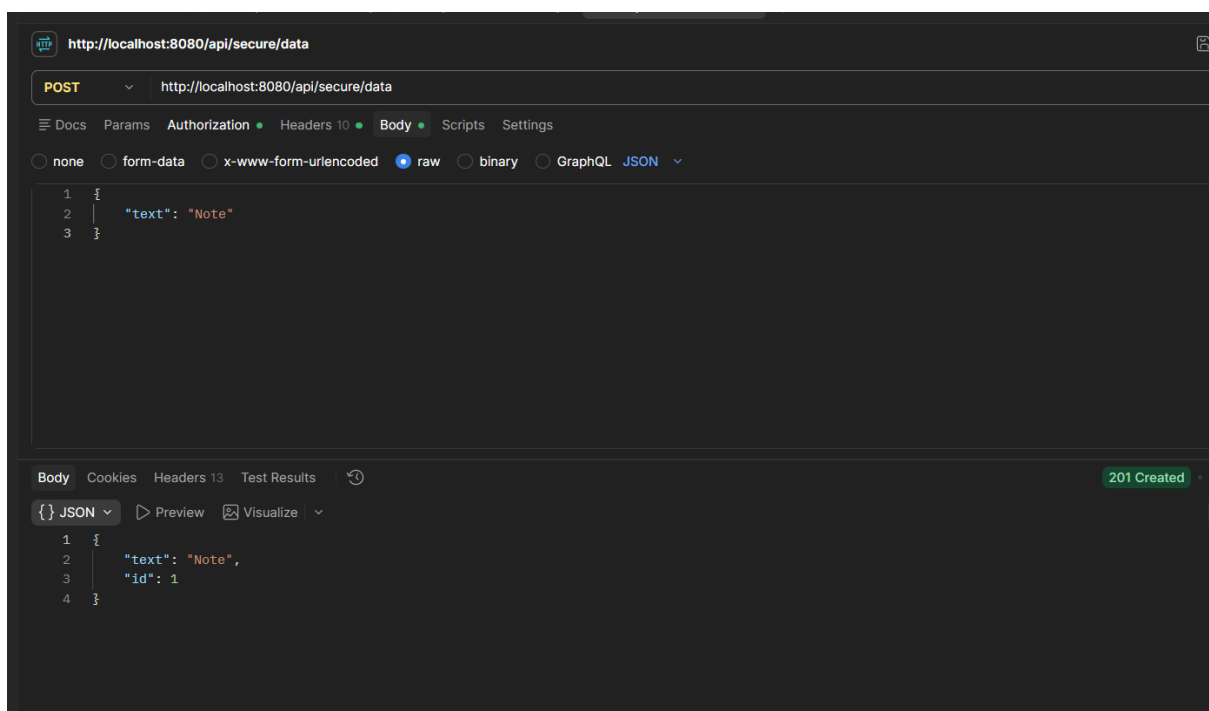
# JPA
spring.jpa.hibernate.ddl-auto=create
spring.jpa.show-sql=true
```

API'S TESTING USING POSTMAN

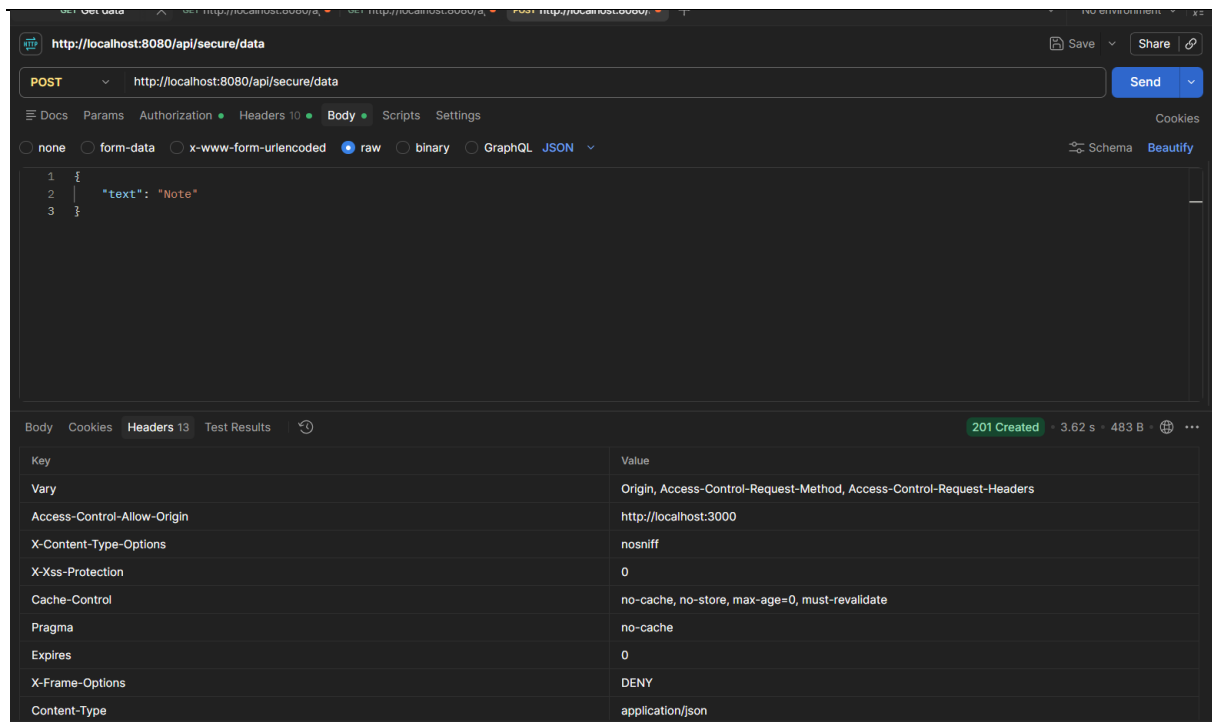
Before the POST API , entered username and password which defined in **SecurityConfig.java**

1. **POST API** – A POST request was sent to `/api/secure/data` with Basic Authentication and the Origin: `http://localhost:3000` header. The note was successfully created with **201 Created**, and the CORS header was present in the response.

POST <http://localhost:8080/api/secure/data>

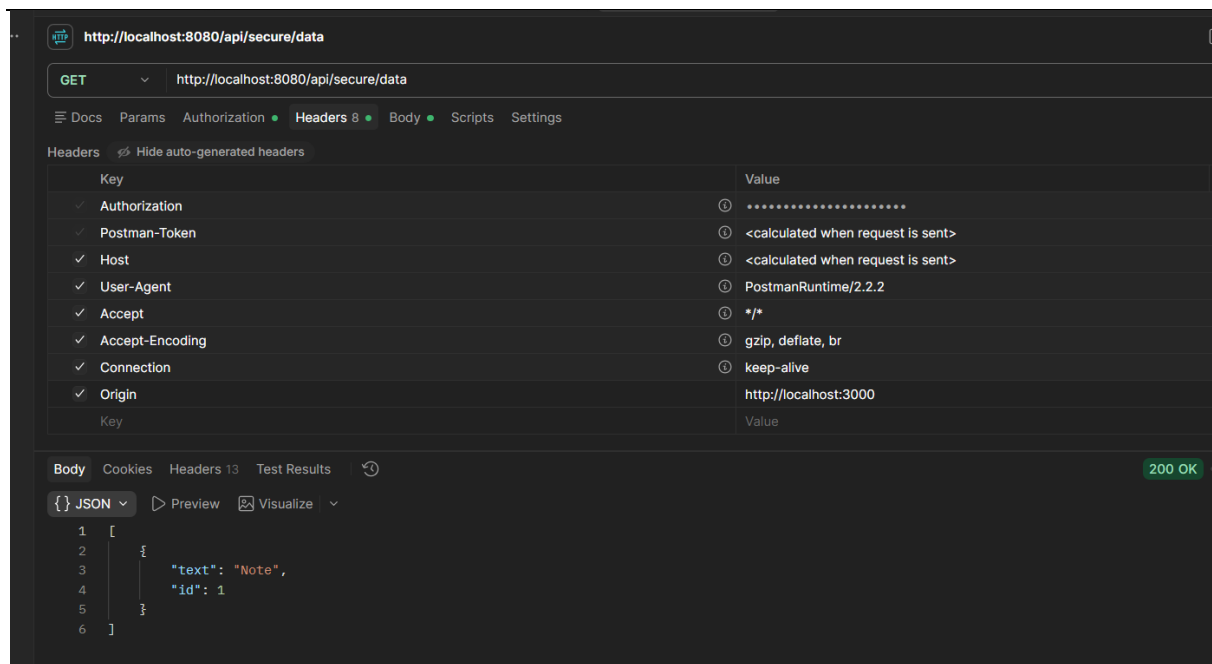


Verify in Header part in response show Access-control-Allow-origin.



2. **GET – A Retrive NOTE:-** A GET request was sent to `/api/secure/data` with Basic Authentication and the allowed origin. The server successfully returned the stored notes with **200 OK**, along with the Access-Control-Allow-Origin header.

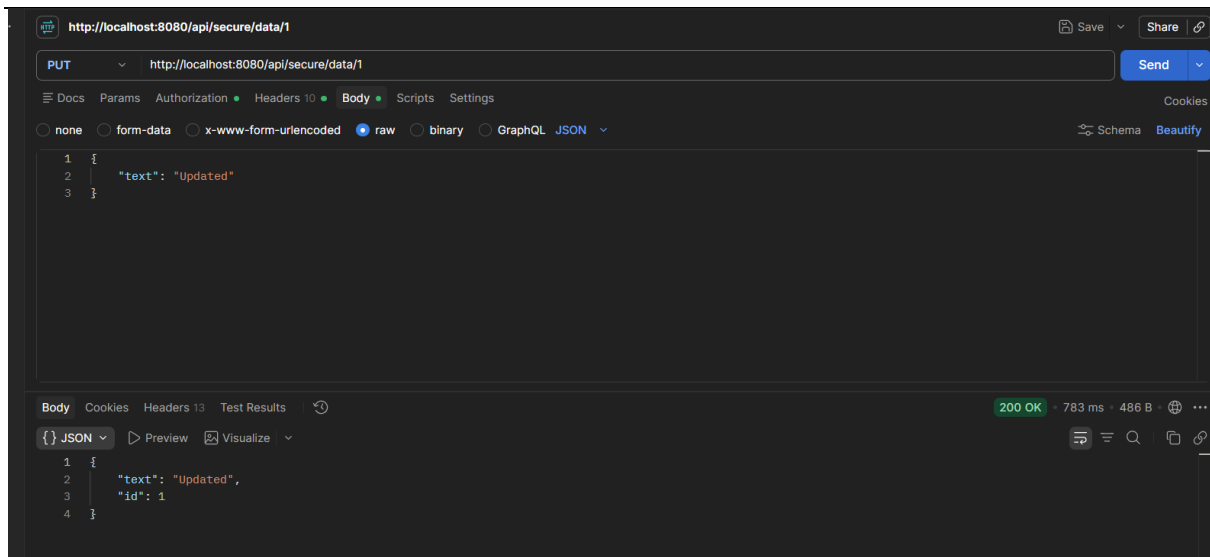
GET <http://localhost:8080/api/secure/data>



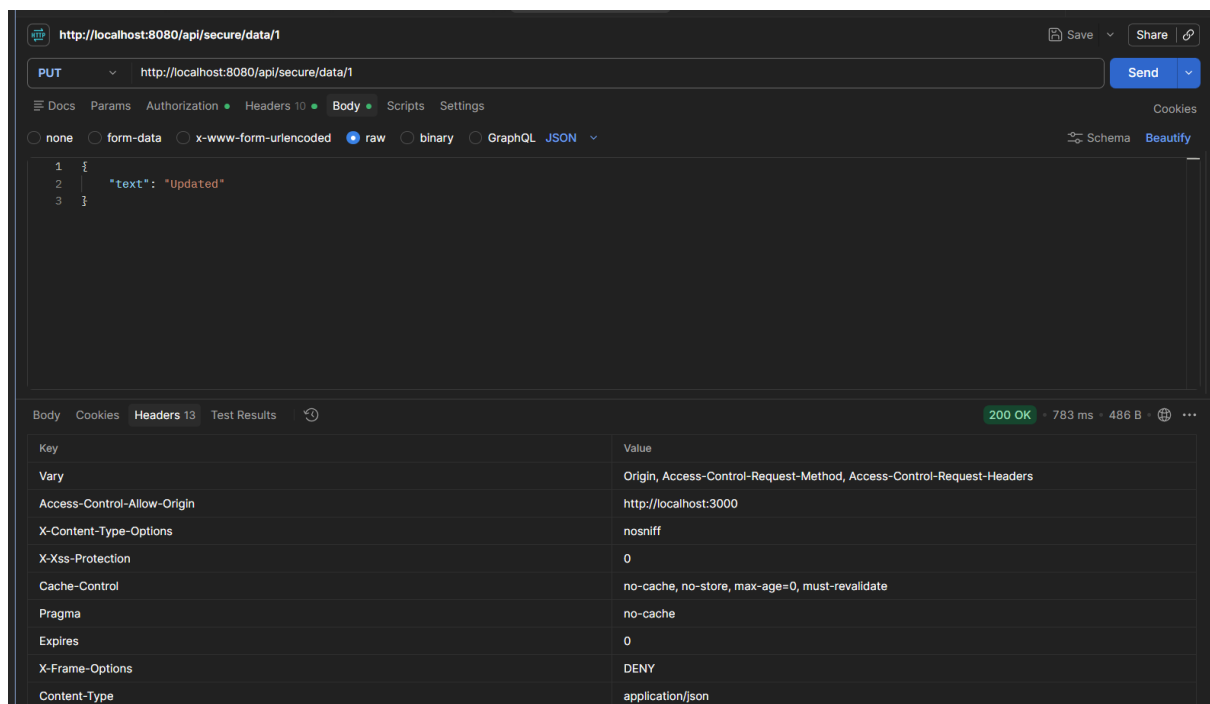
3. PUT – Update Note

A PUT request was sent to `/api/secure/data/1` with the updated note data. The note was successfully updated with **200 OK**, and the CORS response header was present.

PUT <http://localhost:8080/api/secure/data/1>



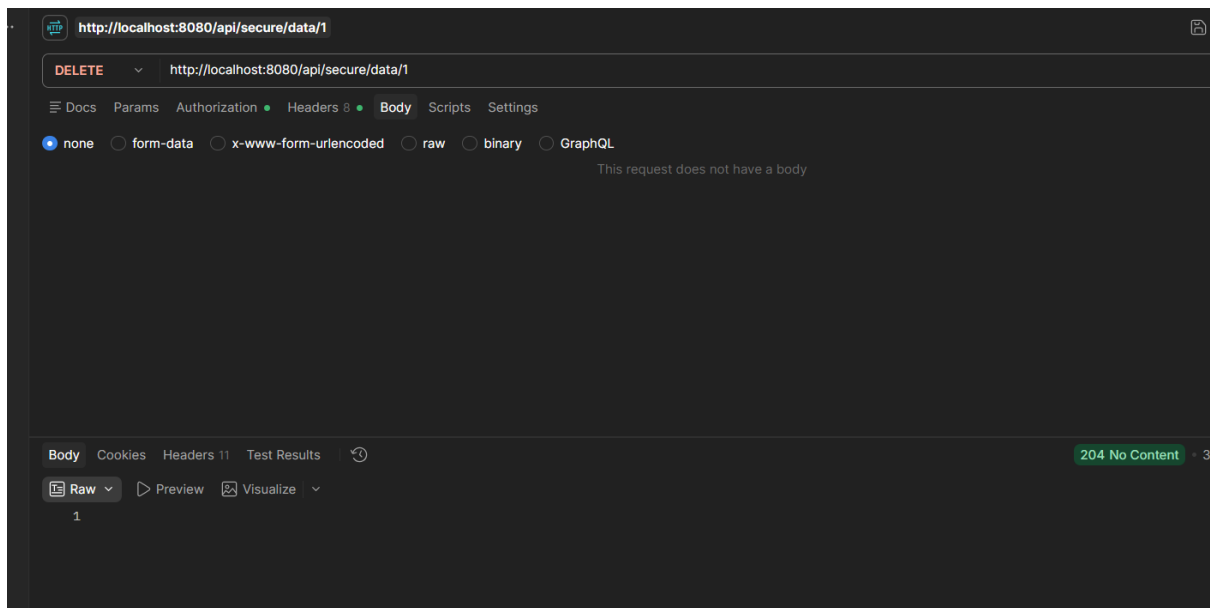
Also verify in Response under Header The url.



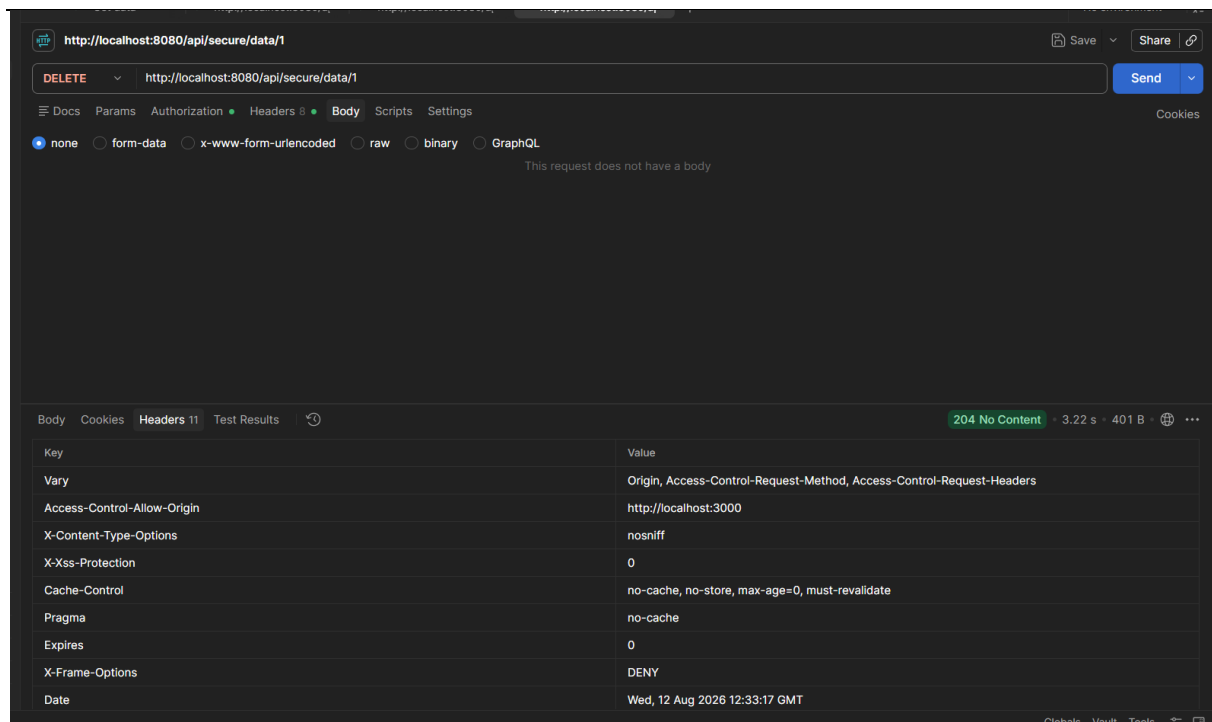
4. DELETE – Delete Note

A DELETE request was sent to `/api/secure/data/1` with Basic Authentication and the allowed origin. The note was successfully deleted, returning **204 No Content** with the CORS header present.

DELETE <http://localhost:8080/api/secure/data/1>



After verify also In Header part of response.

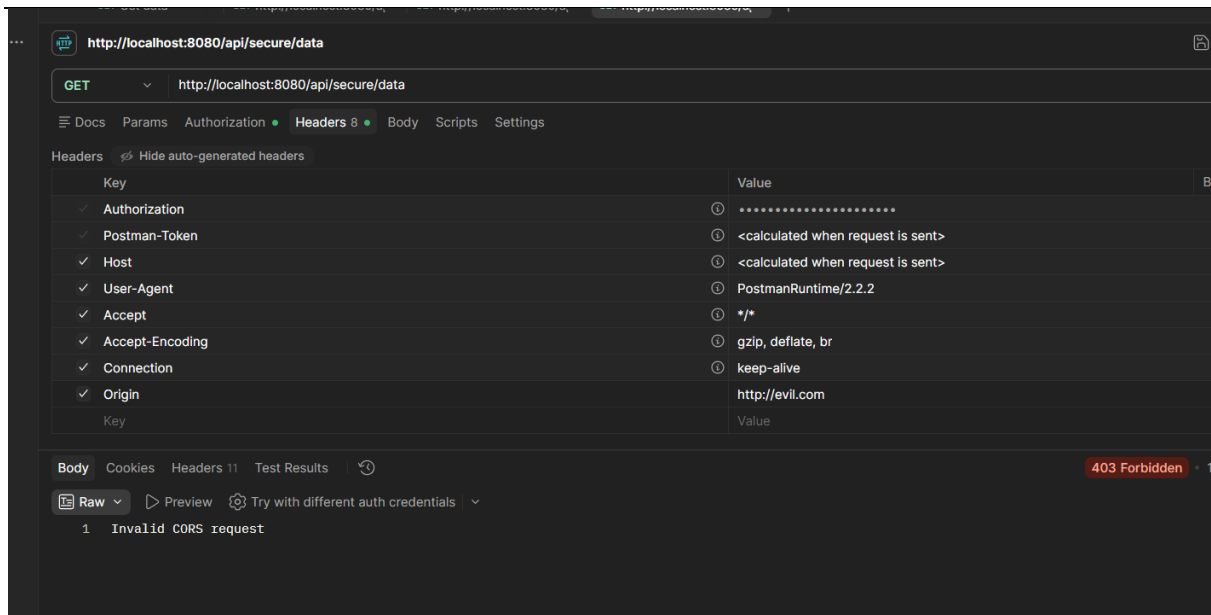


5. Disallowed Origin:-

First add url in origin under header <http://evil.com>

A GET request was tested using Origin: `http://evil.com` instead of the allowed origin. The request was rejected with **403 Forbidden**, and the `Access-Control-Allow-Origin` header for `evil.com` was not present.

GET <http://localhost:8080/api/secure/data>



.....THE END.....